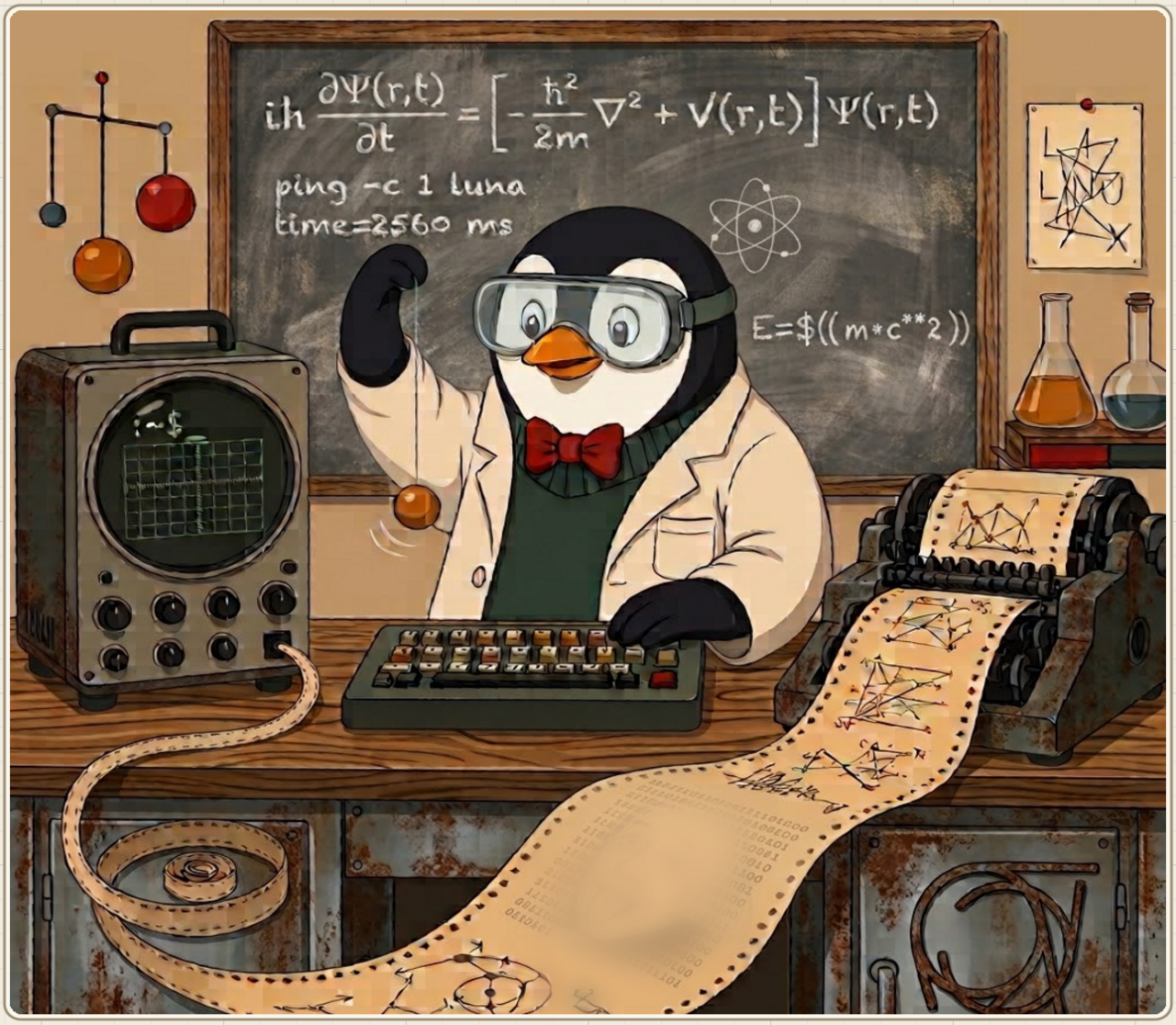
LINUX · REDES · SISTEMAS · PARA JÓVENES CIENTÍFICOS

# Estofado de Linux

## con guarnición de IA y redes




Trabajar en Linux y conceptos de redes, explicado tan «fácil» que hasta un físico lo puede entender

Claudio M. Martínez Velasco

VERSIÓN AGOSTO 2026

# Estofado de Linux

**con guarnición de IA y redes**

Claudio M. Martínez Velasco

agosto de 2026

**Estofado de Linux**
con guarnición de IA y redes

Trabajar en Linux y conceptos de redes, explicado tan «fácil» que hasta un físico lo puede entender

Versión: agosto 2026.

© 2026 Claudio M. Martínez Velasco

El texto de este libro se publica bajo licencia Creative Commons Atribución–CompartirIgual 4.0 Internacional (CC BY-SA 4.0): puedes copiarlo, redistribuirlo y adaptarlo, citando al autor y manteniendo la misma licencia.
https://creativecommons.org/licenses/by-sa/4.0/deed.es

Las fotografías conservan las licencias indicadas en su pie de figura (Wikimedia Commons y dominio público); las capturas del instalador de Debian son GPL.

Edición web, con ejercicios interactivos y datos descargables:
https://clamaveruma.github.io/curso-linux-fisica/

Comentarios y sugerencias: mailto:clamaver@uma.es

Compuesto con Quarto y LaTeX.

# Tabla de contenidos

**Bienvenida** **8**
Cómo leer este curso: lo básico, lo avanzado y la IA . . . 8
Qué necesitas para empezar . . . 9

**1 Abrir la caja negra** **10**
1.1 Qué es esto que parpadea . . . 10
1.2 Primeras palabras . . . 13
1.3 Anatomía de un comando . . . 14
1.4 El territorio: moverse por el árbol . . . 14
1.4.1 Rutas absolutas y relativas . . . 15
1.5 Los dos superpoderes . . . 16
1.5.1 El tabulador: nunca escribas nombres enteros . . . 16
1.5.2 El historial: nunca escribas nada dos veces . . . 17
1.6 Pedir ayuda sin salir del sistema . . . 17
1.7 Tu cuaderno de laboratorio: `script` . . . 19
1.8 Práctica: la expedición . . . 19
1.9 Autocomprobación . . . 20
Soluciones del capítulo . . . 21

**2 Ficheros: tocar sin romper** **23**
2.1 Mirar dentro sin miedo . . . 23
2.2 Crear: `mkdir` y `touch` . . . 24
2.3 Copiar y mover: `cp` y `mv` . . . 25
2.4 Borrar: el comando sin papelera . . . 26
2.5 Comodines: hablar de familias enteras . . . 26
2.6 Encontrar: `find` . . . 27
2.7 Editar: `nano` (y la verdad sobre `vim`) . . . 28
2.8 Tus primeros *dotfiles* . . . 28
2.9 Práctica: poner orden en el laboratorio . . . 30
2.10 Autocomprobación . . . 31
Soluciones del capítulo . . . 32

**3 El flujo de datos** **35**
3.1 Los tres chorros . . . 35
3.2 Redirigir: `>` y `>>` . . . 36
3.3 La tubería: `|` . . . 37
3.4 Lo que usarás de verdad . . . 38
3.5 La caja de herramientas . . . 38

3.6 El análisis, pieza a pieza . . . 39
3.7 ¿Y Python? . . . 41
3.8 Práctica: la noche en el laboratorio . . . 42
3.9 Autocomprobación . . . 44
Soluciones del capítulo . . . 44

**4 El árbol y los permisos 48**
4.1 El mapa del territorio . . . 48
4.2 Quién eres: usuarios y grupos . . . 49
4.3 `root` y `sudo`: llamar a la puerta del administrador . . . 51
4.4 Leer `rwx` de un vistazo . . . 51
4.5 Tu primer script ejecutable . . . 52
4.6 La expedición de las configuraciones . . . 53
4.7 Práctica: inspección general . . . 53
4.8 Autocomprobación . . . 55
Soluciones del capítulo . . . 55

**5 Paquetes y procesos 58**
5.1 Instalar por debajo del envoltorio . . . 58
5.2 Qué es una distro (ahora ya lo puedes entender) . . . 59
5.3 Qué está pasando ahora mismo: procesos . . . 60
5.4 Terminar lo colgado: señales y `kill` . . . 61
5.5 Segundo plano: el `&` . . . 62
5.6 El `$PATH`: la deuda del `./` . . . 62
5.7 Práctica: el taller . . . 64
5.8 Autocomprobación . . . 65
Soluciones del capítulo . . . 65

**6 Discos, particiones y arranque 68**
6.1 El mapa de tus discos: `lsblk` . . . 68
6.2 Sistemas de ficheros: el orden dentro del bloque . . . 69
6.3 Montar: enganchar discos al árbol . . . 69
6.4 ¿Cuánto espacio queda? `df` y `du` . . . 70
6.5 La partición EFI y el arranque . . . 71
6.6 Práctica: inspección del quirófano . . . 72
6.7 Autocomprobación . . . 74
Soluciones del capítulo . . . 74

**7 Tu primera máquina virtual 77**
7.1 Un ordenador dentro del ordenador . . . 77
7.2 Montar el taller . . . 79
7.3 La imagen de instalación . . . 80
7.4 Crear la máquina . . . 81
7.5 El superpoder: instantáneas . . . 82
7.6 Dos formas de conectarla: NAT y puente . . . 84
7.7 Qué te dan cuando te dan «una VM» . . . 84

7.8 Práctica: el taller en marcha . . . 85
7.9 Autocomprobación . . . 87
Soluciones del capítulo . . . 87

**8 Instalar Linux de verdad** **90**
8.1 El plan, antes que el instalador . . . 90
8.2 Arrancar el instalador . . . 91
8.3 Particionar a mano . . . 93
8.4 Espejo, programas y GRUB . . . 93
8.5 El primer arranque: comprobar que respira . . . 94
8.6 Romper, mirar, restaurar . . . 95
8.7 Del ensayo al ordenador de verdad . . . 95
8.8 Práctica: la sala de operaciones . . . 96
8.9 Autocomprobación . . . 99
Soluciones del capítulo . . . 99

**9 Qué es una red** **103**
9.1 Cómo se hablan dos ordenadores . . . 103
9.2 Nivel 2 y nivel 3: switches y routers . . . 105
9.3 Tu ficha de red: `ip a` . . . 105
9.4 El `/24`: red y máquina . . . 106
9.5 ¿Estás ahí? `ping` . . . 108
9.6 Práctica: leer la red . . . 110
9.7 Autocomprobación . . . 110
Soluciones del capítulo . . . 111

**10 Tu red de casa: el router por dentro** **113**
10.1 ¿Quién reparte las direcciones? DHCP . . . 113
10.2 Privada, pública, y el NAT entre medias . . . 115
10.3 El router por dentro (solo lectura) . . . 117
10.4 Cable o Wi-Fi . . . 117
10.5 El protocolo del cambio reversible . . . 118
10.6 Práctica: el censo de casa . . . 118
10.7 Autocomprobación . . . 120
Soluciones del capítulo . . . 120

**11 Nombres, puertos y protocolos** **123**
11.1 Qué pasa al escribir un nombre: DNS . . . 123
11.2 El camino: `tracepath` . . . 124
11.3 Una dirección, muchas puertas: los puertos . . . 125
11.4 Cortafuegos, lo básico . . . 128
11.5 Qué habla cada puerta: los protocolos que usarás . . . 129
11.6 Práctica: la guía y las puertas . . . 129
11.7 Autocomprobación . . . 131
Soluciones del capítulo . . . 131

**12 SSH: entrar en otra máquina** **134**
12.1 Primero con ratón: otro ordenador en tu gestor de archivos . . . 135
12.2 Cliente y servidor . . . 135
12.3 Nivel 0: tu propia máquina como servidor . . . 136
12.4 Claves en vez de contraseñas . . . 137
12.5 Un nombre para cada servidor: `~/.ssh/config` . . . 139
12.6 Mover ficheros: `scp` y `rsync` . . . 140
12.7 Ejecutar allí, sin quedarse . . . 140
12.8 Nivel 2: cuando el servidor es de verdad . . . 141
12.9 Práctica: la escalera . . . 142
12.10 Autocomprobación . . . 144
Soluciones del capítulo . . . 145

**13 Vivir en remoto** **148**
13.1 La sesión que muere . . . 148
13.2 `tmux`: la sesión que no muere . . . 149
13.3 El túnel: un servicio remoto en tu navegador . . . 150
13.4 Qué es «la nube», en concreto . . . 151
13.5 Y si algún día te dan un clúster . . . 152
13.6 Práctica: vivir en el servidor . . . 153
13.7 Autocomprobación . . . 154
Soluciones del capítulo . . . 155

**Proyecto final: tu primer servidor** **157**
Las reglas . . . 157
Los hitos . . . 157
El análisis de partida . . . 158
Hoja de verificación . . . 159
Después . . . 160

**Apéndices** **161**

**A La IA en la terminal: ayudante, no piloto** **161**
El fontanero . . . 161
Instalar Antigravity CLI . . . 161
Configurarlo para aprender, desde el principio . . . 162
Las reglas para seguir mandando tú . . . 162
Cómo pedirle las cosas . . . 163
Lo que la IA no sustituye . . . 164
Práctica . . . 164
Soluciones del capítulo . . . 165

**B Vengo de Windows (o de macOS)** **166**
Los dos caminos . . . 166
Camino A: Mint en VirtualBox . . . 166

Camino B: WSL2 . . . 167
Dos caminos más: el USB persistente y el arranque dual . . . 167
Camino C · Un Linux *live* en un pendrive, con persistencia . . . 167
Camino D · Arranque dual: solo con alguien que sepa . . . 168
Lo que no cambia . . . 169

**C Instalar en un PC real: la lista de comprobación 170**
C.1 Antes de tocar nada . . . 170
C.2 Grabar el pendrive . . . 170
C.3 Lo que cambia respecto a la VM: el arranque es UEFI . . . 171
C.4 Arrancar e instalar . . . 171
C.5 Después del primer arranque . . . 172

**D Scripting: bucles, condiciones y `cron` 173**
Variables y sustitución . . . 173
El bucle `for` . . . 173
Condiciones: `if` . . . 174
`while` y leer línea a línea . . . 174
Un script completo . . . 175
`cron`: que se ejecute sin ti . . . 175
Práctica . . . 176
Soluciones del capítulo . . . 176

**E Montar de todo: discos, pendrives y unidades remotas 178**
Ver qué hay montado . . . 178
Montar a mano un disco que conectas . . . 178
Montaje permanente: `/etc/fstab` . . . 179
Carpetas de otra máquina, I: SSHFS . . . 180
Carpetas de otra máquina, II: SMB y NFS . . . 180
Windows y la nube . . . 181
Servir una carpeta desde tu VM por NFS . . . 181
Práctica . . . 182
Soluciones del capítulo . . . 182

# Bienvenida

Terminal · sistemas · redes · trabajo remoto — **13 sesiones prácticas · versión agosto 2026**

Llevas años usando Linux sin haber hablado nunca con él directamente. Este curso es esa conversación: trece sesiones para pasar de usuario de escritorio a alguien que instala su sistema, entiende su red y trabaja con soltura en un servidor remoto.

**Por qué tú.** No eres un usuario normal de un ordenador: eres un científico, y no puedes conformarte con el manejo básico — tener una herramienta tan potente y no saber usarla es un desperdicio. Pero tampoco eres informático, ni tienes tiempo de dominar un sistema operativo: eso lleva años. Así que hay que ponerse en medio, y ese punto medio es este libro: con él aventajas, y con mucho, al común de los mortales; a partir de ahí decides tú cuánto más quieres aprender. Dedicarle unos días es una muy buena inversión de tu tiempo.

Escrito para estudiantes de ciencias e ingeniería — física, matemáticas, química, cualquier ingeniería — con los ejemplos tomados de la física: los ejercicios usan datos de laboratorio, las analogías vienen de la mecánica, y el destino final es el flujo de trabajo real del cálculo científico.

**Regla única e innegociable:** no copies y pegues los comandos. Tecléalos. Los dedos aprenden lo que los ojos solo reconocen.

## Cómo leer este curso: lo básico, lo avanzado y la IA

Cada capítulo distingue dos niveles, y conviene tenerlos claros desde el principio. **Lo básico** es lo que un físico usa de verdad en su día a día y debe salir con soltura: moverse por el sistema, instalar Linux en un PC, entender particiones, sistemas de ficheros y formateo, mover información entre Linux, Windows, la nube y otros ordenadores, leer su red de casa — direcciones, DHCP, router, NAT y cortafuegos — y entrar en otra máquina. **Lo avanzado** — claves SSH, `rsync`, sesiones que sobreviven a la desconexión, túneles — hay que *conocerlo*, no dominarlo: saber que existe, para qué sirve y qué puede romper. La razón es de 2026: esas tareas las harás con una IA al lado que te dé las instrucciones, y solo quien conoce las herramientas mantiene el control de lo que la IA le propone. El anexo A — cómo instalar un agente de IA en tu terminal y cómo seguir mandando tú — es, en ese sentido, la otra mitad de este curso. Los ejercicios llevan siempre el nivel marcado: los esenciales no se saltan; los avanzados se leen, y se hacen si apetece.

## Qué necesitas para empezar

**Lo que suponemos que ya sabes — y para quién es «fácil».** El subtítulo lleva la palabra entre comillas a propósito. Este curso está pensado para un perfil concreto: estudiante de **ciencias, matemáticas o ingeniería** que ha cursado y aprobado la asignatura de programación que suele haber en primero de grado, y preferiblemente que esté en segundo curso o lo haya pasado. Para ese perfil, el curso es fácil: manejas un ordenador con soltura por la vía gráfica — ventanas, carpetas, navegador, instalar un programa — y no te asusta una variable ni un bucle. No hace falta haber abierto una terminal en tu vida: ese es justo el punto de partida. Con menos formación se puede seguir, pero te costará más y algunas partes te parecerán complicadas; el curso no lo esconde.

**¿Y si estoy en secundaria o bachillerato?** Si te atreves, adelante: vas a aprender cosas muy útiles. Lo que aprendas de Linux no caduca — cuando llegues a la universidad, todo lo que sepas seguirá funcionando igual, que es una de las virtudes de Linux: es estable en el tiempo, y lo básico lleva décadas funcionando prácticamente sin cambios. Eso sí, antes de meterte en la terminal conviene que hayas manejado un rato un entorno gráfico de Linux, como Linux Mint o similares, para no estrenarte en dos cosas a la vez.

**¿Y si ya no soy tan joven?** El curso habla de «jóvenes científicos», pero también cuentan los jóvenes de espíritu: no es un libro solo para estudiantes, es para quien tenga ganas de aprender, tenga la edad que tenga. Si tienes esas ganas, vas a aprender.

**Lo que necesitas tener instalado.** Un Linux de escritorio funcionando. El curso está escrito y probado sobre **Linux Mint**, y cualquier pariente cercano — Ubuntu y sus derivados, Debian con escritorio — sirve exactamente igual: los comandos son los mismos y solo cambiará algún detalle cosmético de los menús. Vale cualquier ordenador de la última década y una conexión a internet para descargar los materiales.

¿Que tu máquina tiene Windows o macOS? El apéndice Vengo de Windows te prepara un Linux dentro de tu sistema, con dos caminos según lo que quieras cubrir.

Algunos bloques pedirán algo más, y se avisará al llegar: unos 8 GB de RAM y 25 GB libres para la máquina virtual del bloque C (virtualización e instalación; el software, KVM y virt-manager, viene en los repositorios de tu Linux) y acceso al router de tu casa en el bloque D (redes) — nada que comprar, nada que arriesgar.

© 2026 Claudio M. Martínez Velasco. Versión: agosto 2026. El texto de este curso se publica bajo licencia Creative Commons Atribución–CompartirIgual 4.0 (CC BY-SA 4.0). Las fotografías conservan las licencias indicadas en su pie de figura. ¿Comentarios o sugerencias? Escribe a clamaver@uma.es.

# 1 Abrir la caja negra

Llevas años usando Linux sin haber hablado nunca con él directamente. Hoy empieza la conversación: qué es realmente esa ventana negra, por qué sigue siendo la herramienta central del cálculo científico medio siglo después, y tus primeros pasos dentro.

**2–3 h** con ejercicios · requisitos: **ninguno** · máquina: **tu Linux de diario**

**Al terminar esta sesión sabrás**

- Distinguir terminal, shell y consola (y por qué casi todo el mundo los confunde).
- Leer y escribir la anatomía de cualquier comando: programa, opciones, argumentos.
- Moverte por el árbol de directorios con `pwd`, `ls` y `cd`, con rutas absolutas y relativas.
- Usar los dos superpoderes que separan a quien sufre la terminal de quien la disfruta: el tabulador y el historial.
- Pedir ayuda al propio sistema con `man` y `--help`, sin salir a internet.
- Registrar tu sesión con `script`: tu cuaderno de laboratorio de este curso.

## 1.1. Qué es esto que parpadea

Abre la terminal de tu equipo (en Mint: menú → Terminal, o Ctrl+Alt+T). Verás algo así:

```
marcos@portatil:~$ _
```

Esa línea se llama **prompt**, y es una frase completa aunque no lo parezca: «soy el usuario `marcos`, en la máquina `portatil`, estoy en el directorio `~`, y el `$` significa que te escucho». Cada símbolo informa. El cursor parpadeante es la pregunta: *¿qué quieres?*

Antes de responder, deshagamos una confusión que arrastra todo el mundo. Aquí hay **dos programas distintos** colaborando, no uno:

| Pieza | Qué es | Analogía de laboratorio |
|---|---|---|
| **Terminal** | La ventana: dibuja texto, recoge tus teclas. No entiende nada de lo que escribes. | La pantalla del osciloscopio |

| Pieza | Qué es | Analogía de laboratorio |
|---|---|---|
| **Shell** | El intérprete que vive dentro: lee tu orden, la ejecuta, devuelve el resultado. El tuyo se llama `bash`. | La electrónica que procesa la señal |
| **Consola** | Históricamente, el hardware físico conectado a la máquina. Hoy se usa como sinónimo informal. | El instrumento completo sobre la mesa |

Y dos siglas que te perseguirán en toda la documentación, así que estrenémoslas ya: esta forma de trabajar — órdenes de texto — es una **CLI** (*command-line interface*, interfaz de línea de comandos); la de ventanas y ratón de todos los días es una **GUI** (*graphical user interface*). No son rivales: son dos puertas a la misma casa, y este curso te entrega la que te faltaba.

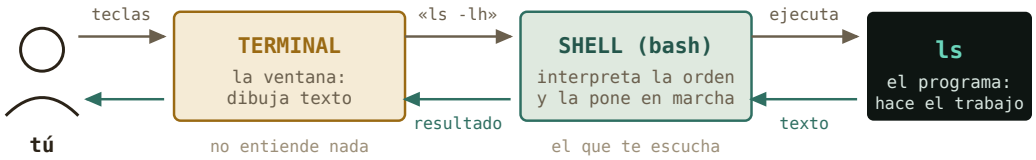



Figura 1.1: El circuito completo de una orden. La terminal solo dibuja; quien entiende es el shell.

**Historia · Por qué tu terminal se llama tty**

En los años 30, mucho antes de los ordenadores, las agencias de noticias transmitían texto con *teletipos* (*teletypewriters*, «tty»): máquinas de escribir que enviaban cada tecla por un cable e imprimían lo que llegaba. Cuando en los 60 hubo que hablar con los primeros ordenadores compartidos, nadie diseñó una interfaz nueva: se enchufó el teletipo que ya existía. El ordenador «contestaba» imprimiendo en papel.

Las pantallas sustituyeron al papel en los 70 —el DEC VT100 de 1978 fijó el estándar que tu terminal aún emula— pero el nombre se quedó. Escribe `tty` en tu terminal y el sistema te dirá qué «teletipo» eres. Hay medio siglo de arqueología funcionando en tu portátil.




Figura 1.2: Un teletipo ASR-33 (años 60): teclado, rollo de papel, y ni rastro de pantalla. *Foto: ArnoldReinhold, CC BY-SA 3.0, Wikimedia Commons.*




Figura 1.3: El DEC VT100 (1978): el papel se vuelve pantalla. Tu terminal de hoy lo sigue emulando. *Foto: Jason Scott, CC BY 2.0, Wikimedia Commons.*




Figura 1.4: Noventa años de la misma idea: teclear texto, recibir texto. Tu terminal «emula» por software lo que fue una máquina física.

**Por dentro · ¿Por qué sobrevive una interfaz de los años 60?**

No es nostalgia. El texto tiene tres propiedades que ninguna interfaz gráfica iguala: es **componible** (la salida de un programa puede ser la entrada de otro, lo verás en el capítulo 3), es **reproducible** (un comando se copia en un correo, un artículo o un script y produce exactamente lo mismo), y **viaja bien** (manejar un superordenador a 2 000 km por SSH cuesta lo mismo que manejar tu portátil). En física computacional, esas tres propiedades son la diferencia entre un resultado y un resultado *reproducible*.

## 1.2. Primeras palabras

Vamos a hablarle. Una regla para todo el curso, y es innegociable: **no copies y pegues los comandos — tecléalos**. Los dedos aprenden lo que los ojos solo reconocen. Escribe cada línea y pulsa Intro:

```
marcos@portatil:~$ whoami
marcos
marcos@portatil:~$ date
domingo, 23 de agosto de 2026, 18:04:11 CEST
marcos@portatil:~$ echo "hola, máquina"
hola, máquina
marcos@portatil:~$ tty
/dev/pts/0
```

Cuatro comandos, cuatro respuestas instantáneas. `whoami` — quién soy. `date` — qué hora es. `echo` — repite lo que te digo. `tty` — qué terminal soy (ahí está el teletipo fantasma). Fíjate en el patrón: escribes una orden, el shell la ejecuta, te devuelve texto, y el prompt reaparece pidiendo la siguiente. Ese ciclo es toda la terminal — y tiene nombre técnico: **REPL** (*read–eval–print loop*: leer, ejecutar, imprimir, repetir). No es nuevo para ti: el intérprete de Python, con su `>>>`, es exactamente otro REPL. Cambia el idioma; el ciclo es el mismo.

¿Y si te equivocas? Prueba a escribir `whoami` mal a propósito:

```
marcos@portatil:~$ woami
bash: woami: orden no encontrada
```

Nada explota. El shell te dice exactamente qué pasó: no conoce ningún programa con ese nombre. **Los mensajes de error de la terminal se leen, no se temen** — casi siempre contienen el diagnóstico completo. Aprender a leerlos es la mitad de este curso.

## 1.3. Anatomía de un comando

Casi todo lo que escribirás en tu vida en una terminal sigue una única gramática de tres piezas:

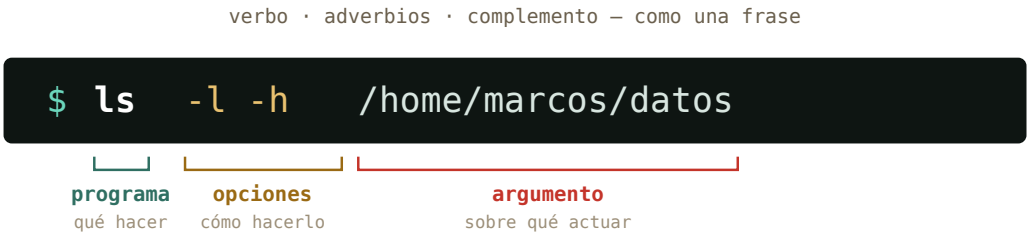



Figura 1.5: La gramática universal: «lista (`ls`), en formato largo y legible (`-l -h`), el contenido de esa carpeta (`/home/marcos/datos`)».

Las **opciones** empiezan por guion y modifican el comportamiento. Las cortas se pueden agrupar: `ls -l -h` y `ls -lh` son idénticos. Muchas tienen versión larga y legible: `-h` es `--human-readable`. Los **argumentos** son el objeto de la acción — normalmente ficheros o carpetas. Si no das argumento, cada programa tiene un valor por defecto razonable: `ls` a secas lista *donde estás*.

**Truco · Los espacios importan**

El shell separa las piezas por espacios. `ls-lh` (sin espacio) es para él un solo programa llamado «ls-lh», que no existe. Y una carpeta llamada `mis datos` son *dos* argumentos salvo que la protejas con comillas: `ls "mis datos"`. Por eso los científicos veteranos nombran todo `sin_espacios` — adopta la costumbre desde hoy.

## 1.4. El territorio: moverse por el árbol

Todo el sistema — programas, configuración, tus datos, tus fotos — vive en un único árbol de directorios que nace en la raíz, escrita `/`. No hay «unidades C: y D:»: todo cuelga del mismo tronco. Tu casa está en `/home/tu_usuario`, y como es el sitio donde pasarás la vida, tiene apodo: la virgulilla `~`.

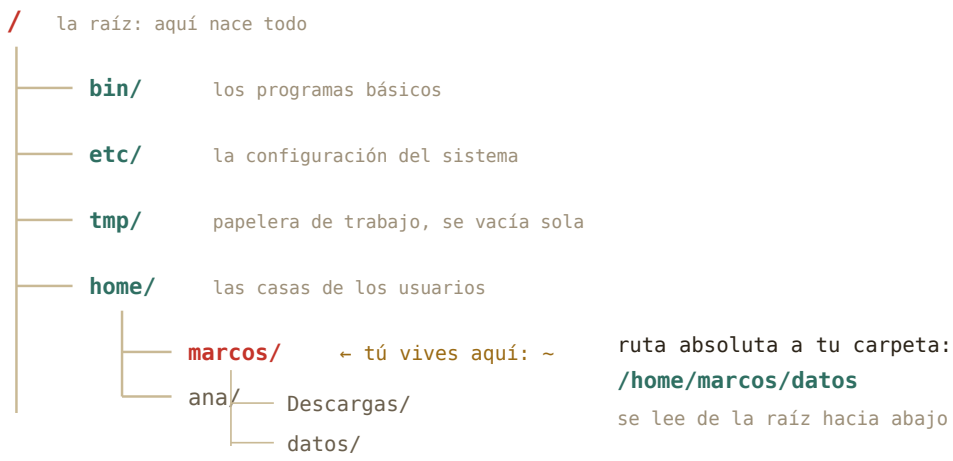



Figura 1.6: El árbol, podado a lo esencial. En el capítulo 4 exploraremos qué guarda cada rama; hoy basta con saber orientarse.

Tres comandos son tu GPS:

```
marcos@portatil:~$ pwd
/home/marcos
marcos@portatil:~$ ls
Descargas  Documentos  Escritorio  Imágenes  Música  Vídeos
marcos@portatil:~$ cd Descargas
marcos@portatil:~/Descargas$ pwd
/home/marcos/Descargas
```

`pwd` (*print working directory*) responde «¿dónde estoy?». `ls` (*list*) responde «¿qué hay aquí?». `cd` (*change directory*) te mueve. Observa que el prompt cambió: ahora dice `~/Descargas`. El prompt es tu brújula permanente.

### 1.4.1. Rutas absolutas y relativas

Hay dos maneras de indicar cualquier lugar del árbol, y la diferencia es idéntica a la de un sistema de referencia en mecánica:

Una **ruta absoluta** empieza por `/` y se mide desde la raíz: `/home/marcos/datos`. Vale desde cualquier sitio, siempre, como unas coordenadas del sistema de laboratorio. Una **ruta relativa** se mide desde donde estás ahora: si estás en `~`, escribir `datos` basta. Es el sistema de referencia del observador — más corto, pero depende de tu posición.

Dos atajos completan el mapa: `..` es «el directorio padre» (un nivel hacia la raíz) y `.` es «aquí mismo». Y `cd` tiene dos gestos que usarás mil veces: sin argumentos te lleva a casa desde cualquier lugar, y `cd -` vuelve al sitio donde estabas justo antes.

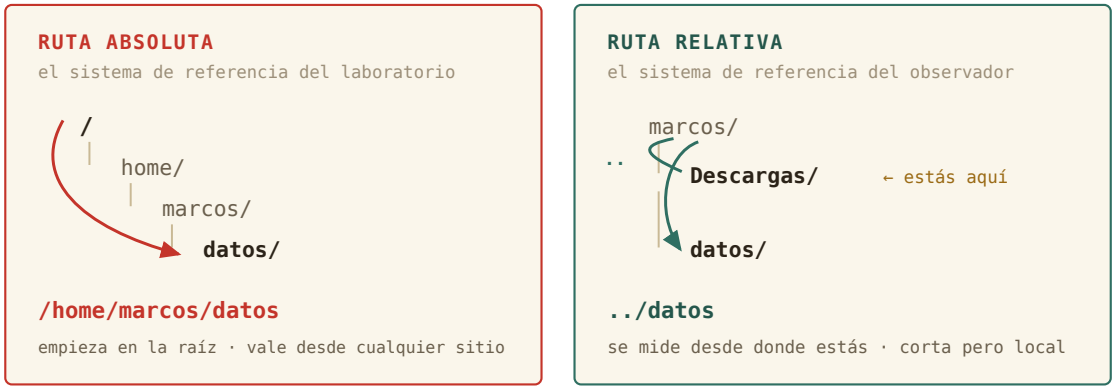



Figura 1.7: Mismo destino, dos sistemas de referencia. La absoluta se mide desde la raíz; la relativa, desde ti.

```
marcos@portatil:~/Descargas$ cd ..
marcos@portatil:~$ cd /etc
marcos@portatil:/etc$ cd
marcos@portatil:~$ cd -
/etc
marcos@portatil:/etc$
```

**Tranquilidad · Hoy no puedes romper nada**

Todos los comandos de este capítulo son de *solo lectura*: miran, no tocan. Aún no sabes borrar ni modificar nada — disfruta de esta era de inocencia mientras dure. Cuando lleguen los comandos con consecuencias (capítulo 2), llegarán también las redes de seguridad.

## 1.5. Los dos superpoderes

Lo que de verdad separa a quien sufre la terminal de quien la habita no es saber más comandos. Son dos teclas.

### 1.5.1. El tabulador: nunca escribas nombres enteros

Escribe `cd Des` y pulsa Tab. El shell completa: `cd Descargas/`. Si hay varias opciones que empiezan igual, pulsa Tab *dos veces* y te las muestra todas. Esto no es solo comodidad: **si el tabulador no completa, el fichero no está donde crees** — acabas de detectar un error antes de cometerlo. El tabulador es tu comprobación experimental continua.

### 1.5.2. El historial: nunca escribas nada dos veces

La flecha ↑ recupera comandos anteriores, uno a uno. Pero el gesto profesional es Ctrl+R: búsqueda inversa. Púlsalo, teclea un fragmento de cualquier comando que hayas usado («des», por ejemplo), y aparecerá el más reciente que lo contenga. Intro lo ejecuta; Ctrl+R otra vez busca más atrás. El comando `history` te enseña la lista completa numerada.

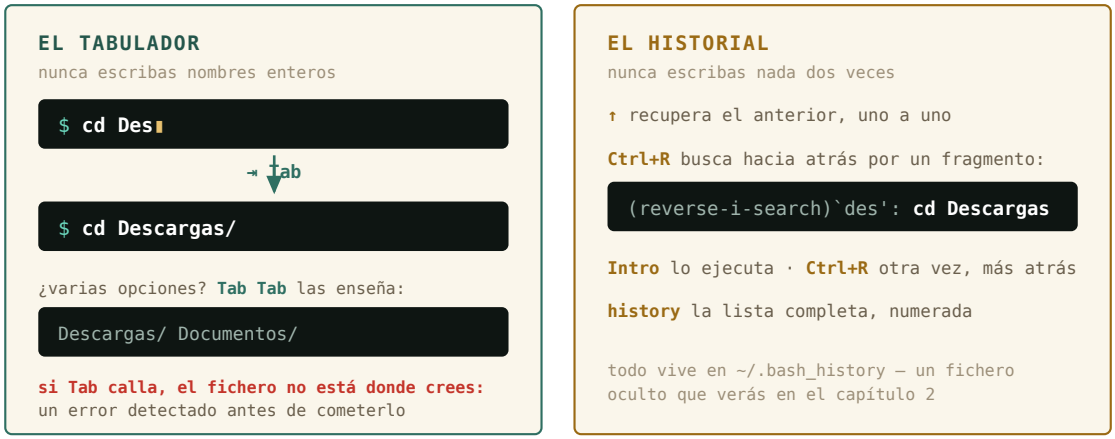



Figura 1.8: Dos teclas que separan a quien sufre la terminal de quien la habita. Ninguna de las dos te pide memorizar nada.

**Por dentro · Dónde vive tu memoria**

El historial no es magia: bash lo va guardando en un fichero oculto llamado `.bashhistory`… casi: `.bash_history`, dentro de tu casa. Los ficheros cuyo nombre empieza por punto están ocultos por convención — `ls` no los muestra salvo que le pidas `ls -a` (*all*). Pruébalo en `~`: descubrirás docenas de ficheros de configuración que llevan años viviendo contigo sin que los hayas visto. En el capítulo 4 aprenderás qué hacen.

## 1.6. Pedir ayuda sin salir del sistema

Aquí está la clave de la autonomía, el motivo por el que este curso existe. Ante una duda, el reflejo entrenado es preguntarle a Google o a una IA. Pero tu sistema lleva dentro la documentación completa y exacta de cada comando *de tu versión concreta*, disponible sin conexión:

```
marcos@portatil:~$ ls --help
Modo de empleo: ls [OPCIÓN]... [FICHERO]...
Lista información sobre los FICHEROs...
marcos@portatil:~$ man ls
```

`--help` da el resumen rápido en pantalla. `man` (*manual*) abre la página completa en un visor con sus propias teclas: ↑/↓ y Espacio para moverte, / para *buscar* dentro (escribe el término y Intro; n salta a la siguiente aparición), y q para salir. Esa tecla / convierte cada página de manual en un documento consultable en segundos.




Figura 1.9: Las cuatro teclas del visor. Las mismas valen para `less` y para `htop` — las verás durante todo el curso.

**Historia · El manual nació con el sistema**

Unix nació en 1969 en los Bell Labs, escrito por Ken Thompson y Dennis Ritchie en un PDP-7 arrinconado — el proyecto oficial de sistema operativo del laboratorio acababa de cancelarse, y ellos siguieron por gusto. En 1971, su jefe puso una condición para comprarles una máquina mejor: que documentaran el sistema. Así nació el *Unix Programmer's Manual*, y con él la costumbre, viva 55 años después, de que cada programa lleve su página `man`. La notación `ls(1)` que verás por ahí significa «la página de `ls` en la sección 1 del manual» — la numeración original de 1971.

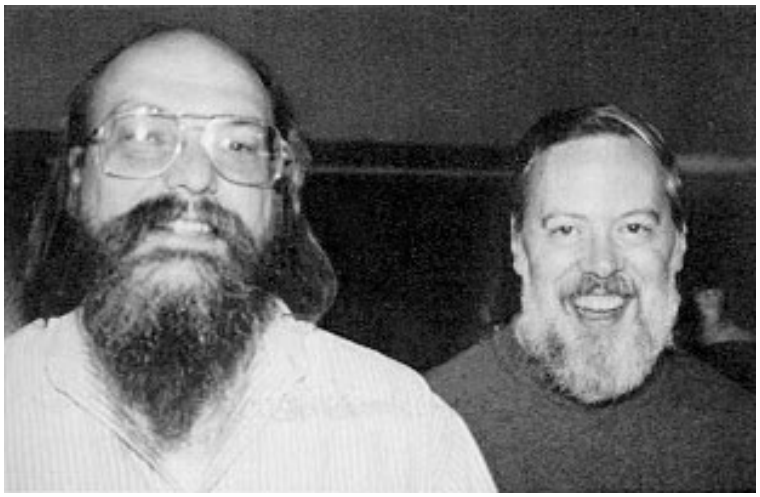


Figura 1.10: Ken Thompson (izquierda) y Dennis Ritchie (derecha), autores de Unix, hacia 1973 — con el manual ya en marcha. *Foto: dominio público, Wikimedia Commons.*

## 1.7. Tu cuaderno de laboratorio: `script`

En el laboratorio no se te ocurriría medir sin cuaderno. En la terminal, tampoco: el comando `script` graba *todo* lo que aparezca en pantalla — tus órdenes y las respuestas — en un fichero, hasta que escribas `exit`.

```
marcos@portatil:~$ script sesion01.log
Script iniciado, el fichero de anotación es 'sesion01.log'
marcos@portatil:~$ pwd
/home/marcos
marcos@portatil:~$ exit
Script terminado, el fichero de anotación es 'sesion01.log'
```

Desde ahora, **cada sesión de este curso empieza con `script sesionNN.log` y termina con `exit`**. Te servirá para revisar qué hiciste, para preguntar con evidencia («esto es exactamente lo que pasó») — y durante el piloto, esos logs son los datos experimentales con los que mejoraremos el curso. Eres a la vez el físico y el experimento.

## 1.8. Práctica: la expedición

Sesión de campo. Arranca tu cuaderno (`script sesion01.log`) y responde a cada pregunta *ejecutando comandos*, no de memoria. Las soluciones están plegadas: inténtalo de verdad antes de abrirlas.

**Ejercicio 1.1 · Reconocimiento del territorio**

Ve a la raíz del sistema y lista lo que hay. Después responde: ¿cuántos directorios de primer nivel tiene tu Mint, aproximadamente? ¿Reconoces `etc`, `home` y `tmp` de la Figura 1.6?

*(Solución al final del capítulo.)*

**Ejercicio 1.2 · Sistemas de referencia**

Estás en `/home/marcos/Descargas`. Escribe **dos** comandos distintos, uno con ruta relativa y otro con absoluta, que te lleven a `/home/marcos/Documentos`.

*(Solución al final del capítulo.)*

**Ejercicio 1.3 · Leer al instrumento**

Ejecuta `ls -lh /etc` y elige cualquier línea. Identifica en ella: el tamaño del fichero y su fecha de modificación. ¿Qué crees que aporta la opción `-h`? Compruébalo repitiendo sin ella.

*(Solución al final del capítulo.)*

**Ejercicio 1.4 · La biblioteca responde**

Sin usar internet: encuentra en `man ls` la opción que ordena los ficheros por fecha de modificación, del más reciente al más antiguo.

*Pista.* Dentro del visor, la tecla / busca.

*(Solución al final del capítulo.)*

**Ejercicio 1.5 · Predicción teórica**

Sin ejecutarlo todavía: partiendo de `/home/marcos`, ¿qué imprimirá `pwd` tras esta secuencia? `cd /tmp` → `cd` → `cd Descargas` → `cd ..` — Escribe tu predicción, luego compruébala experimentalmente.

*(Solución al final del capítulo.)*

**Ejercicio 1.6 · Preparando el material de la próxima sesión**

Descarga el fichero de medidas `pendulo_medidas.csv` — veinte periodos de un péndulo, el clásico de primero — desde la versión web del curso (sección Datos del capítulo 1). Después, *desde la terminal*, localízalo: comprueba con `ls` que está en tu carpeta de descargas, y averigua su tamaño y su fecha con lo aprendido en Sección 1.3.

*(Solución al final del capítulo.)*

## 1.9. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. El programa que interpreta tus órdenes y las ejecuta es…

   a) la terminal
   b) el shell
   c) la consola

2. En `ls -lt /etc`, la pieza `/etc` es…

   a) una opción
   b) el programa
   c) el argumento

3. Estás en `/home/ana/datos` y ejecutas `cd ...` ¿Dónde acabas?

   a) `/home/ana`
   b) `/home`
   c) `/`

4. ¿Cuál de estas es una ruta relativa?

   a) `/home/marcos/datos`
   b) `../figuras/fig01.png`
   c) `/etc`

5. Dentro de `man ls`, ¿qué tecla inicia una búsqueda?

   a) `/`

b) Ctrl+F
c) Tab

6. Escribes `cd Des`, pulsas Tab y no se completa nada. La lectura correcta es…

   a) el tabulador está desactivado
   b) probablemente no existe nada que empiece por «Des» aquí
   c) hay que pulsarlo más fuerte

## Soluciones del capítulo

### Ejercicio 1.1 · Reconocimiento del territorio

Solución razonada

`cd /` y después `ls`. Verás en torno a 20 entradas: `bin  boot  dev  etc  home  lib  ...`
**Por qué así** — También valía `ls /` directamente, sin moverte — recuerda que `ls` acepta como argumento cualquier ruta. Dos caminos correctos para lo mismo: en la terminal casi siempre los hay, y ninguno es «el bueno».

### Ejercicio 1.2 · Sistemas de referencia

Solución razonada

Relativa: `cd ../Documentos` · Absoluta: `cd /home/marcos/Documentos` (o el híbrido `cd ~/Documentos`).
**Por qué así** — `..` sube a `/home/marcos` y desde ahí baja a `Documentos`: es el cambio de sistema de referencia pasando por el origen común. La absoluta funciona idéntica estés donde estés — por eso los scripts que escribas en el futuro preferirán rutas absolutas: no dependen de dónde los ejecutes.

### Ejercicio 1.3 · Leer al instrumento

Solución razonada

En una línea como `-rw-r--r-- 1 root root 3,0K ago 12 10:33 fstab`, el tamaño es `3,0K` y la fecha `ago 12 10:33`. Sin `-h`, el tamaño aparece en bytes crudos: `3044`.
**Por qué así** — `-h` es `--human-readable`: convierte bytes a K, M, G. El resto de la línea (ese `-rw-r--r--` y los dos `root`) es el sistema de permisos y propietarios — protagonista del capítulo 4. Por ahora te basta con saber que está ahí y que significa algo.

### Ejercicio 1.4 · La biblioteca responde

Solución razonada

`man ls`, luego `/modificación` (o `/time` si tu manual está en inglés) e Intro. La opción es `-t`. Prueba: `ls -lt ~/Descargas` — tu descarga más reciente aparece la primera.
**Por qué así** — Este gesto — `man` + `/` — sustituye a la mayoría de búsquedas en Google que hace un principiante. Es más rápido, no requiere conexión, y describe exactamente *tu* versión del programa, no la de un tutorial de 2019.

**Ejercicio 1.5 · Predicción teórica**

Solución razonada

`/home/marcos`.
**Por qué así** — Paso a paso: `cd /tmp` te lleva a `/tmp`; `cd` sin argumentos, a casa (`/home/marcos`); `cd Descargas`, a `/home/marcos/Descargas`; y `..` sube al padre: de vuelta a casa. Si predijiste bien las cuatro, las rutas ya son tuyas. Si no, repite la secuencia mirando el prompt en cada paso: es tu brújula.

**Ejercicio 1.6 · Preparando el material de la próxima sesión**

Solución razonada

`ls -lh ~/Descargas` — y ahí estará `pendulo_medidas.csv`, con menos de 1K y fecha de hoy. Con `ls -lt ~/Descargas` saldrá el primero de la lista.
**Por qué así** — Fíjate en lo que *no* hemos hecho: abrirlo. Sabes llegar hasta cualquier fichero del sistema, pero aún no sabes mirar dentro desde la terminal. Ese es, exactamente, el primer gesto de la próxima sesión.

**Autocomprobación (test de la sección 1.9)** — Respuestas: 1 b · 2 c · 3 a · 4 b · 5 a · 6 b.

### La próxima sesión

Tienes un fichero de medidas esperando en `~/Descargas` y todavía no sabes abrirlo, copiarlo ni ponerlo a salvo. En el capítulo 2 los ficheros dejan de ser intocables: aprenderás a leerlos, copiarlos, moverlos y organizarlos en lote, y conocerás el primer comando capaz de destruir — con su red de seguridad correspondiente. Cierra tu cuaderno con `exit` y guarda `sesion01.log`: es tu primera medida experimental de este curso.

cap. 1 v0.2 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion01.log`

# 2 Ficheros: tocar sin romper

En la primera sesión aprendiste a mirar: dónde estás, qué hay, cómo moverte. Hoy los ficheros dejan de ser intocables: vas a leerlos, crearlos, copiarlos, moverlos y — por primera vez — destruirlos. Con red de seguridad, porque en esta casa el borrado no tiene papelera.

**2–3 h** con ejercicios · requisitos: **capítulo 1** · máquina: **tu Linux de diario**

**Al terminar esta sesión sabrás**

- Mirar dentro de cualquier fichero de texto con `cat`, `less`, `head` y `tail`, sin abrir ningún programa gráfico.
- Crear estructura con `mkdir` y copiar, mover y renombrar con `cp` y `mv` — y por qué mover y renombrar son el mismo gesto.
- Borrar con `rm` sabiendo exactamente por qué no perdona, y qué hacer antes de pulsar Intro.
- Seleccionar familias enteras de ficheros con comodines (`*`, `?`), entendiendo quién los expande de verdad.
- Localizar cualquier cosa con `find`.
- Editar ficheros en la terminal con `nano`, y estrenar tus *dotfiles* con un alias propio.

## 2.1. Mirar dentro sin miedo

Tienes desde la sesión pasada un fichero de medidas esperando en `~/Descargas`. Sabes llegar hasta él; hoy, por fin, vas a abrirlo. El gesto más directo es `cat` (*concatenate*), que vuelca el contenido completo en la pantalla:

```
marcos@portatil:~$ cd ~/Descargas
marcos@portatil:~/Descargas$ cat pendulo_medidas.csv
medida,longitud_m,periodo_s,incertidumbre_s
1,0.50,1.42,0.02
2,0.50,1.40,0.02
3,0.50,1.43,0.02
```

Para veinte líneas, perfecto. Pero un fichero de adquisición real puede tener veinte millones, y `cat` las escupiría todas. Para eso existe `less`: un visor por páginas — exactamente el mismo que usa `man`, así que ya te sabes sus teclas: ↑/↓ y Espacio para avanzar, / para buscar, q para salir.

```
marcos@portatil:~/Descargas$ less pendulo_medidas.csv
```

Y muchas veces no quieres el fichero entero, sino un vistazo: `head` enseña las primeras líneas y `tail` las últimas — la forma más rápida de comprobar cómo empieza y cómo acaba una tanda de medidas:

```
marcos@portatil:~/Descargas$ head -3 pendulo_medidas.csv
medida,longitud_m,periodo_s,incertidumbre_s
1,0.50,1.42,0.02
2,0.50,1.40,0.02
marcos@portatil:~/Descargas$ tail -2 pendulo_medidas.csv
19,1.10,2.10,0.02
20,1.10,2.12,0.02
```

La opción `-3` es el número de líneas. Fíjate en que acabas de leer un fichero de datos con cuatro herramientas distintas sin abrir una sola ventana. ¿Podías haberle hecho doble clic? Claro. Pero el doble clic no escala: cuando tengas doscientos ficheros de un barrido de simulaciones, `head` seguirá costando un segundo.

**Por dentro · Por qué todo esto funciona con cualquier fichero**

Para Linux un fichero es solo una secuencia de bytes con nombre. No hay ficheros «de Word» o «de Excel» a nivel del sistema: la extensión `.csv` es una pista para humanos, no una regla. Por eso `cat` y `less` abren cualquier cosa — aunque si abres un binario (una foto, un ejecutable) verás caracteres sin sentido: los bytes no eran texto. En el capítulo 3 exprimirás esta idea: si todo es texto o bytes, todo se puede encadenar.

## 2.2. Crear: `mkdir` y `touch`

Un experimento ordenado empieza por su estructura de carpetas. `mkdir` (*make directory*) crea directorios, y con la opción `-p` crea de una vez toda una ruta con sus niveles intermedios:

```
marcos@portatil:~$ mkdir fisica
marcos@portatil:~$ mkdir -p fisica/practicas/optica/datos
marcos@portatil:~$ ls fisica/practicas
optica
```

Su compañero menor es `touch`, que crea un fichero vacío (o actualiza la fecha de uno existente). Útil para dejar un marcador o preparar un fichero de notas:

```
marcos@portatil:~$ touch fisica/practicas/optica/notas.txt
```

## 2.3. Copiar y mover: `cp` y `mv`

Los dos siguen la misma gramática de tres piezas del capítulo 1: programa, origen, destino.

```
marcos@portatil:~/Descargas$ cp pendulo_medidas.csv copia_seguridad.csv
marcos@portatil:~/Descargas$ mv copia_seguridad.csv ~/fisica/
marcos@portatil:~/Descargas$ ls ~/fisica
copia_seguridad.csv  practicas
```

`cp` (*copy*) duplica: el original no se toca. `mv` (*move*) traslada: solo queda el destino. Y aquí viene el gesto que confunde a todo el mundo la primera vez: **renombrar es mover**. No hay comando «rename» — mover un fichero a otro nombre dentro de la misma carpeta *es* renombrarlo:

```
marcos@portatil:~/fisica$ mv copia_seguridad.csv pendulo_v1.csv
```

Para copiar carpetas enteras con todo lo que contienen, `cp` necesita la opción `-r` (*recursive*): `cp -r practicas practicas_backup`.

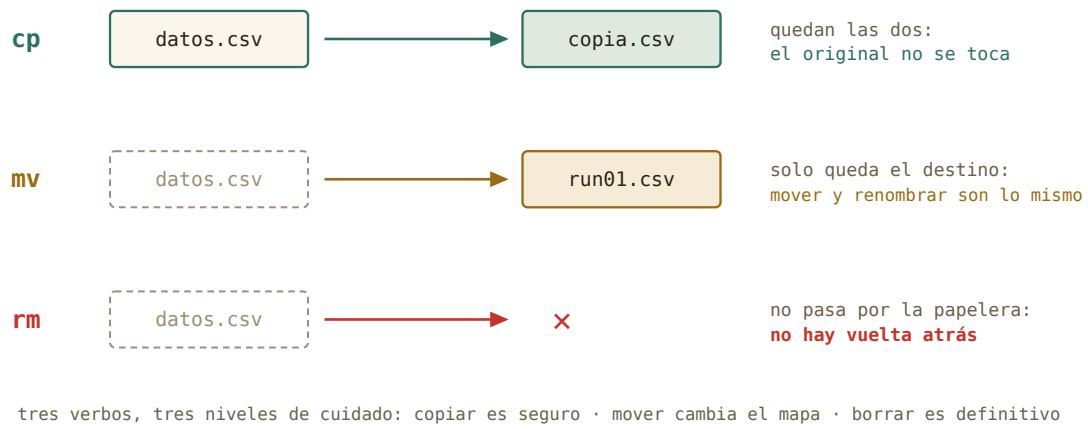



Figura 2.1: Tres verbos, tres niveles de cuidado. `cp` es reversible por naturaleza; `mv` cambia el mapa; `rm` no tiene vuelta.

**Truco · El tabulador vale doble aquí**

En `cp` y `mv` casi todo son nombres de ficheros: territorio ideal del tabulador. `mv penTab` completa el origen; y si el destino es una carpeta, Tab también la completa. Menos teclear, cero erratas — y recuerda: si Tab no completa, el fichero no está donde crees. Esa comprobación gratuita evita la mitad de los `mv` arrepentidos.

## 2.4. Borrar: el comando sin papelera

Llegó el primer comando con consecuencias. `rm` (*remove*) borra ficheros, y `rm -r` borra carpetas con todo lo que contengan:

```
marcos@portatil:~/fisica$ rm pendulo_v1.csv
marcos@portatil:~/fisica$ ls
practicas
```

Ahora lee esto despacio, porque es de las frases importantes del curso: **`rm` no pasa por la papelera**. No hay icono desde el que restaurar, no hay «deshacer». El espacio se marca como libre y el sistema lo reutilizará cuando quiera. En el escritorio llevas años protegido por la papelera del gestor de archivos; en la terminal, la protección eres tú.

Tu red de seguridad son dos costumbres y una opción:

- **Antes de un `rm` con comodines, ensaya con `ls`.** El mismo patrón que vas a borrar, pero listándolo: `ls run*.csv`. Lo que te enseñe `ls` es exactamente lo que borrará `rm`. Es tu predicción teórica antes del experimento irreversible.
- **En `rm -r`, ruta completa y tabulador**, nunca escrita a mano.
- La opción `-i` (*interactive*) pide confirmación fichero a fichero: `rm -i *.tmp`.

**Cuidado · La papelera gráfica sí perdona; `rm` no**

Si arrastras un fichero a la papelera del escritorio de Mint, puedes recuperarlo: esa es la diferencia real entre las dos vías, y no es menor. Mientras aprendes, borra en tus carpetas de prácticas, no en tus datos reales — y recuerda que la regla del curso es que lo destructivo de verdad (particionar, formatear) vivirá siempre en la máquina virtual del bloque C.

## 2.5. Comodines: hablar de familias enteras

Hasta ahora has nombrado los ficheros de uno en uno. Los **comodines** (*wildcards*) nombran familias: `*` significa «cualquier cosa, incluso nada» y `?` «exactamente un carácter». `run*.csv` son todos los CSV que empiecen por `run`; `run?.csv` solo los de un dígito.

```
marcos@portatil:~/Descargas$ ls run*.csv
run1.csv  run2.csv  run3.csv
marcos@portatil:~/Descargas$ cp run*.csv ~/fisica/practicas/optica/datos/
```

Una sola línea acaba de copiar toda una serie de medidas. Este es el momento en que la terminal empieza a pagar el peaje de aprenderla: lo que en el gestor de archivos serían tres viajes de ratón (o treinta, con más series), aquí es una orden.

**Por dentro · El asterisco nunca llega al programa**

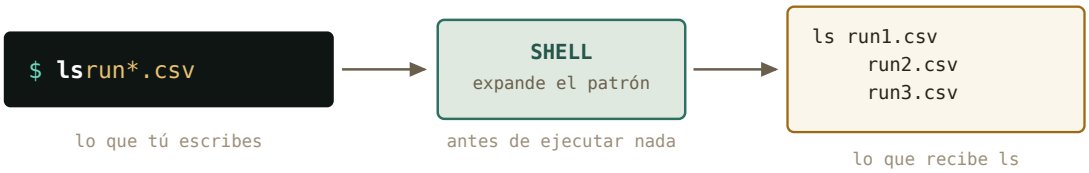



Figura 2.2: Quién expande el patrón: el shell, no el programa. `ls` recibe la lista ya hecha.

La expansión (*globbing*) la hace el **shell**, antes de ejecutar nada: cuando `ls` arranca, ya recibe `run1.csv run2.csv run3.csv` como argumentos, y no sabe que hubo un asterisco. Por eso los comodines funcionan igual con *cualquier* comando, incluso con programas que escribas tú — nadie tuvo que programarlos. Y por eso, si el patrón no casa con nada, el programa recibe el asterisco tal cual y suele protestar: el shell no tenía nada que expandir.

**Cuidado · El espacio asesino**

`rm run*.csv` borra la familia `run…`. Pero `rm run* .csv` — con un espacio colado — son *dos* argumentos: la familia `run*` entera y un fichero llamado `.csv`. El shell expande y `rm` obedece sin preguntar. Es el accidente clásico de esta sesión: antes de cualquier `rm` con patrón, ensáyalo con `ls`. Siempre.

## 2.6. Encontrar: `find`

`ls` enseña lo que hay *aquí*; `find` busca *por todo el árbol*. Sus argumentos son un punto de partida y condiciones:

```
marcos@portatil:~$ find ~/fisica -name "*.csv"
/home/marcos/fisica/practicas/optica/datos/run1.csv
/home/marcos/fisica/practicas/optica/datos/run2.csv
/home/marcos/fisica/practicas/optica/datos/run3.csv
marcos@portatil:~$ find ~/fisica -type d
/home/marcos/fisica
/home/marcos/fisica/practicas
/home/marcos/fisica/practicas/optica
/home/marcos/fisica/practicas/optica/datos
```

`-name` filtra por nombre (fíjate en las comillas: protegen el `*` para que lo expanda `find` y no el shell — acabas de usar lo que aprendiste en la caja anterior). `-type d` pide solo directorios; `-mtime -1`, solo lo modificado en el último día. En el capítulo 3, `find` se convertirá en el generador de listas que alimenta a todos los demás comandos.

## 2.7. Editar: `nano` (y la verdad sobre `vim`)

Leer y organizar está bien; en algún momento hay que *escribir*. El editor de terminal del curso es `nano`, por una razón honesta: se aprende en un minuto y está en todas partes.

```
marcos@portatil:~/fisica$ nano practicas/optica/notas.txt
```

Se abre un editor a pantalla completa. Escribes como en cualquier sitio, y las dos teclas que importan están siempre rotuladas abajo: Ctrl+O guarda (*write Out*, confirma el nombre con Intro) y Ctrl+X sale. El símbolo `^` de la barra inferior significa Ctrl.

¿Y para qué sirve un editor de terminal existiendo el editor gráfico de Mint? Hoy, para poco. Pero dentro de unas sesiones estarás dentro de un servidor remoto donde **no hay ventanas**: allí `nano` no es una opción retro, es la única mesa de trabajo. Lo aprendes hoy, en casa, para que ese día sea trámite.

**Historia · Por qué `vim` asusta (y por qué existe)**

El editor que verás mencionado con reverencia es `vim`. Su antepasado `ed` (1969) se diseñó para teletipos: sin pantalla, editar era dar órdenes a ciegas línea a línea — con papel, refrescar la vista costaba tinta. Cuando llegaron las pantallas nació `vi` (*visual*, 1976), que arrastra ese ADN: tiene *modos* — uno para órdenes y otro para escribir — y por eso millones de personas han tecleado texto y visto cómo el editor lo interpretaba como comandos. Es potentísimo y quizá lo ames algún día; este curso solo te pide saber salir si caes dentro por accidente: Esc y luego `:q!` e Intro. Ya formas parte del club de los que saben salir de vim.

## 2.8. Tus primeros *dotfiles*

En el capítulo 1 descubriste con `ls -a` los ficheros ocultos de tu casa: los que empiezan por punto, los *dotfiles*. No están ocultos por secretos, sino por higiene: son configuración, y estarían en medio todo el día. Hoy vas a editar el más importante: `.bashrc`, el fichero que bash lee cada vez que abres una terminal.

```
marcos@portatil:~$ nano .bashrc
```

Baja hasta el final (con Ctrl+V avanzas página a página) y añade esta línea:

```
alias ll='ls -lh'
```

Guarda y sal. Un **alias** es un atajo: a partir de ahora `ll` significará `ls -lh`. Para que la terminal actual se entere sin cerrarla, recarga la configuración:

```
marcos@portatil:~$ source ~/.bashrc
marcos@portatil:~$ ll
total 44K
drwxr-xr-x 2 marcos marcos 4,0K ago 24 10:12 Descargas
drwxr-xr-x 3 marcos marcos 4,0K ago 24 10:15 fisica
```

Acabas de configurar tu sistema editando un fichero de texto. Ese es el modo Linux de configurar *todo* — el capítulo 4 te enseñará cuántas cosas de tu máquina son exactamente esto: un fichero de texto en el sitio correcto.

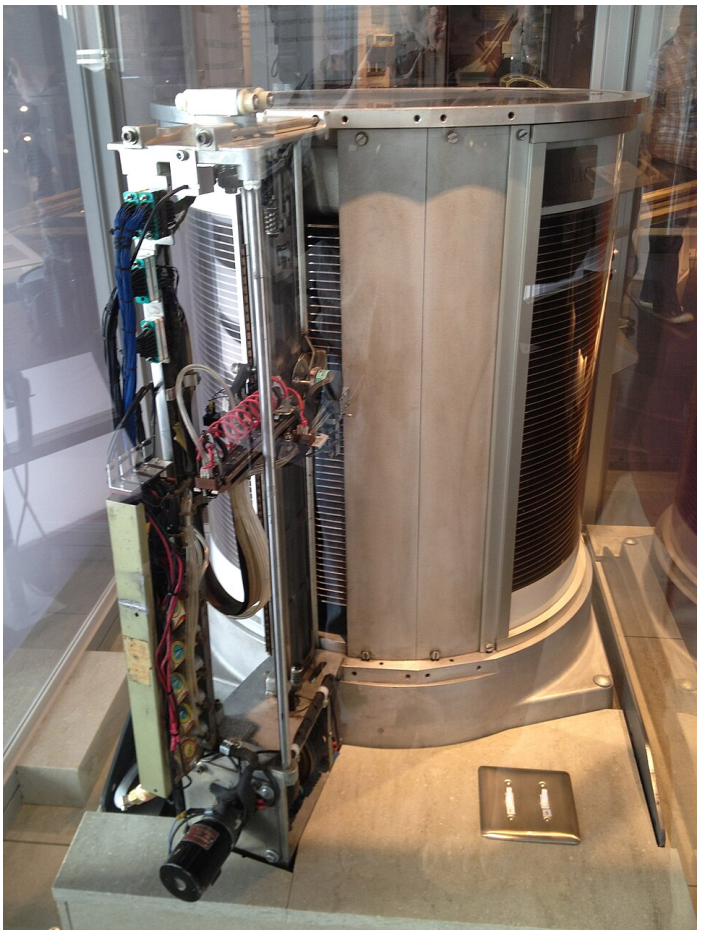


Figura 2.3: El mecanismo de acceso del IBM 350 RAMAC (1956), el primer disco duro comercial: cincuenta platos, cinco megabytes, casi una tonelada. Tus ficheros de hoy viven en el mismo concepto, un billón de veces más denso. *Foto: ArnoldReinhold, CC BY-SA 3.0, Wikimedia Commons.*

## 2.9. Práctica: poner orden en el laboratorio

Sesión de campo. Te ha llegado el directorio de un experimento tal como quedó el último día: duplicados, nombres con espacios, una serie de medidas inservible y ficheros temporales. (Los ficheros se llaman `run…`: es la jerga del laboratorio para cada serie de medidas, y así la verás en cualquier grupo.) Tu misión: dejarlo presentable usando todo lo de esta sesión.

Arranca tu cuaderno (`script sesion02.log`) y descarga `experimento-desordenado.zip` desde esta misma sección en la versión web del curso. Extráelo como prefieras — doble clic en el gestor de archivos vale: descomprimir es una de esas cosas que la vía gráfica hace perfectamente bien. Te quedará una carpeta `experimento-pendulo` en `~/Descargas`.

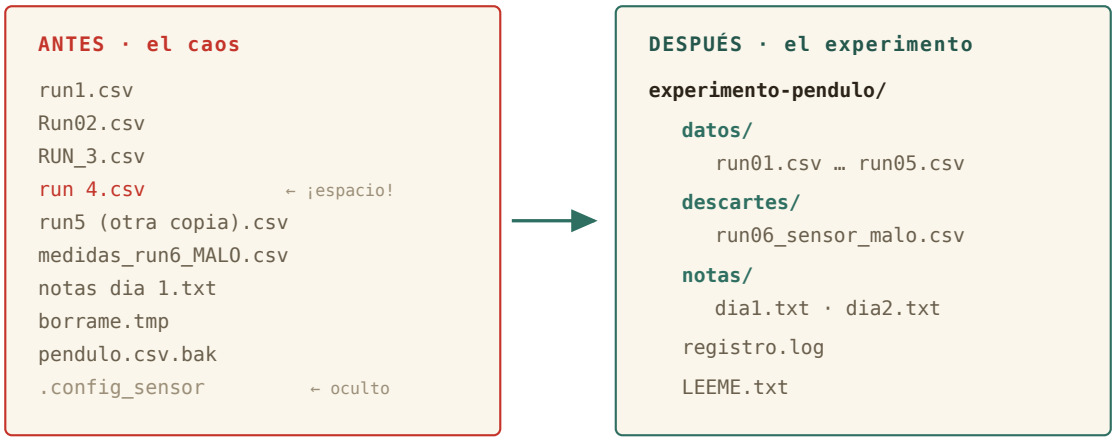



Figura 2.4: El plan de la expedición: de la carpeta heredada al experimento presentable.

**Ejercicio 2.1 · Reconocimiento**

Entra en la carpeta y haz inventario completo — incluido lo oculto. Después mira dentro de `registro.log` y de las notas: ¿qué serie está mal y por qué? ¿Qué dos ficheros son en realidad el mismo?

*(Solución al final del capítulo.)*

**Ejercicio 2.2 · Copia de seguridad primero**

Antes de tocar nada: haz una copia completa de la carpeta, llamada `experimento-pendulo-original`. Comprueba que la copia existe y tiene lo mismo.

*(Solución al final del capítulo.)*

**Ejercicio 2.3 · Estructura**

Dentro de `experimento-pendulo`, crea de una sola orden la estructura `datos/`, `descartes/` y `notas/`.

*(Solución al final del capítulo.)*

**Ejercicio 2.4 · La serie, en limpio**

Lleva las cinco series buenas a `datos/` con nombres uniformes `run01.csv` … `run05.csv` (recuerda las comillas para los nombres con espacios), y la serie mala a `descartes/` con un nombre que avise: `run06_sensor_malo.csv`.

*(Solución al final del capítulo.)*

**Ejercicio 2.5 · Limpieza fina**

Mueve las notas a `notas/` (renombrándolas a `dia1.txt` y `dia2.txt`), y borra lo que no aporta nada: el temporal y el `.bak`. Ensaya el borrado con `ls` antes de ejecutarlo.

*(Solución al final del capítulo.)*

**Ejercicio 2.6 · El censo final**

Sin moverte de `experimento-pendulo`: obtén con un solo comando la lista de todos los `.csv` que quedan, estén en la subcarpeta que estén. ¿Cuántos hay? ¿Coincide con lo esperado?

*(Solución al final del capítulo.)*

## 2.10. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. Quieres cambiar el nombre de `datos.csv` a `run01.csv`. ¿Qué comando usas?

   a) `cp datos.csv run01.csv`
   b) `mv datos.csv run01.csv`
   c) `rename datos.csv run01.csv`

2. Borras un fichero con `rm` y te arrepientes al instante. ¿Qué puedes hacer?

   a) abrir la papelera y restaurarlo
   b) pulsar Ctrl+Z en la terminal
   c) nada razonable: `rm` no pasa por la papelera

3. En `ls run*.csv`, ¿quién convierte `run*.csv` en la lista de nombres?

   a) el shell, antes de ejecutar `ls`
   b) el propio `ls`, al arrancar
   c) la terminal, al dibujar la línea

4. Quieres copiar la carpeta `practicas` con todo su contenido. ¿Qué necesitas?

   a) `cp practicas backup`
   b) `cp -r practicas backup`
   c) `mv practicas backup`

5. Estás dentro de `nano` y quieres guardar y salir. La secuencia es…

   a) Ctrl+O, Intro, Ctrl+X
   b) Esc y luego `:q!`

c) cerrar la ventana de la terminal

6. ¿Por qué `find . -name "*.csv"` lleva comillas en el patrón?

   a) por estética: sin ellas funciona igual siempre
   b) para que el asterisco le llegue a `find`, y no lo expanda antes el shell
   c) porque los nombres con espacios lo exigen

## Soluciones del capítulo

### Ejercicio 2.1 · Reconocimiento

Solución razonada

`cd ~/Descargas/experimento-pendulo`, `ls -a` (aparece `.config_sensor`), y `cat registro.log` o `less` sobre las notas. El log canta: `ERROR sensor congelado en 45.0 (run6)` — y `head medidas_run6_MALO.csv` lo confirma: todos los ángulos valen ~45. Las notas del día 2 confiesan que `run 4.csv` y `run5 (otra copia).csv` son la misma serie guardada dos veces.
**Por qué así** — Antes de mover o borrar nada, un físico lee el cuaderno del experimento. `head` te deja *ver* el fallo del sensor en los datos sin abrir ningún programa: eso es diagnosticar desde la terminal.

### Ejercicio 2.2 · Copia de seguridad primero

Solución razonada

Desde `~/Descargas`: `cp -r experimento-pendulo experimento-pendulo-original`, y comprobación con `ls experimento-pendulo-original` (o `ls -a`, si quieres verificar que el oculto también viajó — sí: `-r` copia todo).
**Por qué así** — Regla de oro antes de reorganizar datos reales: copia primero, ordena después. `cp -r` convierte «espero no equivocarme» en «da igual si me equivoco». El coste es un comando; el seguro, total.

### Ejercicio 2.3 · Estructura

Solución razonada

`mkdir datos descartes notas` — `mkdir` acepta varios argumentos y los crea todos. (También valía tres `mkdir` seguidos; y si hubieras querido niveles anidados, `-p`.)
**Por qué así** — Un comando, varios argumentos: la gramática del capítulo 1 trabajando para ti. La estructura antes que el contenido: primero las estanterías, luego los libros.

### Ejercicio 2.4 · La serie, en limpio

Solución razonada

Cinco `mv` con renombrado incluido — mover y renombrar son el mismo gesto: `mv run1.csv datos/run01.csv` · `mv Run02.csv datos/run02.csv` · `mv RUN_3.csv datos/run03.csv` · `mv "run 4.csv" datos/run04.csv` · `mv "run5 (otra copia).csv" datos/run05.csv` · `mv medidas_run6_MALO.csv descartes/run06_sensor_malo.csv`
**Por qué así** — Las comillas protegen los espacios y paréntesis: sin ellas, `run 4.csv` serían dos argumentos y `mv` protestaría (o algo peor). ¿Tedioso uno a uno? Sí — el renombrado *en lote* de verdad llega cuando sepas encadenar comandos: esta molestia de hoy es la motivación del capítulo 3. Y fíjate: `run01` y no `run1`, para que el orden alfabético coincida con el numérico cuando haya más de nueve.

**Ejercicio 2.5 · Limpieza fina**

Solución razonada

`mv "notas dia 1.txt" notas/dia1.txt` y `mv notas_dia2.txt notas/dia2.txt`. Para el borrado, primero el ensayo: `ls *.tmp *.bak` — lista exactamente `borrame.tmp` y `pendulo.csv.bak`, nada más. Ahora sí: `rm *.tmp *.bak`.
**Por qué así** — El par `ls`-luego-`rm` con el mismo patrón es la red de seguridad de esta sesión: la predicción antes de la medida irreversible. Si el `ls` hubiera enseñado algo inesperado, habrías corregido el patrón sin coste. `calibracion.txt` y `temperatura_lab.csv` se quedan: son datos del experimento, no basura.

**Ejercicio 2.6 · El censo final**

Solución razonada

`find . -name "*.csv"` — deben salir siete: cinco en `datos/`, uno en `descartes/` y `temperatura_lab.csv` en la raíz de la carpeta. (Las comillas en `"*.csv"`: el patrón es para `find`, no para el shell.)
**Por qué así** — `ls` mira una carpeta; `find` recorre el árbol entero. Ese censo final contra lo esperado es el control de calidad del experimentalista: no «creo que está ordenado», sino «lo he contado». Cierra tu cuaderno con `exit` y guarda `sesion02.log`.

**Autocomprobación (test de la sección 2.10)** — Respuestas: 1 b · 2 c · 3 a · 4 b · 5 a · 6 b.

### La próxima sesión

Hoy has renombrado cinco series de medidas a mano y te ha sabido a poco elegante — bien: esa molestia es la puerta del capítulo 3, donde los comandos dejan de trabajar solos y empiezan a encadenarse, de modo que la salida de uno se convierte en la entrada del siguiente. Ahí aparecen `grep`, `sort` y las tuberías. Harás el análisis completo de un registro de adquisición sin abrir Python, y aprenderás cuándo sí conviene abrirlo. Guarda `sesion02.log`: ya tienes dos medidas en tu cuaderno.

cap. 2 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion02.log`

# 3 El flujo de datos

Esta noche tu experimento ha generado un registro de 7 198 líneas. Nadie va a leerlas. Hoy aprendes cómo circula el texto entre programas: de dónde sale, adónde va, cómo se guarda en un fichero y cómo pasa de un programa a otro. Es fontanería, y como toda fontanería se ve poco — pero está debajo de todo lo que harás después: guardar la salida de una simulación, buscar un error en un registro, leer un comando ajeno y entenderlo. El capítulo baja hasta el fondo del mecanismo; hasta dónde bajas tú, lo decides tú.

**2–3 h** con ejercicios · requisitos: **capítulos 1 y 2** · máquina: **tu Linux de diario**

**Al terminar esta sesión sabrás**

- Qué son los tres canales de todo programa — stdin, stdout, stderr — y por qué esa separación es una idea brillante.
- Guardar la salida de cualquier programa — tu simulación incluida — en un fichero con `>`, sin perder los errores por el camino.
- Los cuatro gestos de tubería que un físico usa a diario: `> fichero`, `| less`, `| grep` y `| tail`.
- Leer con soltura cualquier comando encadenado que te encuentres en una documentación, en un foro o en la respuesta de una IA.
- Por dentro, para quien quiera bajar al taller: cómo cinco filtros — `wc`, `grep`, `sort`, `uniq`, `cut` — resuelven un análisis entero, construido pieza a pieza.
- Decidir con criterio qué va en una tubería y qué va en Python.

## 3.1. Los tres chorros

Todo programa de terminal, sin excepción, nace conectado a tres canales. Por uno le entra texto: la **entrada estándar** (*stdin*). Por otro escupe sus resultados: la **salida estándar** (*stdout*). Y por el tercero protesta: la **salida de errores** (*stderr*). Si no dices lo contrario, los tres apuntan a tu terminal: stdin al teclado, y las dos salidas a la pantalla.

¿Por qué *dos* salidas? Piensa en el log de esta noche: si el programa de adquisición mezclara las medidas y los avisos de error en el mismo chorro, guardar los datos limpios sería un suplicio. Separados, puedes enviar cada uno a un sitio. Ese es exactamente el siguiente paso.




Figura 3.1: Los tres canales, numerados 0, 1 y 2. La genialidad está en separar el resultado de las quejas.

## 3.2. Redirigir: > y >>

El símbolo `>` desvía la salida estándar de la pantalla a un fichero. Descarga el registro de esta noche (está al final del capítulo, en la práctica) y prueba:

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log > errores.txt
marcos@portatil:~/Descargas$ head -2 errores.txt
2026-08-18 10:03:00 ERROR s3 sin respuesta del sensor
2026-08-18 10:03:02 ERROR s3 sin respuesta del sensor
```

`grep` (lo conoces en un momento) no imprimió nada en pantalla: su stdout fue a parar a `errores.txt`. Dos reglas de seguridad:

- `>` **sobrescribe** el fichero de destino sin preguntar. Su hermano `>>` **añade** al final sin destruir lo anterior.
- Los errores (stderr) *no* viajan con `>`: siguen saliendo por pantalla, que es donde quieres verlos. Para capturarlos también existe `2>` — el `2` es el número del canal de la figura.

```
marcos@portatil:~/Descargas$ echo "Informe de la noche" > informe.txt
marcos@portatil:~/Descargas$ echo "generado el 19 de agosto" >> informe.txt
marcos@portatil:~/Descargas$ cat informe.txt
Informe de la noche
generado el 19 de agosto
```

`echo`, que conociste el primer día como un juguete, acaba de revelar su oficio real: fabricar líneas de texto para tus ficheros.

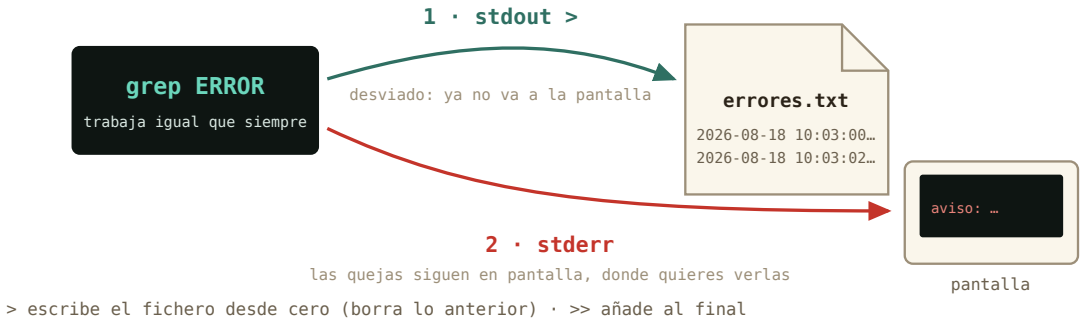



Figura 3.2: La misma escena que la figura anterior, con el desvío puesto: `>` reconecta el stdout al fichero; el stderr ni se entera.

## 3.3. La tubería: |

Y aquí está la idea central del capítulo. Si `>` conecta un programa con un fichero, la **tubería** `|` conecta un programa con *otro programa*: el stdout del de la izquierda se convierte en el stdin del de la derecha. Sin ficheros intermedios, sin pasos manuales.

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log | wc -l
108
```

Léelo como una cadena de montaje: `grep` filtra las líneas con `ERROR`, y su salida — en vez de inundar la pantalla — entra en `wc -l` (*word count*, opción *lines*), que solo cuenta. De 7 198 líneas a un número: esta noche hubo 108 fallos.

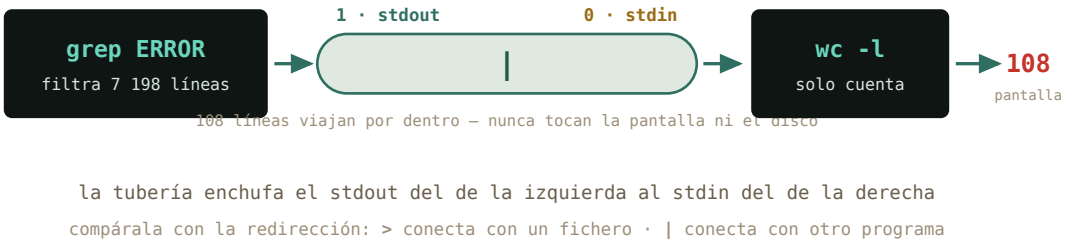



Figura 3.3: El enchufe entre programas. Es la tercera manera de conectar el stdout: a la pantalla (por defecto), a un fichero (`>`), o a otro programa (`|`).

**Historia · Una manguera de jardín en Bell Labs**

La tubería tiene padre: Doug McIlroy, jefe del departamento donde nació Unix. En 1964 — cinco años *antes* de Unix — ya escribía en un memorándum que los programas deberían acoplarse «como los tramos de una manguera de jardín: se enrosca otro segmento cuando hace falta tratar el agua de otra manera». Estuvo casi una década insistiendo, hasta que una noche de 1973 Ken Thompson cedió, la implementó en el kernel, y en una semana el equipo reescribió medio sistema en forma de filtros encadenables. De esa semana sale la **filosofía Unix** que estás usando ahora: haz que cada programa haga una sola cosa bien, y que todos hablen el mismo idioma — texto.

## 3.4. Lo que usarás de verdad

Seamos francos antes de bajar al mecanismo: **un físico no construye tuberías largas.** Lo que sí hace, todos los días de su vida profesional, son cuatro gestos de fontanería básica — y este es el momento de dejarlos grabados:

```
marcos@portatil:~/fisica$ python3 simula.py > resultados.txt
marcos@portatil:~/fisica$ ls -l /usr/share | less
marcos@portatil:~/fisica$ history | grep ssh
marcos@portatil:~/fisica$ tail -3 resultados.txt
```

Uno por uno. **La salida de tu programa, a un fichero** (`>`): cuando la simulación escupe diez mil líneas no quieres verlas, quieres guardarlas — y con un `2> errores.txt` detrás, las quejas van a otro fichero para leerlas en calma. **Una salida larga, paginada** (`| less`): cualquier comando que desborde la pantalla se lee con el visor de `man` del capítulo 1 — su `/` para buscar, su `q` para salir. **Buscar dentro de una salida** (`| grep`): el gesto más frecuente de toda la terminal — ¿estaba ese comando en mi historial?, ¿aparece «usb» en los mensajes del sistema? En los próximos capítulos verás `ps aux | grep`, `lsblk | grep`, `apt list | grep`: siempre el mismo enchufe. Y **solo el final** (`| tail`, o `tail fichero`): las últimas líneas de un registro te dicen cómo acabó la cosa.

Con estos cuatro tienes el noventa por ciento del uso real. Lo que sigue — la caja de herramientas y el análisis completo — es el *mecanismo por dentro*. No hace falta dominarlo; hace falta haberlo visto una vez, para que ningún comando encadenado te intimide cuando lo leas en una documentación o te lo proponga una IA.

## 3.5. La caja de herramientas

Bajamos al taller. Estos cinco filtros son las piezas clásicas de Unix — las verás en scripts ajenos y en respuestas de IA más veces de las que las teclearás. Todos leen texto por stdin (o de un fichero que les des) y escriben texto por stdout — por eso encajan entre sí en cualquier orden:

| Herramienta | Qué hace | La opción estrella |
|---|---|---|
| `wc` | cuenta | `-l` solo líneas |
| `grep PATRON` | deja pasar las líneas que contienen el patrón | `-c` cuenta en vez de mostrar; `-v` invierte: las que NO lo contienen |
| `sort` | ordena las líneas | `-n` numérico; `-r` al revés |
| `uniq` | colapsa líneas repetidas **adyacentes** | `-c` antepone cuántas veces |
| `cut` | recorta columnas | `-d' '` separador, `-f4` cuarto campo |

Y los que ya conoces, `head` y `tail`, también son filtros: al final de una tubería sirven para quedarte con lo primero o lo último.

**Cuidado · `uniq` solo ve lo adyacente**

`uniq` no busca repetidos por todo el fichero: colapsa líneas iguales *consecutivas*. Sueltas por ahí, se le escapan. Por eso el dúo inseparable es `sort | uniq`: primero juntas los iguales, luego colapsas. Si algún día un recuento te sale absurdo, revisa si te faltó el `sort`.

## 3.6. El análisis, pieza a pieza

Una demostración completa, para ver el mecanismo entero funcionando una sola vez. Vamos a responder la pregunta que un físico se hace ante ese log: **¿qué sensor está fallando?** Y vamos a hacerlo con el método profesional: construir la tubería paso a paso, mirando qué sale de cada pieza antes de añadir la siguiente.

Primer paso: aislar los errores y *mirar* unas cuantas líneas.

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log | head -3
2026-08-18 10:03:00 ERROR s3 sin respuesta del sensor
2026-08-18 10:03:02 ERROR s3 sin respuesta del sensor
2026-08-18 10:03:04 ERROR s3 sin respuesta del sensor
```

El nombre del sensor es una columna. Contemos con los dedos sobre una línea: fecha (1), hora (2), nivel (3), **sensor (4)**, mensaje (5 en adelante). Segundo paso: recortar esa columna… y comprobar.

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log | cut -d' ' -f5 | head -3
sin
sin
sin
```

¿«sin»? Hemos cortado el campo 5 — la primera palabra del *mensaje* — en vez del 4. Este tropiezo está puesto aquí a propósito, porque es el error más típico del capítulo y porque el método lo ha cazado al instante: **cada pieza nueva se comprueba con un `head` antes de seguir**. Corregimos y ahora sí:

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log | cut -d' ' -f4 | head -3
s3
s3
s3
```

Tercer paso: de 108 nombres de sensor al recuento. Los iguales juntos (`sort`), colapsados contando (`uniq -c`), y el ranking de mayor a menor (`sort -nr`):

```
marcos@portatil:~/Descargas$ grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -nr
     90 s3
     15 s1
      3 s4
```

Veredicto en una línea: el sensor **s3** es el culpable con 90 fallos (un conector flojo, apostaría un físico), s1 tuvo una racha corta de lecturas corruptas y s4 apenas tres tropiezos. Este es el momento del capítulo: acabas de interrogar a 7 198 líneas con una sola orden.

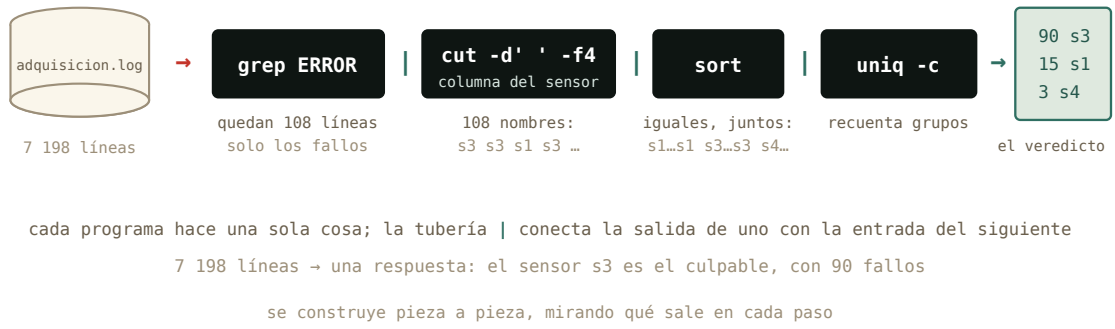



Figura 3.4: La tubería completa del análisis, con lo que fluye en cada tramo.

**Truco · La tubería perfecta no se memoriza: se pregunta**

Que no te intimide la de cinco piezas: es el ejemplo más completo del curso a propósito, para que veas el mecanismo entero de una vez. Tu día a día serán tuberías de dos o tres — y si esta la has entendido, ya sabes cómo funcionan todas. Y una verdad incómoda de toda CLI: **olvidar las herramientas y sus opciones es normal**, hasta para los veteranos. Está permitido pedirle a una inteligencia artificial que te construya el comando («¿cómo cuento los ERROR por sensor en este log?») — con la disciplina del físico: *lee* el comando que te proponga pieza a pieza, verifica con `man` o `--help` lo que no reconozcas, y jamás ejecutes

a ciegas nada que lleve `sudo` o `rm`. Tú ya sabes leer comandos: la IA propone, tú dispones. Este oficio lo afinaremos en un anexo del curso.

¿Y las temperaturas? El valor viene como `temp_C=22.66`: un `=` de separador y `cut` lo recorta igual de bien. Máxima de la noche:

```
marcos@portatil:~/Descargas$ grep INFO adquisicion.log | grep temp_C | cut -d'=' -f2 | sort -n | ta
25.00
```

25.00 justos — sospechosamente redondo. Cruza el dato: `grep -c WARN adquisicion.log` da 357 avisos de «saturada» del sensor s2. El sensor no midió 25.00: se quedó *clavado* en su tope mientras la temperatura real seguía subiendo. Dos tuberías te acaban de dar el diagnóstico completo del montaje: s3 se desconecta, s2 está derivando hasta saturar.

**Por dentro · ¿Y aquel renombrado en lote?**

En el capítulo 2 renombraste cinco series a mano y prometimos algo mejor. La verdad completa: las tuberías transforman *texto*, no ejecutan una orden por cada fichero — para eso hacen falta los bucles del shell, que viven en el anexo de *scripting*. Pero ya tienes la mitad de la solución: `find . -name "*.csv"` te fabrica la **lista** sobre la que un bucle trabajaría. La tubería prepara el trabajo; el bucle lo remata. Todo llega.

## 3.7. ¿Y Python?

La pregunta honesta: tú sabes NumPy — ¿no era más fácil en Python? El mismo recuento, en Python legible:

```
recuento = {}
for linea in open("adquisicion.log"):
    if " ERROR " in linea:
        sensor = linea.split()[3]
        recuento[sensor] = recuento.get(sensor, 0) + 1
for sensor, n in sorted(recuento.items(), key=lambda kv: -kv[1]):
    print(n, sensor)
```

Ocho líneas, mismo resultado (`90 s3 · 15 s1 · 3 s4` — comprobado). Entonces, ¿cuándo cada cosa?

- **Tubería** para el *sistema*: mirar, contar, buscar, guardar. Preguntas de diez segundos cuya respuesta cabe en pantalla — y que funcionan idénticas en el superordenador al que te conectes por SSH, donde quizá no puedas instalar nada.
- **Python** para los *datos*: el análisis de verdad — estadística, ajustes, gráficas — y todo lo que mañana querrás repetir con otros datos. Un script se documenta y se cita; una tubería se teclea y se olvida.

Para un físico la frontera es clara: las tuberías son la herramienta de *administrar* — registros, ficheros, comprobaciones — y Python la de *investigar*. El análisis de este capítulo lo hemos hecho con tuberías para enseñarte el mecanismo; en la vida real lo harías en Python, y las tuberías las reservarías para llegar hasta el fichero, comprobar que es el bueno y guardar el resultado.

## 3.8. Práctica: la noche en el laboratorio

Arranca tu cuaderno (`script sesion03.log`) y descarga el registro de la noche, `adquisicion.log`, desde esta misma sección en la versión web del curso (305 KB de texto — ni se te ocurra abrirlo con doble clic: para eso está la sesión de hoy). Trabaja en `~/Descargas` o muévelo a tu carpeta de física del capítulo 2.

Los ejercicios van en tres niveles. **Lo esencial** (3.1–3.4) es lo que usarás de verdad: no te saltes ninguno. **El mecanismo** (3.5–3.6) es para entender cómo encajan las piezas. **El reto** (3.7–3.9) encadena tres y cuatro tuberías: es opcional y está ahí para quien disfrute — hasta un informático se lo pensaría dos veces. Cada uno lleva su pista, plegada: ábrela sin remordimientos.

**Nivel 1 · Lo esencial**

**Ejercicio 3.1 · Recuento de daños (sin tuberías)**

¿Cuántas líneas tiene el log en total? ¿Cuántas son `ERROR`, y cuántas `WARN`?

*Pista.* `wc -l` para lo primero; para lo demás, `grep -c` cuenta él solo — hoy todavía no necesitas ninguna tubería.

*(Solución al final del capítulo.)*

**Ejercicio 3.2 · Tu primera tubería (una `|`)**

¿A qué hora exacta empezó a fallar el primer sensor, y cuál fue?

*Pista.* El log viene en orden cronológico. Filtra los errores con `grep` y quédate con la primera línea que salga (`head -1`).

*(Solución al final del capítulo.)*

**Ejercicio 3.3 · La salida enorme (lo que harás con tu simulación)**

`ls -lR /usr/share` lista decenas de miles de ficheros: la clase de salida que produce una simulación larga. Primero léela con el paginador: `ls -lR /usr/share | less` — muévete, busca `/png` con la barra, sal con `q`. Después guárdala en un fichero: `ls -lR /usr/share > inventario.txt 2> /dev/null`, y cuenta con `grep -c` cuántas líneas contienen `.png`.

*Pista.* `less` tiene las mismas teclas que `man`; el `2> /dev/null` manda las quejas de permisos a la papelera del sistema; y `grep -c` acepta el fichero como argumento, sin tubería.

*(Solución al final del capítulo.)*

**Ejercicio 3.4 · A fichero (redirección, sin tuberías)**

Crea un fichero `errores.txt` que contenga solo las líneas de error, y comprueba — con un comando sobre ese fichero — que contiene las 108 que esperabas.

*Pista.* `>` desvía el stdout al fichero; y `wc -l` acepta un fichero como argumento, sin tubería.

*(Solución al final del capítulo.)*

**Nivel 2 · El mecanismo**

**Ejercicio 3.5 · La columna del sensor (dos `|`)**

De las líneas `WARN`, extrae solo el nombre del sensor y mira los cinco primeros. ¿Se repite alguno?

*Pista.* Tres piezas — `grep WARN`, `cut -d' ' -f4`, `head -5` — y antes de decidir el `-f`, cuenta los campos con los dedos sobre una línea real (ya sabes lo que pasa si no).

*(Solución al final del capítulo.)*

**Ejercicio 3.6 · Sin repetidos (tres `|`)**

Obtén la lista de sensores *distintos* que emitieron avisos `WARN` — cada nombre una sola vez.

*Pista.* Al ejercicio anterior, quítale el `head` y añádele la pareja inseparable del capítulo: `sort | uniq`.

*(Solución al final del capítulo.)*

**Nivel 3 · El reto (opcional)**

**Ejercicio 3.7 · El jefe final (cuatro `|`)**

El reto de la sesión: reconstruye — de memoria si puedes, con las pistas si no — la tubería del ranking de errores por sensor.

*Pista.* Por pasos: (1) filtra las líneas `ERROR`; (2) recorta el campo 4; (3) junta los iguales; (4) colapsa contando; (5) ordena el recuento de mayor a menor (`sort -nr`). Y en cada paso, tu `head` de comprobación antes de añadir el siguiente.

*(Solución al final del capítulo.)*

**Ejercicio 3.8 · El informe (redirecciones + tu tubería)**

Construye `informe.txt` con tres partes, usando solo `echo` y redirecciones: una línea de título, el total de errores, y el ranking del ejercicio anterior.

*Pista.* La primera orden con `>` (estrena el fichero); las siguientes con `>>` (añaden). La tubería del 3.7 ya la tienes: solo hay que redirigirla.

*(Solución al final del capítulo.)*

**Ejercicio 3.9 · Para nota (opcional) · El termómetro sospechoso**

Solo si te has quedado con ganas: ¿entre qué mínima y máxima se movieron las lecturas *válidas* (las líneas `INFO`)? ¿Y por qué la máxima hay que mirarla con lupa?

*Pista.* El valor viene tras un `=`, y `cut` también sabe cortar por `=`; `sort -n` ordena como números; y los extremos de una lista ordenada son `head -1` y `tail -1`.

*(Solución al final del capítulo.)*

## 3.9. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. En `grep ERROR log | wc -l`, la tubería conecta…

   a) el stdout de `grep` con el stdin de `wc`
   b) el stderr de `grep` con la pantalla
   c) los dos programas con el fichero `log`

2. ¿Por qué `uniq -c` casi siempre lleva un `sort` delante?

   a) porque `uniq` solo colapsa repetidos adyacentes
   b) porque `uniq` exige entrada numérica
   c) por costumbre: sin `sort` da lo mismo

3. Ejecutas dos veces `echo hola > f.txt`. ¿Qué contiene `f.txt`?

   a) dos líneas con «hola»
   b) una sola línea con «hola»
   c) depende del contenido anterior

4. ¿Qué hace `cut -d' ' -f4`?

   a) recorta los 4 primeros caracteres de cada línea
   b) se queda con el cuarto campo, separando por espacios
   c) elimina la cuarta columna y deja el resto

5. ¿Qué canal captura `2>`?

   a) stderr, la salida de errores
   b) stdout, la salida estándar
   c) stdin, la entrada

6. ¿Cuál es el equivalente exacto de `grep ERROR log | wc -l`?

   a) `grep -c ERROR log`
   b) `wc -l ERROR log`
   c) `sort ERROR log | uniq -c`

## Soluciones del capítulo

**Ejercicio 3.1 · Recuento de daños (sin tuberías)**

Solución razonada

`wc -l adquisicion.log` → 7 198. `grep -c ERROR adquisicion.log` → 108. `grep -c WARN adquisicion.log` → 357.
**Por qué así** — Tres preguntas, tres comandos de una sola pieza. Muchos filtros llevan incorporados sus propios atajos (`-c` es «cuenta en vez de mostrar»): antes de encadenar, mira si una pieza sola ya resuelve.

**Ejercicio 3.2 · Tu primera tubería (una |)**

Solución razonada

`grep ERROR adquisicion.log | head -1` → `2026-08-18 10:03:00 ERROR s3 sin respuesta del sensor`. El s3, a las 10:03:00 — tres minutos después de arrancar.
**Por qué así** — «El primer error» es literalmente la primera línea que pase el filtro, porque el orden te lo da hecho el log. Cuando el orden ya existe, aprovéchalo: ni `sort` ni nada.

**Ejercicio 3.3 · La salida enorme (lo que harás con tu simulación)**

Solución razonada

`ls -lR /usr/share | less` (la `R` es *recursivo*: entra en todas las subcarpetas). Después `ls -lR /usr/share > inventario.txt 2> /dev/null` y `grep -c "\.png" inventario.txt` → varios miles, según lo que tengas instalado. (El `\.` fuerza un punto literal; sin la barra, el punto significa «cualquier carácter» para `grep` — un matiz que puedes archivar por ahora.)
**Por qué así** — Este es el patrón que repetirás con tu simulación: salida larga → paginador si solo quieres mirar, fichero si quieres conservar, y `grep` para preguntarle al fichero. Tres reflejos que valen más que cualquier tubería de cinco piezas.

**Ejercicio 3.4 · A fichero (redirección, sin tuberías)**

Solución razonada

`grep ERROR adquisicion.log > errores.txt` y después `wc -l errores.txt` → `108 errores.txt`.
**Por qué así** — Es la figura de la redirección, en vivo: `grep` trabajó igual que siempre, pero su stdout acabó en el fichero. Fíjate en que el recuento coincide con el `grep -c` del 3.1: dos caminos distintos a la misma cifra — la comprobación cruzada de toda la vida en el laboratorio.

**Ejercicio 3.5 · La columna del sensor (dos |)**

Solución razonada

`grep WARN adquisicion.log | cut -d' ' -f4 | head -5` → `s2` cinco veces.
**Por qué así** — Es el método del capítulo en miniatura: filtrar, recortar, y un `head` de comprobación al final. Ese «s2, s2, s2…» ya te está susurrando la respuesta del siguiente ejercicio.

**Ejercicio 3.6 · Sin repetidos (tres |)**

Solución razonada

`grep WARN adquisicion.log | cut -d' ' -f4 | sort | uniq` → `s2`, solo. (Con `uniq -c` verás además que fueron los 357 del 3.1.)
**Por qué así** — `sort | uniq` sobre una columna es la pregunta «¿qué valores distintos hay aquí?». Apréndetela como frase hecha: la usarás mil veces. Y fíjate en cómo has llegado: no escribiste tres tuberías de golpe — le añadiste piezas a una que ya funcionaba.

**Ejercicio 3.7 · El jefe final (cuatro |)**

Solución razonada

`grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -nr` → `90 s3 · 15 s1 · 3 s4`.
**Por qué así** — Si te salió, enhorabuena: cuatro tuberías es más de lo que la mayoría encadena en su día a día. Si no, ningún drama: el objetivo era practicar el método — pieza, `head`, mirar, seguir. Una tubería no se escribe: se *cultiva*. Y recuerda la caja del capítulo: la que no recuerdes, se pregunta — a `man`, o a una IA, leyendo siempre lo que te dé.

**Ejercicio 3.8 · El informe (redirecciones + tu tubería)**

Solución razonada

```
echo "Informe de adquisicion - 18 de agosto" > informe.txt
grep -c ERROR adquisicion.log >> informe.txt
grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -
nr >> informe.txt
```

Comprueba con `cat informe.txt`: título, `108`, y las tres líneas del ranking.
**Por qué así** — Si hubieras usado `>` en las tres, cada orden habría borrado la anterior y el informe tendría solo el ranking. Crear una vez, añadir después: la gramática de todo informe generado por comandos. Cierra el cuaderno con `exit` y guarda `sesion03.log`.

**Ejercicio 3.9 · Para nota (opcional) · El termómetro sospechoso**

Solución razonada

Mínima: `grep INFO adquisicion.log | grep temp_C | cut -d'=' -f2 | sort -n | head -1` → 20.45. Máxima: la misma tubería con `tail -1` → 25.00. Y 25.00 es justo el tope donde s2 satura (los 357 `WARN`): la «máxima» no es una medida, es el techo del instrumento.
**Por qué así** — `sort -n` ordena como números (sin `-n`, `9` iría después de `25`, orden alfabético). Y la lección de física: un valor sospechosamente redondo en el extremo del rango huele a saturación, no a dato. La tubería te dio el número; el criterio lo pones tú.

**Autocomprobación (test de la sección 3.8)** — Respuestas: 1 a · 2 a · 3 b · 4 b · 5 a · 6 a.

### La próxima sesión

Con el bloque A completo, ya *hablas* con la máquina. En el capítulo 4 empieza el bloque B: abrir el capó. Recorrerás el mapa completo del árbol — qué vive de verdad en `/etc`, `/var`, `/usr` —, sabrás quién es `root` y qué significa exactamente ese `sudo` que tecleas con fe, y aprenderás a leer el sistema de permisos `rwx` que llevas viendo en cada `ls -l` sin entenderlo. Guarda `sesion03.log`: tres medidas ya en tu cuaderno.

cap. 3 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion03.log`

# 4 El árbol y los permisos

Ya hablas con la máquina; empieza el bloque B: abrir el capó. Hoy recorres el mapa completo del árbol — qué vive de verdad en cada rama —, descubres que tu sistema es multiusuario hasta la médula, entiendes por fin qué firma exactamente ese `sudo` que tecleas con fe, y aprendes a leer el idioma `rwx` que llevas dos capítulos viendo pasar en cada `ls -l`.

**2–3 h** con ejercicios · requisitos: **bloque A (capítulos 1–3)** · máquina: **tu Linux de diario**

**Al terminar esta sesión sabrás**

- Orientarte por el árbol completo: qué hay en `/etc`, `/home`, `/var`, `/usr`, `/tmp` — y qué rareza es `/proc`.
- Quién eres para el sistema: usuarios, grupos, y el fichero donde está apuntado todo el censo.
- Qué es `root`, qué hace `sudo` de verdad, y por qué te pide *tu* contraseña y no otra.
- Leer de un vistazo los permisos `rwx` de cualquier fichero, y cambiarlos con `chmod`.
- Convertir tus tuberías del capítulo 3 en tu primer *script* ejecutable.
- Localizar dónde guardan su configuración bash, conda y Jupyter — la tuya y la del sistema.

## 4.1. El mapa del territorio

En el capítulo 1 asomaste la cabeza a la raíz; hoy toca el mapa entero. Cada directorio de `/` tiene un oficio, y el reparto no es casualidad: es un estándar — el *Filesystem Hierarchy Standard* — que comparten casi todos los Linux del mundo. Apréndete el plano una vez y te orientarás en cualquier máquina que toques el resto de tu vida, incluido el clúster de cálculo que te espera en unos años.

Compruébalo sobre tu propia máquina, con las herramientas que ya tienes:

```
marcos@portatil:~$ ls /
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  tmp  usr  var
marcos@portatil:~$ ls /etc | wc -l
228
```

Doscientos y pico ficheros de configuración — la tubería del capítulo 3 acaba de contarlos sin despeinarse. Fíjate en la idea de fondo, porque es la misma que viste con `.bashrc`: **en**

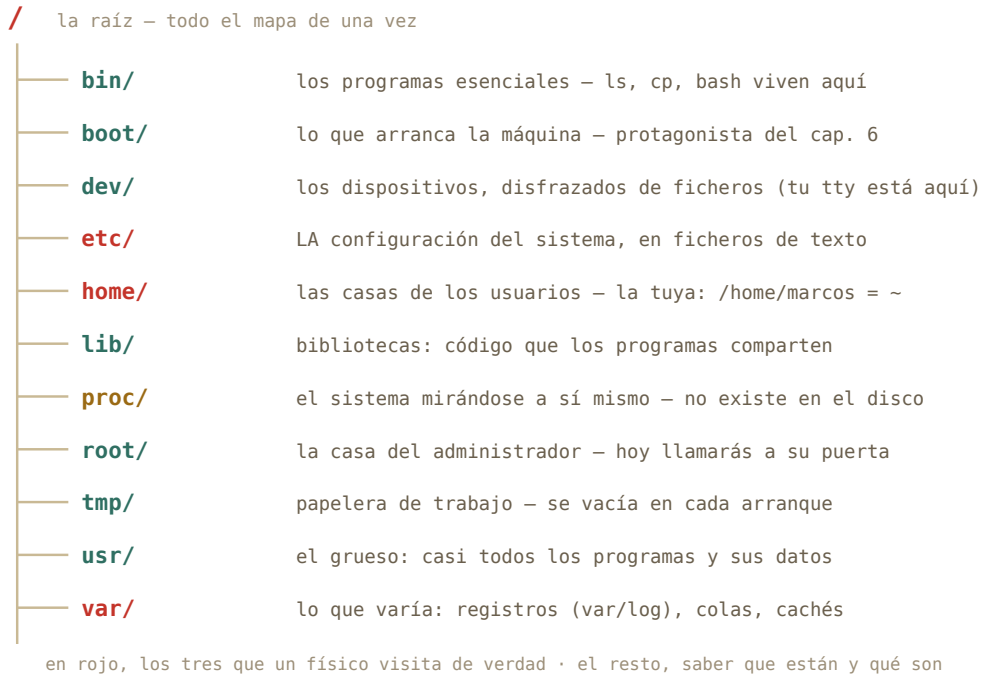



Figura 4.1: El plano de la casa. Tres direcciones en rojo: la configuración (`/etc`), tu casa (`/home`) y los registros (`/var/log`).

**Linux, configurar el sistema es editar ficheros de texto en `/etc`.** No hay registro opaco ni caja negra: está todo ahí, legible con `less`.

**Por dentro · `/proc`, el directorio que no existe**

Haz `cat /proc/uptime`: dos números, los segundos que lleva encendida la máquina. Ese «fichero» no está en tu disco — `/proc` es una ventana que el kernel fabrica al vuelo para contarte su estado, disfrazada de directorio para que puedas usar `cat`, `grep` y compañía sobre ella. Prueba `grep "model name" /proc/cpuinfo`: acabas de interrogar a tu procesador con las herramientas del capítulo 3. «Todo es un fichero» no era una metáfora: es la decisión de diseño más rentable de Unix.

## 4.2. Quién eres: usuarios y grupos

`whoami` te lo dijo el primer día; la respuesta completa la da `id`:

```
marcos@portatil:~$ id
uid=1000(marcos) gid=1000(marcos) grupos=1000(marcos),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev)
```

Para el sistema no eres «marcos»: eres el **uid 1000** — el nombre es un alias para humanos. Y perteneces a **grupos**: bolsas de usuarios a las que se conceden permisos en bloque. Ahí en medio hay uno que va a protagonizar la siguiente sección: `sudo`.

¿Y dónde está apuntado el censo? En un fichero de texto de `/etc`, naturalmente:

```
marcos@portatil:~$ less /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
marcos:x:1000:1000:Marcos,,,:/home/marcos:/bin/bash
```

Cada línea, un usuario; los campos, separados por `:` — uid, casa, shell. Dos sorpresas al leerlo: la primera línea es `root`, con **uid 0**; y hay docenas de usuarios que no son personas (`daemon`, `www-data`…) — cuentas de servicio con las que el sistema se reparte a sí mismo el trabajo, cada una con los permisos justos de su oficio.

**Historia · Los permisos nacieron porque el ordenador era de todos**

En los 70, «tener ordenador» significaba compartir *el* ordenador del departamento: decenas de terminales enchufados a una sola máquina Unix, cada usuario con su cuenta, todos a la vez — el *tiempo compartido*. En ese mundo, que tus ficheros no fueran legibles por el resto no era paranoia: era convivencia. Tu portátil ya no se comparte, pero heredó intacto aquel esqueleto multiusuario — y le sigue sacando partido: cada servicio corre como un usuario distinto para que un fallo en uno no arrastre al sistema. Y en el clúster de cálculo al que te conectarás algún día, el tiempo compartido sigue siendo, literalmente, la forma de vida.



Figura 4.2: Un terminal Unix de tiempo compartido en la Universidad de Wisconsin, 1978. Decenas de personas, una máquina: de ahí vienen usuarios, grupos y permisos. *Foto: Dave Winer, CC BY-SA 2.0, Wikimedia Commons.*

## 4.3. `root` y `sudo`: llamar a la puerta del administrador

`root`, uid 0, es el superusuario: el único para quien los permisos no existen. Puede leerlo todo, borrarlo todo, romperlo todo. Por eso en Mint nadie trabaja *como* root — ni siquiera tú, que administras tu propia máquina. Compruébalo intentando entrar en su casa:

```
marcos@portatil:~$ ls /root
ls: no se puede abrir el directorio '/root': Permiso denegado
marcos@portatil:~$ sudo ls /root
[sudo] contraseña de marcos:
marcos@portatil:~$
```

Ahí está el mecanismo civilizado: `sudo` (*superuser do*) ejecuta **una única orden** con los poderes de root, y deja constancia en el registro del sistema de quién la pidió y cuándo. Pide *tu* contraseña — no una «contraseña de root» — porque lo que comprueba es que tú eres tú y que estás autorizado: miembro del grupo `sudo` que viste en `id`. La orden terminó, y volviste a ser el uid 1000 de siempre. Poderes de administrador de usar y tirar, con recibo.

**Cuidado · `sudo` firma con tu nombre**

Con `sudo` delante, `rm` ya no protege nada: puede vaciar el sistema entero. La regla del curso es doble. Uno: **jamás teclees un `sudo` que no entiendas** — y eso incluye los que te sugiera una IA o un foro (cuando llegue el anexo sobre IA verás cómo verificarlos con `man` antes). Dos: lo destructivo de verdad — particionar, formatear, reinstalar — no se practica aquí, sino en la máquina virtual del bloque C, donde romper es gratis.

## 4.4. Leer `rwx` de un vistazo

Llevas desde el capítulo 1 viendo cosas como `-rw-r--r--` en cada `ls -l` y esquivándolas educadamente. Se acabó: son diez caracteres con una gramática exacta.

Sobre un directorio, `x` no significa «ejecutar» sino **entrar**: sin esa `x` no puedes hacer `cd` dentro. Y ahora ya puedes leer entradas de `ls -l` como un nativo:

```
marcos@portatil:~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1523 ago 20 10:11 /etc/shadow
marcos@portatil:~$ cat /etc/shadow
cat: /etc/shadow: Permiso denegado
```

Traducción: dueño `root` (lee y escribe), grupo `shadow` (solo lee), y los demás — tú — **nada**. Por eso el `cat` rebota. Ese fichero guarda los *hashes* de las contraseñas: es el ejemplo perfecto de un permiso haciendo exactamente su trabajo. Compáralo con su vecino `/etc/passwd` (`-rw-r--r--`): el censo es público, las contraseñas no.

Cambiar permisos es `chmod` (*change mode*), y su dialecto simbólico se lee solo: a quién (`u`, `g`, `o`), qué gesto (`+` da, `-` quita), qué permiso (`r`, `w`, `x`):

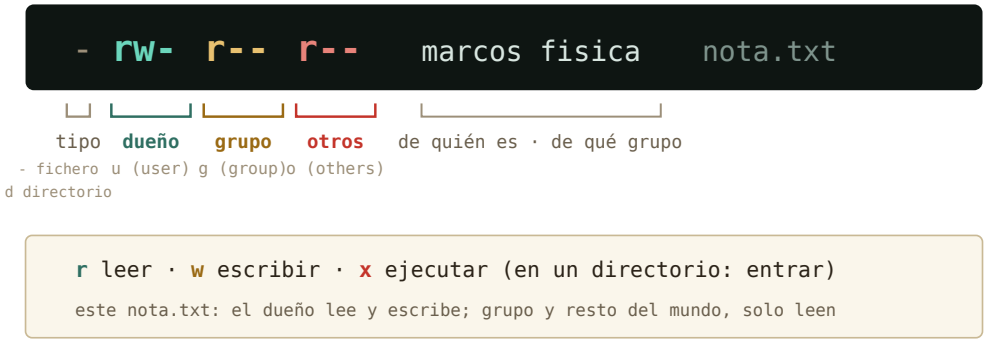



Figura 4.3: Diez caracteres, cuatro bloques: tipo, y permisos del dueño, del grupo y del resto del mundo.

```
marcos@portatil:~/fisica$ chmod go-r pendulo_v1.csv
marcos@portatil:~/fisica$ ls -l pendulo_v1.csv
-rw------- 1 marcos marcos 428 ago 24 11:02 pendulo_v1.csv
```

Grupo y otros acaban de perder la lectura: ese dato ya solo lo ves tú. (También existe `chown` para cambiar el *dueño*, pero regalar ficheros es cosa de root — te cruzarás con él al administrar tu servidor.)

## 4.5. Tu primer script ejecutable

Y aquí es donde los capítulos 2, 3 y 4 se dan la mano. Aquella tubería del análisis que tecleabas una y otra vez — ¿y si fuera un *programa*? Escríbela en un fichero con `nano informe.sh`:

```
#!/bin/bash
echo "Informe de adquisicion"
grep -c ERROR adquisicion.log
grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -nr
```

La primera línea — el *shebang* `#!` — le dice al sistema qué intérprete ejecuta esto: bash. El resto es, literalmente, lo que tecleabas. Dale el permiso que le falta y estrénalo:

```
marcos@portatil:~/Descargas$ chmod u+x informe.sh
marcos@portatil:~/Descargas$ ./informe.sh
Informe de adquisicion
108
     90 s3
```

```
     15 s1
      3 s4
```

Acabas de escribir un programa. El `./` delante significa «el de *este* directorio» — por qué hace falta exactamente eso lo entenderás en el capítulo 5, cuando abramos el `$PATH`. La idea que importa hoy: **un script no es más que comandos guardados, y `x` es lo que lo convierte de apunte en herramienta.**

**Truco · Los permisos, también con ratón**

Todo este sistema `rwx` lo tienes en la vía gráfica: clic derecho sobre cualquier fichero en el gestor de archivos → Propiedades → pestaña Permisos. Son las mismas casillas de leer, escribir y ejecutar para dueño, grupo y otros. Para un fichero suelto es perfectamente digno; la terminal gana cuando son doscientos — o cuando estás en un servidor sin escritorio, que es tu futuro próximo.

## 4.6. La expedición de las configuraciones

El caso práctico del capítulo, y una pregunta que te harás mil veces: *¿dónde guarda su configuración este programa?* La respuesta tiene un patrón — la del **sistema** en `/etc`, la **tuya** en dotfiles de tu casa — y ya tienes herramientas para verificarlo:

```
marcos@portatil:~$ ls -a ~ | grep -i conda
.conda
.condarc
marcos@portatil:~$ ls ~/.jupyter
jupyter_notebook_config.py
marcos@portatil:~$ ls /etc/jupyter 2> /dev/null
```

Ahí está tu ecosistema científico: `.condarc` (tus canales y entornos de conda), `~/.jupyter` (tu Jupyter), `.bashrc` (tu shell, del capítulo 2). Ninguno necesita `sudo`: son tuyos, en tu casa. Cuando un programa «se comporta raro» en tu cuenta pero no en la de otro, la explicación vive casi siempre en uno de estos ficheros ocultos — ahora sabes dónde mirar. (¿Y ese `2> /dev/null` del final? Los tres chorros del capítulo 3: manda las quejas al agujero negro del sistema. Si `/etc/jupyter` no existe, silencio en vez de error — útil cuando solo exploras.)

## 4.7. Práctica: inspección general

Cuaderno en marcha (`script sesion04.log`) y a explorar tu propia máquina. Ningún ejercicio de hoy modifica nada fuera de tu casa.

**Ejercicio 4.1 · El censo de `/etc`**

¿Cuántas entradas tiene tu `/etc`? ¿Qué Linux exacto ejecutas?

*Pista.* La respuesta está en `/etc/os-release`, y `grep PRETTY` la aísla.

*(Solución al final del capítulo.)*

**Ejercicio 4.2 · Leer el censo**

Usando `/etc/passwd` y tuberías: ¿cuántas cuentas hay en total en tu sistema? ¿Cuáles son personas de verdad?

*Pista.* Las personas tienen su casa en `/home` — y `grep` sabe buscar eso.

*(Solución al final del capítulo.)*

**Ejercicio 4.3 · Tus credenciales**

Con un solo comando y sin leer ficheros: comprueba si tu usuario pertenece al grupo `sudo`.

*(Solución al final del capítulo.)*

**Ejercicio 4.4 · Detective de permisos**

Ejecuta `ls -l /etc/passwd /etc/shadow /usr/bin/nano /tmp` (el último con `ls -ld /tmp`: la `d` hace que describa el directorio en vez de su contenido). Traduce a palabras los permisos de cada uno. ¿Cuál puede leer todo el mundo pero escribir solo root? ¿En cuál puedes escribir tú?

*(Solución al final del capítulo.)*

**Ejercicio 4.5 · De tubería a herramienta**

Crea el script `diagnostico.sh` que imprima: quién eres, en qué máquina estás (`hostname`), la fecha, y cuánto lleva encendido el equipo (`/proc/uptime`). Hazlo ejecutable y ejecútalo dos veces — comprueba que la fecha cambia y el uptime crece.

*(Solución al final del capítulo.)*

**Ejercicio 4.6 · `sudo`, con recibo**

Ejecuta `ls /root` sin y con `sudo`. Después reflexiona por escrito en tu cuaderno (una frase vale): ¿por qué es *mejor* este diseño — pedir permiso orden a orden — que trabajar directamente como root todo el rato?

*(Solución al final del capítulo.)*

## 4.8. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. ¿Dónde vive la configuración del sistema en Linux?

   a) en `/etc`, como ficheros de texto
   b) en un registro binario central
   c) en `/tmp`

2. El usuario con uid 0 es…

   a) el primero que se creó al instalar
   b) `root`, para quien no existen los permisos
   c) una cuenta de servicio como `daemon`

3. ¿Qué hace exactamente `sudo ls /root`?

   a) te convierte en root hasta que cierres la terminal
   b) ejecuta solo ese `ls` como root, y lo deja registrado
   c) pide la contraseña de root para desbloquear la carpeta

4. Un fichero muestra `-rw-r-----` con dueño `root` y grupo `shadow`. Tú no estás en ese grupo. ¿Qué puedes hacer con él?

   a) leerlo, pero no modificarlo
   b) nada: ni leerlo
   c) leerlo y ejecutarlo

5. Sobre un *directorio*, el permiso `x` significa…

   a) poder entrar en él
   b) poder ejecutar sus ficheros
   c) nada: solo aplica a ficheros

6. Escribes un script y al hacer `./analisis.sh` el shell dice «Permiso denegado». ¿Qué falta?

   a) ejecutarlo con `sudo`
   b) `chmod u+x analisis.sh`
   c) moverlo a `/bin`

## Soluciones del capítulo

**Ejercicio 4.1 · El censo de /etc**

Solución razonada

`ls /etc | wc -l` (en torno a 200, según lo instalado) y `grep PRETTY /etc/os-release` → `PRETTY_NAME="Linux Mint 22.2"` (o tu versión).
**Por qué así** — Todo son ficheros de texto, así que las herramientas del capítulo 3 valen también para interrogar al sistema. Ese `os-release` es exactamente lo que los programas leen cuando quieren saber dónde están corriendo.

**Ejercicio 4.2 · Leer el censo**

Solución razonada

Total: `wc -l /etc/passwd` (unas 40–50). Personas: `grep /home /etc/passwd | cut -d':' -f1` — probablemente solo tú.
**Por qué así** — El separador aquí es `:`, y `cut -d':' -f1` recorta el nombre igual que recortaba sensores en el log. El resto de cuentas son los usuarios de servicio: el sistema repartiéndose el trabajo con permisos mínimos.

**Ejercicio 4.3 · Tus credenciales**

Solución razonada

`id | grep sudo` — si tu línea de `id` aparece (con `27(sudo)` iluminado), estás dentro. También valía `groups`, que lista solo los nombres.
**Por qué así** — `id` escupe texto y `grep` filtra texto: la componibilidad del capítulo 3 aplicada a la administración. Estar en ese grupo es lo que hace que `sudo` te acepte; en un ordenador del laboratorio, probablemente no lo estarás — y ahora entiendes por qué.

**Ejercicio 4.4 · Detective de permisos**

Solución razonada

`/etc/passwd` → `-rw-r--r--`: censo público, lo escribe solo root. `/etc/shadow` → `-rw-r-----`: ni leerlo puedes. `/usr/bin/nano` → `-rwxr-xr-x`: todo el mundo lo ejecuta, solo root lo modifica — así deben ser los programas del sistema. `/tmp` → `drwxrwxrwt`: escribible por todos (esa `t` final es un matiz — impide borrar lo ajeno — que puedes archivar como curiosidad).
**Por qué así** — Cuatro ficheros, cuatro políticas distintas, y las cuatro se leen de un vistazo con la figura del capítulo. Los permisos no son burocracia: son el diseño de seguridad del sistema, escrito en diez caracteres.

**Ejercicio 4.5 · De tubería a herramienta**

Solución razonada

Con `nano diagnostico.sh`:

```
#!/bin/bash
whoami
hostname
date
cat /proc/uptime
```

Después `chmod u+x diagnostico.sh` y `./diagnostico.sh`.
**Por qué así** — Es el patrón completo de la sección: comandos guardados + shebang + `u+x` + `./`. Cuando administres tu servidor, tus scripts de diagnóstico serán exactamente esto con más líneas. El uptime que crece entre ejecuciones te confirma que `/proc` se fabrica al vuelo: dos lecturas, dos realidades.

**Ejercicio 4.6 · sudo, con recibo**

Solución razonada

Sin `sudo`: permiso denegado. Con él: tu contraseña, y la casa de root (probablemente casi vacía). La reflexión, en la dirección de: como root, *cualquier* errata es potencialmente fatal — un `rm` mal apuntado, un espacio de más como el del capítulo 2. Con `sudo` elevas los privilegios solo el segundo que hacen falta, y cada uso queda registrado con tu nombre.
**Por qué así** — Es el principio de mínimo privilegio, y es idéntico al del laboratorio: la llave del armario de isótopos no se lleva en el bolsillo todo el día; se pide, se usa, se devuelve, y queda apuntado. Cierra el cuaderno: `exit`, y a la colección `sesion04.log`.

**Autocomprobación (test de la sección 4.8)** — Respuestas: 1 a · 2 b · 3 b · 4 b · 5 a · 6 b.

### La próxima sesión

Sabes quién puede tocar qué; el capítulo 5 pregunta *qué está pasando ahora mismo*: procesos, `top`, señales y el arte de matar un programa colgado. Y verás la otra mitad del capó: qué hace de verdad el gestor de software de Mint por debajo — `apt`, repositorios, dependencias — y por qué a veces `python` no es el Python que crees. Guarda `sesion04.log`.

cap. 4 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion04.log`

# 5 Paquetes y procesos

Dos misterios cotidianos caen hoy. Primero: qué pasa de verdad cuando el Gestor de software instala un programa — vas a hacerlo tú por debajo, con `apt`. Segundo: qué está *ejecutándose* ahora mismo en tu máquina, cómo vigilarlo y cómo rematar un programa colgado. Y de postre, la deuda del capítulo 4: por qué `./`, qué es el `$PATH`, y el clásico «este no es mi Python» que algún día te robará una tarde si hoy no lo desactivamos.

**2–3 h** con ejercicios · requisitos: **capítulos 1–4** · máquina: **tu Linux de diario**

**Al terminar esta sesión sabrás**

- Instalar, buscar y desinstalar programas con `apt`, y qué son un repositorio y una dependencia.
- La diferencia exacta entre `apt update` y `apt upgrade` (y por qué van siempre en pareja).
- Qué es una distro de verdad — kernel, utilidades, paquetes, escritorio — y de qué familia es tu Mint.
- Ver los procesos vivos con `ps`, `top` y `htop`, y terminar uno colgado con `kill`.
- Lanzar tareas en segundo plano con `&`.
- Leer el `$PATH`: por qué `./`, y por qué a veces `python` no es el Python que crees.

## 5.1. Instalar por debajo del envoltorio

Cada vez que el Gestor de software de Mint instala algo, quien trabaja es `apt` (*Advanced Package Tool*). Un programa en Linux se distribuye como **paquete** — el programa, más sus metadatos: versión, de qué otros paquetes depende — y los paquetes viven en **repositorios**: almacenes en internet mantenidos por tu distro, con decenas de miles de paquetes verificados. Tu máquina guarda un **catálogo local** con la lista de lo que existe; instalar es elegir del catálogo.

Vamos a instalar algo útil para el resto del capítulo. Tres pasos: refrescar el catálogo, buscar, instalar — los dos que tocan el sistema piden `sudo`, y ahora sabes exactamente por qué:

```
marcos@portatil:~$ sudo apt update
[sudo] contraseña de marcos:
Obj:1 http://packages.linuxmint.com xia InRelease
Des:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Leyendo lista de paquetes... Hecho
marcos@portatil:~$ apt search ^htop
htop/noble 3.3.0-4 amd64
```




Figura 5.1: El triángulo de `apt`. La confusión clásica del principiante es creer que `update` actualiza programas: solo refresca la lista.

```
  interactive processes viewer
marcos@portatil:~$ sudo apt install htop
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  htop
Configurando htop (3.3.0-4) ...
```

Fíjate en «árbol de dependencias»: si el paquete necesita bibliotecas que no tienes, `apt` las localiza, las trae y las instala en el orden correcto, solo. Esa resolución automática — que hoy parece obvia — es el gran invento de este sistema. Desinstalar es `sudo apt remove htop`; y la pareja `sudo apt update && sudo apt upgrade` es exactamente lo que el escudito de actualizaciones de Mint hace por ti cada semana.

**Truco · Las dos caras del mismo catálogo**

El Gestor de software y Synaptic (el gestor gráfico veterano, más detallista) son *interfaces* del mismo `apt`: mismo catálogo, mismos paquetes, mismos botones de fondo. Para descubrir software con capturas y reseñas, la vía gráfica es estupenda. La terminal gana en precisión — versiones exactas, `apt show` — y sobre todo donde no hay botones: tu futuro servidor. Usa la que pida el momento.

## 5.2. Qué es una distro (ahora ya lo puedes entender)

Con el vocabulario de hoy, la pregunta «¿qué Linux tienes?» por fin tiene respuesta técnica. **Linux**, en rigor, es solo el kernel: el núcleo que habla con el hardware. Encima van las utilidades del sistema (bash, `ls`, `grep`… — el proyecto GNU), encima el sistema de paquetes




Figura 5.2: Las capas, y la familia. Cambiar de escritorio es cambiar la capa de arriba; cambiar de familia de paquetes ya es mudarse.

con sus repositorios, y encima el escritorio. Una **distribución** es una elección concreta y empaquetada de todas esas capas.

Tu Mint pertenece a la familia Debian: hereda de Ubuntu, que hereda de Debian, y por eso este capítulo te sirve igual en las tres — y en el Debian *sin escritorio* que instalarás en el bloque C. Cuando leas «Fedora usa `dnf`» o «Arch usa `pacman`», ya sabes qué significa: otra familia, otro gestor de paquetes, mismas ideas. Y los escritorios — Cinnamon en tu Mint, GNOME, KDE, Xfce — son la capa más visible y la menos profunda: mismo sistema debajo, distinta carrocería.

**Historia · Debra, Ian, y el infierno de las dependencias**

Debian nació en 1993, bautizado por su creador con su nombre y el de su pareja: **Deb**ra + **Ian** Murdock. Su gran legado llegó en 1998 con `apt`: hasta entonces, instalar un programa era perseguir a mano la cadena de bibliotecas que necesitaba — el «infierno de las dependencias» que todavía traumatiza a los veteranos. La resolución automática de `apt` lo enterró, y la idea es hoy universal: `pip` y `conda` en Python son descendientes directos de ese concepto. Cuando esta tarde `apt` te resuelva un árbol de dependencias en un segundo, levanta la vista un momento: estás viendo el invento de Ian.

## 5.3. Qué está pasando ahora mismo: procesos

Un **proceso** es un programa *en ejecución*: el fichero ejecutable era la partitura, el proceso es el concierto. Cada uno tiene un número único — el **PID** — y un dueño, con sus permisos del capítulo 4. `ps` los lista; la variante que verás en todas partes es `ps aux` (todos los procesos, de todos los usuarios, con detalle):

```
marcos@portatil:~$ ps aux | wc -l
312
marcos@portatil:~$ ps aux | grep firefox
marcos     8113  4.2  3.1 ... /usr/lib/firefox/firefox
```

Trescientos procesos y solo tienes tres ventanas abiertas: el resto es el sistema trabajando — servicios, el escritorio, controladores. Y sí: acabas de filtrar procesos con las tuberías del capítulo 3, porque `ps` escupe texto como todo el mundo.

Para *vigilar* en directo está `top`, y su versión moderna — la que instalaste hace un rato — `htop`:

```
marcos@portatil:~$ htop
```

Un panel vivo: barras de CPU por núcleo, memoria, y la lista de procesos ordenada por consumo, actualizándose sola. Se sale con q, como en `less` — y con F6 eliges por qué columna ordenar. Cuando el ventilador de tu portátil despegue en mitad de una simulación, `htop` es la primera ventanilla de información: qué proceso, cuánta CPU, cuánta memoria.

**Truco · La versión con gráficas**

El Monitor del sistema de Mint (menú → Administración) es esta misma información con gráficas de colores — perfectamente digno para un vistazo rápido. `htop` gana en detalle, en velocidad… y en que funciona idéntico dentro del servidor sin ventanas al que te conectarás por SSH. Aprendido una vez, vale en todas partes.

## 5.4. Terminar lo colgado: señales y `kill`

El caso real: tu script de análisis lleva veinte minutos en un bucle del que no va a salir. Los procesos se gobiernan mandándoles **señales**. Una ya la usas sin saberlo: Ctrl+C en la terminal envía SIGINT — «interrúmpete» — al proceso que ocupa el primer plano. Para un proceso que no está en tu terminal, la señal se envía por PID:

```
marcos@portatil:~$ ps aux | grep analisis
marcos    9204 98.7  0.4 ... python analisis.py
marcos@portatil:~$ kill 9204
```

`kill`, pese al nombre, es educado: envía SIGTERM — «termina, por favor» — y el proceso puede cerrar ficheros y despedirse. Si lo ignora, queda el recurso final: `kill -9`, SIGKILL, que no se puede ignorar ni atender — el kernel lo elimina en seco. La diferencia importa: es apagar la centrífuga con su rampa de frenado frente a cortarle la corriente. La segunda funciona siempre, pero lo que hubiera a medio escribir se queda a medias. Protocolo: primero `kill`, cinco segundos de cortesía, y solo entonces `kill -9`.

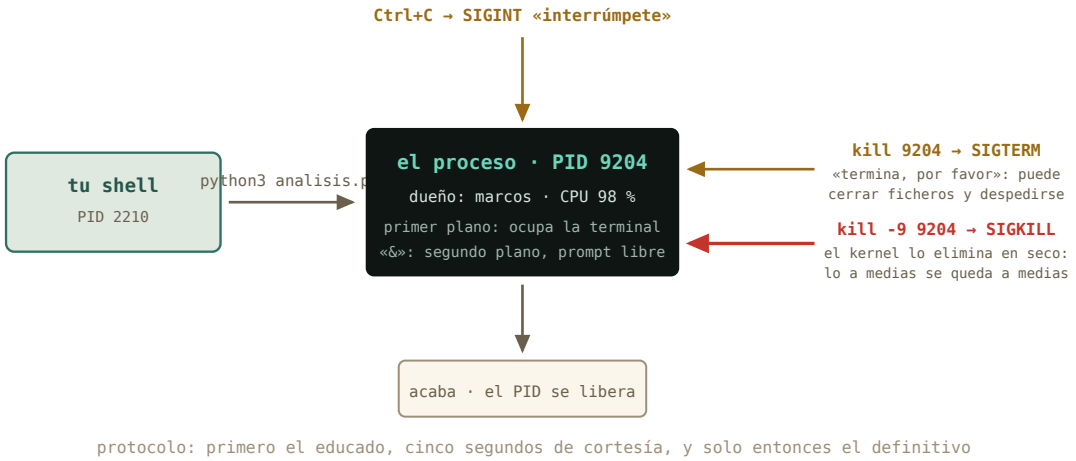



Figura 5.3: La vida de un proceso. Nace del shell, corre en primer o segundo plano, y termina cuando acaba — o cuando una señal se lo pide.

## 5.5. Segundo plano: el `&`

Hasta hoy, cada comando secuestra tu terminal hasta terminar. Un `&` al final lo lanza al **segundo plano** y te devuelve el prompt:

```
marcos@portatil:~$ sleep 600 &
[1] 9871
marcos@portatil:~$ jobs
[1]+  Ejecutando              sleep 600 &
```

El shell te dice su número de trabajo `[1]` y su PID. `jobs` lista lo que tienes en marcha; `fg` lo trae de vuelta al primer plano; y si un comando ya lanzado te está reteniendo, Ctrl+Z lo pausa y `bg` lo reanuda detrás. Para una simulación de diez minutos mientras sigues trabajando, esto basta. Para una de diez *horas* en un servidor remoto hará falta algo más robusto — se llama `tmux`, y es el plato fuerte del capítulo 13. El `&` de hoy es su semilla.

## 5.6. El `$PATH`: la deuda del `./`

Toca pagar lo prometido. El shell guarda **variables de entorno** — datos con nombre que los programas heredan. Míralas: `env` las lista todas; `echo` imprime una:

```
marcos@portatil:~$ echo $HOME
/home/marcos
marcos@portatil:~$ echo $PATH
/home/marcos/.local/bin:/usr/local/bin:/usr/bin:/bin:/usr/games
```

Esa lista separada por `:` es el **PATH**: los directorios donde el shell busca, *en orden*, cada comando que tecleas. Escribes `ls`, el shell recorre la lista, lo encuentra en `/usr/bin` y lo ejecuta. `which` te dice quién ganó la búsqueda:

```
marcos@portatil:~$ which ls
/usr/bin/ls
marcos@portatil:~$ which python3
/usr/bin/python3
```

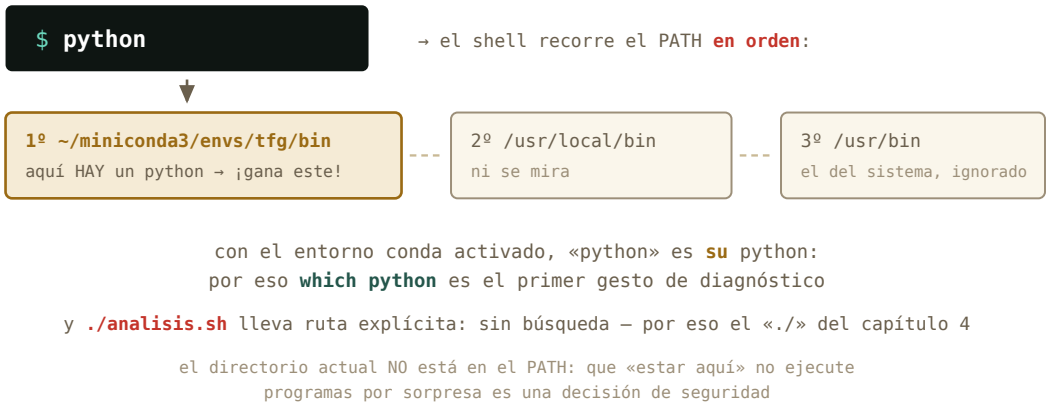



Figura 5.4: La búsqueda del PATH, y sus dos moraleja: el orden decide, y `./` se la salta.

Y aquí las dos respuestas pendientes. **Por qué `./`**: el directorio actual *no* está en el PATH — es una decisión de seguridad, para que entrar en una carpeta ajena no pueda ejecutarte programas por sorpresa —, así que a tus scripts se les da la ruta explícita: `./informe.sh`. **Y el «no es mi Python»**: herramientas como conda funcionan *anteponiendo* su directorio al PATH cuando activas un entorno. Desde ese momento, `python` es el suyo — con sus paquetes, su versión — y no el del sistema:

```
marcos@portatil:~$ conda activate tfg
(tfg) marcos@portatil:~$ which python
/home/marcos/miniconda3/envs/tfg/bin/python
```

**Cuidado · El primer gesto ante «aquí falta un paquete»**

El clásico: «en mi terminal funciona, en Jupyter no encuentra numpy» — o al revés. Antes de reinstalar nada a lo loco, dos comandos de diagnóstico: `which python` (¿*qué* Python está respondiendo?) y `echo $PATH` (¿quién va primero en la cola?). El noventa por ciento de esos misterios es un entorno activado — o sin activar — cambiando quién gana la búsqueda. Ahora tienes el instrumento para verlo en diez segundos.

## 5.7. Práctica: el taller

Cuaderno en marcha (`script sesion05.log`). Hoy instalarás software de verdad — todo reversible con `apt remove`.

**Ejercicio 5.1 · Tu primera instalación completa**

Instala el paquete `tree` con el ciclo completo: refresca el catálogo, búscalo, léelo (`apt show tree` — mira su descripción y su campo `Depends:`), instálalo. Después estrénalo: `tree ~/Descargas/experimento-pendulo`.

*(Solución al final del capítulo.)*

**Ejercicio 5.2 · El censo de procesos**

¿Cuántos procesos corren en tu máquina? ¿Y cuántos pertenecen a cada usuario? Para lo segundo, `ps -eo user` te da la columna de dueños limpia — termina tú la tubería (capítulo 3) que produce el ranking.

*(Solución al final del capítulo.)*

**Ejercicio 5.3 · Caza controlada**

Lanza `sleep 600 &`. Localiza su PID de dos maneras distintas (`jobs -l` y `ps aux | grep`), termínalo educadamente, y verifica que ya no está.

*(Solución al final del capítulo.)*

**Ejercicio 5.4 · update no es upgrade**

Sin ejecutarlos: escribe en tu cuaderno qué hará exactamente cada mitad de `sudo apt update && sudo apt upgrade`. Después ejecuta la pareja y comprueba tu predicción con la salida real.

*(Solución al final del capítulo.)*

**Ejercicio 5.5 · Un `bin` propio**

Objetivo: que tu `diagnostico.sh` del capítulo 4 se ejecute desde cualquier sitio, sin `./` ni rutas. Pasos: crea `~/bin`, mueve el script dentro, añade el directorio al PATH de la sesión (`export PATH="$HOME/bin:$PATH"`), y prueba `diagnostico.sh` a secas desde varios directorios. Comprueba con `which` quién responde.

*(Solución al final del capítulo.)*

**Ejercicio 5.6 · ¿Quién es tu Python?**

Averigua qué `python3` responde en tu sistema (`which`), y su versión (`python3 --version`). Si usas conda o entornos virtuales: actívalo y repite `which python` — documenta en tu cuaderno el antes y el después del PATH (`echo $PATH | cut -d':' -f1` enseña quién va primero).

*(Solución al final del capítulo.)*

## 5.8. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. `sudo apt update`…

   a) actualiza todos los programas instalados
   b) refresca el catálogo: qué existe y en qué versión
   c) actualiza el kernel

2. Una «dependencia» de un paquete es…

   a) otro paquete que necesita para funcionar
   b) una versión antigua del mismo paquete
   c) el repositorio del que procede

3. El PID de un proceso es…

   a) su prioridad de CPU
   b) su número identificador único mientras vive
   c) el puerto de red que usa

4. Diferencia entre `kill PID` y `kill -9 PID`:

   a) ninguna: -9 solo es más rápido
   b) `kill` pide terminar con orden; `-9` elimina en seco, sin opción a limpiar
   c) `-9` mata también los procesos vecinos

5. Tecleas `python` con un entorno conda activado. ¿Cuál se ejecuta?

   a) el primero que aparezca recorriendo el PATH en orden — el del entorno
   b) siempre el del sistema, `/usr/bin/python3`
   c) el más moderno de los instalados

6. Tu script está en el directorio actual, pero `informe.sh` da «orden no encontrada». ¿Por qué?

   a) le falta `chmod u+x`
   b) el directorio actual no está en el PATH: es `./informe.sh`
   c) los scripts requieren `sudo`

## Soluciones del capítulo

**Ejercicio 5.1 · Tu primera instalación completa**

Solución razonada

`sudo apt update`, `apt search ^tree`, `apt show tree`, `sudo apt install tree`. Y el estreno dibuja tu trabajo del capítulo 2:

```
experimento-pendulo
  datos
    run01.csv
    ...
  descartes
  notas
  LEEME.txt
  registro.log
```

**Por qué así** — `apt show` antes de instalar es el hábito sano: qué es, cuánto ocupa, de qué depende (`Depends: libc6` — la biblioteca C que comparte casi todo el sistema). Buscar, leer, instalar: el mismo rigor que con un reactivo nuevo en el laboratorio.

### Ejercicio 5.2 · El censo de procesos

Solución razonada

`ps aux | wc -l` (unos 300) y `ps -eo user | sort | uniq -c | sort -nr | head -3` — root encabeza, tú segundo, y el resto son cuentas de servicio del capítulo 4.
**Por qué así** — ¿Por qué `ps -eo user` y no `ps aux | cut -d' ' -f1`? Prueba el `cut` y verás rarezas: `ps aux` alinea columnas con *varios* espacios seguidos, y para `cut` cada espacio separa un campo. La lección del log del capítulo 3, reapareciendo: antes de cortar, mira cómo está separado de verdad. Aquí es más limpio pedirle a `ps` la columna exacta.

### Ejercicio 5.3 · Caza controlada

Solución razonada

`sleep 600 &`, luego `jobs -l` (muestra el PID junto al `[1]`) o `ps aux | grep sleep`. Con el número: `kill 9871` (el tuyo será otro). Verificación: `jobs` vacío, o `ps aux | grep sleep` sin resultado — salvo, quizá, ¡el propio `grep sleep` cazándose a sí mismo en la lista! Es el clásico bucle del aprendiz: el grep aparece porque él también es un proceso cuya línea de comando contiene «sleep».
**Por qué así** — `sleep` es el conejillo perfecto: un proceso inofensivo que hace exactamente nada durante N segundos. El protocolo practicado — localizar, `kill`, verificar — es idéntico para la simulación colgada de verdad.

### Ejercicio 5.4 · update no es upgrade

Solución razonada

`update` descarga las listas nuevas del repositorio: al final resume cuántos paquetes «se pueden actualizar» — pero no toca ninguno. `upgrade` compara esas listas con lo instalado y trae las versiones nuevas (te enseña la lista y pide confirmación). El `&&`

encadena: «y si lo anterior salió bien, entonces…».
**Por qué así** — Predicción teórica y contraste experimental, como siempre. Esta pareja es el mantenimiento básico de tu máquina — y la versión terminal exacta de lo que el escudito de actualizaciones de Mint te ofrece con un clic.

**Ejercicio 5.5 · Un bin propio**

Solución razonada

`mkdir -p ~/bin`, `mv ~/Descargas/diagnostico.sh ~/bin/` (ajusta el origen), `export PATH="$HOME/bin:$PATH"`, y desde donde quieras: `diagnostico.sh`. `which diagnostico.sh` → `/home/marcos/bin/diagnostico.sh`. El `export` vale solo para esta terminal; en Mint, `~/bin` se añade solo al PATH en cuanto existe — a partir de tu próximo inicio de sesión será permanente sin tocar nada.
**Por qué así** — Acabas de hacer lo que hacen todos los profesionales: una carpeta propia de herramientas, delante del PATH. Tus scripts pasan de «ficheros que hay que buscar» a comandos de pleno derecho. Es el patrón de conda a escala casera — y ahora entiendes ambos.

**Ejercicio 5.6 · ¿Quién es tu Python?**

Solución razonada

Sin entorno: `/usr/bin/python3`, el del sistema. Con un entorno conda activado, `which python` salta a `~/miniconda3/envs/<nombre>/bin/python`, y el primer campo del PATH (ese `cut -d':' -f1`) es ahora el directorio del entorno: la figura del capítulo, en vivo.
**Por qué así** — Este par de comandos — `which python`, `echo $PATH` — es el estetoscopio de los entornos de Python. El día del misterioso «ModuleNotFoundError» los agradecerás. Cierra el cuaderno: `exit`, `sesion05.log` a la colección.

**Autocomprobación (test de la sección 5.8)** — Respuestas: 1 b · 2 a · 3 b · 4 b · 5 a · 6 b.

### La próxima sesión

El capítulo 6 es el último del bloque B y el que abre la puerta a todo lo demás: verás discos y particiones — `lsblk`, montar y desmontar, qué es esa partición EFI — y la película completa de lo que pasa entre el botón de encendido y tu escritorio: UEFI, GRUB, kernel. Es el prerrequisito directo de la instalación del bloque C: entender el quirófano antes de operar. Guarda `sesion05.log`.

cap. 5 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion05.log`

# 6 Discos, particiones y arranque

Última sesión del bloque B, y la que abre la puerta de todo lo que viene: entender dónde viven físicamente tus ficheros — discos, particiones, sistemas de ficheros — y qué pasa exactamente entre el botón de encendido y tu escritorio. Hoy no se toca nada: se aprende a leer el quirófano. Operar — particionar, formatear, instalar — será en la máquina virtual del bloque C, donde equivocarse es gratis.

**2–3 h** con ejercicios · requisitos: **capítulos 1–5** · máquina: **tu Linux de diario (solo lectura)** · opcional: **un pendrive**

**Al terminar esta sesión sabrás**

- Leer el mapa de tus discos y particiones con `lsblk`, e identificar cada pieza.
- Qué es un sistema de ficheros, y quién es quién: ext4, FAT32, NTFS, exFAT.
- Qué significa *montar*: cómo los discos se enganchan al árbol único — y por qué «expulsar con seguridad» no es un ritual supersticioso.
- Vigilar el espacio con `df` y encontrar qué lo devora con `du`.
- La película completa del arranque: UEFI → GRUB → kernel → systemd, y medirla en tu máquina.
- Qué es la partición EFI que te encontrarás al instalar Debian en un PC real.

## 6.1. El mapa de tus discos: `lsblk`

Los discos son **dispositivos de bloque**: almacenes que leen y escriben en bloques de bytes. Como todo en Linux, aparecen disfrazados de fichero en `/dev` — `nvme0n1` si tu disco es NVMe moderno, `sda` si es SATA — y sus **particiones**, las divisiones que un mismo disco puede tener, se numeran detrás: `nvme0n1p1`, `nvme0n1p2`… El plano completo lo dibuja `lsblk` (*list block devices*):

```
marcos@portatil:~$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 476,9G  0 disk
 nvme0n1p1 259:1    0   512M  0 part /boot/efi
 nvme0n1p2 259:2    0 476,4G  0 part /
```

Léelo como un físico lee un diagrama: un disco de 477 GB con dos particiones — una pequeñita de 512 MB montada en `/boot/efi` (enseguida sabrás quién es), y el resto montado en `/`: ahí vive todo tu sistema y tu casa. Tu máquina puede variar — más particiones, un `sda` — pero la gramática es esta.

## 6.2. Sistemas de ficheros: el orden dentro del bloque

Un disco recién salido de fábrica es solo una pizarra de bloques vacíos. Para que haya *ficheros* — con nombres, carpetas, permisos, fechas — hace falta imponerle una estructura: el **sistema de ficheros**. Formatear una partición es exactamente eso: estrenar su estructura (borrando lo anterior, de ahí el respeto).

| Sistema | Dónde lo verás | Lo esencial |
|---|---|---|
| **ext4** | tu `/`, todo Linux | robusto, con permisos y dueños (cap. 4) |
| **FAT32** | la partición EFI, pendrives viejos | antiguo y universal; sin permisos, ficheros de 4 GB máximo |
| **exFAT** | pendrives y tarjetas SD modernas | el FAT sin límite de tamaño; ideal para compartir |
| **NTFS** | discos con Windows | el de Windows; Linux lo lee y escribe sin drama |

La tabla explica media convivencia con Windows: un pendrive exFAT lo leen ambos mundos; tu ext4, Windows ni lo ve. Y fíjate en el detalle del capítulo 4: los permisos `rwx` son una *prestación del sistema de ficheros* — por eso un pendrive FAT no los tiene, y todo aparece como `rwxrwxrwx` cuando lo montas.

**Truco · El mapa con ratón: «Discos» y GParted**

Todo lo que `lsblk` te cuenta en texto lo dibujan dos aplicaciones gráficas que conviene conocer por su nombre, porque las verás en cualquier foro. **Discos** (menú → Accesorios; `gnome-disks`) viene con Mint: enseña cada disco con sus particiones como una barra de colores, monta y desmonta con un botón, formatea un pendrive y edita las opciones de montaje (anexo E). **GParted** (`sudo apt install gparted`, y viene de serie en el pendrive *live* de Mint) es *el* editor de particiones: crea, borra, redimensiona y mueve particiones — es la herramienta con la que se le hace sitio a Linux junto a Windows, y su hermano de texto es lo que usarás en el instalador del capítulo 8. Hoy, por la regla del curso, las abres solo para *mirar*: GParted sobre tu disco real es tan destructivo como potente. Para tocar, la máquina virtual del bloque C o un pendrive que puedas borrar.

## 6.3. Montar: enganchar discos al árbol

Y aquí se cierra por fin el círculo abierto el primer día, cuando te dijimos «no hay unidades C: y D:». En Linux, un disco no aparece como una isla con su letra: se **monta** — se engancha — en un directorio del árbol único, su **punto de montaje**, y desde ese momento su contenido *es* esa rama.

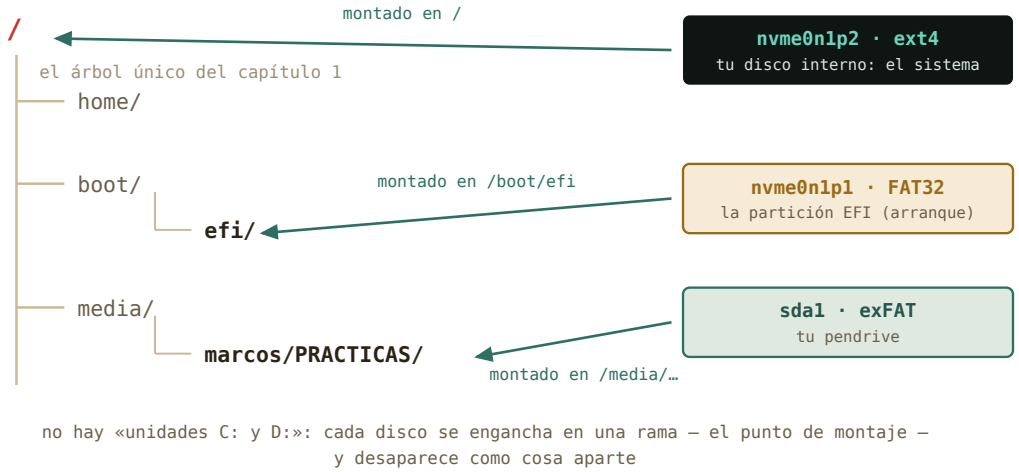



Figura 6.1: El árbol único, con sus enganches. La columna MOUNTPOINTS de `lsblk` es exactamente esto.

Cuando pinchas un pendrive, Mint lo monta solo en `/media/tu_usuario/NOMBRE` y te abre la ventana — cómodo y correcto. La versión manual es `mount` (montar) y `umount` (desmontar; sí, sin la primera *n*), y la usarás en el servidor sin escritorio. Hoy te basta con saber *leerlo* — montar a mano discos de otros formatos, hacerlo permanente y enganchar carpetas de otras máquinas es el anexo E, para cuando hayas visto SSH:

```
marcos@portatil:~$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    1  28,9G  0 disk
 sda1         8:1    1  28,9G  0 part /media/marcos/PRACTICAS
nvme0n1     259:0    0 476,9G  0 disk
...
```

**Por dentro · Por qué «expulsar» no es superstición**

Escribir en disco es lento, así que el sistema hace trampa legal: apunta tus cambios en RAM — la *caché* — y los vuelca al dispositivo cuando le conviene. Si arrancas el pendrive del puerto justo después de copiar, puede que parte siga en la caché: fichero corrupto. «Expulsar con seguridad» (o `umount`) significa «vuelca todo lo pendiente y cierra». Es un compromiso físico clásico: velocidad a cambio de un protocolo de salida. Respétalo como respetas la rampa de frenado de la centrífuga.

## 6.4. ¿Cuánto espacio queda? `df` y `du`

Dos preguntas distintas, dos herramientas. `df` (*disk free*) responde «¿cómo van de llenos mis sistemas de ficheros?»:

```
marcos@portatil:~$ df -h /
S.ficheros      Tamaño Usados Disp Uso% Montado en
/dev/nvme0n1p2   468G   187G 258G  43% /
```

Y `du` (*disk usage*) responde «¿cuánto ocupa *esto*?» — y combinado con las tuberías del capítulo 3, «¿*quién* me está llenando el disco?»:

```
marcos@portatil:~$ du -sh ~/* 2> /dev/null | sort -h | tail -3
2,1G    /home/marcos/Descargas
8,7G    /home/marcos/simulaciones
64G     /home/marcos/Vídeos
```

`-s` resume cada carpeta en una línea, `-h` en unidades humanas, y `sort -h` — fíjate — sabe ordenar esas unidades humanas (sabe que 8,7G > 900M). El `2> /dev/null` es el silenciador de quejas del capítulo 4. Culpable localizado en una línea.

**Truco · El mismo censo, con gráfica de quesitos**

Mint trae el Analizador de uso de disco (menú → Accesorios): el mismo `du` con anillos de colores clicables — francamente bueno para explorar. Y en la terminal existe su equivalente interactivo, `ncdu` (un `apt install` de distancia), que es lo que usarás en el servidor. Tres herramientas, una idea.

## 6.5. La partición EFI y el arranque

Queda explicar aquella partición enana de 512 MB. Para eso, la pregunta que este curso te prometió responder: **¿qué pasa cuando pulsas el botón de encendido?**

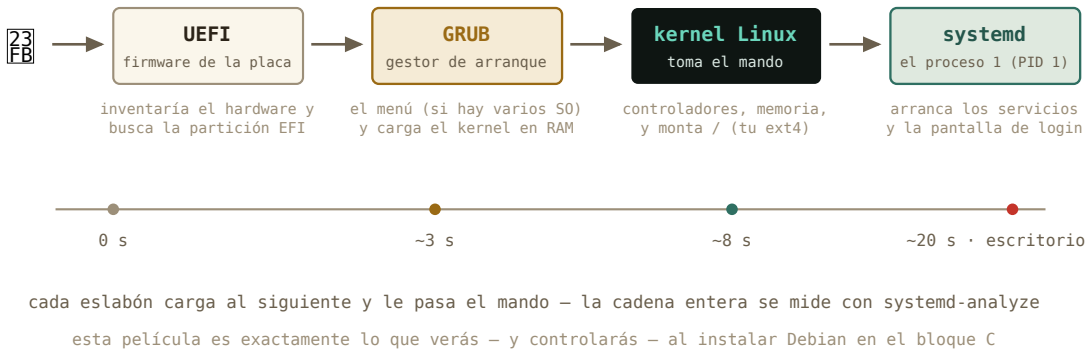



Figura 6.2: La cadena de mando del arranque. Cada eslabón carga al siguiente; conocerla convierte «no arranca» de tragedia en diagnóstico.

1. **UEFI** — el firmware grabado en la placa base — despierta, inventaría el hardware, y busca un disco que tenga *partición EFI*: una partición FAT32 con cargadores de arranque dentro. Ahí está el misterio resuelto: `nvme0n1p1` es el buzón estándar donde los sistemas operativos dejan su llave de arranque.
2. **GRUB**, el cargador que Linux dejó en ese buzón, toma el relevo: si tienes varios sistemas te enseña el menú, y carga el **kernel** en memoria.
3. El **kernel** toma el mando de verdad: controladores, memoria, procesos — y monta tu ext4 en `/`.
4. El kernel lanza el primer proceso, **systemd** — mira su PID: `ps -p 1` — que arranca en cascada los servicios y, al final, tu pantalla de entrada.

Y como todo en esta casa, la película se puede *medir*:

```
marcos@portatil:~$ systemd-analyze
Startup finished in 4.211s (firmware) + 2.108s (loader) + 3.998s (kernel) + 8.870s (userspace) = 19
```

Ahí están los cuatro actos con sus tiempos. Cuando instales Debian en el bloque C vas a *construir* esta cadena — particionar el disco, ver a GRUB instalarse — y cuando algo no arranque (a todos nos pasa), sabrás en qué eslabón se rompió.

**Historia · Levantarse tirando de los cordones**

«Arrancar» en inglés es *to boot*, de *bootstrapping*: levantarse en el aire tirando de los cordones de las propias botas. El chiste es exacto: para cargar un programa hace falta un programa ya cargado — ¿y el primero? En los ordenadores de los 70, como el Altair 8800, el problema lo resolvía un humano: metía a mano el cargador inicial, byte a byte, con los interruptores del panel frontal, para que ese programita minúsculo pudiera cargar el siguiente desde la cinta. Tu UEFI es ese operador, fosilizado en la placa base: la cadena entera — firmware que carga al cargador que carga al kernel — sigue siendo la misma idea, medio siglo después.

## 6.6. Práctica: inspección del quirófano

Cuaderno en marcha (`script sesion06.log`). Todo lo de hoy es lectura: ningún comando cambia nada en tu disco.

**Ejercicio 6.1 · Tu mapa**

Dibuja en tu cuaderno (en texto, como el ejemplo del capítulo) el mapa completo que da `lsblk` de tu máquina: discos, particiones, tamaños y puntos de montaje. Identifica: ¿cuál es tu partición EFI? ¿Dónde está montado tu sistema?

*(Solución al final del capítulo.)*

**Ejercicio 6.2 · Parte de capacidad**

¿A qué porcentaje de llenado está tu disco? ¿Cuánto espacio libre te queda en números absolutos? Una línea de `df`.

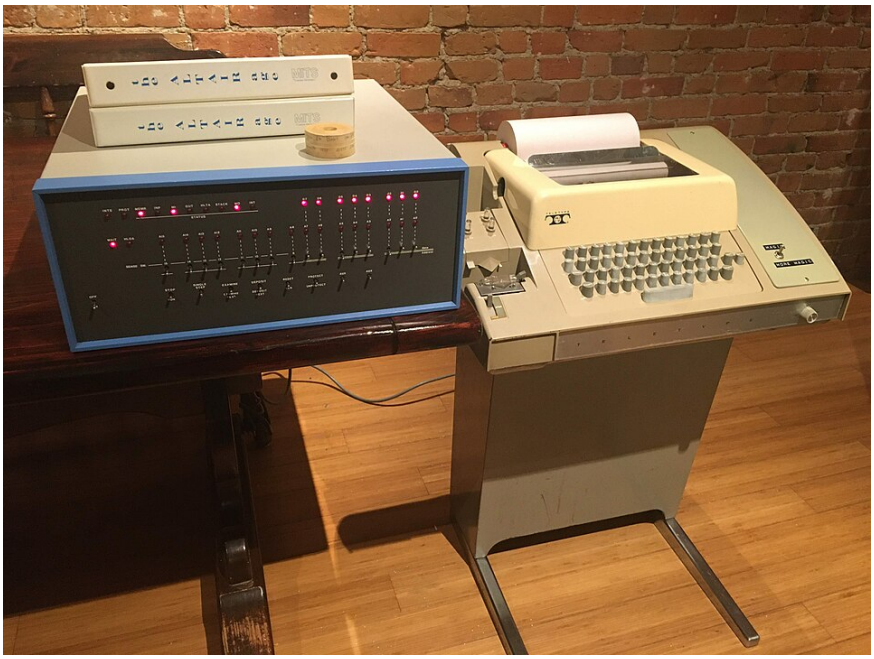


Figura 6.3: Un Altair 8800 (1975) con su panel de interruptores — y detrás, un teletipo ASR-33 como el del capítulo 1. El cargador de arranque se introducía a mano, bit a bit, con esos interruptores. *Foto: Tim Colegrove, CC BY-SA 4.0, Wikimedia Commons.*

*(Solución al final del capítulo.)*

**Ejercicio 6.3 · ¿Quién llena tu casa?**

Con `du`, las tuberías y `sort -h`: obtén el top 5 de carpetas más pesadas de tu home. ¿Te sorprende el resultado?

*(Solución al final del capítulo.)*

**Ejercicio 6.4 · El pendrive (si tienes uno a mano)**

Conéctalo. Con `lsblk`, localiza: qué nombre de dispositivo recibió, qué sistema de ficheros trae (`lsblk -f` lo muestra) y dónde se montó. Copia cualquier fichero dentro, y desmóntalo desde la terminal con `umount /media/tu_usuario/NOMBRE`. Verifica con `lsblk` que el punto de montaje desapareció antes de extraerlo.

*(Solución al final del capítulo.)*

**Ejercicio 6.5 · Cronometrar el arranque**

¿Cuánto tardó tu último arranque, y qué acto de la película fue el más lento? Después: ¿qué *servicio* concreto tardó más? (`systemd-analyze blame` da el ranking; quédate con el top 3.)

*(Solución al final del capítulo.)*

**Ejercicio 6.6 · El buzón de las llaves**

Asómate (solo lectura) a la partición EFI: `sudo ls /boot/efi/EFI`. ¿Qué carpetas hay? ¿Reconoces quién dejó cada llave de arranque?

*(Solución al final del capítulo.)*

## 6.7. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. En `lsblk`, `nvme0n1p2` es…
   a) el segundo disco NVMe del equipo
   b) la partición 2 del disco nvme0n1
   c) un pendrive
2. Formatear una partición significa…
   a) borrarla de la tabla de particiones
   b) estrenarle un sistema de ficheros (perdiendo lo que hubiera)
   c) comprobarla en busca de errores
3. Montar un disco es…
   a) engancharlo a un directorio del árbol único
   b) asignarle una letra de unidad
   c) copiarlo a la RAM
4. ¿Por qué «expulsar con seguridad» un pendrive?
   a) para descargar la electricidad estática
   b) para que la caché pendiente se vuelque al dispositivo antes de sacarlo
   c) es un vestigio: hoy ya no hace falta
5. La partición EFI es…
   a) una FAT32 pequeña donde los sistemas dejan sus cargadores de arranque
   b) la partición donde vive el kernel en ejecución
   c) la memoria de intercambio (swap)
6. El orden correcto del arranque es…
   a) GRUB → UEFI → systemd → kernel
   b) UEFI → GRUB → kernel → systemd
   c) kernel → UEFI → GRUB → systemd

## Soluciones del capítulo

**Ejercicio 6.1 · Tu mapa**

Solución razonada

`lsblk` — la EFI es la pequeña (unos cientos de MB) montada en `/boot/efi`; el sistema, la grande montada en `/`. Si ves una tercera de tipo `swap`, es el «desahogo» de la RAM (en los Mint recientes esa función la hace un fichero, `/swapfile`, y no verás partición).
**Por qué así** — Este mapa es el que te pedirá el instalador de Debian en el capítulo 8. Dibujarlo de tu propia máquina hoy — con calma y sin riesgo — es el ensayo de leerlo allí, donde las decisiones cuentan.

**Ejercicio 6.2 · Parte de capacidad**

Solución razonada

`df -h /` — las columnas Usados, Disp y Uso % responden todo. (Un `df -h` a secas lista además los pseudo-sistemas en RAM tipo `tmpfs`: ignóralos hoy sin remordimiento.)
**Por qué así** — El hábito del `df -h` antes de una descarga o simulación grande evita el clásico «disco lleno al 99 % de la ejecución». En los clústeres, además, el espacio suele tener cuota: esta lectura será rutina.

**Ejercicio 6.3 · ¿Quién llena tu casa?**

Solución razonada

`du -sh ~/* 2> /dev/null | sort -h | tail -5`. Para incluir también las ocultas del capítulo 4: `du -sh ~/.[a-z]* 2> /dev/null | sort -h | tail -5` — no es raro que `~/.cache` o un `~/miniconda3` den la sorpresa.
**Por qué así** — `du` mide, `sort -h` ordena magnitudes humanas, y la tubería los compone: el capítulo 3 aplicado a la administración. El silenciador `2>` evita el ruido de carpetas sin permiso de lectura.

**Ejercicio 6.4 · El pendrive (si tienes uno a mano)**

Solución razonada

Aparecerá como `sda1` (o `sdb1`), típicamente `vfat` o `exfat`, montado en `/media/marcos/NOMBRE`. Tras `umount`, su columna MOUNTPOINTS queda vacía: la caché se ha volcado y es seguro extraer. Si `umount` protesta con «objetivo ocupado», algo lo está usando — casi siempre una terminal con `cd` dentro del pendrive o la ventana del gestor de archivos abierta ahí.
**Por qué así** — Es el ciclo completo de la sección de montaje, con el matiz del «ocupado» incluido, que te encontrarás seguro. El botón de expulsar del escritorio hace exactamente este `umount` — dos caras, un mecanismo.

**Ejercicio 6.5 · Cronometrar el arranque**

Solución razonada

`systemd-analyze` desglosa firmware, cargador, kernel y userspace — lo normal es que gane userspace. Y `systemd-analyze blame | head -3` señala a los servicios lentos (a menudo NetworkManager esperando red, o algún escaneo de disco).
**Por qué así** — «No arranca» o «tarda muchísimo» dejan de ser misterios cuando el proceso se puede medir por actos y por actores. Es instrumentación pura: la misma mentalidad del laboratorio aplicada a la máquina. PID 1, por cierto: compruébalo con `ps -p 1`.

**Ejercicio 6.6 · El buzón de las llaves**

Solución razonada

Típicamente `ubuntu` (la llave de tu Mint — hereda el nombre de su familia, capítulo 5) y `BOOT` (el cargador de respaldo estándar). Si tu máquina convivió con Windows, verás también `Microsoft`: los dos sistemas, sus dos llaves, el mismo buzón FAT32.
**Por qué así** — Ver el buzón con tus ojos desmitifica el arranque dual: no hay magia, hay una carpeta con cargadores y un firmware que elige. Y el `sudo` — capítulo 4 — es porque el buzón del arranque no es zona de paseo. Cierra el cuaderno: `exit`. Bloque B completo: seis sesiones, seis logs.

**Autocomprobación (test de la sección 6.8)** — Respuestas: 1 b · 2 b · 3 a · 4 b · 5 a · 6 b.

**La próxima sesión**

Empieza el bloque C, y por fin está permitido romper cosas. En el capítulo 7 creas tu primera máquina virtual con el taller que Linux trae de serie — KVM y virt-manager: CPU, RAM, disco y red *de mentira* dentro de tu máquina de verdad — y conoces el superpoder que lo cambia todo: las instantáneas, el «deshacer» que la vida real no tiene. Es el laboratorio de riesgo cero donde harás la instalación del capítulo 8 y los experimentos de red de los bloques D y E. Guarda `sesion06.log`.

cap. 6 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion06.log`

# 7 Tu primera máquina virtual

Empieza el bloque C, y con él la parte del curso en la que por fin está permitido romper cosas. Hoy construyes un ordenador de mentira dentro de tu ordenador de verdad — con su procesador, su memoria, su disco y su cable de red, todos fingidos — y conoces el superpoder que la vida real no tiene: guardar el estado de una máquina y volver a él cuando algo salga mal. Es el laboratorio de riesgo cero donde harás la instalación del capítulo 8 y los experimentos de red de los bloques D y E.

**2–3 h** con ejercicios · requisitos: **capítulos 1–6** · máquina: **tu Linux de diario** (8 GB de RAM y 25 GB libres recomendados)

**Al terminar esta sesión sabrás**

- Qué es una máquina virtual, qué es un hipervisor, y qué se virtualiza exactamente: CPU, memoria, disco y red.
- Instalar el taller de virtualización de Linux — KVM, QEMU, libvirt y `virt-manager` — desde el Gestor de software o con un solo `apt`.
- Descargar la imagen de instalación de Debian y comprobar que no viene corrupta.
- Crear una máquina virtual desde cero, arrancarla y apagarla sin miedo.
- Usar instantáneas: guardar el estado de la máquina y restaurarlo en segundos.
- Distinguir las dos formas de conectar la VM a la red — NAT y puente — y por qué importarán en los bloques D y E.
- Entender qué te están dando cuando te asignen «una máquina virtual» en la universidad o en una nube.

## 7.1. Un ordenador dentro del ordenador

Una **máquina virtual** (VM) es un ordenador completo que existe solo como software: arranca, instala un sistema operativo, se conecta a la red, se cuelga y se reinicia — todo dentro de una ventana de tu escritorio. El programa que hace el truco es el **hipervisor**: toma el hardware real de tu máquina — el **anfitrión** (*host*) — y le presenta al sistema invitado — el **huésped** (*guest*) — un hardware de mentira que este cree suyo.

Cuatro cosas se fingen. **CPU**: la VM recibe un par de núcleos prestados — no simulados: el procesador real ejecuta sus instrucciones directamente, por eso va rápido. **Memoria**: un trozo de tu RAM reservado para ella. **Disco**: un fichero en tu casa que la VM cree que es un disco físico — con sus particiones, su sistema de ficheros, su GRUB, todo lo del capítulo 6 dentro. **Red**: una tarjeta de red virtual, conectada a un cable virtual que llega hasta tu

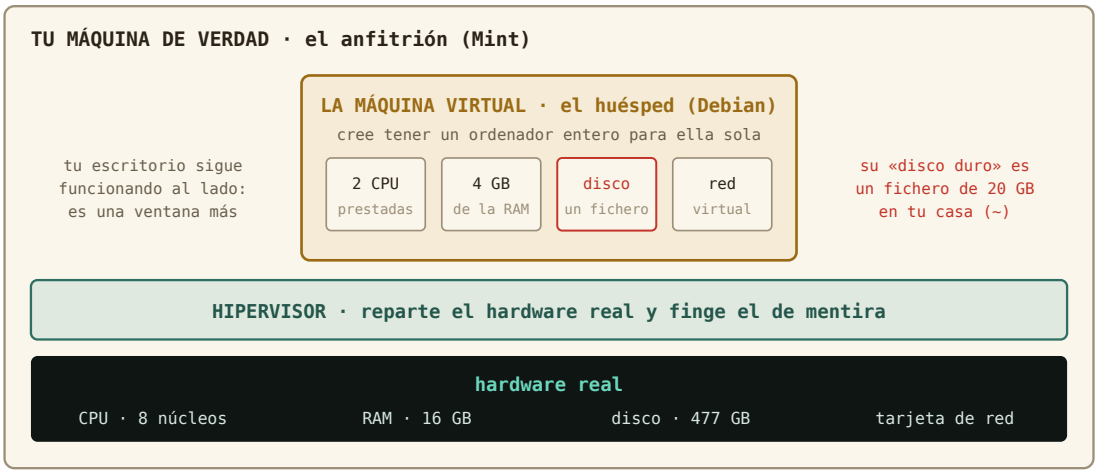



Figura 7.1: Qué se virtualiza. El detalle que más sorprende: el «disco duro» de la VM es un fichero normal en tu disco.

conexión real. Mientras tanto tu escritorio sigue funcionando al lado: la VM es una ventana más.

En Linux, el taller de virtualización viene de serie y se llama por sus piezas:

| Pieza | Qué hace | Analogía |
|---|---|---|
| **KVM** | el hipervisor, dentro del propio kernel: presta la CPU real | el motor |
| **QEMU** | finge los dispositivos: disco, red, pantalla | la carrocería |
| **libvirt** | el gestor: define, arranca, guarda y vigila las máquinas | el cuadro de mandos |
| **virt-manager** | la ventana gráfica sobre libvirt (y `virsh`, su CLI) | el volante |

Nada que descargar de una web ajena, nada que se rompa con la próxima actualización del kernel: KVM *es* parte de Linux. Y es exactamente lo que hay debajo de la mayoría de las nubes del mundo — dato que cobrará sentido al final del capítulo.

**Historia · Ordenadores de mentira desde 1967**

La virtualización es más vieja que Unix. En 1967, en el centro de investigación de IBM en Cambridge, un equipo escribió CP-67 para el mainframe System/360-67: un sistema que en vez de repartir *tiempo* de un ordenador entre muchos usuarios — el tiempo compartido del capítulo 4 — le daba a cada uno un ordenador *entero* de mentira, con su propio sistema operativo dentro. La idea funcionó tan bien que IBM la vendió durante décadas como VM/370. Luego el PC la olvidó casi treinta años: resucitó en 1999 con VMware y en 2007

entró en el kernel de Linux con el nombre de KVM. Hoy, cuando alquilas «un servidor en la nube», estás alquilando una máquina de mentira exactamente en el sentido de 1967.

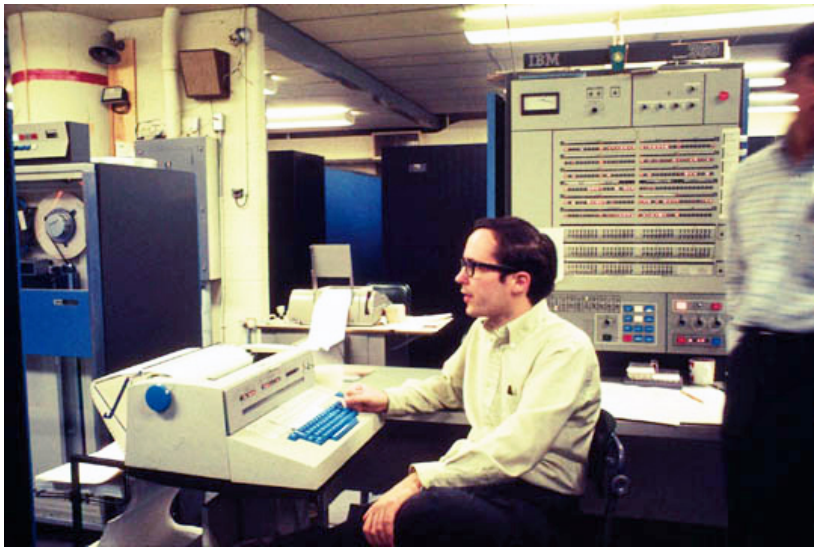



Figura 7.2: Un IBM System/360 modelo 67 en la Universidad de Michigan, hacia 1968: la máquina para la que se escribió el primer hipervisor. Todo lo que hagas hoy en una ventana ocupaba esta sala. *Foto: Dave Mills, dominio público, Wikimedia Commons.*

## 7.2. Montar el taller

Primero, comprueba que tu procesador sabe virtualizar — casi todos los de la última década saben, pero preguntar cuesta un segundo, y lo haces con las herramientas de los capítulos 3 y 4:

```
marcos@portatil:~$ grep -c -E 'vmx|svm' /proc/cpuinfo
8
marcos@portatil:~$ lscpu | grep -i virtualiz
Virtualización:                      VT-x
```

Un número mayor que cero (`vmx` es Intel, `svm` es AMD) significa que sí. Si saliera 0, la virtualización suele estar apagada en el UEFI del capítulo 6: se activa allí, en la opción llamada *VT-x*, *AMD-V* o *SVM Mode*.

El taller entero se instala como cualquier programa: abre el **Gestor de software** de Mint, busca «Gestor de máquinas virtuales» (o `virt-manager`) e **Instalar**. Fíjate en la lista que te enseña antes de aceptar: además de la ventana, trae `qemu`, `libvirt` y compañía — son las dependencias del capítulo 5, y el gestor las resuelve solo. La misma operación, en una línea de terminal, es el gesto de siempre:

```
marcos@portatil:~$ sudo apt install virt-manager
[sudo] contraseña de marcos:
Se instalarán los siguientes paquetes NUEVOS:
  libvirt-daemon-system qemu-system-x86 virt-manager virt-viewer ...
```

Al abrir por primera vez el Gestor de máquinas virtuales (menú → Administración) te pedirá tu contraseña: es el sistema comprobando que eres administrador antes de dejarte tocar el hipervisor. Funciona tal cual — pero hay un paso que la instalación gráfica *no* hace por ti y que conviene dar: añadirte al grupo `libvirt`.

```
marcos@portatil:~$ sudo usermod -aG libvirt marcos
```

Capítulo 4: los grupos son bolsas de permisos, y este es el que autoriza a manejar máquinas virtuales sin `sudo` — virt-manager deja de pedirte la contraseña, y `virsh`, su cara de terminal, funciona directamente. Como con todo cambio de grupos, hace efecto al **cerrar sesión y volver a entrar**. Después, la comprobación de que todo respira:

```
marcos@portatil:~$ groups
marcos adm cdrom sudo dip plugdev libvirt
marcos@portatil:~$ virsh list --all
 Id   Nombre   Estado
------------------------
marcos@portatil:~$ virsh net-list --all
 Nombre    Estado   Inicio automático   Persistente
-----------------------------------------------------
 default   activo   sí                  sí
```

`virsh` es la cara CLI de libvirt — el mismo cuadro de mandos que `virt-manager` enseña con ventanas. Una lista vacía de máquinas y una red `default` activa: el taller está montado.

**Truco · Si tu anfitrión es Windows o macOS**

KVM vive en el kernel de Linux, así que en otros sistemas no existe. Su equivalente allí es **VirtualBox**, gratuito y muy parecido en la práctica: las mismas ideas de este capítulo — hipervisor, disco como fichero, instantáneas, NAT frente a puente — con otros nombres en los menús. Si sigues el curso desde el anexo «Vengo de Windows», lee este capítulo con VirtualBox abierto al lado: te reconocerás en cada paso.

## 7.3. La imagen de instalación

Para instalar un sistema en una máquina — real o virtual — hace falta el medio de instalación. Antes era un CD; hoy es un fichero **ISO**: la imagen exacta, byte a byte, de aquel disco. En un PC real se graba en un pendrive (capítulo 8); en una VM, el fichero *es* el disco, y se «inserta» sin más.

Descarga la imagen *netinst* de Debian desde debian.org — unos 800 MB: lo mínimo para arrancar el instalador, que después descargará el resto de la red. El navegador la dejará en `~/Descargas`. Y antes de usarla, un hábito de físico: comprueba que llegó intacta. Junto a la ISO, Debian publica un fichero `SHA256SUMS` con la **huella digital** de cada imagen — un número que cambia si cambia un solo bit del fichero:

```
marcos@portatil:~/Descargas$ sha256sum debian-13.7.0-amd64-netinst.iso
a7ef94ac…  debian-13.7.0-amd64-netinst.iso
marcos@portatil:~/Descargas$ grep netinst SHA256SUMS
a7ef94ac…  debian-13.7.0-amd64-netinst.iso
```

Si las dos huellas coinciden, el fichero es exactamente el que Debian publicó (el número de versión será el que toque cuando lo descargues). Si no, se descarga de nuevo — nunca se instala desde una imagen sospechosa. Es la misma lógica que verificar la calibración antes de medir.

Verificada, un último paso que **no es opcional**: muévela al almacén de libvirt, `/var/lib/libvirt/images/` — el mismo directorio donde vivirá el disco de la máquina:

```
marcos@portatil:~/Descargas$ sudo mv debian-13.7.0-amd64-netinst.iso /var/lib/libvirt/images/
marcos@portatil:~/Descargas$ sudo ls -l /var/lib/libvirt/images/
-rw-rw-r-- 1 marcos marcos 792723456 ago 25 12:31 debian-13.7.0-amd64-netinst.iso
```

**Por dentro · Por qué la ISO no puede quedarse en tu casa**

La máquina virtual no la ejecutas tú: la ejecuta un proceso que corre como `libvirt-qemu`, un usuario de sistema de los del capítulo 4. Y en tu casa no entra nadie que no seas tú — míralo: `ls -ld ~` dice `drwxr-x---`, sin permiso de paso para «otros». Si dejas la ISO en `~/Descargas`, el asistente la acepta sin rechistar, pero al arrancar la máquina QEMU tropieza con un «Permission denied» y la instalación no llega a empezar. En `/var/lib/libvirt/images/` mandan root y libvirt, y cualquiera puede leer. Es el capítulo 4 en carne propia: el proceso que abre un fichero no siempre eres tú.

## 7.4. Crear la máquina

Abre `virt-manager` (menú → Administración → Gestor de máquinas virtuales, o tecléalo en la terminal). La ventana muestra la conexión `QEMU/KVM` con la lista vacía. Pulsa **Archivo → Nueva máquina virtual** y sigue el asistente — es un formulario de cinco pasos, y cada uno es una decisión que ya entiendes:

1. **Medio de instalación local (imagen ISO)** → *Explorar*: se abre el almacén `default` — es `/var/lib/libvirt/images/` — con tu `debian-13.x-amd64-netinst.iso` dentro (si la lista sale vacía, pulsa el botón de refrescar). Elígela: comprobará solo que es Debian. No la busques en `~/Descargas` con *Buscar local*: la instalación fallaría por permisos, como cuenta la caja anterior.

2. **Memoria y CPU**: 4096 MB y 2 CPU es un buen taller. Regla: nunca más de la mitad de tu RAM real — la otra mitad la necesita tu Mint para seguir vivo al lado.
3. **Disco**: 20 GB. Es el fichero de la figura. Y un detalle elegante: el fichero *crece según se usa* — hoy ocupará unos cientos de MB aunque diga 20 GB.
4. **Nombre**: `debian-lab`. Sin espacios (capítulo 1: los espacios importan). **Selección de red**: *Red virtual 'default': NAT* — la opción por defecto, y la correcta hoy.
5. Esta última pantalla tiene una casilla, *Personalizar la configuración antes de instalar*: déjala **sin marcar** — la máquina nace con el firmware clásico, el BIOS, y hoy es justo lo que quieres (la caja de la sección siguiente cuenta por qué no UEFI). **Finalizar.** La máquina arranca sola: verás en su ventana el menú del instalador de Debian sobre fondo azul, con «BIOS mode» en el título.

**Y aquí paras.** La instalación es la sesión del capítulo 8 entera; hoy solo querías ver la máquina *nacer*. Apágala desde el menú de su ventana: **Máquina virtual → Apagar → Forzar apagado**. En un PC real, cortar la corriente a un instalador a medias sería una barbaridad; aquí no hay nada instalado que perder — primer aviso de que las reglas dentro de la VM son otras.

```
marcos@portatil:~$ virsh list --all
 Id   Nombre       Estado
----------------------------
 -    debian-lab   apagado
marcos@portatil:~$ sudo ls -lh /var/lib/libvirt/images/
-rw------- 1 root root 20G ago 25 12:40 debian-lab.qcow2
marcos@portatil:~$ sudo du -h /var/lib/libvirt/images/debian-lab.qcow2
197M    /var/lib/libvirt/images/debian-lab.qcow2
```

Ahí está el ordenador entero, en un fichero de `/var` — el directorio de «lo que varía», capítulo 4 — que dice 20 GB pero ocupa 197 MB de verdad. Ese `du` frente a `ls -l` es el capítulo 6 enseñándote tamaño nominal frente a espacio real: el fichero es **disperso** (*sparse*), y solo consume disco a medida que la VM escribe en él.

**Cuidado · Lo que vale y lo que no vale en la máquina de mentira**

Dentro de la VM, todo vale: particionar, formatear, borrar `/etc`, reinstalar — para eso está. Pero dos cosas siguen siendo reales. Su **consumo**: mientras corre, se lleva la RAM y los núcleos que le diste (por eso se apaga cuando no se usa). Y su **fichero**: borrarlo es borrar la máquina; si un día quieres deshacerte de ella, se hace desde virt-manager (*Eliminar*, marcando borrar el disco) — no con un `rm` a ciegas en `/var/lib/libvirt/images`. Y la ISO no es la máquina: es el CD de instalación; vive en ese mismo directorio y puede borrarse (con `sudo`) cuando ya no la necesites.

## 7.5. El superpoder: instantáneas

Una **instantánea** (*snapshot*) guarda el estado completo de la máquina virtual — su disco, y si está encendida, también su memoria — con un nombre. Después puedes hacer lo que

quieras y **volver a ese estado exacto en segundos**. Es el «deshacer» que un ordenador real no tiene, y cambia por completo cómo se aprende: el miedo a romper desaparece, porque romper cuesta cinco segundos de arreglar.

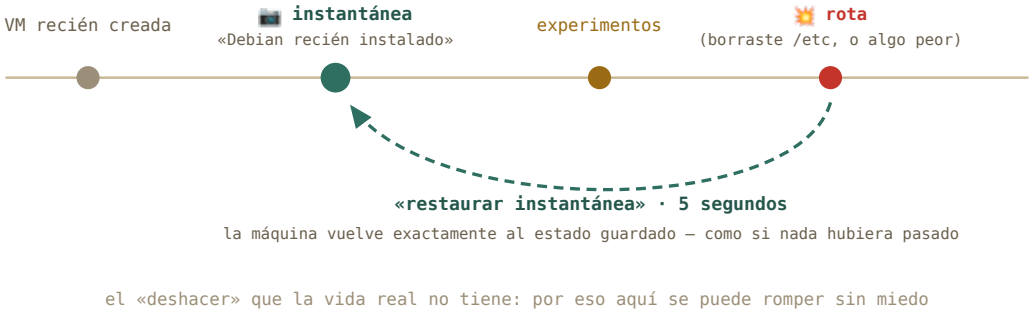



Figura 7.3: El flujo que usarás durante todo el bloque C: instantánea del estado bueno, experimentar, romper, restaurar.

En virt-manager: abre la ventana de la máquina, **Ver → Instantáneas**, y el botón + crea una nueva con el nombre que le pongas. Para volver: selecciónala y pulsa **Ejecutar instantánea seleccionada**. En la CLI es igual de sencillo:

```
marcos@portatil:~$ virsh snapshot-create-as debian-lab vm-recien-creada
Instantánea de dominio vm-recien-creada creada
marcos@portatil:~$ virsh snapshot-list debian-lab
 Nombre              Hora de creación             Estado
------------------------------------------------------------
 vm-recien-creada    2026-08-25 12:52:10 +0200    shutoff
```

El protocolo del curso, a partir del capítulo 8: en cuanto Debian esté instalado y funcionando, instantánea «recién instalado». Cada experimento arriesgado, instantánea antes. Todo lo que se rompa, se restaura. Nunca más reinstalar por un error — salvo cuando reinstalar sea el ejercicio.

### Por dentro · Por qué la VM arranca «a la antigua»

Tu Mint arranca por UEFI (capítulo 6); `debian-lab` arranca con el firmware clásico, el BIOS. No es nostalgia: es una limitación de versiones. El UEFI de una máquina virtual guarda su memoria — la lista de arranque que en tu PC vive en un chip de la placa base — en un segundo fichero, aparte del disco, y el libvirt de Mint 22 no sabe fotografiar los dos a la vez: si eliges UEFI, la instantánea se niega con «internal snapshots of a VM with pflash based firmware are not supported». Entre un arranque moderno y el «deshacer» de cinco segundos, el curso elige el deshacer: lo que hay *después* del arranque es idéntico. La limitación está levantada en libvirt 10.10 y posteriores — el de Mint 23, previsto para diciembre de 2026; LMDE 7 ya lo lleva —, aunque aun ahí hará falta pedir expresamente que la memoria del UEFI se guarde en un formato con «versiones» (qcow2, el del disco). En el capítulo 8 hay un reto para ver el UEFI en una máquina de usar y tirar; y en tu PC real, el anexo C.

## 7.6. Dos formas de conectarla: NAT y puente

Al crear la máquina elegiste *Red virtual 'default': NAT*. Conviene entender qué elegiste, porque los bloques D y E se apoyan en ello.

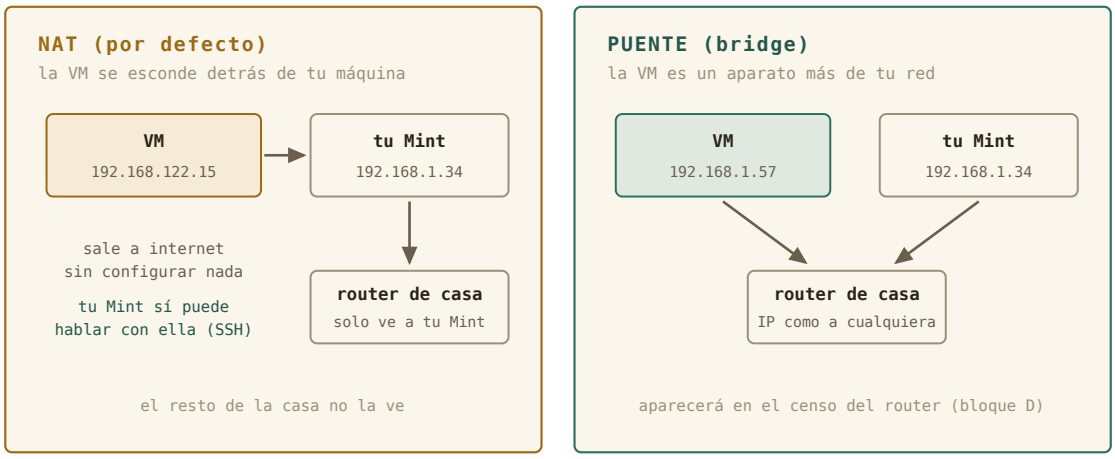



Figura 7.4: Las dos topologías. Hoy usas NAT; el puente queda para cuando quieras que la VM sea un aparato más de tu casa.

Con **NAT**, libvirt monta una red privada minúscula dentro de tu máquina — la `default`, con direcciones `192.168.122.x` — y hace de router entre ella y el mundo: la VM sale a internet sin configurar nada, tu Mint puede hablar con ella (lo harás por SSH en el capítulo 11), y el resto de la casa ni sabe que existe. Con un **puente** (*bridge*), la VM se conecta directamente a tu red doméstica y el router de casa le da una dirección como a cualquier portátil o móvil: aparecería en el censo de dispositivos del bloque D. Es más «real» y también más delicado — en portátiles con Wi-Fi da guerra —, así que el curso trabaja en NAT y deja el puente como experimento opcional. Puedes ver la red `default` desde el anfitrión, con un comando que será el protagonista del capítulo 9:

```
marcos@portatil:~$ ip a | grep virbr0
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 ...
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
```

Tu Mint tiene ahora una segunda tarjeta de red — virtual — con la dirección `192.168.122.1`: es el router de la red de mentira. Guarda este dato; cuando en el capítulo 9 hablemos de direcciones y máscaras, ya tendrás una red propia sobre la que experimentar.

## 7.7. Qué te dan cuando te dan «una VM»

La escena que el diseño de este curso tenía en mente: el grupo de investigación o el servicio de informática te dice «te hemos creado una máquina virtual». Ahora sabes exactamente qué

es: lo mismo que acabas de hacer, sobre un anfitrión grande de la universidad o de una nube, con KVM o un primo suyo debajo. Te darán una dirección, un usuario y probablemente una clave SSH (capítulo 12), y no verás ninguna ventana con su pantalla: la máquina existe, pero solo hablarás con ella por la red. Los núcleos y la memoria que tiene son «prestados» como los de tu `debian-lab`, y compartes el anfitrión con otros — el tiempo compartido de 1967, otra vez. Y si te hablan de **contenedores** (Docker y compañía): son el primo ligero — no fingen un ordenador entero, comparten el kernel del anfitrión y aíslan solo un programa. Más rápidos, menos independientes. Vocabulario para reconocerlos; el curso se queda con las máquinas virtuales.

**Truco · Ventanas o comandos, la misma máquina**

`virt-manager` y `virsh` son dos caras de libvirt: todo lo que haces con clics tiene su orden. Para crear y explorar, la ventana es más amable; para el servidor sin escritorio del bloque E — donde gestionarás máquinas solo por SSH — `virsh` es la única cara que hay. El principio de siempre: aprende el concepto una vez, y elige la puerta según el momento.

## 7.8. Práctica: el taller en marcha

Cuaderno en marcha (`script sesion07.log`). Hoy instalas software y creas tu laboratorio — nada de lo que hagas toca tu sistema más allá de instalar paquetes y crear un fichero grande en `/var`.

**Nivel 1 · Lo esencial**

**Ejercicio 7.1 · ¿Sabe virtualizar tu procesador?**

Comprueba de dos formas que tu CPU tiene soporte de virtualización, y anota en el cuaderno cuál de las dos tecnologías tiene (Intel o AMD).

*Pista.* `/proc/cpuinfo` con `grep -c`, y `lscpu` con `grep -i`.

*(Solución al final del capítulo.)*

**Ejercicio 7.2 · Montar el taller**

Instala el Gestor de máquinas virtuales por la vía que prefieras — Gestor de software o `apt` —, ábrelo una vez (te pedirá la contraseña), añádete al grupo `libvirt`, cierra y vuelve a abrir sesión, y comprueba con `groups` y `virsh list --all` que estás dentro y que el taller responde.

*Pista.* Las dos vías instalan exactamente los mismos paquetes; `usermod -aG` añade sin quitar los grupos que ya tienes; los grupos nuevos no aparecen hasta reiniciar la sesión.

*(Solución al final del capítulo.)*

**Ejercicio 7.3 · La imagen, verificada**

Descarga la ISO *netinst* de Debian para amd64 y su fichero `SHA256SUMS`, comprueba que la huella coincide, y deja la ISO donde libvirt pueda leerla.

*Pista.* `sha256sum fichero.iso` calcula; `grep netinst SHA256SUMS` te aísla la línea correcta para compararla a ojo — o deja que `sha256sum -c SHA256SUMS 2> /dev/null | grep netinst` lo compare por ti. «Donde libvirt pueda leerla» es su almacén, `/var/lib/libvirt/images/`.

*(Solución al final del capítulo.)*

**Ejercicio 7.4 · Nace `debian-lab`**

Crea la máquina con el asistente (4 GB, 2 CPU, 20 GB, red NAT), deja que arranque hasta el menú del instalador, mírala un minuto — y apágala con *Forzar apagado*. Comprueba con `virsh` que existe y está apagada.

*Pista.* El asistente son cinco pantallas; si algo no encaja con el capítulo, avanza igual — cualquier decisión se cambia después en la configuración de la máquina.

*(Solución al final del capítulo.)*

**Ejercicio 7.5 · Un disco de 20 GB que pesa 200 MB**

Localiza el fichero de disco de tu máquina y mide su tamaño de las dos maneras del capítulo 6: el nominal (`ls -lh`) y el real (`du -h`). Explica en tu cuaderno la diferencia.

*Pista.* Los discos de libvirt viven en `/var/lib/libvirt/images/`, y ese directorio es de root.

*(Solución al final del capítulo.)*

**Ejercicio 7.6 · La primera instantánea**

Con la máquina apagada, crea una instantánea llamada `vm-recien-creada` desde virt-manager, y compruébala desde la terminal. Después, borra esa instantánea (será inútil en cuanto instales Debian) y verifica que desapareció.

*Pista.* `virsh snapshot-list debian-lab` lista; `virsh snapshot-delete debian-lab NOMBRE` borra.

*(Solución al final del capítulo.)*

**Nivel 2 · Para curiosos (opcional)**

**Ejercicio 7.7 · La máquina, por dentro de libvirt**

libvirt guarda la definición de cada máquina como texto. Con `virsh dumpxml debian-lab` y las tuberías esenciales del capítulo 3, averigua cuánta memoria tiene asignada, cuántas CPU, y dónde está su disco.

*Pista.* `| grep -i memory`, `| grep vcpu`, `| grep -i qcow2`.

*(Solución al final del capítulo.)*

## 7.9. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. El hipervisor es…

   a) el sistema operativo que instalas dentro de la VM
   b) el programa que reparte el hardware real y presenta hardware de mentira al huésped
   c) la ventana donde se ve la máquina virtual

2. El disco duro de tu máquina virtual es…

   a) una partición que se crea en tu disco real
   b) un fichero (`.qcow2`) en `/var/lib/libvirt/images`
   c) un trozo de la RAM

3. Una instantánea sirve para…

   a) guardar el estado de la VM y volver a él cuando quieras
   b) hacer una captura de pantalla de la VM
   c) copiar la VM a otro ordenador

4. Con la red NAT por defecto de libvirt…

   a) la VM recibe una IP del router de casa como cualquier otro aparato
   b) la VM sale a internet y el anfitrión puede hablar con ella, pero el resto de la casa no la ve
   c) la VM no tiene red

5. Te añaden al grupo `libvirt` y `virsh` sigue dando «permiso denegado». Lo más probable:

   a) hay que reinstalar virt-manager
   b) no has cerrado y vuelto a abrir sesión
   c) necesitas usar `sudo` siempre con `virsh`

6. Un contenedor (Docker) se diferencia de una VM en que…

   a) es lo mismo con otro nombre
   b) comparte el kernel del anfitrión y aísla solo un programa
   c) necesita hardware especial

## Soluciones del capítulo

**Ejercicio 7.1 · ¿Sabe virtualizar tu procesador?**

Solución razonada

`grep -c -E 'vmx|svm' /proc/cpuinfo` (un número > 0) y `lscpu | grep -i virtualiz` (`VT-x` para Intel, `AMD-V` para AMD). Si sale 0 o vacío, la opción está apagada en el UEFI: reinicia, entra en la configuración (capítulo 6) y actívala.
**Por qué así** — Dos instrumentos distintos para la misma medida, como siempre. Y fíjate en `/proc`: el capítulo 4 te enseñó que ahí el kernel te cuenta lo que sabe del hardware — hoy le preguntas por una capacidad concreta de la CPU.

**Ejercicio 7.2 · Montar el taller**

Solución razonada

Gestor de software → «Gestor de máquinas virtuales» → Instalar (o `sudo apt install virt-manager`), `sudo usermod -aG libvirt $USER`, cerrar sesión y entrar. `groups` debe listar `libvirt`; `virsh list --all` devuelve una tabla vacía sin quejarse. Si protesta con «permiso denegado» hablando de un *socket*, es que no reiniciaste la sesión.
**Por qué así** — Un paquete, un grupo, una comprobación: el capítulo 5 y el 4 trabajando juntos. `$USER` es la variable de entorno del capítulo 5 con tu nombre — escribirlo así vale para cualquier usuario.

**Ejercicio 7.3 · La imagen, verificada**

Solución razonada

Las dos huellas deben ser idénticas carácter a carácter. Con `sha256sum -c`, la línea buena dice `debian-13.x-amd64-netinst.iso: La suma coincide`; el `2> /dev/null` silencia las quejas por las otras imágenes del fichero que no descargaste. Y después, `sudo mv` de la ISO a `/var/lib/libvirt/images/`.
**Por qué así** — Nunca se instala desde una imagen sin verificar: una descarga cortada o manipulada produce instalaciones que fallan de formas misteriosas. El `2>` del capítulo 3 y el `grep` de siempre te dan la comprobación en una línea.

**Ejercicio 7.4 · Nace debian-lab**

Solución razonada

`virsh list --all` → `debian-lab  apagado`. Si la máquina no arranca con un error sobre KVM, vuelve al 7.1: la virtualización está apagada en el UEFI. Si al arrancar la máquina sale «Permission denied» sobre la ISO, es que la elegiste en `~/Descargas`: muévela al almacén de libvirt (sección de la imagen) y en la configuración de la máquina apunta la unidad de CD al nuevo sitio.
**Por qué así** — Acabas de hacer, con clics, exactamente lo que el servicio de informática hace cuando te «da una VM». La única diferencia es la escala del anfitrión.

**Ejercicio 7.5 · Un disco de 20 GB que pesa 200 MB**

Solución razonada

`sudo ls -lh /var/lib/libvirt/images/` dice 20G; `sudo du -h /var/lib/libvirt/images/debian-lab.qcow2` dice unos cientos de MB. El fichero es *disperso*: reserva 20 GB de tamaño lógico pero solo ocupa lo que la VM ha escrito de verdad. Crecerá durante la instalación del capítulo 8.
**Por qué así** — Tamaño nominal frente a ocupación real: un físico distingue una capacidad de una medida sin pestañear. Y `sudo` porque el directorio es del sistema (`/var`, capítulo 4): las máquinas de todos los usuarios viven ahí.

**Ejercicio 7.6 · La primera instantánea**

Solución razonada

En virt-manager: ventana de la máquina → Ver → Instantáneas → +. En terminal: `virsh snapshot-list debian-lab` la muestra; `virsh snapshot-delete debian-lab vm-recien-creada` la elimina, y la lista vuelve a estar vacía.
**Por qué así** — Has recorrido el ciclo completo — crear, comprobar, borrar — con las dos caras de libvirt. El que importa, «recién instalado», lo harás al final del capítulo 8, y será el punto de retorno de todo el resto del curso.

**Ejercicio 7.7 · La máquina, por dentro de libvirt**

Solución razonada

`virsh dumpxml debian-lab | grep -i memory` → `<memory unit='KiB'>4194304</memory>` (4 GB en KiB); `| grep vcpu` → 2; `| grep qcow2` → la ruta en `/var/lib/libvirt/images/`.
**Por qué así** — «Todo es un fichero de texto» llega hasta aquí: tu ordenador de mentira es, en última instancia, un documento XML que describe su hardware. Cambiar la RAM de la máquina es editar un número — y `grep` es tu lupa para encontrarlo. Cierra el cuaderno: `exit`, `sesion07.log` a la colección.

**Autocomprobación (test de la sección 7.9)** — Respuestas: 1 b · 2 b · 3 a · 4 b · 5 b · 6 b.

**La próxima sesión**

Tienes una máquina vacía con el instalador de Debian esperando en su bandeja. En el capítulo 8 la instalas de verdad — y no con el botón de «siguiente, siguiente»: particionando a mano con lo que aprendiste en el capítulo 6, viendo a GRUB instalarse en el disco. Después la romperás y la reinstalarás hasta que aburra, y saldrás con la lista de comprobación para el día en que lo hagas en un PC real. Guarda `sesion07.log`.

cap. 7 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion07.log`

# 8 Instalar Linux de verdad

Hasta hoy, tu Linux te lo instalaron. Esta sesión lo instalas tú — en la máquina virtual del capítulo 7, sin red de seguridad más que las instantáneas, y sin el botón de «siguiente, siguiente»: eligiendo particiones a mano con el mapa del capítulo 6, mirando a GRUB instalarse en el primer sector del disco. Después la romperás a propósito y la reinstalarás, porque la segunda vez tarda un cuarto de hora y la tercera ya aburre. Y saldrás con la lista de comprobación para el día en que lo hagas en un ordenador de verdad.

**3 h** con ejercicios (la instalación en sí, 20–30 min) · requisitos: **capítulos 6 y 7** · máquina: **la VM `debian-lab`** — nada de hoy toca tu Mint

**Al terminar esta sesión sabrás**

- Decidir un plan de particiones *antes* de tocar el instalador — raíz + swap en la VM, y por qué tu PC real añade una tercera pieza, la EFI.
- Recorrer el instalador de Debian entero: idioma, red, usuarios, particionado manual, espejo, selección de programas, GRUB.
- Por qué se deja la contraseña de `root` vacía, y qué consigue eso.
- Instalar un Debian *sin escritorio* — un servidor — con SSH desde el primer día, y comprobar que respira.
- Qué se rompe cuando se rompe el arranque, y cómo la instantánea lo convierte en un susto de cinco segundos.
- Qué cambia — y qué no — cuando lo hagas en un PC real desde un pendrive.

## 8.1. El plan, antes que el instalador

Los instaladores modernos permiten no pensar: «usar el disco entero», siguiente, siguiente. Hoy no vas a pulsar ese botón, y la razón es la misma por la que un físico no arranca un montaje sin un esquema: **cuando lo hagas en tu PC de verdad, con un Windows al lado o un disco con datos, «usar el disco entero» será exactamente lo que no quieres.** Se decide el plan en papel, se ejecuta en la VM, se repite hasta que sea rutina.

Lo que instalas: **Debian 13** — la madre de tu Mint (capítulo 5) —, imagen *netinst*, **sin escritorio**. Sí, sin escritorio: un servidor al que hablarás por terminal y, a partir del capítulo 12, por SSH. Es lo que te encontrarás en el clúster y en la nube, y es lo que construye el proyecto final. Nombre de la máquina: `debian-lab`. Usuario: el tuyo.

Dos particiones, y cada decisión la tomas con el capítulo 6 en la cabeza: la **raíz** (`/`, ext4) se lleva casi todo; y **swap** (2 GB) es el desahogo de la RAM cuando se llena — en una VM de 4 GB, un seguro barato. ¿Y la **partición EFI** del capítulo 6, el buzón de GRUB? Aquí no

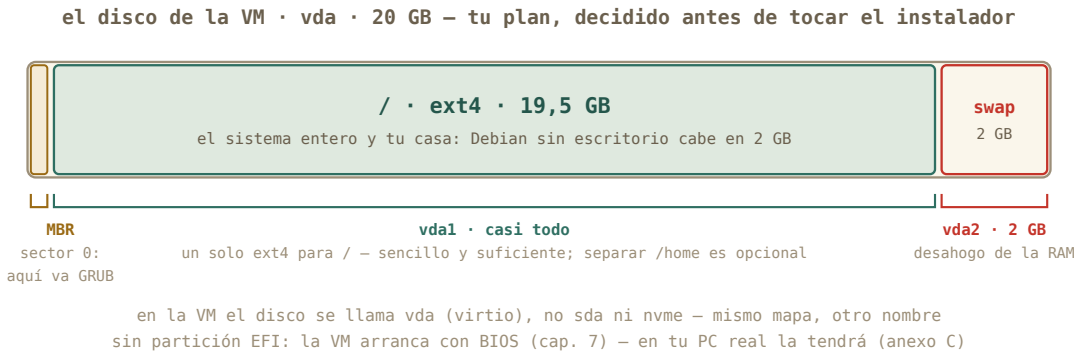



Figura 8.1: El plan de particiones para el disco virtual. Dos particiones y un sector: cada pieza con su oficio del capítulo 6.

hay: la VM arranca con el firmware clásico, el BIOS (capítulo 7), y en ese mundo GRUB no va en una partición sino en el **primer sector del disco** — el MBR, *master boot record* —, que el BIOS lee a ciegas al encender. Treinta años funcionó así, y en la VM sigue valiendo; tu PC real, que arranca por UEFI, sí tendrá la EFI, y el anexo C te lo recuerda. ¿Y `/home` separado? Es una opción respetable — permite reinstalar el sistema sin tocar tus datos — pero complica el reparto de espacio; para el taller, un solo `/` basta. Un detalle nuevo: dentro de la VM el disco no se llama `sda` ni `nvme0n1` sino `vda` — *virtio disk*, el disco de mentira del hipervisor. Mismo mapa, otro nombre.

## 8.2. Arrancar el instalador

Abre virt-manager, selecciona `debian-lab` y pulsa el triángulo de *Ejecutar* (o, desde la terminal, `virsh start debian-lab` y abre su ventana). La ISO sigue en la bandeja, así que la VM arranca en el menú del instalador: *Graphical install*, *Install*, y opciones avanzadas (si no tocas nada en treinta segundos, arranca solo con síntesis de voz: una flecha lo para). Los dos primeros son el mismo instalador con dos caras — la gráfica es más cómoda con ratón; la de texto funciona en cualquier sitio, incluida una consola remota sin ventanas. Elige la que quieras; las pantallas y las decisiones son idénticas.

Las primeras pantallas son las fáciles, y una de ellas esconde la decisión más importante del día:

1. **Idioma, país, teclado**: español, España, español. (Un profesional a veces deja el *sistema* en inglés para buscar errores en internet con más éxito; para el curso, español, que es como verás los mensajes en los ejemplos.)
2. **Red**: la VM recibe dirección sola de la red NAT — es el DHCP del capítulo 10, adelantado. Nombre de la máquina: `debian-lab`. Dominio: vacío.
3. **Contraseña de root: déjala vacía.** No es un descuido, es un truco documentado de Debian: si `root` queda sin contraseña, el instalador **desactiva la cuenta de root**

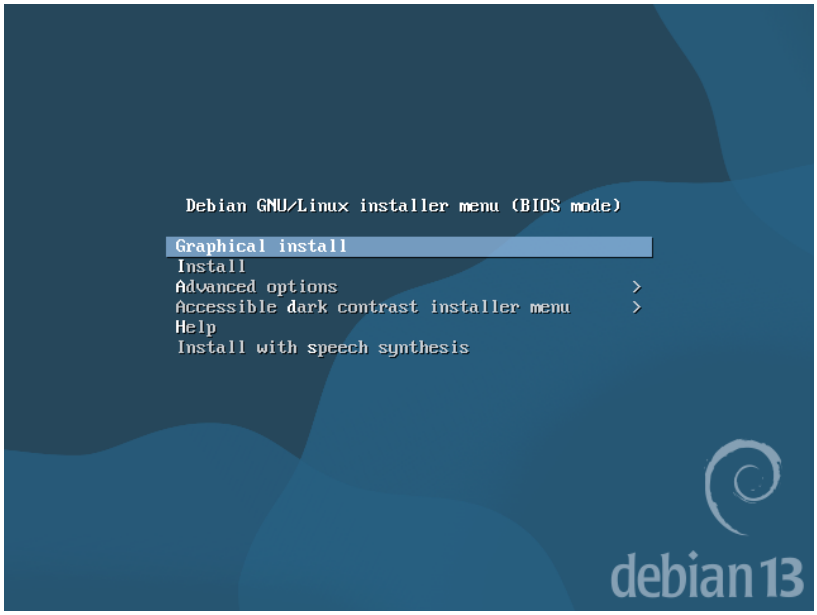



Figura 8.2: El menú de arranque de la ISO en la VM: «BIOS mode» en el título. Quien sirve este menú es *isolinux*, un cargador pequeño que viaja en la ISO; GRUB — el del capítulo 6 — llegará cuando lo instales en el disco. *Captura del instalador de Debian 13 (GPL).*

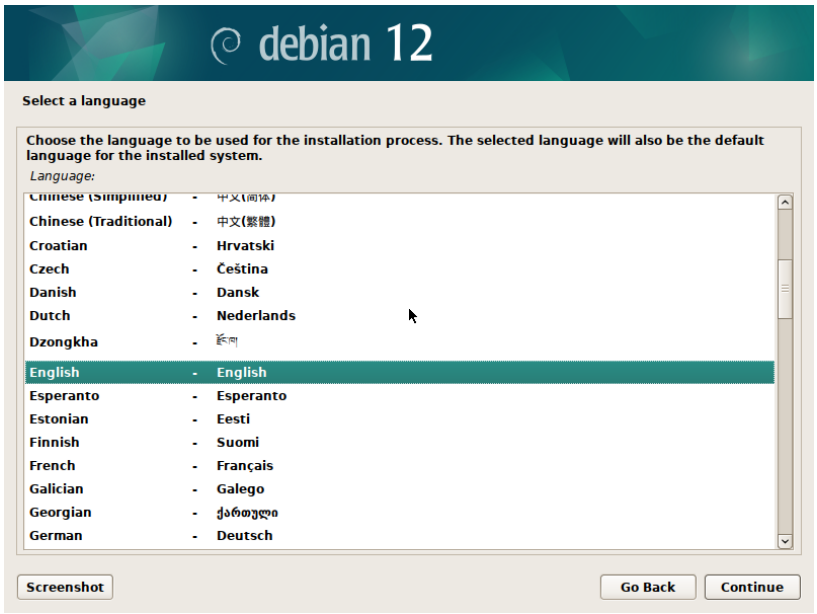



Figura 8.3: El instalador de Debian en modo gráfico, en su primera pregunta: el idioma. Cada pantalla es una pregunta, y ya sabes responderlas todas. *Captura de la versión 12: Paowee, GPL, Wikimedia Commons.*

**y añade a tu usuario al grupo `sudo`** — exactamente el modelo de tu Mint, el del capítulo

   4. Si pones contraseña, tendrás un root activo y un usuario *sin* sudo, y el primer `sudo apt` te dará un disgusto.

4. **Tu usuario**: nombre completo, nombre de usuario (`marcos`), contraseña. Zona horaria: la tuya.

## 8.3. Particionar a mano

Llega la pantalla que asusta a todo el mundo, y que tú vas a leer con calma. Elige **Manual**. Verás el disco — «Disco virtual 1 (vda) - 21.5 GB Virtio Block Device» — sin particiones. ¿21,5? El instalador cuenta en GB decimales, y tus 20 GiB del capítulo 7 son exactamente eso. El procedimiento, pantalla a pantalla:

1. Selecciona el disco `vda` → **Crear una nueva tabla de particiones vacía** → sí. En modo BIOS no pregunta el tipo: crea una `msdos`, la clásica (`gpt` es la que espera UEFI, capítulo 6). Aparece una línea «ESPACIO LIBRE» con los 21,5 GB.
2. Selecciona el espacio libre → **Crear una partición nueva** → tamaño `19.5 GB` (quedan 2 para la swap) → **Primaria** → **Principio** → la pantalla de ajustes ya propone *ext4* con punto de montaje `/`: compruébalo → **Se ha terminado de definir la partición**.
3. Espacio libre restante → partición nueva → tamaño: el que propone (todo lo que queda) → **Primaria** — ojo, esta vez el instalador señala *Lógica* por defecto — → en ajustes, *Utilizar como:* cambia *ext4* por **área de intercambio** → terminar.
4. Compara la tabla con la figura: `#1 primaria 19.5 GB ext4 /` y `#2 primaria 2.0 GB swap` — serán `vda1` y `vda2`. **Finalizar el particionado y escribir los cambios en el disco** → «¿Desea escribir los cambios?» → **Sí**.

Ese último *Sí* es el punto sin retorno de cualquier instalación: a partir de ahí el disco se formatea. En tu VM el disco es un fichero vacío y el susto es gratis; en tu PC real será el momento de haber leído tres veces qué disco has elegido — vuelve a esta frase cuando llegue el día.

**Cuidado · Lo que este capítulo no hace en tu Mint**

Ni una sola orden de hoy se ejecuta en tu máquina real: todo pasa dentro de la ventana de `debian-lab`. Si en algún momento ves el prompt `marcos@portatil`, estás fuera de la VM — para en seco. Dentro, el prompt dirá `marcos@debian-lab`. Ese nombre de máquina que pusiste en el instalador acaba de convertirse en tu cinturón de seguridad.

## 8.4. Espejo, programas y GRUB

Con el disco listo, el instalador copia el sistema base y hace las últimas preguntas:

- **Réplica de Debian** (*mirror*, el espejo): el repositorio del capítulo 5, en su versión Debian. `deb.debian.org` sirve; proxy, vacío. De aquí descargará todo lo que no cabía en la netinst.
- **Concurso de popularidad**: no.
- **Selección de programas** — la pantalla decisiva del *servidor*. **Desmarca** *Entorno de escritorio Debian* y *GNOME* (vienen marcados). **Marca** *Servidor SSH* — el bloque E te lo agradecerá — y *Utilidades estándar del sistema*. Nada más.
- **Cargador de arranque**: la VM arranca con BIOS, así que el instalador pregunta *en qué disco* instalar GRUB: `/dev/vda` — el disco entero, no una partición. Es el primer sector de la figura del plan, el sitio que el BIOS mira al encender.

Reinicia cuando lo pida. La ISO se expulsa sola, y en el primer arranque desde el disco recién estrenado verás, durante unos segundos, el menú de GRUB — el eslabón del capítulo 6 que hoy has puesto tú. Ya no es el isolinux de la ISO: es GRUB, leído del primer sector de `vda`, sirviendo *tu* sistema.

## 8.5. El primer arranque: comprobar que respira

Sin escritorio, el sistema te recibe con una consola de texto y un `login:`. Entra con tu usuario, y pásale la revisión del bloque B — son tus comandos, sobre tu instalación:

```
marcos@debian-lab:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda    254:0    0   20G  0 disk
 vda1 254:1    0 18,2G  0 part /
 vda2 254:2    0  1,8G  0 part [SWAP]
marcos@debian-lab:~$ df -h /
S.ficheros     Tamaño Usados  Disp Uso% Montado en
/dev/vda1         18G   1,3G   16G   8% /
marcos@debian-lab:~$ ip a | grep "inet "
    inet 127.0.0.1/8 scope host lo
    inet 192.168.122.15/24 brd 192.168.122.255 scope global dynamic enp1s0
marcos@debian-lab:~$ systemctl status ssh | head -3
  ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled)
     Active: active (running) since Tue 2026-08-25 19:02:11 CEST
```

Léelo como el parte de un experimento que ha salido bien: las dos particiones montadas donde las pusiste; un sistema completo que ocupa 1,3 GB; una dirección `192.168.122.x` — la red NAT del capítulo 7, repartida por tu Mint —; y un servidor SSH ya activo, esperando al capítulo 12. Pon la instalación al día con el gesto del capítulo 5 (`sudo apt update && sudo apt upgrade`) — y fíjate en que `sudo` funciona: el truco de la contraseña vacía de root ha hecho su efecto.

Y ahora, el protocolo del capítulo 7. Apaga la máquina con `sudo poweroff` — un servidor se apaga desde dentro, no desde el botón —, y en virt-manager crea la instantánea **`recien-instalado`**. Es el punto de retorno del resto del curso.

**Truco · Dos cosas que verás y no te esperabas**

La consola de texto no muestra nada mientras tecleas la contraseña — ni asteriscos: es normal, escribe y pulsa Intro. Y la ventana de la VM se queda con tu ratón y tu teclado cuando haces clic dentro: la tecla para soltarlos es Ctrl derecho (virt-manager lo indica en la barra de título). Ambas cosas son mucho más frecuentes de lo que parece: la primera pasa en todo servidor, la segunda en toda VM.

## 8.6. Romper, mirar, restaurar

Hay una cosa que un instalador nunca enseña: **qué pasa cuando se rompe**. Hoy sí. Restaura la instantánea si has hecho pruebas, arranca la VM, entra, y haz lo impensable — con el capítulo 6 en mente:

```
marcos@debian-lab:~$ sudo rm -rf /boot
marcos@debian-lab:~$ ls /boot
ls: no se puede acceder a '/boot': No existe el fichero o el directorio
marcos@debian-lab:~$ sudo reboot
```

Acabas de borrar el kernel — y casi todo GRUB: sus módulos y su menú viven en `/boot/grub`; solo sobrevive su primer trozo, el del sector cero. El sistema sigue funcionando *hasta que se apaga*: todo lo que necesitaba ya estaba cargado en RAM. Pero al reiniciar, la cadena del capítulo 6 se rompe en el segundo eslabón: ese trozo de GRUB arranca, busca el resto de sí mismo en `/boot/grub`, no lo encuentra, y te deja en su consola de emergencia — `grub rescue>`, con un «file not found». Míralo bien. Es la pantalla que aterroriza a quien no sabe qué es GRUB — y tú sabes exactamente qué falta y por qué.

Ahora, desde virt-manager: *Forzar apagado* → Instantáneas → `recien-instalado` → *Ejecutar*. Arranca. Cinco segundos, y `/boot` está de vuelta como si nada. Ese es el bloque C entero en una escena: el miedo a romper ha desaparecido porque romper cuesta cinco segundos.

## 8.7. Del ensayo al ordenador de verdad

Todo lo que has hecho hoy es, paso a paso, lo que harás el día que instales Linux en un PC físico. Cambian cuatro cosas — la ISO viaja en un pendrive, el disco tiene nombre real y quizá un Windows dentro, el arranque es UEFI y pide una partición más (la EFI del capítulo 6), y no hay instantánea que te salve — y por eso existe el anexo C: la lista de comprobación completa para ese día, con la grabación del pendrive por la vía gráfica y por terminal.

**Historia · De ochenta disquetes a un pendrive**

Instalar Linux en 1994 empezaba por descargar decenas de imágenes de disquete — Slackware llegó a necesitar más de cincuenta, en series con nombres como *A*, *AP*, *D*, *N* — y grabarlas una a una, rezando para que ninguna viniera mal. El CD (1995) lo redujo a un disco; el pendrive de arranque, a un fichero copiado byte a byte. Lo que no ha cambiado en treinta años

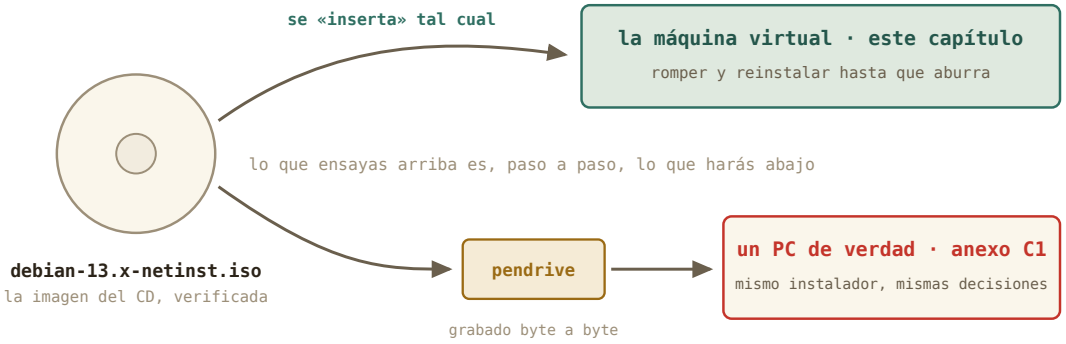



Figura 8.4: Una misma imagen, dos destinos. Lo que ensayas en la VM es exactamente lo que ejecutarás en el PC real.

es la parte que hoy has practicado: decidir las particiones, elegir el cargador de arranque, y aguantar el pulso en el «¿desea escribir los cambios?». Los soportes se encogen; las decisiones son las mismas.

## 8.8. Práctica: la sala de operaciones

Cuaderno en marcha *en tu Mint* (`script sesion08.log`): anota ahí tus decisiones y tiempos; dentro de la VM no hace falta grabar nada.

**Nivel 1 · Lo esencial**

**Ejercicio 8.1 · El plan, por escrito**

Antes de arrancar el instalador, escribe en tu cuaderno la tabla de particiones que vas a crear: nombre de dispositivo, tamaño, sistema de ficheros y punto de montaje de cada una. Y dos decisiones más: nombre de la máquina y contraseña de root (vacía, y por qué).

*Pista.* La figura del plan tiene todo; el «por qué» de root está en la sección de arrancar el instalador.

*(Solución al final del capítulo.)*

**Ejercicio 8.2 · La instalación**

Ejecuta la instalación completa siguiendo las secciones del capítulo — particionado manual, sin escritorio, con servidor SSH — y anota en el cuaderno cuánto ha tardado y en qué pantalla dudaste.

*Pista.* Si en el particionado te pierdes, «Deshacer los cambios realizados a las particiones» vuelve al principio sin coste; y si algo sale mal de verdad, *Forzar apagado* y volver a arrancar la VM reinicia el instalador desde cero.




Figura 8.5: Una caja de disquetes de 5¼ pulgadas junto a un pendrive con más de cien veces la capacidad de toda la caja. Una instalación de los 90 cabía justo en una caja así. *Foto: JIP, CC BY-SA 3.0, Wikimedia Commons.*

*(Solución al final del capítulo.)*

**Ejercicio 8.3 · El parte del primer arranque**

Entra en `debian-lab` y ejecuta la revisión del capítulo: `lsblk`, `df -h /`, `ip a | grep "inet "` y `systemctl status ssh`. Copia los cuatro resultados al cuaderno y compáralos con tu plan del 8.1. Después, actualiza el sistema con `apt`.

*Pista.* Son exactamente los comandos de los capítulos 5 y 6, ahora sobre una instalación tuya.

*(Solución al final del capítulo.)*

**Ejercicio 8.4 · La instantánea que importa**

Apaga la VM desde dentro (`sudo poweroff`) y crea la instantánea `recien-instalado`. Compruébala con `virsh` desde tu Mint.

*Pista.* Capítulo 7, sección de instantáneas — con la máquina apagada, la instantánea guarda el disco y es la más limpia de restaurar.

*(Solución al final del capítulo.)*

**Nivel 2 · El reto**

**Ejercicio 8.5 · Romper el arranque**

Con la instantánea a salvo: borra `/boot`, reinicia, y observa exactamente dónde se rompe la cadena del capítulo 6. Describe la pantalla en el cuaderno. Después restaura la instantánea y comprueba que `/boot` está de vuelta.

*Pista.* La orden está en la sección «Romper, mirar, restaurar». Antes de teclearla, mira el prompt: debe decir `debian-lab`.

*(Solución al final del capítulo.)*

**Ejercicio 8.6 · La segunda instalación, cronometrada**

Borra la máquina entera desde virt-manager (Eliminar, marcando borrar el disco), vuelve al capítulo 7 para crearla de nuevo, y reinstala Debian desde cero — esta vez con `/home` en su propia partición de 5 GB — sin mirar el capítulo más que para lo imprescindible. Anota el tiempo.

*Pista.* El plan cambia en una línea: la raíz se queda con `14.5 GB` y `/home` recibe `5 GB`, ext4, punto de montaje `/home` (el instalador lo propone solo para la segunda ext4). Crea las tres particiones en orden: raíz, home, swap — todas **primarias**, aunque te sugiera lógica.

*(Solución al final del capítulo.)*

**Nivel 3 · Opcional · el pendrive real**

**Ejercicio 8.7 · El pendrive de arranque (sin instalar nada)**

Siguiendo el anexo C, graba la ISO en un pendrive que puedas borrar, arranca tu portátil desde él hasta el menú del instalador — **y sal sin elegir nada**: apaga y arranca de nuevo tu Mint. Objetivo: haber visto el menú de arranque del UEFI y el instalador en hardware real sin tocar un solo byte de tu disco.

*Pista.* En el anexo tienes las dos vías — la aplicación «Discos» de Mint y la orden `dd` — y la regla de oro: `lsblk` antes y después de pinchar el pendrive para estar seguro de su nombre.

*(Solución al final del capítulo.)*

**Ejercicio 8.8 · La misma instalación, con UEFI (de usar y tirar)**

`debian-lab` arranca con BIOS por la razón del capítulo 7. Para ver con tus manos qué cambia con UEFI — que es lo que tendrás en un PC real —, crea una **segunda** máquina, `debian-uefi` (2 GB de RAM bastan), marcando en el último paso del asistente *Personalizar la configuración antes de instalar* → *Vista general* → *Firmware:* **UEFI** → *Aplicar* → *Comenzar la instalación*. Instálala con el mismo plan y anota las tres diferencias que veas. Después bórrala (*Eliminar*, con su disco): sin instantáneas no es sitio para vivir.

*Pista.* Las diferencias están en el menú de arranque de la ISO, en el particionado (el instalador querrá una partición más, y de un tipo concreto) y en el `lsblk` final. Dentro de la máquina, `efibootmgr -v` enseña dónde apuntó GRUB su llave.

*(Solución al final del capítulo.)*

## 8.9. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. La imagen «netinst» de Debian…

   a) contiene el sistema completo con escritorio
   b) contiene solo lo mínimo para arrancar el instalador; el resto lo descarga del espejo
   c) solo sirve para máquinas virtuales

2. Dejar vacía la contraseña de root en el instalador de Debian…

   a) es un fallo de seguridad grave
   b) desactiva root y da `sudo` a tu usuario, como en Mint
   c) hace que el instalador se detenga

3. En `debian-lab`, que arranca con BIOS, GRUB se instala en…

   a) el primer sector del disco entero (`/dev/vda`, el MBR)
   b) la swap
   c) una partición EFI (FAT32, montada en `/boot/efi`)

4. Marcaste «Servidor SSH» en la selección de programas para…

   a) poder navegar por internet desde la VM
   b) que la máquina acepte conexiones remotas desde tu Mint (bloque E)
   c) acelerar la instalación

5. Tras `sudo rm -rf /boot`, el sistema sigue funcionando hasta reiniciar porque…

   a) el kernel ya estaba cargado en RAM; solo falta al arrancar
   b) Linux protege `/boot` y no lo borra de verdad
   c) GRUB tiene una copia

6. La diferencia más peligrosa entre instalar en la VM y en un PC real es…

   a) que el instalador es distinto
   b) que en el PC real hay discos con datos y no hay instantánea que restaurar
   c) que en el PC real no se puede particionar a mano

## Soluciones del capítulo

**Ejercicio 8.1 · El plan, por escrito**

Solución razonada

`vda1` ~18 GB · ext4 · `/` — `vda2` 2 GB · swap; GRUB en el primer sector de `vda`. Máquina `debian-lab`. Root vacío: Debian desactiva root y da `sudo` a tu usuario, como en Mint.

**Por qué así** — El plan escrito es lo que separa una instalación de una apuesta. En el PC real añadirás una columna: «¿qué había antes en esa partición?» — y ahí es donde el hábito te salvará los datos.

### Ejercicio 8.2 · La instalación

Solución razonada

Entre 20 y 40 minutos la primera vez, gran parte esperando descargas del espejo. Las dudas típicas: la pantalla de root (vacío), el tamaño de la raíz (lo que quede menos el swap) y la selección de programas (desmarcar el escritorio). Al terminar, el `login:` en consola de texto es la señal de éxito.
**Por qué así** — La primera instalación siempre es lenta y torpe; la segunda (ejercicio 8.6) es donde se nota que has aprendido. Por eso el diseño de este capítulo insiste en repetir.

### Ejercicio 8.3 · El parte del primer arranque

Solución razonada

Las dos particiones montadas según el plan; alrededor de 1,3 GB usados; una dirección `192.168.122.x`; y `ssh.service` en *active (running)*. `sudo apt update && sudo apt upgrade` debe funcionar sin pedir nada raro: la prueba de que el truco de root funcionó.
**Por qué así** — Cada comando verifica una decisión del instalador: particiones, tamaño, red, SSH. Un físico no da por bueno un montaje sin medirlo; tú tampoco das por buena una instalación.

### Ejercicio 8.4 · La instantánea que importa

Solución razonada

Ver → Instantáneas → + → nombre `recien-instalado`. En Mint: `virsh snapshot-list debian-lab` la muestra con estado `shutoff`.
**Por qué así** — Es el punto de retorno de todo lo que queda de curso: los experimentos de red del bloque D y el servidor del bloque E parten de aquí. Restaurarla es siempre una opción legítima.

### Ejercicio 8.5 · Romper el arranque

Solución razonada

`sudo rm -rf /boot`, `sudo reboot` → el primer trozo de GRUB arranca (sigue en el sector cero, que no has tocado) pero no encuentra ni el resto de GRUB ni el kernel: te deja en `grub rescue>` con un «file not found». Forzar apagado → restaurar `recien-instalado` → `ls /boot` muestra el kernel (`vmlinuz-…`) y la carpeta `grub` como si nada.

**Por qué así** — Ver romperse un eslabón concreto de la cadena convierte «no arranca» en «falta `/boot`; el sector cero sigue ahí»: un diagnóstico. Y la restauración es la demostración práctica de para qué sirve el bloque C.

**Ejercicio 8.6 · La segunda instalación, cronometrada**

Solución razonada

Normalmente la mitad del tiempo de la primera vez, y `lsblk` mostrará `vda2` montado en `/home`. Renueva la instantánea `recien-instalado` sobre esta máquina — es la definitiva.
**Por qué así** — «Romperla y reinstalar hasta que aburra» era el objetivo declarado del capítulo. La variante del `/home` separado enseña el matiz que más se discute en foros — y te deja decidir con criterio propio, que es lo único que cuenta.

**Ejercicio 8.7 · El pendrive de arranque (sin instalar nada)**

Solución razonada

El pendrive aparece como `sda` (o `sdb`) al pinchar; se graba con «Discos → Restaurar imagen de disco» o con `sudo dd if=debian….iso of=/dev/sdX bs=4M status=progress` (con la X correcta, verificada dos veces); se reinicia pulsando la tecla del menú de arranque (F12, F2 o Esc según la marca) y se elige el pendrive. Verás el menú del instalador — en modo UEFI tiene otro aspecto que en la VM (anexo C), pero las mismas opciones; *Ctrl+Alt+Supr* o apagar te devuelve a Mint intacto.
**Por qué así** — Es el ensayo general con el vestuario de verdad, y la única parte del capítulo con un riesgo real: equivocar la letra del pendrive en `dd` borra el disco equivocado. Por eso la vía gráfica es perfectamente honorable — elige por su nombre y tamaño, sin letras — y por eso el anexo insiste en comprobar dos veces.

**Ejercicio 8.8 · La misma instalación, con UEFI (de usar y tirar)**

Solución razonada

(1) El menú de la ISO ya no es isolinux sino GRUB, con «UEFI» en el título. (2) Al particionar, la tabla es `gpt` y el instalador espera una **partición del sistema EFI** (512 MB, FAT32) antes que nada: créala tú, o el paso «Cargador de arranque» protestará. (3) `lsblk` muestra `vda1 … /boot/efi` además de la raíz y la swap, y `efibootmgr -v` lista una entrada `debian` que apunta a `\EFI\debian\shimx64.efi` dentro de esa partición — la llave en el buzón del capítulo 6. Y si en virt-manager intentas una instantánea, verás el aviso de la caja del capítulo 7. Borra la máquina al acabar.

**Por qué así** — Es el ensayo exacto de lo que te encontrarás en el anexo C, en una máquina donde equivocarse sigue siendo gratis. Las tres diferencias caben en una línea del cuaderno, y con eso el UEFI deja de ser un misterio.

**Autocomprobación (test de la sección 8.9)** — Respuestas: 1 b · 2 b · 3 a · 4 b · 5 a · 6 b.

### La próxima sesión

Tienes un servidor instalado por ti, con una dirección `192.168.122.x` que todavía no sabes leer. El bloque D empieza ahí: aprenderás qué es una dirección IP, qué es esa máscara `/24` y quién reparte las direcciones en tu casa, y harás la expedición al router de la compañía, en modo solo lectura, para censar todos los aparatos que viven en tu red. Guarda `sesion08.log`: bloque C completo.

cap. 8 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion08.log`

# 9 Qué es una red

Tu servidor recién instalado tiene una dirección — `192.168.122.15` — que todavía no sabes leer. Este es el primero de tres capítulos sobre redes, y se ocupa de lo más básico y lo más importante: cómo se hablan dos ordenadores, qué significan esos cuatro números y la barra que los sigue, quién es tu router, y cómo saber si una máquina está ahí — y a qué distancia. Nada de configurar todavía: hoy se aprende a leer.

**2 h** con ejercicios · requisitos: **capítulos 1–8** · máquina: **tu Linux de diario y la VM**

**Al terminar esta sesión sabrás**

- Cómo se hablan dos ordenadores: el modelo de capas, qué sobre pone cada una, y por qué la MAC no sale de tu casa mientras la IP recorre el mundo.
- Qué hace un switch y qué hace un router — y por qué la caja de tu compañía es varias cosas a la vez.
- Leer tu ficha de red con `ip a` e `ip route`: interfaz, dirección MAC, dirección IP, máscara y puerta de enlace.
- Qué significa el `/24` de una dirección, en sus tres notaciones, y cómo decide quién puede hablar con quién directamente.
- Medir con `ping` si una máquina responde y cuánto tarda — y qué tiene que ver ese tiempo con la velocidad de la luz.

## 9.1. Cómo se hablan dos ordenadores

Antes de teclear nada, el modelo mental — porque con él, todo lo que sigue es leer; sin él, es memorizar. Cuando tu portátil envía algo a otro ordenador — una página, un `ping`, una sesión SSH — no manda «un mensaje»: lo trocea en **paquetes** de unos pocos miles de bytes, y cada paquete viaja por su cuenta, metido en varios sobres, uno dentro de otro. Cada sobre lo escribe y lo lee una **capa** distinta, y eso es todo lo que hay que saber del famoso *modelo de capas*: los libros lo cuentan con siete (el modelo OSI); internet usa en la práctica cinco, y son estas:

| Capa | Qué resuelve | Con qué | Dónde la verás |
|---|---|---|---|
| **Aplicación** | qué se dice | HTTP, SSH, DNS | capítulos 11–13 |

| Capa | Qué resuelve | Con qué | Dónde la verás |
|---|---|---|---|
| **Transporte** | a qué *programa* de la máquina va, y si llega entero y en orden | TCP, UDP, los **puertos** | capítulo 11 |
| **Red** | a qué *máquina* del mundo va, aunque esté a diez saltos | **IP**, direcciones, routers | este capítulo |
| **Enlace** | al vecino de al lado, por este cable o esta Wi-Fi | Ethernet, Wi-Fi, direcciones **MAC** | `ip a`, línea `link/ether` |
| **Física** | bits convertidos en voltios, luz o radio | cobre, fibra, aire | tu asignatura |

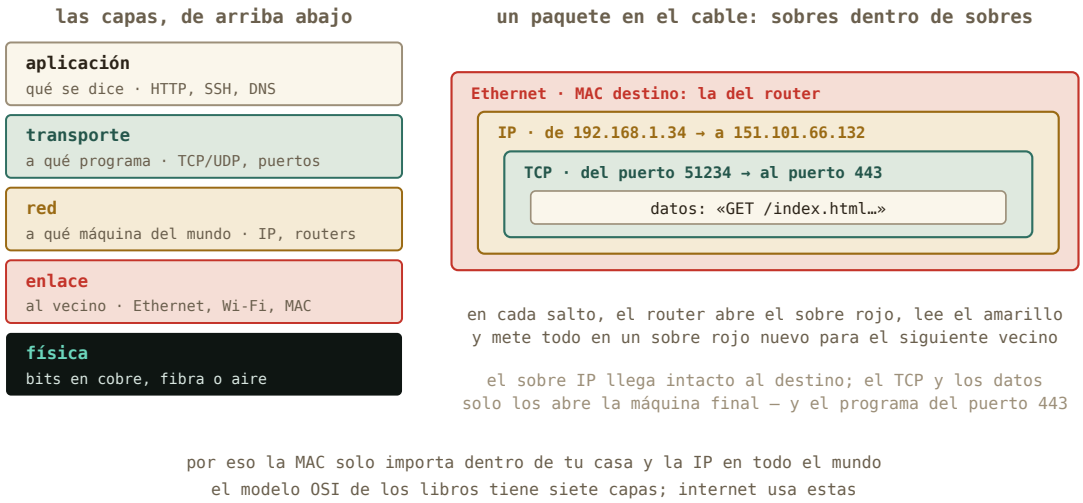



Figura 9.1: Cada capa añade su sobre. Los routers del camino solo abren el de fuera; el sobre IP llega intacto, y lo que hay dentro solo lo lee el programa de destino.

La analogía postal lo deja claro. Tú escribes la carta (los **datos**). La metes en un sobre con el número de piso (el **puerto**: qué programa, entre los muchos de esa máquina, debe abrirla). Ese sobre va dentro de otro con la dirección de la calle (la **IP**: qué máquina del mundo). Y ese, dentro de un sobre más que solo entiende el repartidor de tu barrio (la **MAC**: qué aparato de *esta* red, por este cable). En cada salto, el repartidor abre el sobre exterior, lee la calle del siguiente, y vuelve a envolver para el siguiente repartidor. Por eso la MAC **solo importa dentro de tu casa** y la IP importa en todo el mundo; por eso un **router** no es más que una máquina que lee el sobre IP y decide por qué puerta sale; y por eso los puertos, que verás en el capítulo 11, son el número de piso que hace posible que una dirección sirva a muchos programas — y, como descubrirás dentro de un rato, a muchas casas.

## 9.2. Nivel 2 y nivel 3: switches y routers

Dos de esas capas tienen aparato propio, y conviene distinguirlos porque en la caja de tu compañía van juntos. Un **switch** trabaja en el nivel 2, el de enlace: tiene varias bocas de cable, aprende qué MAC hay detrás de cada una y reparte los sobres *dentro de tu red* sin mirar la IP — es el repartidor del barrio. El punto de acceso Wi-Fi es un switch sin cables. Un **router** trabaja en el nivel 3, el de red: tiene una pata en cada red, lee el sobre IP y decide por cuál sale el paquete — es el que te saca del barrio. La regla que ya conoces, dicha con los aparatos: **misma red, switch; otra red, router.**

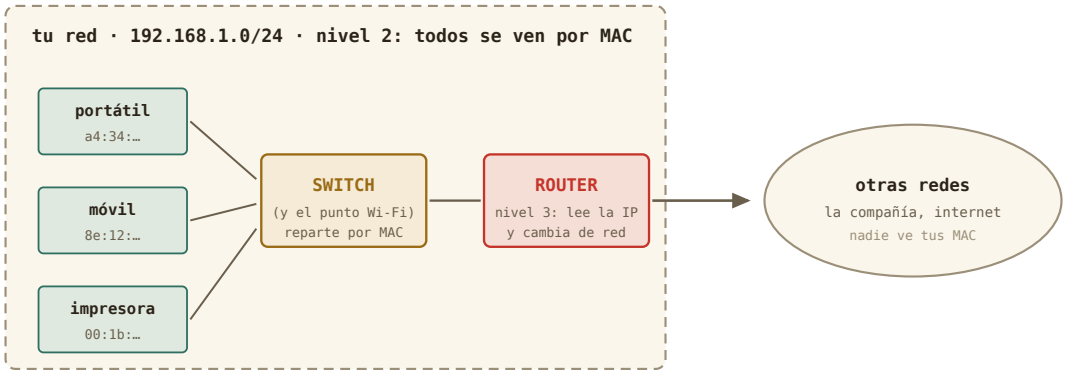



Figura 9.2: Dos papeles, una caja. En casa van juntos en el aparato de la compañía; en un campus verás decenas de switches y unos pocos routers.

La «caja del router» de tu salón es en realidad cuatro o cinco aparatos en uno: un switch (las bocas amarillas de detrás), un punto de acceso Wi-Fi, un router, un servidor DHCP y un pequeño servidor de nombres — y un cortafuegos (capítulo 11). Cuando su página web te enseñe pestañas separadas para LAN, Wi-Fi, WAN y DHCP, estás viendo esos papeles por separado. Si algún día necesitas más bocas de cable, lo que compras es un switch de veinte euros: se enchufa al router y todo sigue siendo una sola red.

## 9.3. Tu ficha de red: `ip a`

El comando de este bloque es `ip`, y su primera pregunta es «¿quién soy en la red?»: `ip a` (*address*). La salida asusta la primera vez; léela por bloques, que cada bloque es una **interfaz** — una tarjeta de red, real o virtual:

```
marcos@portatil:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 ...
    inet 127.0.0.1/8 scope host lo
```

```
2: wlp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
    link/ether a4:34:d9:1c:7e:02 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.34/24 brd 192.168.1.255 scope global dynamic wlp3s0
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 ...
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
```

Tres interfaces. `lo` es el *loopback*: la máquina hablando consigo misma (`127.0.0.1` siempre eres tú — la usarás en el capítulo 12). `wlp3s0` es tu Wi-Fi (`enp…` sería el cable), con dos direcciones distintas — una por capa: la **MAC** (`link/ether a4:34:…`, el número de serie grabado en la tarjeta de fábrica: capa de enlace, el sobre del barrio) y la **IP** (`inet 192.168.1.34/24`, la que te ha tocado en esta red: capa de red, el sobre con la calle). Y `virbr0` es el router de mentira del capítulo 7 — la red NAT de tus máquinas virtuales. Ese `dynamic` al final de tu IP es una pista para dentro de dos secciones.

**Truco · La misma ficha, en el icono de red**

El icono de red de tu escritorio (clic → *Información de la conexión*) muestra exactamente esto: dirección IP, máscara, puerta de enlace, DNS. Es la vía gráfica de `ip a`, y perfectamente digna para un vistazo. La terminal gana cuando la máquina no tiene escritorio — tu `debian-lab` — o cuando quieres pegarlo en un correo pidiendo ayuda: «esto es lo que dice `ip a`» es la frase con la que empiezan todos los diagnósticos de red.

## 9.4. El /24: red y máquina

Una dirección IP son cuatro números de 0 a 255 — cuatro bytes, tú sabes de esto — y el `/24` es la **máscara**: dice cuántos bits del principio identifican la *red* y cuántos quedan para la *máquina*.




Figura 9.3: La anatomía de tu dirección. Los tres primeros números son el «prefijo» que comparte toda la casa; el último es el tuyo.

Con `/24`, los 24 primeros bits — tres números — son la red: `192.168.1`. Queda un byte para las máquinas: 254 direcciones útiles (la `.0` nombra a la red y la `.255` es la de «todos a la vez»).

La misma máscara se escribe de tres maneras, y te encontrarás las tres: la **barra** (`/24`) que usa `ip a`; la **decimal con puntos** (`255.255.255.0`) que enseñan el icono de red y el router; y la **binaria**, que es lo que la máquina usa de verdad — unos donde hay red, ceros donde hay máquina. Basta con escribirla en binario una vez para que las otras dos dejen de ser arbitrarias:

| Barra | Decimal con puntos | Binario | Máquinas útiles | Típico de… |
|---|---|---|---|---|
| `/24` | `255.255.255.0` | `11111111.11111111.11111111.00000000` | 254 | una casa, un laboratorio |
| `/16` | `255.255.0.0` | `11111111.11111111.00000000.00000000` | 65 534 | un campus |
| `/8` | `255.0.0.0` | `11111111.00000000.00000000.00000000` | 16 777 214 | la red `10.x.x.x` de una multinacional |
| `/25` | `255.255.255.128` | `11111111.11111111.11111111.10000000` | 126 | media casa: la máscara ya no cae en un punto |
| `/30` | `255.255.255.252` | `11111111.11111111.11111111.11111100` | 2 | un enlace entre dos routers |

Dos ejemplos leídos del todo. `192.168.1.34/24`: máscara `255.255.255.0`, en binario 24 unos y 8 ceros; red `192.168.1.0`, máquina `34`, vecinos del `.1` al `.254`. Y `10.20.30.40/8`: máscara `255.0.0.0`, 8 unos y 24 ceros; red `10.0.0.0`, y los otros tres números — `20.30.40` — identifican la máquina entre casi 17 millones. El `/25` es el que delata a quien no ha visto el binario: el `128` de `255.255.255.128` es simplemente `10000000` — un bit más de red, robado a las máquinas.

Y la regla que lo explica todo: **dos máquinas de la misma red se hablan directamente; para llegar a otra red hace falta un intermediario**, el **router**, al que tu máquina llama *puerta de enlace* (*gateway*). `ip route` te dice cuál es la tuya:

```
marcos@portatil:~$ ip route
default via 192.168.1.1 dev wlp3s0 proto dhcp metric 600
192.168.1.0/24 dev wlp3s0 proto kernel scope link src 192.168.1.34
192.168.122.0/24 dev virbr0 proto kernel scope link src 192.168.122.1
```

Léelo como instrucciones de un GPS: «para `192.168.1.0/24` (mi casa), sal directo por la Wi-Fi; para `192.168.122.0/24` (mis VMs), por `virbr0`; para *todo lo demás* (`default`), entrégaselo a `192.168.1.1`». Ese `192.168.1.1` es tu router — el que abrirás al final del capítulo.

## 9.5. ¿Estás ahí? `ping`

`ping` manda un paquete diminuto a una dirección y cronometra la respuesta. Es el estetoscopio de las redes: responde a «¿esa máquina existe y me oye?» y de propina mide *cuánto tarda*:

```
marcos@portatil:~$ ping -c 3 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=2.31 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=2.08 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=2.44 ms
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
marcos@portatil:~$ ping -c 3 192.168.122.15
64 bytes from 192.168.122.15: icmp_seq=1 ttl=64 time=0.41 ms
...
marcos@portatil:~$ ping -c 3 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=14.7 ms
...
```

Tres medidas, tres escalas: el router de casa a 2 ms, la máquina virtual a 0,4 ms (no hay cable que recorrer: está dentro de tu RAM), y un servidor de Google a 15 ms. `-c 3` limita a tres paquetes — sin él, `ping` sigue hasta que lo pares con Ctrl+C (la señal del capítulo 5). Y ese `8.8.8.8` sin nombre es a propósito: hoy hablamos con números; los nombres llegan en el capítulo 11.

**Por dentro · `ping` y la velocidad de la luz**

Ese tiempo es de *ida y vuelta*, y tiene un mínimo físico que puedes calcular. En fibra la luz va a unos dos tercios de $c$ — 200 000 km/s —, así que cada milisegundo de ida y vuelta corresponde como mucho a 100 km de distancia. Un servidor a 15 ms está, como mínimo, en un radio de 1 500 km; uno a 150 ms está seguro al otro lado de un océano, y nada lo bajará de ahí — ni la fibra más cara. Es la razón por la que los centros de datos se construyen cerca de sus usuarios, y por la que un clúster «en la nube» a 5 000 km nunca se sentirá tan inmediato como la máquina de al lado. Pocas veces un comando de terminal te deja medir una constante fundamental.

**Historia · La primera palabra de la red fue «LO»**

El 29 de octubre de 1969, un estudiante de UCLA llamado Charley Kline intentó escribir `LOGIN` en un ordenador de Stanford, a 500 km, a través de dos armarios llamados IMP — los primeros routers de la historia, fabricados por BBN a partir de un miniordenador Honeywell blindado para uso militar. Tecleó la L, llegó. Tecleó la O, llegó. Al teclear la G, el sistema de Stanford se cayó. La primera palabra transmitida por la red que acabaría siendo internet fue, por accidente, «LO». Aquella red, ARPANET, tenía cuatro nodos; las reglas que inventó para que máquinas distintas se entendieran — trocear los mensajes en paquetes, darle a cada máquina una dirección, dejar que los intermediarios decidan el camino — son las que tu `ip route` está aplicando ahora mismo.




Figura 9.4: El primer IMP de ARPANET (1969), el abuelo de la caja con lucecitas de tu salón. *Foto: Steve Jurvetson, CC BY 2.0, Wikimedia Commons.*

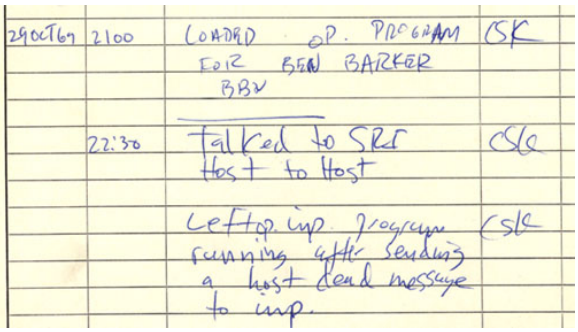



Figura 9.5: La página del cuaderno de UCLA con el registro de aquel «LO», 22:30 del 29 de octubre de 1969. *Dominio público, Wikimedia Commons.*

## 9.6. Práctica: leer la red

Cuaderno en marcha (`script sesion09.log`). Todo lo de hoy es lectura: ni un cambio en tu máquina.

**Nivel 1 · Lo esencial**

**Ejercicio 9.1 · Tu ficha de red**

Con `ip a` e `ip route`, rellena en tu cuaderno la ficha de tu portátil: nombre de la interfaz, dirección MAC, dirección IP, máscara y puerta de enlace. Después contrástala con la *Información de la conexión* del icono de red.

*Pista.* La MAC es la línea `link/ether`; la IP y la máscara van juntas en `inet`; la puerta de enlace es el `default via` de `ip route`.

*(Solución al final del capítulo.)*

**Ejercicio 9.2 · Tres distancias**

Haz `ping -c 5` a tu router, a tu `debian-lab` (arráncala si está apagada) y a `8.8.8.8`. Anota el tiempo medio de cada uno, y con la regla de «100 km por milisegundo» estima a qué distancia *máxima* está ese servidor de Google.

*Pista.* La línea final `rtt min/avg/max` te da la media sin calcular nada.

*(Solución al final del capítulo.)*

**Nivel 3 · El reto (opcional)**

**Ejercicio 9.3 · Leer máscaras**

Sin ejecutar nada: (a) ¿cuántas máquinas caben en una red `/8` como `10.0.0.0/8`? (b) ¿Y en una `/30`? (c) `192.168.1.34/24` y `192.168.2.10/24`, ¿se hablan directamente o necesitan un router?

*Pista.* Bits para máquinas = 32 − máscara; caben 2 elevado a eso, menos las dos direcciones reservadas.

*(Solución al final del capítulo.)*

## 9.7. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. La dirección MAC de tu portátil…

   a) viaja en cada paquete hasta el servidor de destino
   b) solo importa dentro de tu propia red: es el sobre del barrio
   c) es lo mismo que la dirección IP

2. Dos máquinas de la misma red se hablan…

a) a través del router, siempre
b) directamente, a través del switch o la Wi-Fi
c) solo si tienen IP pública

3. En `192.168.1.34/24`, el `/24` indica…

a) que hay 24 máquinas en la red
b) que los 24 primeros bits (tres números) identifican la red
c) el número de saltos hasta internet

4. La «puerta de enlace» (`default via` en `ip route`) es…

a) la máquina a la que se entregan los paquetes que van a otras redes
b) el servidor de nombres
c) tu propia dirección

5. Un `ping` de 150 ms a un servidor te dice que…

a) tu Wi-Fi está estropeada
b) el servidor está, como poco, a miles de kilómetros: ninguna mejora lo bajará mucho
c) el servidor está apagado

6. Un router del camino, al recibir un paquete tuyo…

a) abre todos los sobres y lee tus datos
b) abre solo el sobre de enlace, lee el destino IP y reenvía
c) lo devuelve si no es para él

## Soluciones del capítulo

### Ejercicio 9.1 · Tu ficha de red

Solución razonada

Algo como: `wlp3s0` · `a4:34:d9:1c:7e:02` · `192.168.1.34` · `/24` · `192.168.1.1`. El icono de red dice lo mismo con otras palabras («máscara de subred 255.255.255.0» es el `/24` escrito en decimal).
**Por qué así** — Esta ficha es lo primero que te pedirá cualquiera a quien pidas ayuda con la red, y lo primero que tú pedirás cuando seas el que ayuda. Dos vías, una ficha: el principio de siempre.

### Ejercicio 9.2 · Tres distancias

Solución razonada

Típicamente: router 1–5 ms, VM $< 1$ ms, `8.8.8.8` 10–30 ms. Con 15 ms de ida y vuelta, el servidor está como mucho a unos 1 500 km — y probablemente mucho más cerca, porque el tiempo incluye el trabajo de cada router del camino, no solo el vuelo de la

luz.
**Por qué así** — `ping` mide; la física acota. Distinguir «el mínimo que impone $c$» del «tiempo real con sus retrasos» es exactamente lo que hace un físico con cualquier medida — y te será útil cuando un clúster remoto «vaya lento»: primero, ¿cuánto tarda el `ping`?

**Ejercicio 9.3 · Leer máscaras**

Solución razonada

(a) $32 - 8 = 24$ bits $\rightarrow 2^2 - 2 = 16\,777\,214$ máquinas — el tamaño de una multinacional. (b) $32 - 30 = 2$ bits $\rightarrow 4 - 2 = 2$ máquinas: la red mínima, típica de un enlace punto a punto. (c) Con `/24`, sus redes son `192.168.1` y `192.168.2`: distintas $\rightarrow$ necesitan un router aunque estén en la misma mesa.

**Por qué así** — La máscara es aritmética binaria, que dominas de la asignatura de computación; solo cambia el vocabulario. Y la (c) es la avería de red más común del mundo: dos máquinas «al lado» que no se ven porque alguien escribió mal un número.

**Autocomprobación** — Respuestas: 1 b · 2 b · 3 b · 4 a · 5 b · 6 b.

**La próxima sesión**

Ya sabes leer una dirección y medir una distancia. En el capítulo 10 abres la caja con lucecitas: verás quién reparte las direcciones en tu casa y con qué parámetros, entenderás por qué todos tus aparatos salen a internet con un solo número — el NAT, paso a paso —, harás el censo de todo lo que vive en tu red, y tocarás el router por primera vez con un cambio reversible. Guarda `sesion09.log`.

cap. 9 v0.2 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion09.log`

# 10 Tu red de casa: el router por dentro

Ya sabes leer una dirección. Hoy abres la caja con lucecitas que te dio la compañía: quién reparte las direcciones en tu casa y con qué parámetros, por qué todos tus aparatos salen a internet con un solo número, y cómo hacer el censo de tu red sin tocar nada. Y, por primera vez, tocas el router — un solo cambio, útil, con su vuelta atrás escrita antes de pulsar guardar.

**2–3 h** con ejercicios · requisitos: **capítulo 9** · máquina: **tu Linux de diario, la VM y el router (solo lectura, y un cambio reversible)**

**Al terminar esta sesión sabrás**

- Quién reparte las direcciones — DHCP —, los cinco parámetros que entrega, y qué son el rango y las reservas de tu router.
- Por qué casi todas las casas usan `192.168.1.x`: los tres bloques de direcciones privadas.
- La diferencia entre tu dirección privada y la pública, y cómo el router traduce entre ambas con su tabla NAT — paso a paso, con la respuesta de vuelta incluida.
- Entrar en la página de administración de tu router, orientarte en ella y hacer el censo de tu red.
- Por qué el cable gana a la Wi-Fi para un servidor — y medirlo.
- El protocolo del cambio reversible: anotar, escribir la vuelta atrás, cambiar, verificar — aplicado a una reserva DHCP.

## 10.1. ¿Quién reparte las direcciones? DHCP

Nunca le has dicho a tu portátil que es el `.34`. Se lo dijo el router, mediante **DHCP** (*Dynamic Host Configuration Protocol*): al conectarte, tu máquina grita a la red «¿alguien me da una dirección?» y el servidor DHCP del router le responde con un *préstamo* — una dirección libre, la máscara, la puerta de enlace y el servidor de nombres (capítulo 11) — por un tiempo limitado, que se renueva solo. De ahí el `dynamic` de `ip a`, y de ahí que tu móvil cambie de número de vez en cuando.

Y ya lo has visto funcionar dos veces sin saberlo: la red `default` de libvirt lleva su propio servidor DHCP — por eso `debian-lab` apareció con `192.168.122.15` sin configurar nada —, y el instalador de Debian pidió su dirección exactamente así. El mismo mecanismo a tres escalas: tu casa, tu hipervisor, el campus de la universidad.

Lo que reparte un DHCP es siempre el mismo lote, y son los cuatro o cinco parámetros que verás en cualquier pantalla de red del mundo:

| Parámetro | Ejemplo | Para qué |
|---|---|---|
| **Dirección IP** | `192.168.1.34` | tu número en la red |
| **Máscara** | `/24` = `255.255.255.0` | hasta dónde llega «mi red» |
| **Puerta de enlace** | `192.168.1.1` | a quién dar lo que va a otra red: el router |
| **Servidor DNS** | `192.168.1.1` o `9.9.9.9` | quién traduce nombres (capítulo 11) |
| **Duración del préstamo** | 24 h | cada cuánto se renueva |

Y con eso ya puedes leer — y al final de este capítulo, tocar — la pestaña DHCP de tu router, que tiene siempre la misma forma: la dirección del propio router, un **rango** de direcciones que reparte (algo como `192.168.1.100` a `.200`), la duración, y una lista de **reservas**: «a esta MAC, siempre esta IP». Lo usual en una casa bien puesta es dejar el rango para lo que va y viene (móviles, visitas) y reservar direcciones fijas para lo que debe encontrarse siempre en el mismo sitio: la impresora, un NAS, tu `debian-lab` si algún día vive en un PC físico. Lo que no se toca sin motivo: la dirección del router y la máscara.

¿Y por qué `192.168.1.x` en casi todas las casas? Porque hay tres bloques de direcciones reservados para uso privado, y los fabricantes eligen del más pequeño:

| Bloque privado | Cuántas redes /24 caben | Dónde lo verás |
|---|---|---|
| `192.168.0.0/16` | 256 | casas y oficinas pequeñas (`192.168.0.x`, `192.168.1.x`) |
| `172.16.0.0/12` | 4 096 | empresas, y la red interna de Docker |
| `10.0.0.0/8` | 65 536 | universidades, multinacionales, redes de la compañía |

Si dos casas usan `192.168.1.x` no chocan: cada una está detrás de su propio NAT, que es la siguiente sección. Y si un día montas una VPN con la universidad y las dos redes se llaman igual, sabrás por qué no funciona — y que la solución es cambiar la tuya a `192.168.37.x`, cualquier número que nadie use.

**Truco · Los mismos parámetros, en la ventana de red**

El icono de red de Mint → *Configuración de red* (o *Editar conexiones*) → tu Wi-Fi → pestaña **IPv4**: ahí están, con botones, los cinco parámetros de la tabla. *Automático (DHCP)* es lo normal; *Manual* te deja escribir IP, máscara, puerta de enlace y DNS a mano — lo que necesitarías en un laboratorio sin DHCP. Para que tu portátil tenga siempre la misma dirección en casa, la vía limpia no es *Manual* aquí sino una reserva en el router (al final de este capítulo): así la decisión vive en un solo sitio y no choca con el reparto del DHCP.

## 10.2. Privada, pública, y el NAT entre medias

Hay algo raro en tu `192.168.1.34`: millones de casas del mundo tienen exactamente esa misma dirección. No es un error: las redes `192.168.x.x`, `10.x.x.x` y `172.16-31.x.x` son **direcciones privadas**, reservadas para uso interno y repetidas en cada casa, oficina o laboratorio. Internet nunca las ve. Lo que internet ve es la **dirección pública** de tu router — una sola para toda la casa — y la traducción entre ambas es el **NAT** (*Network Address Translation*):

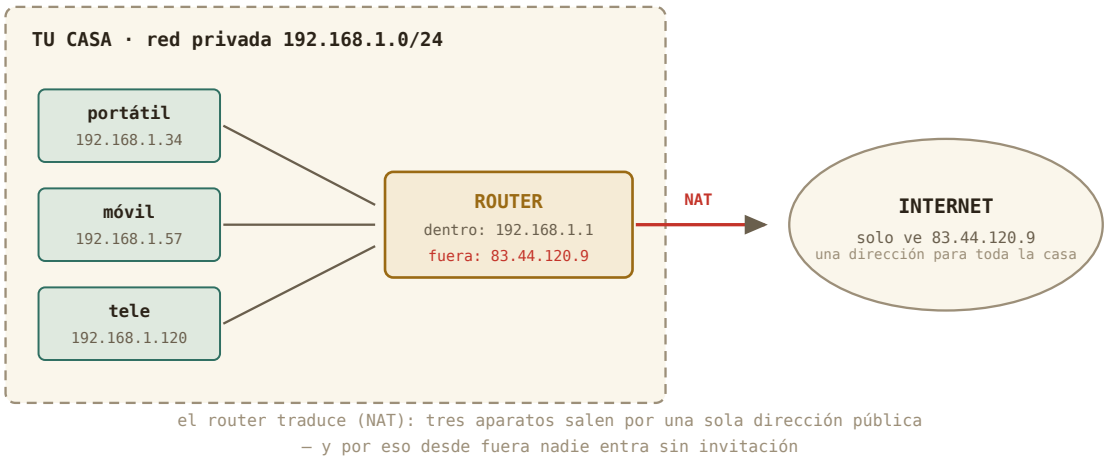



Figura 10.1: Tres aparatos dentro, una dirección fuera. El router reescribe cada paquete al salir y recuerda a quién devolver la respuesta.

¿Cómo puede una sola dirección servir a toda la casa? Con los sobres de la primera sección, paso a paso. Tu portátil pide una página web: sale un paquete que dice *«de 192.168.1.34, puerto 51234 → a 151.101.66.132, puerto 443»*. El router lo intercepta antes de que salga a internet y hace dos cosas: **reescribe el remitente** — ahora dice *«de 83.44.120.9, puerto 51234»*, su dirección pública — y **apunta una línea en su libreta**: «lo que vuelva dirigido a 83.44.120.9:51234 es en realidad para 192.168.1.34:51234». El servidor responde a la única dirección que conoce, la pública; el router recibe la respuesta, busca la línea, y **reescribe el destinatario**: → a 192.168.1.34:51234. Tu portátil recibe su página y nunca supo que le cambiaron el nombre. Mientras, tu móvil hace lo mismo con otro puerto (58001) y tiene su propia línea. Esa libreta es la **tabla NAT**:

| Dentro (privado) | Fuera (público) | Hablando con |
|---|---|---|
| `192.168.1.34:51234` | `83.44.120.9:51234` | `151.101.66.132:443` |
| `192.168.1.57:58001` | `83.44.120.9:58001` | `142.250.184.14:443` |

De esta libreta salen todas las consecuencias que importan. Un paquete que llega desde internet *sin que nadie de casa lo haya pedido* — alguien probando suerte en tu puerto 22 —

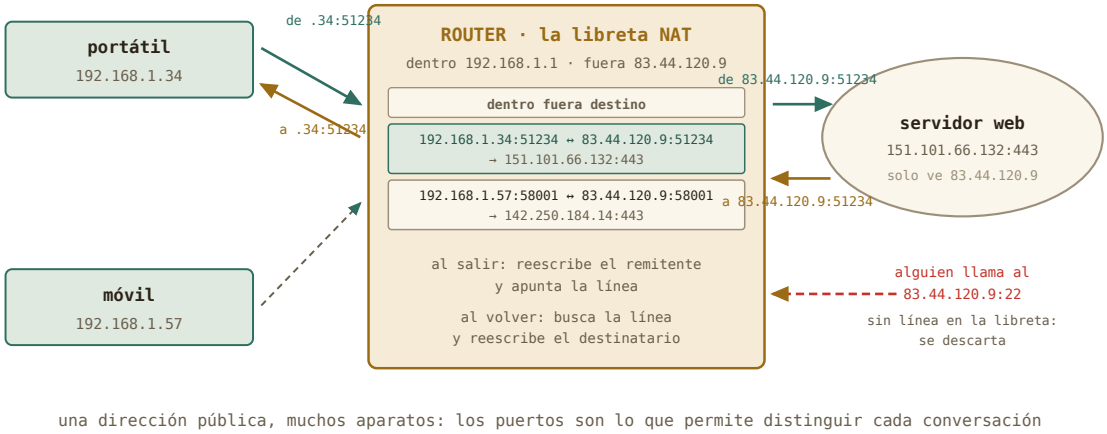



Figura 10.2: La libreta del router. Cada conversación que sale deja una línea; cada respuesta que vuelve se busca en ella. Lo que llega sin línea, se tira.

no tiene línea en la tabla: el router no sabe a quién dárselo y lo descarta. Por eso **desde fuera nadie entra sin invitación**. Cuando *quieras* que alguien entre — tú mismo por SSH desde la universidad —, tendrás que escribir a mano una línea permanente: «lo que llegue al puerto 22 público, dáselo siempre a 192.168.1.34» — es el *reenvío de puertos* del capítulo 11. Y fíjate en el detalle que explica el capítulo siguiente: la traducción funciona **gracias a los puertos**. La dirección pública es la calle; los puertos son los pisos, y el router puede permitirse una sola calle porque tiene sesenta y cinco mil pisos que repartir entre las conversaciones de toda la casa.

Puedes ver las dos caras desde tu terminal: la privada en `ip a`, y la pública preguntándole a un servidor de fuera qué dirección ve:

```
marcos@portatil:~$ curl ifconfig.me
83.44.120.9
```

Esa es tu casa vista desde internet — y coincidirá con la que el router llama *WAN* en su página. Dos remates. Tu `debian-lab` vive detrás de *dos* libretas — la de tu Mint (para la red `192.168.122.x`) y la del router —: cada paquete suyo cambia de remitente dos veces al salir y de destinatario dos veces al volver. Y algunas compañías añaden una tercera libreta en sus propias instalaciones para miles de casas a la vez (el CG-NAT del ejercicio 9.3): entonces ni la línea escrita a mano en tu router sirve, porque el intruso — o tú desde la universidad — nunca llega a tu router. Muñecas rusas, todas con la misma lógica.

## 10.3. El router por dentro (solo lectura)

El `192.168.1.1` de `ip route` no es solo una dirección: es una **página web**. Escríbela en el navegador y aparecerá la administración de tu router — usuario y contraseña suelen estar en la pegatina de la caja (y si son los de fábrica, el final de este capítulo tendrá algo que decir). Cada compañía diseña la suya, pero todas tienen los mismos cajones; tu tarea hoy es *encontrarlos sin tocar nada*:

- **Estado / Información**: la dirección WAN (compárala con `curl ifconfig.me`), el tiempo encendido, la versión del firmware.
- **Red local / LAN**: la dirección del router (`192.168.1.1`), y la configuración del **servidor DHCP** — el rango de direcciones que reparte (algo como `.2` a `.254`) y la duración de los préstamos.
- **Dispositivos conectados / Tabla DHCP / Lista de clientes**: el tesoro del capítulo. Cada aparato con su nombre, su IP y su MAC — el censo de tu red.
- **Wi-Fi**: nombre de la red (SSID), canal, tipo de cifrado.

**Cuidado · Primero se mira; al final, se toca**

La regla de seguridad del curso, aplicada al router: esta sección es **de solo lectura**. No cambies contraseñas, no reinicies, no toques el DHCP ni la Wi-Fi — un clic de más deja a toda la casa sin internet, y no hay instantánea que restaurar. Al final del capítulo harás cambios *reversibles*, cada uno con su procedimiento de vuelta atrás escrito antes de tocar. Hoy, si te pica el dedo: `ip a` en la terminal, que no rompe nada.

¿Y si tu router no enseña la tabla de dispositivos, o la enseña a medias? Tu propia máquina lleva un censo parcial: `ip neigh` (*los vecinos*) lista las máquinas de tu red con las que has hablado últimamente, con sus MAC:

```
marcos@portatil:~$ ip neigh
192.168.1.1 dev wlp3s0 lladdr 3c:84:6a:b2:10:f1 REACHABLE
192.168.1.57 dev wlp3s0 lladdr 8e:12:5b:0a:77:c3 STALE
192.168.122.15 dev virbr0 lladdr 52:54:00:9a:4e:21 REACHABLE
```

Y para un censo completo por tu cuenta existe `nmap`, el explorador de redes: `sudo apt install nmap` y `nmap -sn 192.168.1.0/24` toca a la puerta de las 254 direcciones y lista quién responde. Es una herramienta legítima **sobre tu propia red**; usarla sobre redes ajenas es, según dónde, un delito. Aquí, en casa, es tu censo de respaldo.

## 10.4. Cable o Wi-Fi

Un dato práctico que separa el portátil del servidor: por Wi-Fi, los paquetes comparten el aire con los vecinos, se pierden y tardan más de forma *irregular*; por cable, no. Y como todo aquí, se mide. Tu tarjeta Wi-Fi te dice cuánta señal recibe — en dBm, una unidad que reconoces:

```
marcos@portatil:~$ iw dev wlp3s0 link | grep signal
        signal: -58 dBm
marcos@portatil:~$ ping -c 20 192.168.1.1 | tail -2
20 packets transmitted, 19 received, 5% packet loss, time 19031ms
rtt min/avg/max/mdev = 1.9/6.4/48.2/11.7 ms
```

Un −58 dBm es buena señal (−30 sería pegado al router; −80, cobertura mala). Y fíjate en la línea de `ping`: un paquete perdido de veinte, y un tiempo que oscila entre 2 y 48 ms — ese `mdev` es la desviación típica, y por Wi-Fi es grande. Por cable serían 20 de 20, 1 ms, y `mdev` de décimas. Moraleja para el futuro: **un servidor va por cable**; y si tu `debian-lab` un día vive en un PC físico, ese PC tendrá un cable.

## 10.5. El protocolo del cambio reversible

Hoy tocas el router por primera vez, y lo haces con un método que vale para cualquier sistema que no tenga instantáneas — es decir, para casi todo en la vida real:

1. **Anota el valor actual** — captura de pantalla o línea en el cuaderno. Sin esto, no hay vuelta atrás posible.
2. **Escribe la vuelta atrás** *antes* de tocar: «para deshacer, ir a X y poner Y».
3. **Cambia una sola cosa.**
4. **Verifica** con un comando que el cambio hizo lo que esperabas.
5. **Decide**: se queda, o se ejecuta la vuelta atrás.

El cambio de hoy es uno, elegido por útil y reversible: una **reserva de dirección** — pedirle al DHCP del router que a tu portátil le dé siempre la misma IP, identificándolo por su MAC (en el capítulo 11 harás el segundo: el servidor de nombres que el router reparte a la casa). Y uno previo, si procede: si la contraseña de administración del router sigue siendo la de la pegatina, cámbiala — apuntando la nueva en un sitio seguro *antes* de pulsar guardar. Lo que **no** se toca hoy: la Wi-Fi, el rango del DHCP, y nada que diga «restablecer».

**Cuidado · Los dos cambios que dejan a la casa sin internet**

Cambiar la dirección del propio router (`192.168.1.1`) o desactivar su DHCP son cambios *legales* pero que cortan la red a todos los aparatos hasta que se arregla — y arreglarlo exige saber más de lo que sabes hoy. Ni siquiera con vuelta atrás escrita: no en el piloto. El curso los deja para quien administre un router en serio, con un segundo equipo por cable para rescatarlo.

## 10.6. Práctica: el censo de casa

Cuaderno en marcha (`script sesion10.log`). Todo es lectura salvo el ejercicio 10.4, que sigue el protocolo del cambio reversible.

**Nivel 1 · Lo esencial**

**Ejercicio 10.1 · Las dos caras**

Averigua tu dirección pública con `curl ifconfig.me` y compárala con la privada de `ip a`. Después, en la página del router, localiza la dirección *WAN* y comprueba que coincide con la pública.

*Pista.* Si `curl` no está instalado, ya sabes: `sudo apt install curl`.

*(Solución al final del capítulo.)*

**Ejercicio 10.2 · El censo**

En la página del router, encuentra la tabla de dispositivos conectados y pásala a tu cuaderno: nombre, IP y MAC de cada aparato. Identifica cuál es cada uno (portátil, móviles, tele, impresora…) y anota los que no reconozcas.

*Pista.* Si el router no la muestra o está incompleta, `ip neigh` en tu Mint es el plan B, y `nmap -sn` sobre tu `/24`, el plan C.

*(Solución al final del capítulo.)*

**Nivel 2 · El mecanismo**

**Ejercicio 10.3 · La VM en el mapa**

Dentro de `debian-lab`, ejecuta `ip a` e `ip route`. Compara con tu Mint: ¿cuál es la puerta de enlace de la VM? ¿Por qué la VM puede hacer `ping` a tu Mint y a `8.8.8.8`, pero el router de casa no la tiene en su censo?

*Pista.* Mira el `default via` de la VM y búscalo en el `ip a` de tu Mint.

*(Solución al final del capítulo.)*

**Ejercicio 10.4 · Una reserva en el router, con vuelta atrás**

Con el protocolo completo — valor actual anotado, vuelta atrás escrita, cambio, verificación —: crea en el router una **reserva DHCP** para tu portátil, de modo que siempre reciba la misma dirección (la que tiene ahora vale). Verifica reconectando la Wi-Fi y mirando `ip a`.

*Pista.* El menú se llama «Reserva de dirección», «DHCP estático» o «Asignación fija», en la sección LAN/DHCP; necesita tu MAC (`ip a`, línea `link/ether`). Si tu router no lo ofrece, deja constancia en el cuaderno y sigue: no todos lo exponen.

*(Solución al final del capítulo.)*

**Nivel 3 · El reto (opcional)**

**Ejercicio 10.5 · Wi-Fi contra cable, con datos**

Si tienes un cable de red a mano: mide con `ping -c 30` al router por Wi-Fi y por cable, y compara media, máximo, `mdev` y paquetes perdidos. Añade la señal Wi-Fi en dBm (`iw dev … link`). Escribe en el cuaderno una conclusión de dos líneas, como en un informe de prácticas.

*Pista.* `ip a` te dirá qué interfaz está activa en cada caso (`wlp…` o `enp…`); desactiva la Wi-Fi mientras mides por cable para no mezclar.

*(Solución al final del capítulo.)*

## 10.7. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. Tu portátil sabe su dirección IP porque…

   a) viene grabada de fábrica, como la MAC
   b) el servidor DHCP del router se la prestó al conectarse
   c) la eligió al azar

2. Una «reserva» en el DHCP del router sirve para…

   a) que un aparato reciba siempre la misma dirección, identificándolo por su MAC
   b) acelerar la Wi-Fi
   c) dar dirección pública a un aparato

3. ¿Cuál de estas es una dirección privada?

   a) `8.8.8.8`
   b) `10.20.30.40`
   c) `83.44.120.9`

4. Gracias al NAT del router…

   a) cada aparato de casa tiene su propia dirección pública
   b) toda la casa sale con una sola dirección pública, y desde fuera nadie entra sin invitación
   c) internet va más rápido

5. El router de casa no ve a tu máquina virtual en su censo porque…

   a) las VM no tienen red
   b) está detrás de un segundo NAT: el de tu Mint, que hace de router para ella
   c) el router bloquea a Debian

6. El primer paso del protocolo de cambio reversible es…

   a) hacer el cambio y ver qué pasa
   b) anotar el valor actual y escribir la vuelta atrás
   c) reiniciar el router

## Soluciones del capítulo

**Ejercicio 10.1 · Las dos caras**

Solución razonada

Privada `192.168.1.x`; pública algo como `83.44.120.9`, idéntica a la WAN del router. Si la WAN del router es *también* del tipo `100.64.x.x` o `10.x.x.x`, tu compañía te tiene detrás de un NAT adicional (se llama CG-NAT), y eso condicionará el reenvío de puertos del capítulo 11.
**Por qué así** — Ver las dos direcciones con tus propios ojos convierte el NAT de concepto en hecho: toda la casa, un número.

**Ejercicio 10.2 · El censo**

Solución razonada

Una tabla de entre 5 y 20 filas es lo normal en una casa. Los aparatos «misteriosos» suelen ser el propio router, un enchufe inteligente, la consola o el móvil de alguien de visita — los tres primeros bytes de la MAC identifican al fabricante, y buscarlos en internet («MAC lookup») resuelve casi todos.
**Por qué así** — Es el ejercicio central del capítulo: saber quién vive en tu red es el principio de saber administrarla. En el capítulo 10, con esta lista, podrás hacer los cambios con criterio.

**Ejercicio 10.3 · La VM en el mapa**

Solución razonada

Puerta de enlace de la VM: `192.168.122.1` — que es `virbr0`, tu propio Mint. La VM está en la red privada de libvirt; tu Mint hace de router y NAT para ella, igual que el de casa hace para tu Mint. El router de casa solo ve salir paquetes de `192.168.1.34`: la VM está escondida detrás, en un segundo NAT.
**Por qué así** — Es la figura del NAT aplicada dos veces, y la demostración de que un «router» no es una caja: es un *papel* que cualquier máquina Linux puede hacer. Tu Mint lleva haciéndolo desde el capítulo 7.

**Ejercicio 10.4 · Una reserva en el router, con vuelta atrás**

Solución razonada

La reserva asocia tu MAC a una IP; tras reconectar, `ip a` la muestra — y a partir de ahora tu `/etc/hosts` (capítulo 11) o el SSH del capítulo 12 pueden contar con ella. Vuelta atrás: borrar la reserva; tu portátil volverá a coger la que le toque del rango.
**Por qué así** — Un cambio pequeño, útil y deshecho en un minuto si molesta: exactamente el tipo de administración que se hace en una casa. Y el protocolo que has seguido es el mismo con el que tocarás un servidor de verdad — donde tampoco habrá instantáneas.

**Ejercicio 10.5 · Wi-Fi contra cable, con datos**

Solución razonada

Cable: 30 de 30, ~1 ms, `mdev` de décimas. Wi-Fi: alguna pérdida posible, media mayor y `mdev` de varios ms — más cuanto peor la señal (por debajo de −70 dBm empieza a notarse). Conclusión típica: «la media apenas cambia; la *dispersión* y las pérdidas sí — por eso el servidor va por cable».
**Por qué así** — Media, desviación y pérdidas: el mismo trío que resume cualquier medida de laboratorio. Has convertido una opinión de foro («el cable va mejor») en un resultado con incertidumbre.

**Autocomprobación** — Respuestas: 1 b · 2 a · 3 b · 4 b · 5 b · 6 b.

### La próxima sesión

Hoy has hablado con números; el capítulo 11 les pone nombre. Verás qué pasa exactamente al escribir `debian.org` — el DNS, la guía telefónica de internet —, seguirás el camino paquete a paquete hasta la universidad con `tracepath`, entenderás qué es exactamente un puerto y por qué SSH es el 22 y la web el 443, pondrás un cortafuegos básico, y harás el segundo cambio reversible en el router: el servidor de nombres. Guarda `sesion10.log`.

cap. 10 v0.2 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion10.log`

# 11 Nombres, puertos y protocolos

Tercer y último capítulo de redes. Hasta ahora has hablado con números; hoy descubres todo lo que pasa en el segundo que transcurre entre escribir `debian.org` y ver la página: la guía telefónica de internet, el camino que recorren tus paquetes salto a salto hasta la universidad, y por qué una máquina tiene una dirección pero miles de puertas. Cierras con un portero básico para tu servidor y con el segundo cambio reversible en el router.

**2–3 h** con ejercicios · requisitos: **capítulo 10** · máquina: **tu Linux de diario, la VM y el router (un cambio reversible)**

**Al terminar esta sesión sabrás**

- Qué es el DNS, quién responde cuando preguntas por un nombre, y cómo ver la respuesta con `host`.
- El fichero `/etc/hosts`: la guía telefónica más antigua, y cómo usarla para bautizar a tu máquina virtual.
- Seguir el camino de tus paquetes hasta la universidad con `tracepath`, salto a salto.
- Qué es exactamente un puerto TCP, qué cuatro números identifican una conexión, por qué SSH es el 22 y la web el 443, y cómo ver qué puertas tiene abiertas una máquina.
- TCP frente a UDP en diez minutos, y qué es un cortafuegos — el del router y el de tu máquina.
- Qué protocolo usar para mover datos según lo que haya al otro lado: Linux, Windows, un NAS, el laboratorio, la nube.

## 11.1. Qué pasa al escribir un nombre: DNS

Tu máquina solo sabe hablar con números; tú solo recuerdas nombres. Entre ambos está el **DNS** (*Domain Name System*): un servicio distribuido por todo el planeta que traduce `debian.org` en `151.101.66.132`. Lo usas cada vez que escribes un nombre, y `ping` te lo enseña sin pedírselo:

```
marcos@portatil:~$ ping -c 1 debian.org
PING debian.org (151.101.66.132) 56(84) bytes of data.
64 bytes from 151.101.66.132: icmp_seq=1 ttl=57 time=18.2 ms
marcos@portatil:~$ host debian.org
debian.org has address 151.101.66.132
marcos@portatil:~$ resolvectl status | grep "DNS Servers"
       DNS Servers: 192.168.1.1
```

`host` hace la pregunta a secas y te da la respuesta. Y `resolvectl status` te dice **a quién** se la hace tu máquina: a `192.168.1.1` — el router, que a su vez pregunta a los servidores de tu compañía. Es uno de los datos que el DHCP del capítulo 10 te prestó junto con la dirección.

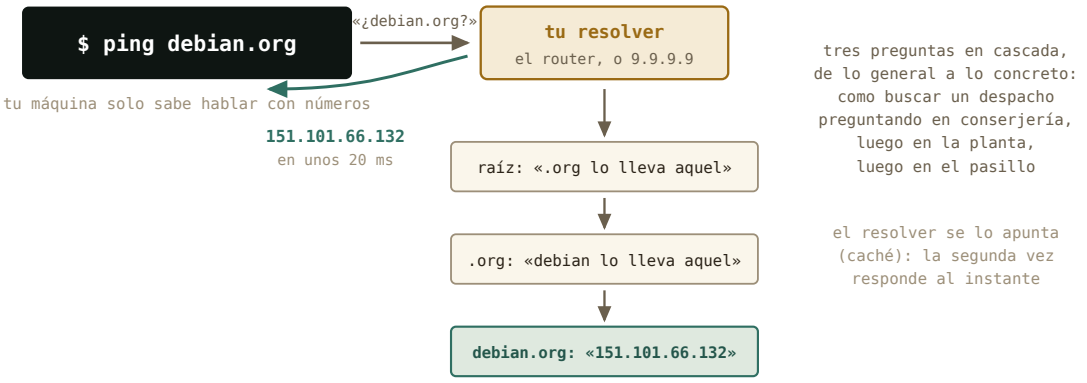



Figura 11.1: La cadena de preguntas, de lo general a lo concreto. Nadie sabe todas las direcciones del mundo; cada servidor sabe a quién preguntar.

¿Y antes del DNS? Un fichero de texto — en Linux siempre hay un fichero de texto — que sigue existiendo y que tu máquina consulta *antes* que a cualquier servidor: `/etc/hosts`.

```
marcos@portatil:~$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       portatil
```

Dos líneas: `localhost` eres tú, y `portatil` también. Lo interesante es lo que puedes *añadir*: una línea `192.168.122.15  debian-lab` y, desde ese momento, `ping debian-lab` — y en el capítulo 12, `ssh debian-lab` — funciona sin números. Es la guía telefónica privada de tu máquina, y la práctica de hoy la estrena.

**Historia · Cuando internet cabía en un folio**

En los años 70, la «guía» de ARPANET era literalmente un fichero, `HOSTS.TXT`, que una oficina de Stanford mantenía a mano y que cada máquina de la red descargaba periódicamente: nombre, dirección, y ya. Funcionó mientras internet tenía docenas de máquinas; en 1983, con cientos y creciendo, Paul Mockapetris diseñó el DNS — un sistema *distribuido* en el que nadie tiene la lista entera y cada uno responde por su parcela. Tu `/etc/hosts` es el descendiente directo de aquel fichero: la guía de bolsillo que sobrevivió a la guía telefónica.

## 11.2. El camino: `tracepath`

`ping` te decía *cuánto* tardaba un paquete; `tracepath` te enseña *por dónde* pasa. Cada router del camino es un **salto**, y la herramienta los va descubriendo uno a uno:

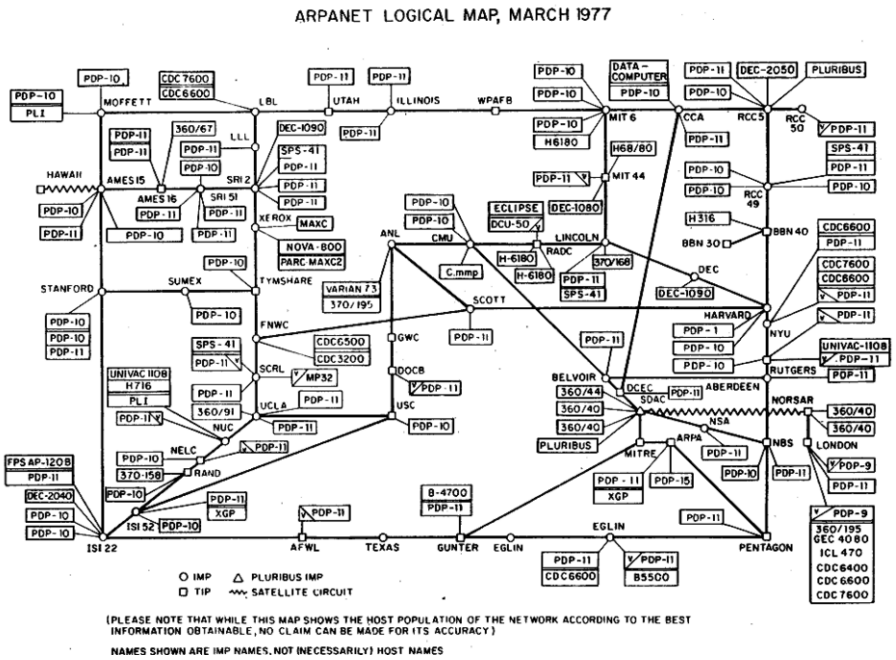



Figura 11.2: El mapa lógico de ARPANET en marzo de 1977: cada máquina de la red, en una hoja. Hoy no cabría en un país. *Dominio público, Wikimedia Commons.*

```
marcos@portatil:~$ tracepath -n uma.es
 1?: [LOCALHOST]                      pmtu 1500
 1:  192.168.1.1                        2.104ms
 2:  10.40.128.1                        9.870ms
 3:  81.46.12.201                      12.335ms
 ...
 9:  150.214.40.10                     21.482ms reached
```

El primer salto es siempre tu router; el segundo ya está en la compañía (fíjate: una dirección `10.x` — privada, capítulo 10 — que la operadora usa por dentro); y unos saltos después, la universidad. El tiempo crece con cada salto, y el mínimo lo pone la física de ayer. Cuando «no va internet», `tracepath` te dice *hasta dónde* llega: si se corta en el salto 1, el problema es tuyo; si en el 2 o el 3, de la compañía. (`traceroute`, su hermano clásico, hace lo mismo con más opciones — `apt install traceroute` si lo echas de menos.)

## 11.3. Una dirección, muchas puertas: los puertos

Vuelve a la analogía postal del capítulo 9: la IP es la calle; el **puerto** es el número de piso. Es, literalmente, un número de 16 bits — de 0 a 65 535 — escrito en el sobre de la capa de transporte (TCP o UDP), y sirve para que la máquina que recibe el paquete sepa a *qué programa* dárselo, porque en una máquina hay decenas corriendo a la vez. Un programa que quiere recibir le dice al kernel «lo que llegue al 22, es mío»: eso es **escuchar** en un

puerto, y solo un programa puede escuchar en cada número. Conectarse a una máquina es siempre conectarse a *dirección:puerto*, y los de los servicios clásicos están acordados desde hace décadas:

| Puerto | Servicio | Dónde lo verás |
|---|---|---|
| **22** | SSH | el capítulo 12 entero |
| **53** | DNS | cada nombre que resuelves |
| **80** / **443** | web sin cifrar / cifrada (https) | cada página que abres |
| **8888** | Jupyter | el caso estrella del capítulo 13 |

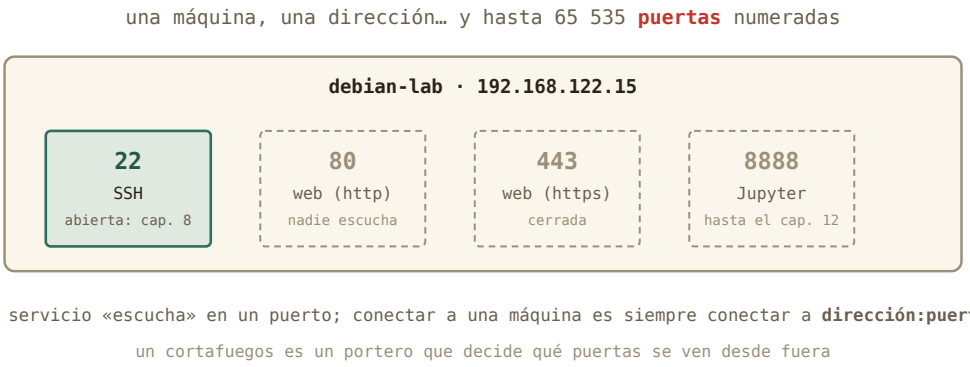



Figura 11.3: Los puertos de tu servidor: una sola puerta abierta, la que dejaste en la instalación.

Y puedes *mirar* las puertas. Desde dentro, `ss` (*socket statistics*) lista lo que escucha; desde fuera, el `nmap` que instalaste ayer toca a la puerta de cada una:

```
marcos@debian-lab:~$ ss -tln
State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
LISTEN  0       128           0.0.0.0:22         0.0.0.0:*
```

```
marcos@portatil:~$ nmap 192.168.122.15
PORT   STATE SERVICE
22/tcp open  ssh
```

Tu servidor, visto desde dentro y desde fuera, cuenta lo mismo: una puerta abierta, la 22, y detrás el servidor SSH que marcaste en el instalador. Mañana entrarás por ella.

Queda la pregunta que desconcierta a todo el mundo: si `sshd` escucha en *un* puerto, ¿cómo atiende a cien clientes a la vez sin mezclar sus conversaciones? Porque una **conexión** no la identifica un puerto, sino **cuatro números**: la IP y el puerto de *cada* extremo. El servidor pone los suyos (`192.168.122.15`, `22`); el cliente pone su IP y elige, para cada conexión, un puerto **efímero** libre y alto — el `51234` de la tabla NAT del capítulo 10 era exactamente

eso. Dos terminales tuyas conectadas al mismo servidor son dos cuartetos distintos (`…:51234 → …:22` y `…:51240 → …:22`), y el kernel del servidor reparte cada paquete a la conversación correcta por su cuarteto. Es también lo que permite que tu navegador tenga diez pestañas abiertas contra el mismo servidor, y lo que hacía posible la libreta del router: sin puertos distintos, el NAT no tendría cómo distinguir tu portátil de tu móvil.

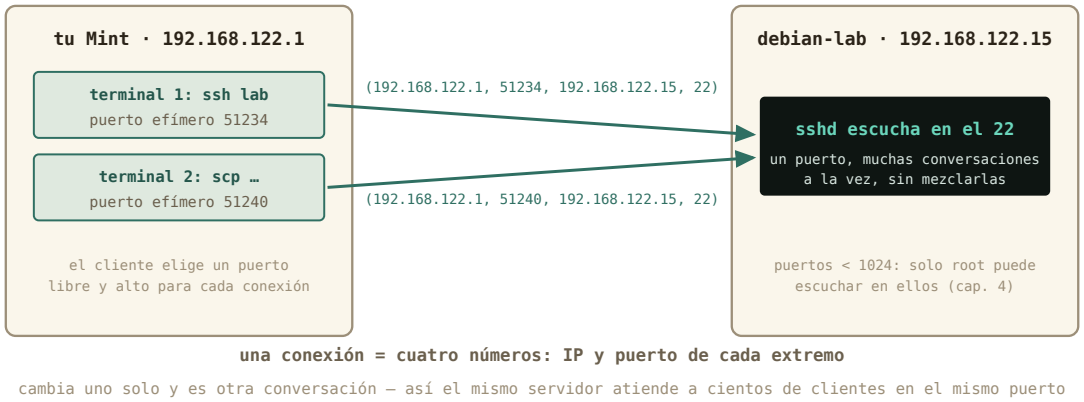



Figura 11.4: Cuatro números por conexión. El servidor repite los suyos; el cliente estrena un puerto efímero cada vez.

Y se ve en vivo: `ss -tn` (sin la `l`) lista las conexiones *establecidas* con sus dos extremos, cuarteto completo:

```
marcos@portatil:~$ ss -tn
State  Recv-Q Send-Q Local Address:Port    Peer Address:Port
ESTAB  0      0      192.168.122.1:51234   192.168.122.15:22
ESTAB  0      0      192.168.1.34:44810    151.101.66.132:443
```

Tu sesión SSH con la VM y una pestaña del navegador, cada una con su puerto efímero. Un último detalle que conecta con el capítulo 4: los puertos **por debajo del 1024** — los de los servicios clásicos — solo puede reservarlos `root`. Así, un usuario cualquiera no puede poner un programa suyo a escuchar en el 22 y hacerse pasar por el SSH de la máquina.

### Por dentro · TCP y UDP en diez minutos

Los paquetes viajan de dos maneras. **TCP** es correo certificado: antes de enviar nada, los dos extremos se saludan en tres pasos — «¿estás?», «estoy, ¿y tú?», «aquí» (el *handshake*: SYN, SYN-ACK, ACK) — y a partir de ahí cada paquete va numerado, el receptor confirma lo que recibe, lo que se pierde se reenvía y todo se entrega en orden. Más lento, pero fiable: SSH, la web y el correo van así. **UDP** es una postal: se envía y ya, sin conexión ni confirmación — rapidísimo, y si una se pierde, se pierde. Las consultas DNS, el vídeo en directo y los juegos van así: prefieren perder un paquete a esperar. Por eso `nmap` decía `22/tcp`: el puerto 22 *de TCP*. Los dos sistemas conviven en la misma dirección con numeraciones independientes —

el 53 de UDP (DNS) y el 53 de TCP son puertas distintas, aunque el mismo programa suela atender las dos.

## 11.4. Cortafuegos, lo básico

Un **cortafuegos** (*firewall*) es un portero: mira cada paquete que llega y decide, por reglas, si pasa. Tienes dos, y conviene saber qué hace cada uno.

El **del router** lo llevas puesto sin saberlo: la libreta NAT del capítulo 10 ya deja fuera todo lo que nadie pidió, y encima el router lleva un cortafuegos que vigila el *estado* de cada conversación (lo llaman SPI): solo entra lo que es respuesta a algo que salió. Abrir una puerta desde fuera es escribir una regla — el reenvío de puertos — y hacerlo a conciencia. Para una casa, ese portero basta.

El **de cada máquina** es la segunda línea, y en Linux se llama `ufw` (*uncomplicated firewall*): una lista de reglas del tipo «lo que entre, se rechaza, salvo lo que yo diga». En tu Mint viene instalado y apagado — detrás del router no hace falta. En un servidor con dirección pública, en cambio, es obligatorio, y la forma de ponerlo es siempre la misma, **en este orden**:

```
marcos@debian-lab:~$ sudo apt install ufw
marcos@debian-lab:~$ sudo ufw allow OpenSSH
marcos@debian-lab:~$ sudo ufw enable
marcos@debian-lab:~$ sudo ufw status verbose
Status: active
Default: deny (incoming), allow (outgoing)
To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
```

Primero se abre la puerta por la que entras — SSH —, *después* se enciende: al revés, te dejas fuera de tu propio servidor. La política por defecto lo dice todo: entrar, nada salvo lo permitido; salir, todo. Cada servicio nuevo será una línea (`sudo ufw allow 8888/tcp` si algún día abres un Jupyter a la red, que no lo harás: capítulo 12). Y `nmap` desde tu Mint es la comprobación: la 22 abierta, el resto `filtered`, que es el portero diciendo que no.

**Truco · El cortafuegos con ventana**

Mint trae la cara gráfica de `ufw`: menú → Preferencias → **Configuración del cortafuegos** (`gufw`). Un interruptor de encendido, tres perfiles (casa, oficina, público) y una lista de reglas donde se añade «permitir SSH» eligiéndolo de una lista de aplicaciones. Es exactamente el mismo `ufw` — las reglas que crees ahí las verás con `sudo ufw status` — y para un portátil que se conecta a la Wi-Fi de una cafetería, activarlo con el perfil *público* es un gesto razonable y reversible.

## 11.5. Qué habla cada puerta: los protocolos que usarás

Detrás de cada puerto hay un **protocolo**: el idioma acordado entre los dos extremos. Para un físico, la pregunta práctica es «¿cómo paso datos de aquí a allí?», y la respuesta depende de qué hay en el otro extremo. Esta es la tabla que resuelve el noventa por ciento de los casos, con el capítulo o anexo donde se practica cada uno:

| Protocolo | Puerto | Otro extremo típico | Con qué lo usarás | Dónde |
|---|---|---|---|---|
| **SSH / SFTP** | 22 | otro Linux, un servidor, la nube — y un Windows moderno | terminal, `scp`, `rsync`, «Conectar al servidor», WinSCP | caps. 11–12, anexo E |
| **SMB** | 445 | un PC con Windows, un NAS, una impresora | «Conectar al servidor» (`smb://`), montar por `cifs` | anexo E |
| **NFS** | 2049 | otro Linux, el laboratorio, un clúster | montar con `-t nfs` | anexo E |
| **HTTP / HTTPS** | 80 / 443 | la web, la nube, Nextcloud, Jupyter | el navegador, `curl`, `rclone` | caps. 9, 12, anexo E |
| **FTP** | 21 | servidores antiguos de datos públicos | «Conectar al servidor» (`ftp://`) | — sin cifrar: solo lectura y solo si no hay otra |

Una regla para elegir: **si hay SSH en el otro extremo, usa SSH** — sirve para terminal y para ficheros, va cifrado, y lo hablan hoy hasta los Windows. SMB es el idioma de Windows y de los NAS; NFS, el del laboratorio; HTTPS, el de la nube. Todo lo demás son variantes.

## 11.6. Práctica: la guía y las puertas

Cuaderno en marcha (`script sesion11.log`). El cambio en el router se hace con el protocolo del capítulo 10; el cortafuegos, en la VM y con instantánea.

**Nivel 1 · Lo esencial**

**Ejercicio 11.1 · El DNS en acción**

Con `host`, averigua la dirección de `uma.es`, de `debian.org` y de `google.com` (esta última te dará varias: anota cuántas). Averigua también a qué servidor le pregunta tu máquina (`resolvectl status`).

*Pista.* `host nombre` a secas; si `host` no existe, `sudo apt install bind9-host`.

*(Solución al final del capítulo.)*

**Ejercicio 11.2 · Bautizar a la VM**

Añade a `/etc/hosts` de tu Mint una línea con la IP de `debian-lab` (la de `ip a` dentro de la VM) y su nombre, y comprueba que `ping debian-lab` funciona. Antes de editar, aplica el protocolo: copia de seguridad del fichero y vuelta atrás escrita.

*Pista.* `sudo cp /etc/hosts /etc/hosts.bak`, `sudo nano /etc/hosts`, y una línea como `192.168.122.15  debian-lab` al final. Vuelta atrás: `sudo cp /etc/hosts.bak /etc/hosts`.

*(Solución al final del capítulo.)*

**Ejercicio 11.3 · El camino a la universidad**

Ejecuta `tracepath -n uma.es` (o el dominio de tu universidad). ¿Cuántos saltos hay? ¿Cuál es el primero? ¿En qué salto aparece la primera dirección que *no* es privada — es decir, dónde sale tu paquete a internet?

*Pista.* Privadas son `10.x`, `192.168.x` y `172.16-31.x`; repasa la figura del NAT del capítulo 10.

*(Solución al final del capítulo.)*

**Ejercicio 11.4 · Las puertas de tu servidor**

Desde dentro de `debian-lab`, lista los puertos que escuchan (`ss -tln`); desde tu Mint, tócalos desde fuera (`nmap debian-lab` — ya tiene nombre). ¿Coinciden las dos vistas?

*Pista.* `-t` TCP, `-l` solo los que escuchan, `-n` números en vez de nombres.

*(Solución al final del capítulo.)*

**Nivel 2 · El mecanismo**

**Ejercicio 11.5 · El servidor de nombres, con vuelta atrás**

Con el protocolo del capítulo 10 — valor actual anotado, vuelta atrás escrita, cambio, verificación —: cambia el **servidor DNS** que tu router reparte a la casa por `9.9.9.9` (Quad9) o `1.1.1.1` (Cloudflare), reconecta la Wi-Fi y verifica con `resolvectl status` que tu máquina pregunta ahora al nuevo. Comprueba que `host debian.org` sigue resolviendo.

*Pista.* El DNS suele estar en la pestaña LAN/DHCP o en WAN/Internet del router; el valor anterior será a menudo «automático» o la IP de la compañía — anótalo tal cual.

*(Solución al final del capítulo.)*

**Ejercicio 11.6 · Un portero para `debian-lab`**

Con instantánea previa de la VM: instala `ufw`, permite SSH, enciende el cortafuegos, y comprueba desde tu Mint con `nmap debian-lab` que la 22 sigue abierta y el resto aparece como `filtered`. Después apágalo (`sudo ufw disable`) o restaura la instantánea.

*Pista.* El orden importa — `allow OpenSSH` antes de `enable`, o te quedas fuera; y ten una sesión SSH abierta de reserva mientras pruebas.

*(Solución al final del capítulo.)*

## 11.7. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. El DNS sirve para…

   a) traducir nombres como `debian.org` en direcciones IP
   b) repartir direcciones IP a los aparatos de casa
   c) cifrar la conexión

2. Añades `192.168.122.15 debian-lab` a `/etc/hosts`. Al hacer `ping debian-lab`…

   a) tu máquina pregunta al DNS y falla, porque ese nombre no existe en internet
   b) tu máquina usa la línea de `/etc/hosts` sin preguntar a ningún servidor
   c) el router aprende el nombre

3. En `tracepath`, el primer salto es siempre…

   a) el servidor de destino
   b) tu router, la puerta de enlace
   c) un servidor de la compañía

4. Una conexión TCP la identifican…

   a) solo el puerto del servidor
   b) cuatro números: la IP y el puerto de cada extremo
   c) la dirección MAC de las dos máquinas

5. SSH va sobre TCP y no sobre UDP porque…

   a) TCP garantiza que todo llega, en orden, reenviando lo perdido
   b) UDP no funciona en Linux
   c) TCP es más rápido

6. Al poner un cortafuegos `ufw` en un servidor al que entras por SSH, el orden correcto es…

   a) `ufw enable` y luego `ufw allow OpenSSH`
   b) `ufw allow OpenSSH` y luego `ufw enable`
   c) da igual: `ufw` nunca corta SSH

## Soluciones del capítulo

**Ejercicio 11.1 · El DNS en acción**

Solución razonada

`host uma.es`, `host debian.org`, `host google.com` — Google devuelve varias direcciones (y una de tipo IPv6, más larga: es la nueva numeración de internet, que hoy dejamos de lado). El resolver será tu router, `192.168.1.1`, salvo que tu compañía configure otro.
**Por qué así** — Ver que un nombre tiene *varias* direcciones desmonta la idea de «la dirección de Google»: son granjas de servidores, y el DNS reparte. Y saber a quién preguntas es el primer paso para cambiarlo con criterio en el 10.5.

**Ejercicio 11.2 · Bautizar a la VM**

Solución razonada

Tras guardar, `ping -c 1 debian-lab` responde desde `192.168.122.15` sin haber preguntado a ningún DNS: `/etc/hosts` se consulta primero. Si la IP de la VM cambiara algún día (el DHCP de libvirt suele mantenerla, pero no lo jura), basta actualizar la línea.
**Por qué así** — Es tu primera edición de un fichero de `/etc` con `sudo` — capítulo 4 — hecha con red: copia antes, vuelta atrás escrita. Y el premio llega mañana: `ssh debian-lab` en vez de una ristra de números.

**Ejercicio 11.3 · El camino a la universidad**

Solución razonada

Entre 8 y 15 saltos es lo normal. El primero es `192.168.1.1`, tu router; a menudo el segundo o tercero sigue siendo privado (la red interna de la compañía), y la primera pública marca la salida real a internet. Algún salto puede aparecer como `no reply`: routers que no contestan por política — no es un fallo.
**Por qué así** — Tu paquete acaba de contarte su viaje. La próxima vez que «no vaya internet», este comando te dirá en qué salto muere — y por tanto a quién llamar.

**Ejercicio 11.4 · Las puertas de tu servidor**

Solución razonada

Dentro: `LISTEN` en `0.0.0.0:22` (y quizá `[::]:22`, su versión IPv6). Fuera: `22/tcp open ssh`. Una puerta, dos puntos de vista, el mismo resultado — y `nmap` funcionando ya por nombre gracias al 10.2.
**Por qué así** — Comparar la vista interior con la exterior es el diagnóstico básico de cualquier servicio: si dentro escucha pero fuera no se ve, hay un portero en medio (cortafuegos o NAT). Hoy no lo hay; en la universidad lo habrá.

**Ejercicio 11.5 · El servidor de nombres, con vuelta atrás**

Solución razonada

`resolvectl status` pasa de `192.168.1.1` (o el de la compañía) a `9.9.9.9`; `host debian.org` responde igual que antes. Vuelta atrás: devolver el campo al valor anotado.
**Por qué así** — Es un cambio pequeño, deshecho en un minuto, y con una consecuencia real: quién ve las preguntas de tu casa. Y es tu segundo cambio en el router con el protocolo completo — a partir de aquí, cualquier ajuste doméstico se hace así.

**Ejercicio 11.6 · Un portero para debian-lab**

Solución razonada

`sudo apt install ufw`, `sudo ufw allow OpenSSH`, `sudo ufw enable`, `sudo ufw status verbose` → `deny (incoming)`, `allow (outgoing)`, `22/tcp ALLOW IN`. Desde Mint, `nmap debian-lab` muestra `22/tcp open` y nada más (los demás puertos, filtrados).
**Por qué así** — Detrás del NAT de casa la VM no lo necesita; en un servidor con dirección pública es la primera línea. Haberlo puesto una vez, en el orden correcto y con red de seguridad, es lo que hace que el día que importe no te dejes fuera.

**Autocomprobación** — Respuestas: 1 a · 2 b · 3 b · 4 b · 5 a · 6 b.

**La próxima sesión**

Empieza el bloque E: trabajar en remoto. Tienes un servidor con nombre (`debian-lab`), una puerta abierta (la 22) y, desde hoy, el mapa entero de cómo llegar a ella. En el capítulo 12 entras: primero con ratón, desde el gestor de archivos; después con `ssh` y tu contraseña; y para quien quiera ir más allá, claves en vez de contraseñas y copiar ficheros entre máquinas con `scp` y `rsync`. Guarda `sesion11.log`: bloque D completo.

cap. 11 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion11.log`

# 12 SSH: entrar en otra máquina

Bloque E: trabajar en remoto. Todo lo anterior conducía aquí — un servidor instalado por ti, con nombre, con una puerta abierta, y el mapa de cómo llegar. Hoy entras por esa puerta. Aprendes el comando con el que un físico computacional pasa media vida, sustituyes las contraseñas por claves — más seguras y más cómodas a la vez —, y mueves ficheros entre máquinas como si estuvieran en la misma mesa. Primero contigo mismo, luego con tu VM, y después con cualquier servidor del mundo.

**2–3 h** con ejercicios · requisitos: **capítulos 8 y 10** · máquina: **tu Linux de diario y `debian-lab`** · nivel: **el capítulo más avanzado del curso — lo esencial cabe en las dos primeras secciones**

**Al terminar esta sesión sabrás**

- Conectarte a otro ordenador *con ratón*, desde el gestor de archivos: SSH, Samba, FTP y NFS por el mismo diálogo.
- Qué son el cliente y el servidor SSH, y qué pasa exactamente al conectar por primera vez (la huella, `known_hosts`).
- Convertir tu propia máquina en servidor con `openssh-server`, y entrar en ella desde ella misma.
- Crear un par de claves ed25519, instalarlo en el servidor con `ssh-copy-id` y no volver a teclear una contraseña.
- Bautizar servidores en `~/.ssh/config` para conectar con una palabra.
- Copiar ficheros entre máquinas con `scp` y sincronizar carpetas con `rsync`.
- Ejecutar un comando en otra máquina sin abrir sesión — y por qué eso cambia tu forma de trabajar.
- Qué cambia cuando el servidor es de verdad: la universidad, una nube, o tu casa desde fuera.

**Aviso · Este capítulo tiene dos velocidades**

Lo **esencial** — y suficiente para el proyecto final — son las dos primeras secciones: conectarte a otro ordenador con el gestor de archivos, y abrir una sesión SSH con tu contraseña. Todo lo demás — claves, agenda, `rsync`, comandos remotos, endurecer el servidor — es lo que usa un profesional a diario, pero es **avanzado**: si en algún punto te pierdes, quédate con lo esencial, sigue al capítulo 13 (que solo necesita saber entrar) y vuelve aquí cuando el resto te haga falta. Nadie aprende SSH entero en una tarde — y cuando vuelvas, hazlo con una IA al lado, como cuenta el anexo A: lo avanzado se conoce para entender lo que la IA te proponga, no para teclearlo de memoria.

## 12.1. Primero con ratón: otro ordenador en tu gestor de archivos

Antes de teclear un solo comando, la versión que ya sabes usar. Tu gestor de archivos — Nemo en Mint (Caja, Nautilus o Thunar en otros escritorios) — sabe abrir carpetas que viven en *otra máquina*, y lo hace con un solo diálogo para todos los protocolos que te vas a encontrar. Menú **Archivo → Conectar al servidor…** y verás un desplegable con el tipo de conexión:

| Tipo | Qué es | Cuándo te lo encontrarás |
|---|---|---|
| **SSH** (SFTP) | ficheros a través de una sesión SSH cifrada | cualquier máquina Linux con SSH — tu `debian-lab`, el clúster |
| **Windows / Samba** (SMB) | el «compartir carpeta» de Windows y de los NAS | la impresora de casa, el disco de red, el PC con Windows |
| **FTP** | el protocolo veterano de transferir ficheros, sin cifrar | servidores públicos de datos, alojamientos web antiguos |
| **WebDAV** | ficheros sobre una web | nubes universitarias, Nextcloud |
| **NFS** (escribiendo `nfs://…` en la barra de dirección) | el compartir de Unix | laboratorios y clústeres (anexo E) |

Pruébalo ahora con lo que ya tienes: tipo **SSH**, servidor `debian-lab` (el nombre que le diste en el capítulo 11), carpeta `/home/marcos`, usuario `marcos`, y tu contraseña. La casa del servidor aparece en la barra lateral como una carpeta más: arrastra un fichero, ábrelo, edítalo, bórralo — todo viaja por el servidor SSH que dejaste instalado en el capítulo 8. Eso que acaba de pasar *es* SSH; el resto del capítulo es el mismo mecanismo visto desde la terminal, donde además de mover ficheros podrás *ejecutar* cosas en la otra máquina. Si hoy no pasas de esta sección, ya sabes conectarte a otro ordenador — que era el objetivo.

## 12.2. Cliente y servidor

**SSH** (*Secure Shell*) es un shell — el del capítulo 1 — que corre en *otra* máquina, con tu teclado y tu pantalla conectados a él por la red, cifrado de punta a punta. Tiene dos mitades: el **servidor** (`sshd`, el que escucha en el puerto 22 que viste ayer) y el **cliente** (`ssh`,

el comando que tecleas). Tu `debian-lab` lleva el servidor desde la instalación; tu Mint trae el cliente de serie. Y gracias al `/etc/hosts` de ayer, la máquina tiene nombre:

```
marcos@portatil:~$ ssh marcos@debian-lab
The authenticity of host 'debian-lab (192.168.122.15)' can't be established.
ED25519 key fingerprint is SHA256:kX9v2Lm4pQ7rT1sW8yZaBcDeFgHiJkLmNoPqRsTuVwX.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'debian-lab' (ED25519) to the list of known hosts.
marcos@debian-lab's password:
Linux debian-lab 6.12.0-amd64 #1 SMP Debian 6.12.41-1 x86_64
marcos@debian-lab:~$ hostname
debian-lab
marcos@debian-lab:~$ exit
Connection to debian-lab closed.
marcos@portatil:~$
```

Léelo con calma, porque cada línea enseña algo. La pregunta de la **huella** (*fingerprint*) solo aparece la primera vez: SSH no conoce a esa máquina y te pide que confirmes que es quien dice ser; al decir `yes`, apunta su huella en `~/.ssh/known_hosts` y a partir de ahí la reconoce sola. (Si algún día esa huella *cambia*, SSH se negará a conectar con un aviso en mayúsculas: o han reinstalado el servidor, o alguien se está haciendo pasar por él. Nunca se ignora.) Después, la contraseña de tu usuario *en el servidor*, y el prompt cambia: `marcos@debian-lab`. Tu terminal está ahora en la otra máquina — cada comando que teclees se ejecuta allí, con sus ficheros, sus permisos, su `apt`. `exit` te devuelve a casa. Ese cambio de prompt es tu brújula del bloque E: mira siempre dónde estás antes de teclear.

**Historia · Del teléfono al cifrado**

Entrar en un ordenador lejano es más viejo que internet: en los 70 se hacía colgando el auricular del teléfono en un *acoplador acústico* que convertía los bits en pitidos, a 300 bits por segundo — un folio de texto por minuto. Los protocolos de entonces, `telnet` y `rlogin`, enviaban tu contraseña tal cual por la línea; nadie pensó que importara hasta que importó: en 1995, tras descubrir que alguien había pinchado la red de la Universidad de Helsinki y estaba recolectando contraseñas, el investigador Tatu Ylönen escribió en unos meses un sustituto cifrado y lo liberó. Lo llamó Secure Shell. Cuatro años después el proyecto OpenBSD lo reescribió como software libre — OpenSSH — y es el que corre hoy en tu `debian-lab`, en el clúster de la universidad y en cada servidor de cada nube.

## 12.3. Nivel 0: tu propia máquina como servidor

Antes de irte lejos, el experimento más corto posible: convertir tu Mint en servidor y entrar en él *desde él mismo*. Suena absurdo, pero tiene dos utilidades — practicar sin red de por medio, y abrir tu portátil a otras máquinas de tu casa (o a tu móvil, con cualquier app de SSH):



Figura 12.1: Un acoplador acústico de los años 70: el auricular del teléfono se encajaba en las dos ventosas, y por ahí viajaba tu sesión — a 300 bits por segundo y sin cifrar. *Foto: G. Edward Johnson, dominio público, Wikimedia Commons.*

```
marcos@portatil:~$ sudo apt install openssh-server
marcos@portatil:~$ systemctl status ssh | head -3
  ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled)
     Active: active (running) since Tue 2026-08-25 18:40:02 CEST
marcos@portatil:~$ ssh marcos@localhost
marcos@localhost's password:
marcos@portatil:~$ exit
```

`localhost` es el `127.0.0.1` del capítulo 9: tú mismo. El prompt no cambia — estás en la misma máquina — pero la sesión es una sesión SSH completa, cifrada y con servidor en medio.

## 12.4. Claves en vez de contraseñas

Teclear la contraseña en cada conexión es incómodo, y en un servidor expuesto a internet es peligroso: hay robots probando contraseñas en el puerto 22 de todas las máquinas del mundo, a todas horas. La solución es a la vez más cómoda y más segura, y es física de la buena: un **par de claves**. La **privada** se queda en tu máquina y no sale de ella jamás; la **pública** se regala a cada servidor en el que quieras entrar. El servidor te lanza un reto que solo la privada puede resolver, comprueba la respuesta con la pública, y te deja pasar. Ninguna contraseña viaja; ningún secreto se copia.

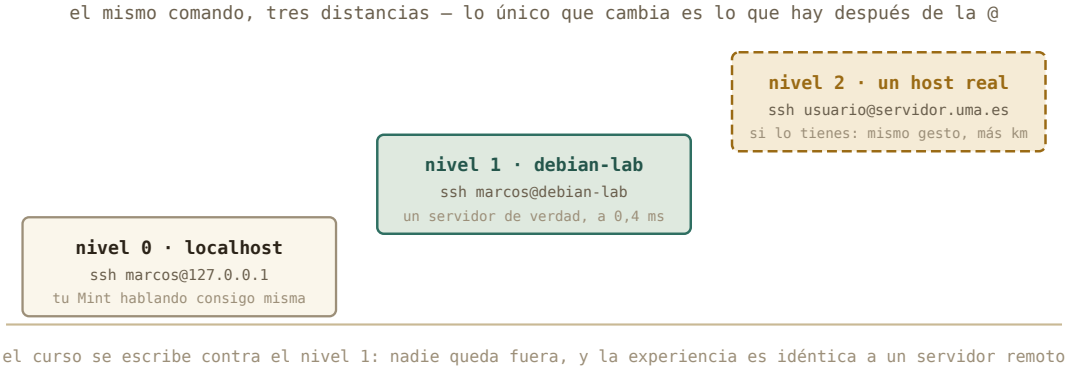



Figura 12.2: Los tres niveles del capítulo. El curso se escribe contra el nivel 1; el 2 es para quien tenga dónde.

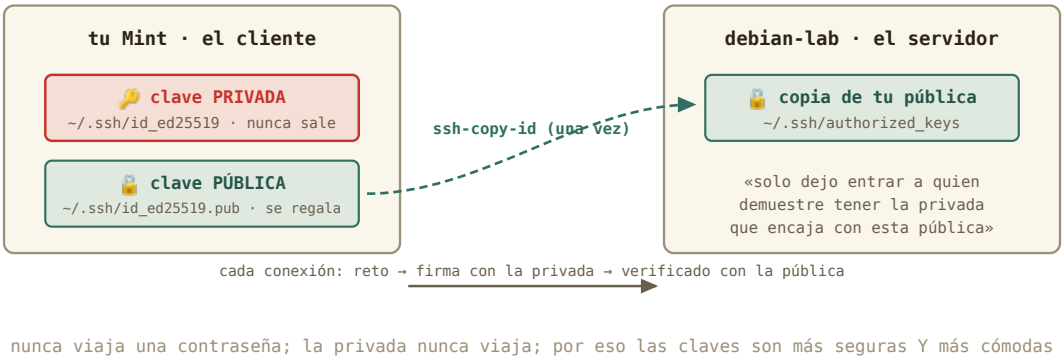



Figura 12.3: Privada en casa, pública en el servidor. La cerradura se comparte; la llave, nunca.

Tres comandos y no vuelves a teclear una contraseña:

```
marcos@portatil:~$ ssh-keygen -t ed25519 -C "marcos@portatil"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/marcos/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Your identification has been saved in /home/marcos/.ssh/id_ed25519
Your public key has been saved in /home/marcos/.ssh/id_ed25519.pub
marcos@portatil:~$ ssh-copy-id marcos@debian-lab
marcos@debian-lab's password:
Number of key(s) added: 1
marcos@portatil:~$ ssh marcos@debian-lab
marcos@debian-lab:~$
```

`ssh-keygen` crea el par (`ed25519` es el algoritmo moderno: claves cortas, rápidas, seguras). Pon una **frase de paso** (*passphrase*) cuando la pida: es la contraseña *de la clave*, por si alguien te roba el portátil; tu escritorio la recuerda durante la sesión, así que solo la escribirás una vez al día. `ssh-copy-id` hace el reparto: entra con tu contraseña una última vez y deja tu clave pública en el servidor, en `~/.ssh/authorized_keys`. Y la tercera línea es el premio: dentro sin preguntar nada.

**Por dentro · Los permisos vuelven a mandar**

Mira en `debian-lab`: `ls -la ~/.ssh` → `drwx------` para la carpeta y `-rw-------` para `authorized_keys`. Y en tu Mint, `ls -l ~/.ssh/id_ed25519` → `-rw-------`. SSH **se niega a usar claves que otros puedan leer**: si un día tu clave «deja de funcionar» tras copiarla o restaurarla, el diagnóstico es casi siempre `chmod 600` sobre la privada y `chmod 700` sobre `~/.ssh`. El capítulo 4 no era teoría: es la razón por la que tu llave está a salvo de los demás usuarios de la máquina.

## 12.5. Un nombre para cada servidor: `~/.ssh/config`

Con varias máquinas — la VM, la universidad, un servidor de cálculo — recordar usuarios, direcciones y puertos cansa. `~/.ssh/config` es la agenda de SSH: un fichero de texto (¿qué si no?) que crea alias. `nano ~/.ssh/config`:

```
Host lab
    HostName debian-lab
    User marcos

Host uni
    HostName servidor.uma.es
    User m.garcia
    Port 22
```

Desde ahora, `ssh lab` basta — y también vale para `scp` y `rsync`. Cuando la universidad te dé una cuenta, será una entrada más en esta agenda, y todo lo que sigue funcionará igual con `uni` que con `lab`.

## 12.6. Mover ficheros: `scp` y `rsync`

Copiar entre máquinas usa la misma gramática que `cp` del capítulo 2, con un `máquina:` delante de la ruta remota:

```
marcos@portatil:~/Descargas$ scp adquisicion.log lab:~/
adquisicion.log                           100%  298KB  41.2MB/s   00:00
marcos@portatil:~/Descargas$ scp -r experimento-pendulo lab:~/
marcos@portatil:~/Descargas$ scp lab:~/informe.txt .
```

Subir, subir una carpeta (`-r`), bajar: tres formas del mismo gesto. Para carpetas grandes que cambian — los resultados de una simulación que vas recogiendo — el profesional es **`rsync`**: copia solo lo que ha cambiado, se puede interrumpir y reanudar, y muestra el progreso:

```
marcos@portatil:~$ rsync -av --progress simulaciones/ lab:~/simulaciones/
sending incremental file list
run07.csv
        1,204,112 100%   38.12MB/s    0:00:00
sent 1,204,589 bytes  received 42 bytes  2,409,262.00 bytes/sec
```

`-a` conserva fechas y permisos, `-v` cuenta lo que hace. Y un matiz que muerde a todo el mundo una vez: **la barra final importa**. `simulaciones/` (con barra) sincroniza *el contenido* de la carpeta; `simulaciones` (sin barra) copia *la carpeta entera* dentro del destino. La segunda vez que ejecutes el mismo `rsync` tardará un segundo: solo viaja lo nuevo.

**Truco · Arrastrar y soltar sigue valiendo**

La conexión con ratón de la primera sección es SFTP — el primo de `scp` que viaja por el mismo túnel — y también se abre escribiendo `sftp://debian-lab/home/marcos` en la barra de direcciones (Ctrl+L). Para un fichero suelto es perfecto; `rsync` gana cuando son mil ficheros o hay que repetir la copia cada día. Y si prefieres que la casa del servidor aparezca como una carpeta *de verdad* en tu árbol — para abrirla desde cualquier programa —, eso se llama montar por SSHFS, y está en el anexo E.

## 12.7. Ejecutar allí, sin quedarse

El detalle que convierte SSH de «terminal lejana» en herramienta de trabajo: puedes pedirle a otra máquina que ejecute **un comando** y te devuelva la salida, sin abrir sesión:

```
marcos@portatil:~$ ssh lab hostname
debian-lab
marcos@portatil:~$ ssh lab "grep -c ERROR adquisicion.log"
108
marcos@portatil:~$ ssh lab df -h / | tail -1
/dev/vda1         18G   1,4G   16G   9% /
```

Ahí están los tres bloques anteriores dándose la mano: el `grep` del capítulo 3 corriendo en el servidor del capítulo 8, con el resultado llegando a tu terminal por la red del bloque D — y ese `| tail -1` se ejecuta *en tu Mint*, sobre lo que el servidor devolvió. Cuando tengas una simulación en un clúster, «¿ha terminado?» será `ssh cluster ls resultados/`, sin moverte de tu editor.

## 12.8. Nivel 2: cuando el servidor es de verdad

Todo lo anterior funciona idéntico con un servidor a mil kilómetros; solo cambia lo que hay después de la `@`. Dónde conseguir uno, mientras el curso se escribe (comprueba disponibilidad y condiciones, que cambian):

| Vía | Qué es | Coste |
|---|---|---|
| **Universidad o alguien conocido** | el servicio de informática, tu grupo, o el servidor de un profesor: una máquina Linux con SSH | gratis; pídela — es la mejor opción |
| **Azure for Students / GitHub Student Developer Pack** | con tu correo de la universidad: créditos para VMs *reales* con IP pública (Azure 100 $, DigitalOcean 200 $ y más), sin tarjeta | gratis para estudiantes |
| **Oracle Cloud «Always Free»** | una VM real y permanente (ARM, hasta 4 núcleos) con IP pública | gratis; pide tarjeta para identificarte, y a veces no hay plazas |
| **tilde.town, SDF** | un servidor Unix compartido por SSH; la cuenta se pide *entregando tu clave pública* — el ejercicio 12.3 en la vida real | gratis, sin tarjeta; sin `sudo`: para practicar |
| **GitHub Codespaces** | un contenedor Ubuntu con terminal (60 h/mes en la cuenta gratuita); se entra con `gh codespace ssh` | gratis; es un contenedor, no una VM |
| **Google Cloud Shell** | una terminal en el navegador con 5 GB de casa persistente; `gcloud cloud-shell ssh` desde tu Mint | gratis, sin tarjeta; la máquina es efímera |

(Las condiciones de estos servicios cambian cada pocos meses: comprueba disponibilidad y letra pequeña antes de contar con uno.) Las diferencias prácticas: la dirección será pública o un nombre DNS; el puerto puede no ser el 22 (`-p 2222`, o `Port` en tu `config`); te pedirán tu **clave pública** por adelantado — la que ya sabes generar — y quizá un segundo factor. Y la lección de seguridad que cierra el capítulo: un servidor a la vista de internet recibe miles de intentos de contraseña al día. Se sobrevive con claves y con la contraseña *desactivada* — exactamente lo que harás en el ejercicio 11.6, en tu VM y con instantánea.

**Cuidado · La clave privada es tu identidad**

Tres reglas sin excepción. Uno: la **privada nunca se copia** a otro sitio — ni al servidor, ni a un correo, ni «para tenerla en el portátil del laboratorio»: allí se genera otro par. Dos: **frase de paso** siempre. Tres: si una huella de servidor conocido *cambia* de repente, no se acepta a ciegas — se pregunta a quien lo administra. Con esas tres, SSH es hoy la forma más segura que existe de trabajar a distancia; sin ellas, es una puerta abierta con tu nombre.

**Por dentro · El camino inverso: tu casa desde fuera (opcional)**

¿Y entrar en tu Mint desde la universidad? Ayer viste por qué no se puede sin más: tu casa está detrás del NAT. Haría falta pedirle al router que *reenvíe* el puerto 22 de su dirección pública a tu Mint («reenvío de puertos» o *port forwarding*, en la sección WAN) — un cambio reversible más, que abriría una puerta de tu casa a todo internet. Solo tiene sentido con claves y contraseña desactivada, y si tu compañía te tiene tras un CG-NAT (ejercicio 10.1), ni así. Lo dejamos como experimento avanzado para quien lo necesite; el capítulo 13 enseña una vía más elegante para casi todos los casos.

## 12.9. Práctica: la escalera

Cuaderno en marcha (`script sesion12.log`). Los cambios de hoy en la VM se hacen con instantánea previa; en tu Mint, solo se instala un paquete y se crean ficheros en `~/.ssh`. Lo esencial son los tres primeros ejercicios (con ratón, contigo mismo, el servidor); el nivel 2 es para quien quiera ir más allá hoy — o dentro de un mes.

**Nivel 1 · Lo esencial**

**Ejercicio 12.0 · Con ratón**

Desde el gestor de archivos, conéctate a `debian-lab` por SSH con el diálogo *Conectar al servidor…*, copia un fichero cualquiera de tu Mint a la casa del servidor arrastrándolo, y comprueba en la barra lateral que la conexión queda como marcador. Después desconéctala (botón  junto al marcador).

*Pista.* Tipo SSH, servidor `debian-lab`, usuario `marcos`; si pide aceptar una «identidad» desconocida, es la huella que explica la sección siguiente — acepta.

*(Solución al final del capítulo.)*

**Ejercicio 12.1 · Nivel 0: contigo mismo**

Instala `openssh-server` en tu Mint, comprueba que el servicio escucha (`systemctl status ssh` y, del capítulo 11, `ss -tln`), y entra con `ssh marcos@localhost`. Dentro, ejecuta `who` para ver tu sesión listada, y sal.

*Pista.* `who` muestra las sesiones abiertas; la tuya por SSH aparecerá con un `pts/` y una dirección.

*(Solución al final del capítulo.)*

**Ejercicio 12.2 · Nivel 1: el servidor**

Entra en `debian-lab` con contraseña, demuestra dónde estás con tres comandos (`hostname`, `ip a | grep "inet "`, `whoami`) y mira el fichero de huellas de tu Mint tras salir.

*Pista.* `cat ~/.ssh/known_hosts` — verás la entrada que acabas de aceptar.

*(Solución al final del capítulo.)*

**Nivel 2 · Avanzado (pero muy útil)**

**Ejercicio 12.3 · Las claves**

Genera tu par ed25519 con frase de paso, instálalo en `debian-lab` con `ssh-copy-id`, y comprueba que `ssh marcos@debian-lab` ya no pide contraseña. Después mira, en el servidor, dónde ha quedado tu clave pública y con qué permisos.

*Pista.* `cat ~/.ssh/authorized_keys` y `ls -la ~/.ssh` dentro de la VM.

*(Solución al final del capítulo.)*

**Ejercicio 12.4 · Alias, copia y comando remoto**

Crea `~/.ssh/config` con el alias `lab`. Después: sube `adquisicion.log` con `scp`, y sin abrir sesión, pídele al servidor que cuente los `ERROR` y que te diga el sensor que más falla (la tubería del capítulo 3, entre comillas).

*Pista.* `ssh lab "grep … | … | sort -nr | head -1"` — las comillas hacen que la tubería entera se ejecute allí.

*(Solución al final del capítulo.)*

**Ejercicio 12.5 · `rsync` y la barra**

Crea en tu Mint una carpeta `resultados/` con tres ficheros cualesquiera. Sincronízala a la VM con `rsync -av` — primero con barra final, luego sin ella — y observa con `ssh lab ls -R` dónde ha acabado cada cosa. Después modifica un fichero, repite el `rsync` y fíjate en qué viaja.

*Pista.* `rsync -av resultados/ lab:~/resultados/` frente a `rsync -av resultados lab:~/otra/`.

*(Solución al final del capítulo.)*

**Ejercicio 12.6 · Cerrar la puerta a las contraseñas**

Con instantánea previa de la VM y el protocolo del capítulo 10 (copia del fichero, vuelta atrás escrita): edita en `debian-lab` el fichero `/etc/ssh/sshd_config`, pon `PasswordAuthentication no`, reinicia el servicio y verifica que con tu clave sigues entrando — y que sin ella (prueba `ssh -o PubkeyAuthentication=no lab`) te rechaza.

*Pista.* `sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak`, `sudo nano`, `sudo systemctl restart ssh`. No cierres tu sesión abierta hasta haber comprobado que una nueva entra.

*(Solución al final del capítulo.)*

**Nivel 3 · Opcional · el host real**

**Ejercicio 12.7 · Nivel 2, si tienes dónde**

Si dispones de una cuenta en un servidor real (universidad, nube o shell público): añádelo a tu `~/.ssh/config`, instala tu clave pública por la vía que te indiquen, entra, y repite el parte de «¿dónde estoy?» del 11.2 más un `ping` de vuelta a tu casa desde allí — para ver la distancia con los ojos del capítulo 9.

*Pista.* Si el puerto no es el 22, `Port` en la agenda; si te dan solo un formulario web para la clave, el contenido a pegar es tu `id_ed25519.pub` completo.

*(Solución al final del capítulo.)*

## 12.10. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. Al conectar por primera vez a un servidor, SSH te pregunta si aceptas su huella porque…

   a) todavía no conoce esa máquina y quiere que confirmes que es quien dice ser
   b) tu contraseña es incorrecta
   c) el servidor no tiene clave

2. ¿Qué fichero se copia al servidor con `ssh-copy-id`?

   a) la clave privada, `id_ed25519`
   b) la clave pública, `id_ed25519.pub`, a `authorized_keys`
   c) tu contraseña cifrada

3. ¿Qué hace `ssh lab "grep -c ERROR datos.log"`?

   a) abre una sesión interactiva en `lab`
   b) ejecuta ese comando en `lab` y te devuelve la salida, sin abrir sesión
   c) copia `datos.log` a tu máquina y lo cuenta aquí

4. Para subir la carpeta `datos` entera a `lab`:

   a) `scp datos lab:~/`
   b) `scp -r datos lab:~/`

c) `ssh datos lab`

5. La ventaja de `rsync` sobre `scp` para una carpeta que cambia es que…

   a) va cifrado y `scp` no
   b) solo transfiere lo que ha cambiado, y se puede reanudar
   c) no necesita SSH

6. Tras `PasswordAuthentication no` en el servidor…

   a) ya nadie puede entrar
   b) solo entra quien tenga una clave privada cuya pública esté en `authorized_keys`
   c) las contraseñas se vuelven más largas

## Soluciones del capítulo

### Ejercicio 12.0 · Con ratón

Solución razonada

Archivo → Conectar al servidor → SSH · `debian-lab` · `/home/marcos` · `marcos` → contraseña → la carpeta aparece; arrastrar el fichero;  para desconectar. Desde la VM, `ls ~` muestra el fichero copiado.
**Por qué así** — Es exactamente `scp` con ratón, y para mucha gente es *toda* la SSH que necesitan. El resto del capítulo existe para cuando el ratón se queda corto: ejecutar cosas allí, automatizar, o entrar en máquinas sin ventanas.

### Ejercicio 12.1 · Nivel 0: contigo mismo

Solución razonada

`sudo apt install openssh-server`, `ss -tln | grep :22` muestra `LISTEN`, y tras `ssh marcos@localhost` la primera vez pedirá aceptar la huella (de tu propia máquina). `who` lista dos sesiones: la de tu escritorio y la SSH desde `127.0.0.1`. `exit`.
**Por qué así** — El servidor SSH en tu portátil es lo que permitirá, en el capítulo 13, cosas como entrar desde el móvil o conectar dos portátiles en el laboratorio. Y practicar la huella con una máquina que sabes que es de fiar quita el susto para las siguientes.

### Ejercicio 12.2 · Nivel 1: el servidor

Solución razonada

`ssh marcos@debian-lab` → `yes` → contraseña. Dentro: `debian-lab`, `192.168.122.15`, `marcos`. Fuera: `known_hosts` guarda una línea por máquina conocida — ahora dos, `localhost` y `debian-lab`.
**Por qué así** — Tres comandos que responden «¿dónde estoy?», la pregunta que hay que hacerse antes de cualquier `rm` en una sesión remota. Y `known_hosts` es la memoria

de confianza de tu cliente: saber que existe es saber leer el aviso de huella cambiada.

**Ejercicio 12.3 · Las claves**

Solución razonada

`ssh-keygen -t ed25519 -C "marcos@portatil"`, `ssh-copy-id marcos@debian-lab`, y la siguiente conexión entra directa (pedirá la frase de paso de la clave la primera vez de la sesión de escritorio). En la VM, `authorized_keys` contiene una línea larga que empieza por `ssh-ed25519` — tu pública —, con permisos `-rw-------`.
**Por qué así** — Es el ritual que repetirás con cada servidor nuevo de tu vida: generar una vez, copiar la pública a cada máquina. La línea de `authorized_keys` es exactamente el contenido de tu `id_ed25519.pub`: compáralos con `cat` y verás que coinciden.

**Ejercicio 12.4 · Alias, copia y comando remoto**

Solución razonada

Con el alias: `scp ~/Descargas/adquisicion.log lab:~/`, luego `ssh lab "grep -c ERROR adquisicion.log"` → 108, y `ssh lab "grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -nr | head -1"` → `90 s3`.
**Por qué así** — Sin las comillas, tu Mint interpretaría el `|` y ejecutaría la mitad de la tubería en casa sobre un fichero que no tiene. Con ellas, el servidor recibe la orden completa. Es el patrón del trabajo remoto: los datos y el cálculo allí, el resultado aquí.

**Ejercicio 12.5 · rsync y la barra**

Solución razonada

Con barra, los tres ficheros aparecen dentro de `~/resultados/`; sin barra, aparece la carpeta `resultados` entera *dentro* de `~/otra/`. En la repetición tras el cambio, `rsync` envía solo el fichero modificado — la lista dice un nombre, no tres.
**Por qué así** — La barra es la trampa clásica y la copia incremental es la virtud clásica: entender las dos en un experimento de dos minutos evita una tarde de carpetas duplicadas y horas de transferencias repetidas.

**Ejercicio 12.6 · Cerrar la puerta a las contraseñas**

Solución razonada

La línea suele venir comentada (`#PasswordAuthentication yes`): descoméntala y ponla a `no`. Tras el reinicio del servicio, `ssh lab` entra por clave; `ssh -o PubkeyAuthentication=no lab` devuelve `Permission denied (publickey)`. Vuelta atrás: restaurar el `.bak` y reiniciar `ssh` — o la instantánea.
**Por qué así** — Es *el* endurecimiento básico de cualquier servidor expuesto, y lo has hecho con todas las redes: instantánea, copia, sesión abierta de reserva. La regla de oro

de los administradores — «nunca cierres la sesión que funciona hasta probar la nueva» — acaba de entrar en tu repertorio.

**Ejercicio 12.7 · Nivel 2, si tienes dónde**

Solución razonada

La experiencia es idéntica a `debian-lab` con dos diferencias visibles: el `ping` de vuelta a tu IP pública tarda decenas de milisegundos (kilómetros reales), y probablemente `sudo` no funcione — no eres administrador allí, y ahora entiendes exactamente qué significa eso.
**Por qué así** — El curso se escribió para que llegaras aquí sin necesitar este ejercicio; hacerlo cierra el círculo: la escalera de tres niveles era la misma máquina a tres distancias.

**Autocomprobación** — Respuestas: 1 a · 2 b · 3 b · 4 b · 5 b · 6 b.

**La próxima sesión**

Ya entras y sales de tu servidor. El capítulo 13 es vivir en él: lanzar una simulación, cerrar el portátil, volver mañana y encontrarla terminada — `tmux`, la sesión que no muere. Y llega el caso estrella del físico computacional: un Jupyter corriendo en el servidor, abierto en el navegador de tu portátil a través de un túnel SSH. De postre sabrás qué es «la nube» en dos párrafos, y qué es un clúster en dos frases. Guarda `sesion12.log`.

cap. 12 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion12.log`

# 13 Vivir en remoto

Última sesión numerada, y la que junta todo. Ya entras y sales de tu servidor; hoy aprendes a *quedarte* sin estar: lanzar una simulación, cerrar el portátil, volver mañana desde otra ciudad y encontrarla terminada. Después, el truco favorito del físico computacional — un Jupyter corriendo en el servidor, abierto en el navegador de tu portátil por un túnel cifrado. Y de postre, qué es «la nube» en términos concretos — y por dónde seguir el día que te den cuenta en un clúster.

**2 h** (lo esencial) · requisitos: **capítulo 12** · máquina: **tu Linux de diario y `debian-lab`** · nivel: **avanzado — para conocer**

**Cómo leer este capítulo · conocer, no dominar**

Este capítulo es el más avanzado del curso, y se lee con otra actitud: lo esencial es entender el *problema* (un programa lanzado por SSH muere con la conexión) y saber que existen dos soluciones — `tmux` y los túneles — y para qué sirve cada una. Usarlas con soltura es cosa de quien las necesite, y ese día tendrás una IA al lado que te dé los comandos (anexo A): lo que este capítulo te da es entenderlos. Haz los ejercicios 12.1 a 12.3, que son cortos; el 12.4 es opcional.

**Al terminar esta sesión sabrás**

- Por qué un programa lanzado por SSH muere al cerrar la conexión — y cómo evitarlo.
- Qué es `tmux` y sus cuatro gestos: crear una sesión, desconectarte, listar, volver a engancharla.
- Abrir un túnel SSH y ver en tu navegador un servicio que solo existe dentro del servidor.
- Que Jupyter puede correr en el servidor con los datos y la CPU de allí mientras tú trabajas desde tu portátil — y cómo se monta, para el día que lo necesites.
- Qué es exactamente «una máquina en la nube», qué te dan y qué se paga.
- Qué es un clúster de cálculo, y por qué lo aprendido aquí es lo que su guía dará por sabido.

## 13.1. La sesión que muere

Un experimento de treinta segundos que explica el capítulo. Entra en `debian-lab`, lanza algo largo en segundo plano (capítulo 5), y cierra la conexión:

```
marcos@portatil:~$ ssh lab
marcos@debian-lab:~$ sleep 600 &
[1] 1204
marcos@debian-lab:~$ exit
marcos@portatil:~$ ssh lab "ps aux | grep sleep"
marcos      1231  0.0  0.0   6612  2200 ?  Ss  20:02  0:00 grep sleep
```

El `sleep` ha desaparecido: al cerrar la sesión, el shell envía a todos sus hijos la señal SIGHUP — «se ha colgado la línea», nombre heredado de los módems del capítulo 12 — y los procesos terminan. Tu simulación de diez horas moriría a la primera caída de Wi-Fi. Hay parches (`nohup`, el `&` con `disown`), pero la solución que usa todo el mundo es otra idea: que la sesión viva *en el servidor*, y que tú te enganches y desenganches de ella.

## 13.2. `tmux`: la sesión que no muere

`tmux` (*terminal multiplexer*) crea sesiones de terminal que pertenecen al servidor, no a tu conexión. Te desconectas y siguen; vuelves — desde el mismo portátil o desde otro — y las recuperas tal como las dejaste.

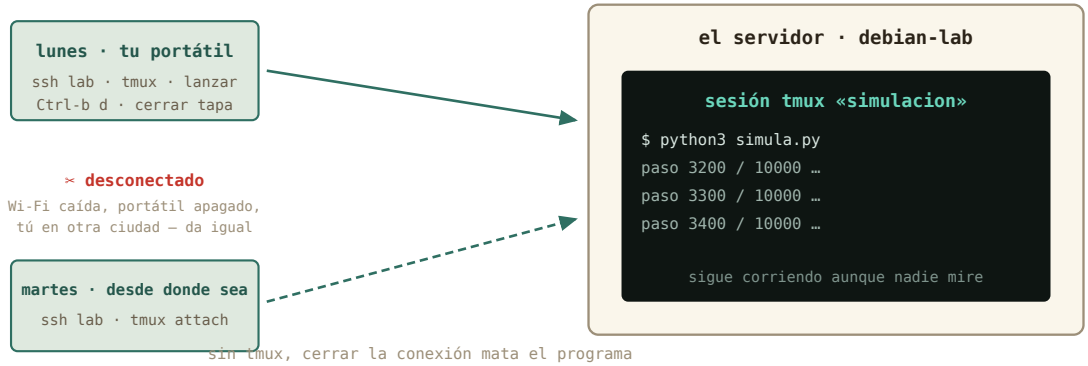



Figura 13.1: Lunes lanzas, martes recoges. La sesión y todo lo que corre dentro viven en el servidor.

Instálalo en `debian-lab` (`sudo apt install tmux`) y repite el experimento, esta vez dentro de una sesión con nombre:

```
marcos@debian-lab:~$ tmux new -s simulacion
```

La pantalla cambia: una barra verde abajo con el nombre de la sesión. Estás *dentro* de tmux. Lanza tu trabajo largo — el `simula.py` de la práctica, o el `sleep 600` de antes — y ahora el gesto clave: Ctrl+b y luego d (*detach*, desenganchar). Vuelves a tu prompt normal; la sesión sigue viva detrás. Cierra SSH, apaga la Wi-Fi si quieres, y vuelve:

```
marcos@portatil:~$ ssh lab
marcos@debian-lab:~$ tmux ls
simulacion: 1 windows (created Tue Aug 25 20:10:41 2026)
marcos@debian-lab:~$ tmux attach -t simulacion
```

Ahí está, con su `sleep` todavía contando. Todo lo que hay que saber de tmux para vivir en remoto cabe en cuatro gestos: `tmux new -s nombre`, Ctrl+b d, `tmux ls`, `tmux attach -t nombre`. (Cuando quieras más: Ctrl+b c abre otra ventana en la misma sesión, Ctrl+b % parte la pantalla en dos, y `exit` dentro de la última ventana cierra la sesión.)

**Truco · La regla de los servidores**

En un servidor ajeno, **lo primero que se teclea tras entrar es `tmux attach || tmux new`**: engánchate a tu sesión si existe, crea una si no. Así, cada trabajo lanzado sobrevive a tu conexión, y cuando el administrador pregunte «¿lo tienes en tmux?», la respuesta será siempre sí. Su hermano mayor, `screen`, hace lo mismo y lo encontrarás en máquinas antiguas: `screen -S nombre`, Ctrl+a d, `screen -r nombre`.

## 13.3. El túnel: un servicio remoto en tu navegador

Segunda idea del capítulo, y la más elegante del curso. En el capítulo 11 viste que un servicio escucha en un puerto, y que un puerto solo abierto a `localhost` es invisible desde fuera. SSH puede **tender un túnel**: hacer que un puerto de *tu* máquina desemboque en un puerto del servidor, cifrado, sin abrir nada a la red.




Figura 13.2: La opción `-L`: puerto local, dos puntos, destino visto desde el servidor. Tu navegador no sabe que está hablando con otra máquina.

La prueba más simple no necesita instalar nada: Python trae un servidor web de juguete. En `debian-lab`, dentro de tmux:

```
marcos@debian-lab:~$ python3 -m http.server 8000 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8000 (http://127.0.0.1:8000/) ...
```

Escucha solo en `127.0.0.1`: desde tu Mint, `nmap debian-lab` no lo vería. Ahora, desde tu Mint, la conexión con túnel:

```
marcos@portatil:~$ ssh -L 8000:localhost:8000 lab
```

Y en tu navegador, `http://localhost:8000`: la lista de ficheros de la casa de `debian-lab`, servida por una máquina que no tiene ese puerto abierto. `-L 8000:localhost:8000` se lee «mi puerto 8000 → lo que el servidor llama `localhost:8000`». Mientras esa sesión SSH esté abierta, el túnel existe; al cerrarla, desaparece.

Y ahora el caso estrella: **Jupyter en el servidor**. Los datos pesados, la CPU de cálculo y el entorno de Python viven allí; tu portátil solo pone el navegador:

```
marcos@debian-lab:~$ sudo apt install jupyter-notebook
marcos@debian-lab:~$ jupyter notebook --no-browser --port 8888
    http://localhost:8888/tree?token=4f2a9c…
```

Con `ssh -L 8888:localhost:8888 lab` abierto en tu Mint, esa URL — copiada con su *token* — funciona en tu navegador. Lo que ves es un Jupyter normal; lo que pasa es que cada celda se ejecuta a kilómetros. Lánzalo dentro de tmux y sobrevivirá a tus desconexiones: mañana, nuevo túnel, misma sesión, todo donde estaba. Este flujo — tmux + túnel + Jupyter — es, literalmente, cómo trabaja la física computacional en 2026.

**Cuidado · Por qué el túnel y no «abrir el puerto»**

La tentación es lanzar Jupyter escuchando en todas las interfaces y entrar directo por `http://debian-lab:8888`. Funcionaría — y dejaría un intérprete de Python con tus permisos abierto a toda la red, sin cifrar. En el servidor de la universidad, además, el portero del capítulo 11 no te dejaría. El túnel resuelve las dos cosas: nada se abre, todo va cifrado, y solo tú — con tu clave — lo ves.

## 13.4. Qué es «la nube», en concreto

Con el curso hecho, la palabra deja de ser humo. **Una máquina en la nube es una VM como tu `debian-lab`, en un anfitrión de KVM (capítulo 7) que vive en un edificio de otro** — y que alquilas por horas. Te dan exactamente tres cosas: una dirección IP *pública* (capítulo 10: sin NAT delante), un usuario, y un sitio donde pegar tu clave pública (capítulo 12). Desde ese momento es `ssh usuario@ip`, y todo este bloque aplica sin cambiar una coma. Lo que se paga es el tiempo encendida, el disco y el tráfico que sale — por eso el primer hábito de la nube es apagar lo que no se usa. Y la física de la sección de `ping` decide dónde: una máquina en Fráncfort responde a 30 ms; la misma en Virginia, a 100.

**Historia · Tiempo compartido, tercera edición**

El curso ha contado la misma historia tres veces. En los 70, muchos usuarios compartían *un* ordenador por turnos de milisegundos (capítulo 4). En 1967, IBM les dio a cada uno un

ordenador de mentira (capítulo 7). Y hoy, un **clúster** de cálculo — miles de nodos como tu `debian-lab`, pero físicos, unidos por una red rapidísima y compartidos por cientos de investigadores — reparte esos nodos con una cola de trabajos. MareNostrum 4, en la capilla de Torre Girona de Barcelona, tenía 3 456 nodos y 165 888 núcleos; su sucesor multiplica eso. Las cuentas de la Red Española de Supercomputación se piden por convocatoria, y tu grupo probablemente ya tenga horas: el día que te den un usuario, este capítulo será tu primera semana.



Figura 13.3: MareNostrum 4 (2017), en la capilla de Torre Girona, Barcelona: miles de `debian-lab` físicos compartidos por una cola. *Foto: Gemmaribasmaspoch, CC BY-SA 4.0, Wikimedia Commons.*

## 13.5. Y si algún día te dan un clúster

Un **clúster** de cálculo — el de la universidad, el de la Red Española de Supercomputación — es la tercera edición del tiempo compartido: miles de máquinas como tu `debian-lab`, pero físicas y unidas por una red rapidísima, repartidas entre cientos de investigadores por una **cola de trabajos**: se entra por SSH, se prepara el trabajo y se le pide a la cola que lo ejecute cuando haya hueco. Cada centro tiene su gestor de colas y su guía de usuario, y esa guía será tu manual el día que toque. Lo que este bloque te ha dado — entrar por SSH con clave, sobrevivir a la desconexión con `tmux`, mover datos con `rsync` — es exactamente lo que esa guía dará por sabido.

## 13.6. Práctica: vivir en el servidor

Cuaderno en marcha (`script sesion13.log`). Los ejercicios de hoy son el flujo de trabajo que usarás en la vida real; hazlos en orden.

**Nivel 1 · Lo esencial**

**Ejercicio 13.1 · La muerte de la sesión**

Reproduce el experimento del capítulo: `sleep 600 &` dentro de una sesión SSH, `exit`, y comprueba desde fuera si sigue vivo. Repite con tmux — y comprueba de nuevo.

*Pista.* `ssh lab "ps aux | grep sleep"` es la comprobación sin entrar; la primera vez desaparecerá, la segunda seguirá ahí.

*(Solución al final del capítulo.)*

**Ejercicio 13.2 · La simulación que sobrevive**

Crea en `debian-lab` un `simula.py` de diez minutos (abajo), lánzalo dentro de una sesión tmux llamada `simulacion`, desengánchate, cierra SSH, espera unos minutos, vuelve a entrar desde tu Mint y comprueba el progreso — sin engancharte, con `ssh lab "tail -2 progreso.txt"` — y luego engánchate para verlo terminar.

*Pista.* Con `nano simula.py`:

```
import time
for paso in range(1, 601):
    with open("progreso.txt", "a") as f:
        f.write(f"paso {paso}\n")
    print(f"paso {paso} / 600")
    time.sleep(1)
```

*(Solución al final del capítulo.)*

**Ejercicio 13.3 · Tu primer túnel**

En `debian-lab` (dentro de tmux), sirve tu casa con `python3 -m http.server 8000 --bind 127.0.0.1`. Comprueba desde tu Mint que el puerto *no* está abierto (`nmap debian-lab`), abre el túnel con `ssh -L 8000:localhost:8000 lab`, y visita `http://localhost:8000` en tu navegador. Cierra la sesión SSH y recarga la página.

*Pista.* El túnel solo existe mientras esa conexión SSH esté abierta.

*(Solución al final del capítulo.)*

**Nivel 3 · Opcional (para quien lo vaya a usar — con una IA al lado si quieres)**

**Ejercicio 13.4 · Jupyter a distancia**

Instala `jupyter-notebook` en `debian-lab`, lánzalo dentro de tmux con `--no-browser --port 8888`, abre el túnel del 8888 y trabaja desde tu navegador: crea un cuaderno que lea

`progreso.txt` y cuente sus líneas. Desconéctate, vuelve, y comprueba que el cuaderno sigue donde estaba.

*Pista.* La URL con el `token` la imprime Jupyter al arrancar; si la pierdes, `jupyter notebook list` la repite.

*(Solución al final del capítulo.)*

## 13.7. Autocomprobación

Responde de memoria; las respuestas están al final del capítulo.

1. Lanzas `python3 simula.py` por SSH y se corta la Wi-Fi. Sin tmux, el programa…

   a) sigue corriendo en el servidor
   b) muere: el cierre de la sesión le envía SIGHUP
   c) se pausa hasta que vuelvas

2. Para desengancharte de una sesión tmux dejándola viva:

   a) Ctrl+C
   b) Ctrl+b y después d
   c) cerrar la ventana de la terminal

3. `ssh -L 8888:localhost:8888 lab` hace que…

   a) el puerto 8888 del servidor se abra a toda la red
   b) tu puerto 8888 local desemboque, cifrado, en el 8888 del servidor
   c) Jupyter se instale en tu portátil

4. Una «máquina en la nube» es, en concreto…

   a) una VM como `debian-lab` en un anfitrión ajeno, con IP pública, alquilada por horas
   b) un ordenador que no necesita sistema operativo
   c) un disco duro en internet

5. Lanzaste Jupyter en el servidor dentro de tmux y cerraste el portátil. Al día siguiente…

   a) el cuaderno y su kernel siguen vivos: basta abrir un túnel nuevo
   b) Jupyter murió al cerrarse el túnel
   c) hay que reinstalar Jupyter

6. `tmux attach || tmux new` significa…

   a) engánchate a tu sesión si existe; si no, crea una
   b) crea siempre una sesión nueva
   c) cierra todas las sesiones

## Soluciones del capítulo

### Ejercicio 13.1 · La muerte de la sesión

Solución razonada

Sin tmux, el `grep` solo se encuentra a sí mismo: SIGHUP mató al `sleep`. Con `tmux new -s prueba`, `sleep 600 &`, Ctrl+b d, `exit`, la comprobación muestra el `sleep` vivo — y `tmux attach -t prueba` te lo devuelve.
**Por qué así** — Ver morir el proceso una vez vale más que cualquier explicación: a partir de hoy el reflejo «¿lo tengo en tmux?» te saldrá solo antes de lanzar nada que dure más que un café.

### Ejercicio 13.2 · La simulación que sobrevive

Solución razonada

`tmux new -s simulacion`, `python3 simula.py`, desenganchar, `exit`. Minutos después: `ssh lab "tail -2 progreso.txt"` muestra un paso avanzado; `ssh lab` + `tmux attach -t simulacion` enseña la cuenta en directo. El programa nunca supo que te fuiste.
**Por qué así** — Es el flujo del físico computacional entero en miniatura: lanzar, irse, consultar sin entrar (el comando remoto del capítulo 12), volver. Sustituye `simula.py` por tu código de verdad y `600` por doce horas: el gesto es idéntico.

### Ejercicio 13.3 · Tu primer túnel

Solución razonada

`nmap` sigue viendo solo el 22 — el 8000 es privado del servidor. Con el túnel abierto, el navegador muestra el listado de ficheros de la VM (ahí estará `progreso.txt`). Al cerrar SSH, la página deja de cargar: el túnel murió con la conexión.
**Por qué así** — Has visto las tres propiedades del túnel: no abre puertas, va por SSH, y vive lo que vive la conexión. Todo lo que sigue (Jupyter, y cualquier panel web de un servidor) es este mismo gesto con otro número.

### Ejercicio 13.4 · Jupyter a distancia

Solución razonada

`sudo apt install jupyter-notebook`; en tmux: `jupyter notebook --no-browser --port 8888`; en Mint: `ssh -L 8888:localhost:8888 lab` y la URL con token en el navegador. El cuaderno con `len(open("progreso.txt").readlines())` da 600. Tras cerrar y reabrir túnel, el cuaderno y su kernel siguen vivos: viven en tmux.
**Por qué así** — Este es el montaje completo: los datos y el cálculo en la máquina remota, tu comodidad en la local. En un clúster el patrón es el mismo, con Jupyter

lanzado en una máquina de cálculo — pregúntalo allí: casi todos tienen una receta escrita para exactamente esto.

**Autocomprobación** — Respuestas: 1 b · 2 b · 3 b · 4 a · 5 a · 6 a.

### Lo que queda

Trece sesiones: sabes hablar con la máquina, abrirle el capó, instalarla, conectarla y trabajar en ella desde lejos. Queda el ensayo general: el **proyecto final**, en el que montas de cero un servidor — VM, instalación, red, SSH con clave, dataset, análisis en tmux — sin más guía que una lista de objetivos. Y quedan los anexos para cuando los necesites: scripting con bucles, montar unidades de todo tipo, la IA como herramienta y no como oráculo, y el camino para quien venga de Windows. Guarda `sesion13.log`: son doce medidas.

cap. 13 v0.1 · piloto — anota tiempo real invertido, dudas y erratas junto a tu `sesion13.log`

# Proyecto final: tu primer servidor

El ensayo general. Durante trece sesiones cada pieza vino con su explicación y su solución plegada; hoy no hay solución: hay una lista de objetivos, la referencia al capítulo donde vive cada pieza, y una hoja de verificación. Montas de cero un servidor Linux, lo conectas, lo aseguras, le subes un dataset, lanzas un análisis que sobrevive a tu ausencia y recoges los resultados. Es, pieza a pieza, el flujo de trabajo remoto de un físico computacional — y al terminar sabrás que sabes hacerlo, que es distinto de haberlo leído.

**~3 h** · requisitos: **los doce capítulos** · máquina: **tu Linux de diario y una VM nueva**

## Las reglas

- **Desde cero.** Crea una VM *nueva* llamada `servidor` (o borra `debian-lab` y empieza limpio). Que el músculo se acuerde sin el capítulo delante.
- **Todo por terminal**, salvo la creación de la VM y sus instantáneas, que se hacen en virt-manager.
- **Cuaderno obligatorio**: `script proyecto.log` en tu Mint antes de empezar. Anota el tiempo de cada hito y qué te atascó: es el dato que el curso quiere.
- **Cada hito se verifica con un comando**, no con una sensación. La hoja del final dice cuál.
- **Si te atascas**: primero el capítulo indicado; después `man`; y si sigues atascado, pide el comando a una IA con la disciplina del curso — léelo entero, verifica lo que no reconozcas, jamás un `sudo` o un `rm` a ciegas.

## Los hitos

**1. La máquina** *(capítulo 7)*. VM `servidor`: 2 CPU, 4 GB, disco de 20 GB, red NAT, con la ISO de Debian verificada en la bandeja.

**2. La instalación** *(capítulos 6 y 8)*. Debian sin escritorio, con servidor SSH. Particionado manual: raíz ext4 y swap — o, si te atreves, `/home` separado. Contraseña de root vacía. Al terminar, apaga desde dentro e instantánea `recien-instalado`.

**3. La red** *(capítulos 9–11)*. Averigua la IP del servidor, bautízalo en el `/etc/hosts` de tu Mint como `servidor`, y comprueba que responde por nombre. De propina: ¿qué puerta tiene abierta?

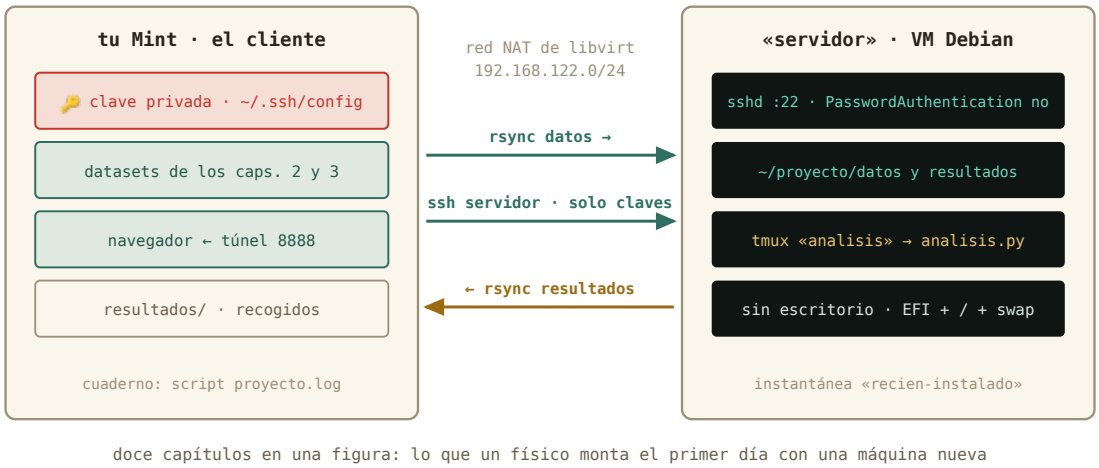



Figura 13.1: La arquitectura que vas a montar. Cada caja y cada flecha es un capítulo.

**4. El acceso** *(capítulo 12)*. Tu clave pública en el servidor, alias `servidor` en `~/.ssh/config`, entrada sin contraseña. Después, cierra la puerta a las contraseñas en `sshd_config` — con copia del fichero y sesión de reserva abierta — y demuestra que te rechaza sin clave.

**5. El sistema** *(capítulos 4 y 5)*. Actualízalo. Instala lo que el análisis necesita: `tmux`, `htop`, `tree`. Comprueba con `which` que están, y con `df -h` cuánto disco queda.

**6. Los datos** *(capítulos 2 y 11)*. Crea en el servidor `~/proyecto/datos` y `~/proyecto/resultados`. Sube con `rsync` el `adquisicion.log` del capítulo 3 y la carpeta `experimento-pendulo` del capítulo 2. `tree ~/proyecto` debe enseñar el orden.

**7. El análisis** *(capítulos 3, 4 y 12)*. Escribe en el servidor `analisis.py` (abajo tienes uno de partida) y un `analiza.sh` con shebang que lo lance y anote la fecha en `resultados/registro.txt`. Hazlo ejecutable. Lánzalo dentro de una sesión tmux `analisis`, desengánchate, **cierra la conexión y apaga la Wi-Fi un minuto**. Vuelve, engánchate, y mira si ha terminado.

**8. La recogida** *(capítulo 12)*. Trae `resultados/` a tu Mint con `rsync`, y abre el informe. Repite el `rsync`: debe no transferir nada.

**9. Propina** *(capítulo 13)*. Jupyter en el servidor a través de un túnel, abriendo el informe desde el navegador. Y un `du -h` del disco de la VM antes y después de todo: ¿cuánto ha crecido?

## El análisis de partida

Sin NumPy — un servidor recién instalado no lo tiene, y no lo necesita para esto —. Solo biblioteca estándar, como en el capítulo 3:

```python
# analisis.py - recuento de errores por sensor y rango de temperaturas
from collections import Counter

errores = Counter()
temps = []
with open("datos/adquisicion.log") as f:
    for linea in f:
        campos = linea.split()
        nivel, sensor = campos[2], campos[3]
        if nivel == "ERROR":
            errores[sensor] += 1
        elif nivel == "INFO" and "temp_C=" in linea:
            temps.append(float(linea.split("temp_C=")[1]))

with open("resultados/informe.txt", "w") as out:
    out.write("Errores por sensor:\n")
    for sensor, n in errores.most_common():
        out.write(f"  {sensor}: {n}\n")
    out.write(f"Temperatura minima: {min(temps):.2f}\n")
    out.write(f"Temperatura maxima: {max(temps):.2f}\n")
print("informe escrito en resultados/informe.txt")
```

Si el informe dice `s3: 90`, `s1: 15`, `s4: 3` y un rango de 20,45 a 25,00, el análisis coincide con las tuberías del capítulo 3 — dos métodos, un resultado, como debe ser. Y si te sobra tiempo: que `analisis.py` genere también un CSV con errores por minuto, y grafícalo en tu Mint.

## Hoja de verificación

| Hito | Comando (desde tu Mint salvo indicación) | Lo que debe salir |
|---|---|---|
| 1 | `virsh list --all` | `servidor` en la lista |
| 2 | `ssh servidor lsblk` | raíz en `/`, `[SWAP]` |
| 2 | `virsh snapshot-list servidor` | `recien-instalado` |
| 3 | `ping -c 1 servidor` | respuesta desde `192.168.122.x` |
| 3 | `nmap servidor` | solo `22/tcp open` |
| 4 | `ssh servidor hostname` | `servidor`, sin pedir contraseña |
| 4 | `ssh -o PubkeyAuthentication=no servidor` | `Permission denied (publickey)` |
| 5 | `ssh servidor "which tmux htop tree"` | tres rutas |

| Hito | Comando (desde tu Mint salvo indicación) | Lo que debe salir |
|---|---|---|
| 6 | `ssh servidor tree ~/proyecto` | `datos/` con el log y el experimento; `resultados/` vacío |
| 7 | `ssh servidor tmux ls` | la sesión `analisis` |
| 7 | `ssh servidor cat ~/proyecto/resultados/informe.txt` | el informe con `s3: 90` |
| 8 | `rsync -av servidor:~/proyecto/resultados/ ~/resultados/` (2.ª vez) | `sent … bytes`, ningún fichero listado |
| 9 | `du -sh` del `.qcow2` (con `sudo`) | más que en el capítulo 7 |

## Después

Escribe en el cuaderno, para ti, tres respuestas: qué hito te costó más y por qué; qué harías distinto la segunda vez; y cuánto tardaste en total. Esas tres líneas y `proyecto.log` son la medida del curso.

Y luego, el paso que ya no es un ejercicio: pide una cuenta en el clúster de tu universidad — o mira qué horas tiene tu grupo en la Red Española de Supercomputación. Cuando te la den, entra por SSH con tu clave, teclea `tmux attach || tmux new`, y empieza a trabajar. Ya sabes dónde estás parado.

proyecto final v0.1 · piloto — el cuaderno `proyecto.log` con tiempos por hito es el dato más valioso que puedes devolver al curso

# A La IA en la terminal: ayudante, no piloto

Este anexo es la otra mitad del curso. Todo lo que has aprendido — qué es un proceso, qué hace `sudo`, qué significa montar, qué es un puerto — no es para que teclees comandos de memoria: es para que **entiendas lo que una IA te proponga** y decidas tú. En 2026, un científico que administra su portátil, su servidor y su red trabaja con una IA al lado; la diferencia entre el que manda y el que copia y pega a ciegas es exactamente el contenido de este curso. Aquí están las reglas, y el paso a paso con una herramienta concreta y gratuita: **Antigravity CLI**, el agente de Google para la terminal.

## El fontanero

Cuando llamas a un fontanero tienes dos maneras de tratarlo. La primera: le dejas hacer, te vas, y vuelves cuando está todo arreglado. Rápido, cómodo, y mañana no sabrás ni qué se rompió ni cómo se arregla. La segunda: te quedas al lado, le preguntas por cada herramienta y cada técnica que va a usar, y le pides que te indique cómo hacerlo tú. Tarda el doble — y la próxima vez no hace falta llamarle. Está claro cuál de las dos vías es la de aprender.

Un **agente** de IA — un programa que ejecuta un modelo de IA *con acceso a tu terminal* — es ese fontanero, y el trato lo eliges tú. Le describes la tarea («monta la carpeta `proyecto` de `debian-lab` en `~/lab` por NFS y hazlo permanente»), y puede leer tus ficheros, proponer los comandos, ejecutarlos, mirar el resultado y corregir. Durante este curso, la segunda vía es la única: el agente **propone**, tú lees, preguntas, verificas con `man` (capítulo 1) y **tecleas**. Cuando el curso termine y estés administrando un servidor con prisa, ya decidirás cuándo dejarle hacer — con las reglas de más abajo y una instantánea a mano.

## Instalar Antigravity CLI

Antigravity CLI vive directamente en tu terminal — sin ventanas ni editor —, funciona con una cuenta de Google (el acceso básico sobra para este curso) y se instala con un guion que Google publica en su web. La instrucción oficial, hoy, es esta:

```
curl -fsSL https://antigravity.google/cli/install.sh | bash
```

Y aquí aparece el reflejo que este curso te ha entrenado: esa línea descarga un programa de internet y lo ejecuta *sin que lo hayas visto*. Funciona, y millones de personas la teclean tal cual — pero tú ya sabes hacerlo con red de seguridad, y cuesta diez segundos más:

```
marcos@portatil:~$ curl -fsSL https://antigravity.google/cli/install.sh -o instalar-antigravity.sh
marcos@portatil:~$ less instalar-antigravity.sh
marcos@portatil:~$ bash instalar-antigravity.sh
```

Descargar, *mirar* (¿dónde escribe?, ¿pide `sudo`?, ¿de qué dirección descarga el programa?), y solo entonces ejecutar. Al terminar, el instalador te dice con qué comando se arranca y cómo iniciar sesión con tu cuenta de Google.

Un consejo más importante que la línea exacta: **estas instrucciones cambian**. La forma más robusta de instalar cualquier herramienta de IA es preguntarle a una IA en línea — o a la propia web de Google — «cómo instalo Antigravity CLI en Linux desde bash», y aplicar a la respuesta la misma disciplina: leer el guion antes de ejecutarlo. Es, de paso, el primer ensayo de todo este anexo: pedir instrucciones a una IA y mantener el control.

## Configurarlo para aprender, desde el principio

Antes de pedirle nada, dos ajustes. El primero, en la configuración del agente: la **ejecución de comandos** tiene varios modos, desde «ejecuta lo que consideres» hasta «pide permiso cada vez» (el nombre exacto de la opción cambia con las versiones; búscala en la ayuda o pregúntale al propio agente cómo se configura). Ponla en el modo que **pide confirmación siempre** — nunca en automático. Así, aunque un día le dejes hacer, nada correrá sin que tú lo hayas leído y aceptado.

El segundo es de trato, no de menú: la primera instrucción que le das en cada sesión fija las reglas del juego. Guárdala en un fichero (`~/fisica/notas/ia.txt`, capítulo 2) y pégala al empezar:

```
Estoy aprendiendo Linux. No ejecutes comandos: propónmelos y explícame
qué hace cada opción; los teclearé yo. Si un comando puede destruir
datos o cambiar el sistema (rm, dd, mkfs, sudo, cambios en /etc),
avísamelo explícitamente antes. Cuando no entienda algo, dímelo con
un ejemplo pequeño que pueda probar.
```

Con eso el agente pasa de fontanero-que-arregla a fontanero-que-enseña. Muchos agentes permiten guardar esa instrucción como regla permanente del proyecto: si el tuyo lo ofrece, úsalo.

## Las reglas para seguir mandando tú

Un agente es un becario brillante, rapidísimo y sin miedo. Las reglas de un buen jefe:

1. **Trabaja en un sitio acotado.** Abre el agente dentro de la carpeta del proyecto, no en `~`: puede leer todo lo que hay debajo.

2. **Modo «pregunta antes»**, como acabas de configurar. Lee lo que propone *como lo que es*: un comando que ya sabes leer.
3. **`sudo` lo tecleas tú.** Que el agente prepare la orden y tú la ejecutes en tu terminal es un buen reparto; lo destructivo (particionar, formatear, `rm -rf`) nunca sale de tus manos.
4. **Instantánea antes** (capítulo 7) si el agente va a tocar la VM; **copia y vuelta atrás escrita** (capítulo 10) si va a tocar tu máquina real o el router.
5. **Ni claves ni contraseñas en el chat.** Tu `id_ed25519` (capítulo 12), los ficheros de credenciales, los *tokens*: el agente puede necesitar *usarlos* desde tu máquina, no *leerlos* para ti.
6. **Pídele que explique.** «¿Qué hace cada opción de ese `rsync`?» es una pregunta legítima, y la respuesta la puedes contrastar con `man`. Si no entiendes lo que va a hacer, no se ejecuta.

## Cómo pedirle las cosas

La pregunta buena describe el objetivo, no el comando que crees que necesitas. Compara: «dame un `grep`» frente a «tengo un registro de adquisición con líneas `2026-08-18 10:03:00 ERROR s3 …`; quiero saber qué sensor tiene más errores». Con la segunda, el agente te devuelve la tubería del capítulo 3 — y aquí empieza tu trabajo:

1. **Léela pieza a pieza.** Cada programa lo reconoces (o lo buscas en `man`); cada opción debe tener sentido.
2. **Pídele que la explique**: «explícame ese `cut -d' ' -f4`». Su respuesta la contrastas con el manual; si no coinciden, gana el manual.
3. **Pruébala en pequeño** antes de en grande — un `head`, un fichero de prueba, la máquina virtual.
4. **Tecléala tú.** Los dedos aprenden lo que los ojos solo reconocen: la regla del primer día vale también con la IA delante.

Y las preguntas que más rinden son las de después: «¿por qué has usado `sort` antes de `uniq`?», «¿qué habría pasado sin las comillas?», «¿cómo lo haría sin tuberías, en Python?». Cada una es un capítulo de este curso contado otra vez, a tu ritmo y sobre tu problema.

Las tareas del curso que más tiempo comen — y donde un agente ayuda de verdad — son las de *montaje*: configurar la conexión SSH con claves y agenda (capítulo 12), escribir la línea de `fstab` correcta (anexo B2), preparar el `rsync` con las opciones justas, leer un `journalctl` de tres mil líneas para encontrar por qué no arranca un servicio, o escribir el script de bash del anexo D que renombra doscientos ficheros. Un ejemplo completo del reparto de papeles, con la tarea del anexo E:

> **Tú:** «Quiero montar `/home/marcos/proyecto` de `debian-lab` en `~/lab` por SSHFS, y que se desmonte limpio. Explícame cada comando antes de proponerlo.»
>
> **Agente:** propone `sudo apt install sshfs` (tú: lo entiendes, capítulo 5, lo ejecutas), `mkdir ~/lab`, `sshfs lab:/home/marcos/proyecto ~/lab` (tú: reconoces

el alias `lab` de tu `~/.ssh/config`), y `fusermount -u ~/lab` para desmontar. Te explica por qué no hace falta `sudo` para el montaje.

**Tú:** verificas con `findmnt ~/lab` — capítulo 6 — y das por buena la tarea.

Fíjate en lo que ha pasado: no has memorizado la sintaxis de `sshfs`, pero has entendido cada paso, has sabido qué comprobar y habrías detectado un `sudo` fuera de sitio. Eso es lo que este curso te ha dado — y la razón por la que lo *avanzado* de cada capítulo había que conocerlo aunque no dominarlo: para que, cuando la IA te lo ponga delante, sepas qué es, para qué sirve y qué puede romper.

**Cuidado · Lo que no se le pide y lo que no se le da**

No le pidas que ejecute «para ver qué pasa» en tu máquina real: para eso está `debian-lab` con su instantánea. No le pegues claves privadas, contraseñas ni ficheros con datos personales: el agente lee lo que le enseñas. Y no aceptes un `sudo` que no entiendas — la regla del capítulo 4, que no cambia porque quien lo proponga sea una IA.

## Lo que la IA no sustituye

El cuaderno (`script`), la instantánea, la copia antes de editar, el `lsblk` antes de `dd`. Un agente se equivoca con la misma seguridad con la que acierta, y no tiene manera de saber que ese disco `sdb` era el de las fotos de tu abuela. Las redes de seguridad de este curso están pensadas para humanos y para agentes por igual — y con un agente al lado, las necesitas más, no menos.

**Truco · Otros agentes, mismas reglas**

Antigravity CLI no es el único. La misma cuenta de Google sirve para **Gemini CLI** (`sudo apt install npm` y `sudo npm install -g @google/gemini-cli`, se arranca con `gemini`), y existen **Claude Code** (Anthropic) y **Codex CLI** (OpenAI), con cuenta de pago o créditos. Cambian los nombres y los planes cada pocos meses; no cambia nada de lo que has leído aquí: instalador leído antes de ejecutar, modo «pregunta antes», instrucción de aprendizaje, y los comandos en tus manos. Y todos funcionan también a distancia — el agente puede ejecutar `ssh lab …` por ti, y entonces lo que administra es tu servidor, con las mismas reglas y más motivo para la número 4.

## Práctica

**Ejercicio A.1 · Domar al fontanero**

Instala Antigravity CLI (guion descargado, leído y ejecutado), inicia sesión, pon la ejecución de comandos en modo «pedir confirmación» y pega tu instrucción de aprendizaje. Después plantéale el problema del capítulo 3 con `adquisicion.log` (qué sensor falla más) *sin* mencionar ningún comando. Compara lo que te propone con la tubería del capítulo, pídele que te explique la pieza que menos entiendas, verifícala en `man`, y ejecútala tú.

*(Solución al final del capítulo.)*

**Ejercicio A.2 · Que te explique una línea que ya tienes**

Dale una línea de `/etc/fstab` de tu máquina (`cat /etc/fstab`, sin tocarla) y pídele que te explique campo a campo qué hace, y qué pasaría si faltara `nofail`. Contrasta con el anexo E.

*(Solución al final del capítulo.)*

## Soluciones del capítulo

**Ejercicio A.1 · Domar al fontanero**

Solución razonada

Casi seguro te propondrá `grep ERROR adquisicion.log | cut -d' ' -f4 | sort | uniq -c | sort -nr` o una variante equivalente (quizá con `awk`, un programa que este curso no cubre: buena ocasión para preguntarle qué es y por qué lo prefiere). El resultado debe coincidir con el tuyo: `90 s3 · 15 s1 · 3 s4`.
**Por qué así** — Has hecho el ciclo completo del fontanero que enseña: problema en tus palabras, propuesta suya, comprensión tuya pieza a pieza, verificación en el manual, y las teclas en tus manos. Ese ciclo, repetido, es lo que convierte a la IA en profesor y no en muleta.

**Ejercicio A.2 · Que te explique una línea que ya tienes**

Solución razonada

Debería identificar dispositivo (UUID), punto de montaje, tipo, opciones y los dos números finales, y explicar que sin `nofail` un disco ausente puede detener el arranque. Si algo no cuadra con el anexo E, pregúntale otra vez con la cita del anexo delante: los agentes se equivocan con seguridad, y saber dudar de ellos con criterio es exactamente lo que este curso te ha dado.
**Por qué así** — Explicar lo que ya existe es el uso más rentable de un agente para quien aprende: cero riesgo, y cada respuesta es una clase sobre tu propio sistema.

# B Vengo de Windows (o de macOS)

El curso está escrito para un Linux de escritorio — Mint, Ubuntu o parientes — instalado en el ordenador. Si tu máquina tiene Windows o macOS, tienes dos caminos para seguirlo sin tocar tu sistema, y conviene elegir bien, porque no cubren lo mismo.

## Los dos caminos

| | **A · Un Mint completo en VirtualBox** | **B · WSL2 (Ubuntu dentro de Windows)** |
|---|---|---|
| Qué es | una máquina virtual con Linux Mint entero, escritorio incluido | una terminal Ubuntu integrada en Windows, sin escritorio |
| Instalar | 30–40 min; ISO de Mint + VirtualBox | 5 min; una orden en PowerShell |
| Bloques A y B (caps. 1–6) | todo | casi todo: el cap. 6 (discos, arranque) se lee, no se practica |
| Bloque C (caps. 7–8) | con un matiz (abajo) | no hay hipervisor dentro: hazlo con VirtualBox en Windows, con la caja del cap. 7 al lado |
| Bloques D y E (caps. 9–12) | | (la red es la de Windows; SSH funciona igual) |
| Recomendado para | seguir el curso entero como está escrito | empezar hoy con la terminal, o si tu PC va justo de RAM |

Si puedes, el **camino A**: es un Linux de verdad — con su `apt`, su `systemd`, su `lsblk` — y todo el curso encaja. Con Windows en un Mac con chip Apple, el equivalente de VirtualBox es **UTM** (gratuito) y la ISO de Mint debe ser la de arquitectura ARM64 (o usa Debian, que la publica).

## Camino A: Mint en VirtualBox

1. Descarga **VirtualBox** (virtualbox.org) e instálalo como cualquier programa. Descarga la ISO de **Linux Mint** (linuxmint.com, edición Cinnamon) y verifica su huella como

en el capítulo 7 — en PowerShell: `Get-FileHash linuxmint-22.iso -Algorithm SHA256`.
2. **Nueva máquina**: nombre `mint`, tipo Linux / Ubuntu (64-bit), 4 GB de RAM (la mitad de la tuya como máximo), 2 CPU, disco de 40 GB. Elige la ISO como medio de arranque.
3. Arranca. Mint entra en modo *live* — un escritorio de prueba —; el icono **Install Linux Mint** lanza el instalador. Aquí sí vale «borrar el disco e instalar»: el disco es el fichero de la VM, y todo lo que aprenderás en el capítulo 8 sobre particionar lo harás después, en otra VM dentro de esta.
4. Tras reiniciar, en el menú *Dispositivos → Insertar imagen de CD de las Guest Additions* e instálalas: dan pantalla completa, portapapeles compartido y carpetas compartidas con Windows.
5. Ya estás en el capítulo 1. Cuando llegues al bloque C, activa en VirtualBox *Configuración → Sistema → Procesador → Habilitar VT-x/ AMD-V anidado* para que KVM funcione dentro del Mint virtual. Si tu CPU no lo permite, haz los capítulos 7 y 8 con VirtualBox directamente en Windows: los conceptos son idénticos, y la caja «Si tu anfitrión es Windows o macOS» del capítulo 7 te guía.

## Camino B: WSL2

En PowerShell como administrador: `wsl --install` — instala WSL2 y un Ubuntu, y pide un usuario y contraseña al primer arranque. Desde ese momento, la aplicación *Ubuntu* del menú de inicio es tu terminal, y los capítulos 1 a 5 funcionan tal cual (`sudo apt install`, `nano`, tuberías, permisos, `htop`). Tres avisos honestos: tus ficheros de Windows están en `/mnt/c/…`, más lentos que tu casa Linux; `lsblk` y el arranque del capítulo 6 no describen tu máquina, sino el disco virtual de WSL — léelo como teoría; y para los bloques C en adelante querrás el camino A.

## Dos caminos más: el USB persistente y el arranque dual

Hay otras dos formas de tener un Linux *de verdad* en tu ordenador sin renunciar a Windows. Una es limpia y fácil; la otra no es para principiantes.

### Camino C · Un Linux *live* en un pendrive, con persistencia

Cualquier ISO de Mint arranca «en vivo» desde un pendrive: un escritorio completo que no toca tu disco… y que olvida todo al apagar. La **persistencia** arregla eso: una partición extra en el mismo pendrive donde se guardan tus ficheros, tu configuración y lo que instales con `apt`. Desde Windows, la forma más sencilla es **Rufus** (rufus.ie, gratuito):

1. Un pendrive **USB 3 de 32 GB** (16 como mínimo) que puedas borrar; cuanto más rápido, mejor: es tu disco duro durante el curso.

2. Rufus → elige el pendrive y la ISO de Mint → aparece el control **«Tamaño de la partición persistente»**: dale todo lo que quepa (16–24 GB) → Empezar. Rufus crea la partición `casper-rw` de persistencia por ti.
3. Reinicia con la tecla del menú de arranque (anexo C) y elige el pendrive. Mint arranca en vivo; lo que hagas, se queda.

Lo que cubre: todo el curso salvo matices — el bloque C (KVM en un sistema *live*) funciona si tienes 8 GB de RAM, y el capítulo 6 lo verás con dos discos en `lsblk`, el tuyo y el de Windows, que es incluso más instructivo. Los *peros*, honestos: va a la velocidad del pendrive (un USB 2 barato desespera); no actualices el kernel (`apt upgrade` sí, pero el sistema *live* arranca siempre con el suyo); y como con cualquier pendrive, si lo arrancas de un tirón en mal momento la persistencia puede corromperse — copia tu carpeta de física al servidor de vez en cuando (capítulo 12 y anexo D: `rsync` con `cron`). Para quien quiera varias ISO en un mismo pendrive existe **Ventoy**, con persistencia configurable por fichero; es más potente y más liado — para más adelante.

Y una variante con más recorrido: en vez de un sistema *live*, **instalar Linux entero en un SSD externo por USB** — con el instalador del capítulo 8, eligiendo ese disco como destino y poniendo en él la partición EFI. Es un Linux completo, rápido y permanente que arranca en cualquier PC desde el menú de arranque. La única pega es seria: al elegir el disco de destino, equivocarse destruye Windows. Es el camino perfecto *después* del capítulo 8 y del anexo C, con el plan escrito y `lsblk` delante — no antes.

### Camino D · Arranque dual: solo con alguien que sepa

Instalar Linux en una partición del disco interno, junto a Windows, y elegir al encender. Es lo que hace mucha gente y lo que cubre el anexo C — pero **no es para hacerlo a solas el primer día**. Las implicaciones que hay que entender antes: reducir la partición de Windows (desde Windows, con su «inicio rápido» y BitLocker desactivados), que GRUB pase a gestionar el arranque de los dos sistemas, que una actualización grande de Windows puede pisar ese arranque y dejarte «sin Linux» hasta repararlo, y que deshacerlo requiere saber restaurar el cargador de Windows. Si tienes a alguien que lo haya hecho varias veces y te lo explique mientras lo hace, adelante — con copia de seguridad. Si no, el camino C te da el mismo Linux sin ninguno de esos riesgos, y el capítulo 8 te enseñará a hacer el dual tú mismo cuando toque.

| | Cubre el curso | Riesgo para Windows | Para quién |
|---|---|---|---|
| **A** Mint en VirtualBox | todo (bloque C con virtualización anidada) | ninguno | la opción por defecto |
| **B** WSL2 | terminal y redes; no discos ni instalación | ninguno | empezar hoy mismo |

| | Cubre el curso | Riesgo para Windows | Para quién |
|---|---|---|---|
| **C** USB persistente (Rufus) | todo, a la velocidad del pendrive | ninguno | quien quiere Linux «real» sin tocar el disco |
| **C'** instalación en SSD externo | todo, rápido | alto al instalar, ninguno después | tras el capítulo 8 y el anexo C |
| **D** arranque dual | todo | medio, y permanente | solo acompañado por alguien que sepa |

## Lo que no cambia

Sea cual sea el camino: cuaderno con `script`, ejercicios en escalera, y la regla de seguridad — lo destructivo, en una VM (y con el USB persistente, además, copia periódica de tu carpeta al servidor). La única diferencia real es que, además de aprender Linux, aprenderás dónde termina Windows y dónde empieza la máquina de mentira. Y cuando quieras dar el paso de verdad, el anexo C te espera con el pendrive.

# C Instalar en un PC real: la lista de comprobación

Todo lo que ensayaste en la máquina virtual del capítulo 8 vale para un ordenador físico. Este anexo es la lista que se lee **antes** de empezar y se sigue **durante**, porque aquí no hay instantánea que restaurar. Léela entera una vez; el día de la instalación, tenla abierta en el móvil.

## C.1. Antes de tocar nada

- ☐ **Copia de seguridad** de todo lo que importe del disco de destino — aunque «vayas a dejar Windows». Un error de letra en el particionado no avisa.
- ☐ **Plan de particiones escrito** (ejercicio 8.1), con una columna más: qué hay ahora en cada partición del disco y qué va a pasarle.
- ☐ Si vas a **convivir con Windows**: reduce su partición *desde Windows* (Administración de discos → Reducir volumen) antes de arrancar el instalador — o con **GParted** desde el pendrive *live* de Mint, que es lo que hace mucha gente; en ambos casos con copia de seguridad hecha —, y desactiva el «inicio rápido» de Windows, que deja el disco a medio cerrar.
- ☐ **ISO verificada** con `sha256sum` (capítulo 7). Debian incluye desde la versión 12 el firmware de Wi-Fi y gráficas en la netinst: ya no hace falta la imagen «non-free».
- ☐ Un **pendrive** de 2 GB o más que puedas borrar por completo, y **cable de red** si es posible: la instalación por cable ahorra sorpresas con la Wi-Fi.
- ☐ La **tecla del menú de arranque** de tu equipo, buscada antes: suele ser F12 (Dell, Lenovo), F9 (HP), Esc o F2 (ASUS, Acer), Opción (Mac Intel). Se pulsa nada más encender.

## C.2. Grabar el pendrive

La ISO se copia al pendrive **byte a byte** — no se «pega» como un fichero. Dos vías igual de válidas:

**Con ratón (recomendada la primera vez).** En Mint, aplicación **Discos**: selecciona el pendrive en la lista de la izquierda — compruébalo por su tamaño y su nombre —, menú ( ) → *Restaurar imagen de disco* → elige la ISO → *Iniciar restauración*. Eliges el destino por su nombre, sin letras que confundir.

**Con terminal.** Primero, la regla de oro: `lsblk` *antes* de pinchar el pendrive y *después*, para ver qué nombre nuevo aparece (`sda`, `sdb`…). Luego, con esa letra y solo con esa:

```
marcos@portatil:~$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    1  14,9G  0 disk
 sda1         8:1    1  14,9G  0 part /media/marcos/KINGSTON
nvme0n1     259:0    0 476,9G  0 disk
marcos@portatil:~$ sudo umount /dev/sda1
marcos@portatil:~$ sudo dd if=/var/lib/libvirt/images/debian-13.7.0-amd64-netinst.iso of=/dev/sda b
```

(La ISO vive en el almacén de libvirt desde el capítulo 7; en la aplicación «Discos» llegas a ella navegando a esa misma ruta.)

`dd` copia bloques de `if` (*input file*) a `of` (*output file*) sin preguntar nada: **si `of=` apunta a tu disco interno, lo destruye en silencio**. Por eso se comprueba la letra dos veces, y por eso el destino es el disco entero (`/dev/sda`), no una partición (`sda1`). Cuando termine, `sync` y expulsar (capítulo 6).

## C.3. Lo que cambia respecto a la VM: el arranque es UEFI

`debian-lab` arranca con el firmware clásico (BIOS) por la razón del capítulo 7; tu PC arranca por UEFI, y eso cambia exactamente cuatro cosas — ninguna más:

1. **El menú de arranque del firmware**: se entra con una tecla al encender y el pendrive aparece con «UEFI:» delante.
2. **Secure Boot**: puede quedarse activado; Debian lo soporta.
3. **Una partición más**: la EFI (FAT32, ~512 MB, montada en `/boot/efi`) — la del capítulo 6. El particionado guiado la crea solo; en manual la creas tú (*Utilizar como: Partición del sistema EFI*), y si ya existe una, se reutiliza sin formatear.
4. **GRUB** se instala en esa partición, no en el primer sector, y deja una entrada de arranque en la memoria de la placa: `efibootmgr -v` la enseña.

Si hiciste el reto 8.8, ya has visto las cuatro en una máquina de usar y tirar.

## C.4. Arrancar e instalar

- ☐ Pendrive pinchado, encender, **tecla del menú de arranque**, elegir el pendrive (la entrada que dice *UEFI:* delante — no la otra, si aparecen dos).
- ☐ Si el equipo no lo ofrece: en la configuración del UEFI, comprueba que el arranque por USB está permitido. **Secure Boot puede quedarse activado**: Debian lo soporta.




Figura C.1: El menú de arranque de la ISO en modo UEFI: es GRUB, con otro aspecto que el isolinux de la VM, pero las mismas opciones. *Captura de la versión 12 (la 13 es igual): Paowee, GPL, Wikimedia Commons.*

- ☐ Instalador: las mismas pantallas que en la VM, con la partición EFI de más. En el **particionado manual**, el disco se llamará `nvme0n1` o `sda` y *tendrá particiones ya*. Con Windows al lado: **no crees una tabla nueva** (borraría todo); crea las particiones de Linux en el espacio libre que dejaste, y reutiliza la partición EFI existente marcándola como *Partición del sistema EFI* sin formatearla.
- ☐ Contraseña de root vacía → tu usuario con `sudo` (capítulo 8).
- ☐ Selección de programas: aquí sí querrás probablemente un escritorio — marca el que uses — y, por costumbre, el servidor SSH.
- ☐ GRUB: en modo UEFI se instala en la partición EFI (no pregunta por un disco, como en la VM); si hay Windows, lo detectará y lo añadirá al menú.

## C.5. Después del primer arranque

- ☐ `lsblk` y `df -h`: el parte del capítulo 8, sobre hardware real.
- ☐ `sudo apt update && sudo apt upgrade`.
- ☐ Wi-Fi, pantalla, sonido: si algo falta, la respuesta suele estar en `lspci | grep -i` (red, VGA, audio) para saber qué chip tienes, y de ahí a la wiki de Debian.
- ☐ Guarda el plan de particiones y tus notas en el cuaderno del curso: la próxima instalación la harás en la mitad de tiempo.

# D Scripting: bucles, condiciones y `cron`

En el capítulo 2 renombraste cinco ficheros a mano y el curso prometió algo mejor; en el capítulo 3 dijimos que las tuberías transforman texto pero no *ejecutan una orden por cada fichero*; en el 4 escribiste tu primer script. Este anexo cierra esa deuda: las cuatro piezas de bash que convierten comandos sueltos en programas — y `cron`, para que se ejecuten sin ti.

## Variables y sustitución

```
marcos@portatil:~/fisica$ dir=resultados_$(date +%F)
marcos@portatil:~/fisica$ echo "$dir"
resultados_2026-08-26
marcos@portatil:~/fisica$ mkdir "$dir"
```

Dos ideas. Una variable se crea con `nombre=valor` (sin espacios alrededor del `=`) y se lee con `$nombre`. Y `$(comando)` ejecuta el comando y pone su salida en el sitio: aquí, la fecha de hoy. La regla que evita el noventa por ciento de los sustos: **las variables van siempre entre comillas dobles** — `"$dir"` — para que un valor con espacios (capítulo 1) no se rompa en dos argumentos.

## El bucle `for`

```
marcos@portatil:~/fisica$ for f in *.csv; do echo "procesando $f"; done
procesando run01.csv
procesando run02.csv
procesando run03.csv
```

`for f in LISTA; do …; done` repite el bloque una vez por elemento, con `$f` valiendo cada uno. La lista puede ser un comodín del capítulo 2 (`*.csv`), una salida de comando (`$(cat lista.txt)`) o palabras sueltas. Y la deuda del capítulo 2, saldada — renombrar en lote quitando el `_final`:

```
marcos@portatil:~/fisica$ ls
run01_final.csv  run02_final.csv  run03_final.csv
marcos@portatil:~/fisica$ for f in *_final.csv; do mv "$f" "${f/_final/}"; done
marcos@portatil:~/fisica$ ls
run01.csv  run02.csv  run03.csv
```

`${f/_final/}` es una *sustitución* dentro de la variable: el valor de `f` con `_final` cambiado por nada. Hay una familia entera: `${f%.csv}` quita la extensión, `${f#run}` quita el prefijo. Antes de un `mv` en lote, el gesto de seguridad es cambiar `mv` por `echo mv` y mirar qué *haría*: la simulación en seco.

## Condiciones: `if`

```
#!/bin/bash
# comprueba.sh - ¿existe el fichero que me pasan?
if [ -f "$1" ]; then
    echo "$1 existe y tiene $(wc -l < "$1") líneas"
else
    echo "no encuentro $1"
    exit 1
fi
```

`$1` es el primer argumento con que se llamó al script (`./comprueba.sh adquisicion.log`), `$2` el segundo, y así. `[ -f "$1" ]` pregunta «¿es un fichero?»; otros: `-d` directorio, `-z` cadena vacía, `-gt` mayor que para números. `exit 1` termina con código de error: es el `$?` que puedes consultar después, y lo que `&&` (capítulo 5) mira para decidir si continúa. Un script que comprueba sus entradas y falla *con mensaje* vale el doble que uno que simplemente revienta.

## `while` y leer línea a línea

```
#!/bin/bash
# por cada sensor de la lista, cuenta sus errores
while read -r sensor; do
    n=$(grep -c "ERROR $sensor" adquisicion.log)
    echo "$sensor: $n errores"
done < sensores.txt
```

`while read -r linea; do …; done < fichero` recorre un fichero línea a línea — la forma correcta de tratar listas que vienen de otro sitio. Y fíjate en que dentro del bucle vive una tubería del capítulo 3: los scripts son el pegamento que convierte comandos sueltos en procedimiento.

## Un script completo

Todo junto, con las dos líneas de cabecera que distinguen a un script serio: `set -e` (para al primer error, en vez de seguir a ciegas) y un uso claro cuando faltan argumentos.

```
#!/bin/bash
# organiza.sh - mueve a datos/ los CSV de una serie y anota el registro
set -e
if [ $# -ne 1 ]; then
    echo "uso: $0 CARPETA_DEL_EXPERIMENTO"
    exit 1
fi
cd "$1"
mkdir -p datos descartes
for f in *.csv; do
    if grep -q "sensor congelado" "$f"; then
        mv "$f" descartes/
    else
        mv "$f" datos/
    fi
    echo "$(date +%T) $f" >> registro.log
done
echo "hecho: $(ls datos | wc -l) válidos, $(ls descartes | wc -l) descartados"
```

Es el ejercicio del capítulo 2 hecho en un segundo y repetible mañana con otro experimento. `$#` es el número de argumentos; `$0`, el nombre del propio script. Con `chmod u+x` y en tu `~/bin` (capítulo 5), es una herramienta tuya.

## `cron`: que se ejecute sin ti

`cron` ejecuta comandos a horas fijas. Tu tabla se edita con `crontab -e` (la primera vez pregunta qué editor: `nano`), y cada línea son cinco campos de tiempo y un comando:

```
# min hora día mes díasemana   comando
  30   2   *   *    *          rsync -a /home/marcos/fisica/ servidor:~/copia_fisica/
   0   8   *   *    1          /home/marcos/bin/informe.sh >> /home/marcos/informes.log 2>
```

La primera línea copia tu carpeta de física al servidor del capítulo 11 cada noche a las 2:30 — una copia de seguridad que ya no depende de acordarte. La segunda lanza tu informe los lunes a las 8. Tres reglas de `cron`: **rutas absolutas** (no hay `cd`, no hay tu `PATH` de la terminal), **redirige la salida** a un fichero (si no, se pierde o se convierte en correo), y prueba el comando *a mano* antes de programarlo. En sistemas con `systemd` existe el equivalente moderno — los *timers* — que verás en servidores; la idea es la misma.

## Práctica

### Ejercicio D.1 · La deuda del capítulo 2

Crea una carpeta con `touch run0{1..5}_final.csv` (las llaves son otro truco de bash: generan la secuencia) y renómbralos en lote quitando `_final` — primero en seco con `echo mv`, luego de verdad.

*(Solución al final del capítulo.)*

### Ejercicio D.2 · Un script con criterio

Escribe `resumen.sh` que reciba un fichero de log como argumento, compruebe que existe (y proteste si no), y escriba cuántas líneas `ERROR`, `WARN` e `INFO` tiene — con un bucle sobre los tres niveles.

*(Solución al final del capítulo.)*

### Ejercicio D.3 · Copia nocturna (opcional)

Programa con `cron` una copia diaria de tu carpeta de física a `debian-lab` con `rsync`, a una hora en la que el portátil suela estar encendido. Prueba primero el comando a mano; después, cambia la hora a «dentro de dos minutos» para ver que `cron` lo ejecuta, y déjalo en su hora definitiva.

*(Solución al final del capítulo.)*

## Soluciones del capítulo

### Ejercicio D.1 · La deuda del capítulo 2

Solución razonada

`for f in *_final.csv; do echo mv "$f" "${f/_final/}"; done` enseña las cinco órdenes; sin el `echo`, las ejecuta. `ls` confirma.
**Por qué así** — El ensayo en seco es el hábito que convierte un bucle con `mv` (o `rm`) de peligro en herramienta. Te costó cinco renombrados en el capítulo 2; con cinco mil costaría lo mismo.

### Ejercicio D.2 · Un script con criterio

Solución razonada

```
#!/bin/bash
set -e
[ -f "$1" ] || { echo "uso: $0 FICHERO.log"; exit 1; }
for nivel in ERROR WARN INFO; do
    echo "$nivel: $(grep -c " $nivel " "$1")"
```

```
done
```

**Por qué así** — Tres piezas del anexo en ocho líneas: argumento, comprobación con salida de error, y bucle con una tubería dentro. `[ … ] || { …; }` es la forma compacta de «si no, protesta y sal».

### Ejercicio D.3 · Copia nocturna (opcional)

Solución razonada

`crontab -e` y una línea como `15 22 * * * rsync -a /home/marcos/fisica/ lab:~/copia_fisica/ >> /home/marcos/copia.log 2>&1`. La prueba «dentro de dos minutos» confirma que la clave SSH (capítulo 12) funciona sin ti — `cron` no puede teclear frases de paso, así que si pusiste una, hará falta un agente de claves o una clave sin frase solo para esto.
**Por qué así** — Es el primer proceso *automático* de tu vida como administrador: una copia que ocurre aunque lo olvides. El detalle de la frase de paso es el clásico tropiezo de las tareas programadas — y ahora ya lo conoces.

# E Montar de todo: discos, pendrives y unidades remotas

El capítulo 6 enseñó la idea — un disco se *engancha* en un directorio del árbol único — y Mint la aplica sola cada vez que pinchas un pendrive. Este anexo la lleva hasta el final: montar a mano, montar discos de otros formatos, hacer un montaje permanente, y enganchar carpetas que viven en *otra máquina* — por SSH, por el protocolo de Windows y por el de los laboratorios. Todo por terminal y todo por ventanas, porque aquí las dos vías se usan de verdad. Requiere los capítulos 6 y 11.

## Ver qué hay montado

Dos comandos que ya conoces y uno nuevo: `lsblk -f` (con `-f`, enseña el sistema de ficheros y la etiqueta de cada partición), `df -h` (lo montado, con su espacio), y `findmnt`, que dibuja el árbol de montajes entero — útil cuando `df` es demasiado largo:

```
marcos@portatil:~$ lsblk -f
NAME        FSTYPE FSVER LABEL     UUID                                 MOUNTPOINTS
sdb
 sdb1       exfat  1.0   VIAJE     6A3F-1B2C                            /media/marcos/VIAJE
nvme0n1
 nvme0n1p1 vfat   FAT32           7C21-9E04                            /boot/efi
 nvme0n1p2 ext4   1.0             3f8a2c1e-…                           /
marcos@portatil:~$ findmnt /media/marcos/VIAJE
TARGET                SOURCE    FSTYPE OPTIONS
/media/marcos/VIAJE  /dev/sdb1 exfat  rw,nosuid,nodev,relatime,uid=1000,gid=1000
```

Fíjate en el **UUID**: el identificador único de cada partición. Los nombres `sdb1` cambian según el orden en que enchufes las cosas; el UUID no. Por eso los montajes permanentes se escriben con él.

## Montar a mano un disco que conectas

Mint monta los pendrives en `/media/tu_usuario/ETIQUETA`. En un servidor sin escritorio no hay nadie que lo haga, y a veces en el portátil quieres controlarlo — montar solo lectura un disco antiguo, por ejemplo, para no tocar nada. El gesto: un directorio vacío como punto de montaje, `mount`, `umount`:

```
marcos@portatil:~$ sudo mkdir -p /mnt/viejo
marcos@portatil:~$ sudo mount -o ro /dev/sdb1 /mnt/viejo
marcos@portatil:~$ ls /mnt/viejo
tesis_2019  fotos  backup_windows
marcos@portatil:~$ sudo umount /mnt/viejo
```

`-o ro` (*read-only*) es el seguro: nada de lo que hagas podrá escribir en ese disco. `/mnt` es el sitio tradicional para montajes manuales — `/media` es de los automáticos. Y **otros formatos** funcionan igual: `mount` detecta solo NTFS (el de Windows; Mint trae el controlador), exFAT y FAT32, y los ext4 de otro Linux. Con ext4 ajeno verás una curiosidad del capítulo 4: los ficheros aparecen con el *número* de usuario que tenían allí (`uid 1001`, quizá sin nombre) — los permisos viajan con el disco, los nombres no. Si `mount` protesta con «tipo de sistema de ficheros desconocido», falta el paquete del formato (`exfatprogs`, `ntfs-3g`); `apt` lo resuelve.

**Truco · La aplicación «Discos»**

Menú → Accesorios → **Discos** (`gnome-disks`) es la cara gráfica de todo este anexo, y muy buena: elige el disco a la izquierda, la partición en el dibujo, y los botones ▶ / ⏏ montan y desmontan. El icono de engranaje tiene *Editar opciones de montaje*, que es — literalmente — la forma con ratón de escribir la línea de `fstab` de la sección siguiente, con el UUID puesto por ti y sin riesgo de errata. También formatea y comprueba discos. Para un disco suelto en tu portátil, empieza por aquí; la terminal es para el servidor y para entender qué hizo el botón. Y para *cambiar* particiones — crear, borrar, redimensionar — la herramienta es **GParted** (capítulo 6), que va un paso más allá de «Discos» y exige el mismo respeto que `dd`.

## Montaje permanente: `/etc/fstab`

Un montaje a mano dura hasta el reinicio. Para que un disco de datos aparezca *siempre* en el mismo sitio, se apunta en `/etc/fstab` (*file system table*) — un fichero de texto de `/etc`, cómo no — con el protocolo del capítulo 10: copia antes, vuelta atrás escrita. Cada línea son seis campos:

```
# dispositivo (por UUID)                 punto de montaje  tipo   opciones          dump
UUID=3f8a2c1e-…                           /                 ext4   errors=remount-ro 0
UUID=7C21-9E04                            /boot/efi         vfat   umask=0077        0
UUID=9d4b7e20-…                           /datos            ext4   defaults,nofail   0
```

Las dos primeras las escribió el instalador del capítulo 8 — ahora puedes leerlas. La tercera es tuya: un segundo disco interno con tus datos, montado en `/datos` en cada arranque. Dos reglas que evitan la avería clásica: **`nofail`** — si el disco no está, el sistema arranca igual en vez de quedarse esperando —, y **probar sin reiniciar** con `sudo mount -a`, que monta todo lo de `fstab` que no esté montado y te enseña el error si te has equivocado. Una línea mal escrita sin `nofail` puede dejar el arranque en modo de emergencia; con la copia de seguridad y `mount -a`, nunca llegarás ahí.

## Carpetas de otra máquina, I: SSHFS

Aquí entra el capítulo 12. Si puedes entrar en una máquina por SSH, puedes montar cualquier carpeta suya en tu árbol como si fuera un disco — sin administrar nada en el servidor:

```
marcos@portatil:~$ sudo apt install sshfs
marcos@portatil:~$ mkdir ~/lab
marcos@portatil:~$ sshfs lab:/home/marcos ~/lab
marcos@portatil:~$ ls ~/lab
adquisicion.log  proyecto  simula.py
marcos@portatil:~$ fusermount -u ~/lab
```

Desde ese momento tu editor, tu Python, tu gestor de archivos ven `~/lab` como una carpeta local — cada lectura y escritura viaja por SSH con tu clave. Se desmonta con `fusermount -u` (no hace falta `sudo`: el montaje es tuyo). Es la herramienta perfecta para editar cómodamente ficheros de un servidor donde no tienes más que SSH — un clúster incluido —; a cambio, es lento para carpetas grandes.

## Carpetas de otra máquina, II: SMB y NFS

Dos protocolos más, cada uno con su mundo. **SMB** (también llamado CIFS o «compartir de Windows») es lo que hablan los NAS domésticos, las impresoras y los PC con Windows. **NFS** es el de Unix: es lo que hay debajo cuando en el laboratorio o el clúster tu casa «aparece» en todas las máquinas a la vez.

Por ventanas, los dos son un solo gesto: en el gestor de archivos (Nemo en tu Mint; Caja, Nautilus o Thunar en otros escritorios), *Archivo → Conectar al servidor…* y una dirección: `smb://nas/datos`, `sftp://lab/home/marcos`, `nfs://servidor/export`. La carpeta aparece en la barra lateral y todo es arrastrar y soltar. Por terminal, cada uno tiene su paquete y su `-t`:

```
marcos@portatil:~$ sudo apt install cifs-utils nfs-common
marcos@portatil:~$ sudo mkdir -p /mnt/nas /mnt/lab
marcos@portatil:~$ sudo mount -t cifs //nas/datos /mnt/nas -o username=marcos,uid=1000,gid=1000
Password for marcos@//nas/datos: ********
marcos@portatil:~$ sudo mount -t nfs debian-lab:/home/marcos/proyecto /mnt/lab
marcos@portatil:~$ findmnt -t cifs,nfs
```

En SMB, `uid=1000,gid=1000` hace que los ficheros aparezcan como tuyos (el protocolo no entiende de usuarios Linux); en NFS los permisos viajan de verdad, así que tu usuario debe tener **el mismo número** en las dos máquinas — `id`, capítulo 4 — o verás ficheros ajenos. Para hacerlos permanentes, `fstab` con dos opciones más: `_netdev` («espera a la red») y, en SMB, un fichero de credenciales con permisos `600` en vez de la contraseña a la vista:

```
//nas/datos                          /mnt/nas  cifs  credentials=/etc/smb-nas,uid=1000,gid=10
debian-lab:/home/marcos/proyecto  /mnt/lab  nfs   defaults,nofail,_netdev
```

¿Cuál usar? **SSHFS** cuando solo tienes SSH y no administras el servidor. **SMB** cuando el otro lado es un NAS, un Windows o la impresora de casa. **NFS** cuando tú (o el laboratorio) administráis ambos lados y quieres rendimiento y permisos de verdad. Y por ventanas, los tres son la misma pantalla de *Conectar al servidor*.

## Windows y la nube

Los dos casos que más se preguntan, resueltos con lo anterior. **De Windows a tu Linux y al revés**: Windows 10 y 11 traen el cliente OpenSSH de serie, así que desde PowerShell funcionan los mismos `scp` y `sftp` del capítulo 12 contra tu Mint (con `openssh-server` instalado, ejercicio 11.1) o contra `debian-lab`:

```
PS C:\Users\Marcos> scp .\medidas.csv marcos@192.168.1.34:~/fisica/
PS C:\Users\Marcos> sftp marcos@192.168.1.34
```

Con ratón, en Windows, **WinSCP** (gratuito) es el «Conectar al servidor» del otro lado: arrastrar y soltar por SFTP. En sentido contrario, la vía natural es SMB: compartir una carpeta de Windows (propiedades → Compartir) y montarla desde Linux como en la sección anterior — o compartir una carpeta de tu Linux desde Nemo (clic derecho → *Opciones de compartición*; Mint instala Samba la primera vez) para que Windows la vea en «Red». **Con la nube**: el navegador siempre vale (Drive, Dropbox, OneDrive, el Nextcloud de la universidad); para hacerlo desde la terminal o montarla como carpeta existe `rclone` (`sudo apt install rclone`, `rclone config` te guía para conectar la cuenta, y después `rclone copy` o `rclone mount`), sin `sudo` ni permanencia — el mismo espíritu que SSHFS, con la nube al otro lado.

## Servir una carpeta desde tu VM por NFS

Para ver NFS entero, el servidor lo puedes montar tú en `debian-lab` — con instantánea antes, como siempre:

```
marcos@debian-lab:~$ sudo apt install nfs-kernel-server
marcos@debian-lab:~$ echo "/home/marcos/proyecto 192.168.122.0/24(rw,sync,no_subtree_check)" | sudo
marcos@debian-lab:~$ sudo exportfs -ra
marcos@debian-lab:~$ sudo exportfs -v
/home/marcos/proyecto  192.168.122.0/24(rw,sync,no_subtree_check)
```

`/etc/exports` dice qué carpeta se comparte y con quién — aquí, con toda la red NAT del capítulo 7 —; `exportfs -ra` la publica. Desde tu Mint, el `mount -t nfs` de la sección anterior la engancha, y con el mismo `uid 1000` en ambas máquinas, lo que escribas en `/mnt/lab` aparece en la VM como tuyo. Es exactamente lo que verás en un clúster: tu casa está en un servidor NFS, y cada nodo de cálculo la monta.

## Práctica

**Ejercicio E.1 · A mano y solo lectura**

Con un pendrive: desmóntalo desde el gestor de archivos, móntalo a mano en `/mnt/usb` en modo solo lectura, intenta crear un fichero dentro (debe fallar), desmonta, y vuelve a montar sin `ro`. Observa `findmnt /mnt/usb` en cada estado.

*(Solución al final del capítulo.)*

**Ejercicio E.2 · Permanente, por las dos vías**

Si tienes una partición de datos (o el propio pendrive, para practicar): añádele un montaje permanente primero con «Discos → Editar opciones de montaje», y mira qué línea ha escrito en `/etc/fstab` (`cat`). Después bórrala con `nano`, escríbela tú a mano con `nofail`, y compruébala con `sudo mount -a`. Copia de `fstab` antes de todo.

*(Solución al final del capítulo.)*

**Ejercicio E.3 · La VM como carpeta**

Monta la casa de `debian-lab` con SSHFS en `~/lab`, edita desde tu Mint (con el editor que quieras) el `informe.txt` del proyecto, y comprueba desde dentro de la VM (`ssh lab cat …`) que el cambio está allí. Desmonta.

*(Solución al final del capítulo.)*

**Ejercicio E.4 · NFS de punta a punta (opcional)**

Con instantánea de la VM: exporta `~/proyecto` de `debian-lab` por NFS, móntalo en tu Mint en `/mnt/lab`, crea un fichero desde cada lado y comprueba con `ls -l` que ambos aparecen como `marcos` en las dos máquinas. Hazlo permanente en `fstab` con `nofail,_netdev` y verifica con `mount -a`.

*(Solución al final del capítulo.)*

## Soluciones del capítulo

**Ejercicio E.1 · A mano y solo lectura**

Solución razonada

`sudo mkdir -p /mnt/usb`, `sudo mount -o ro /dev/sdb1 /mnt/usb` (tu letra según `lsblk`), `touch /mnt/usb/prueba` → «sistema de ficheros de solo lectura»; `sudo umount /mnt/usb`; `sudo mount /dev/sdb1 /mnt/usb` y el `touch` funciona (en FAT/exFAT, con los permisos `rwxrwxrwx` del capítulo 6).
**Por qué así** — El montaje de solo lectura es el gesto con el que se examina un disco ajeno o dañado sin riesgo. Y verlo fallar al escribir es la mejor demostración de que la opción manda.

**Ejercicio E.2 · Permanente, por las dos vías**

Solución razonada

«Discos» escribe algo como `UUID=… /mnt/datos auto nosuid,nodev, nofail,x-gvfs-show 0 0`. La tuya a mano: `UUID=… /mnt/datos ext4 defaults,nofail 0 2` (o `exfat` con `uid=1000,gid=1000` para un pendrive). `sudo mount -a` sin quejas y `findmnt /mnt/datos` la confirma.
**Por qué así** — Ver la línea que genera la aplicación gráfica y escribir la tuya después es la forma honesta de aprender `fstab`: entiendes cada campo y pierdes el miedo. `nofail` y `mount -a` son la red de seguridad que te separa del arranque en modo emergencia.

**Ejercicio E.3 · La VM como carpeta**

Solución razonada

`sshfs lab:/home/marcos ~/lab`, editar `~/lab/proyecto/resultados/informe.txt`, `ssh lab cat proyecto/resultados/informe.txt` muestra el cambio, `fusermount -u ~/lab`.
**Por qué así** — Cada escritura viajó por SSH con tu clave: el mismo túnel de `scp`, presentado como un directorio. Es la forma más cómoda de trabajar con ficheros de un servidor donde no administras nada.

**Ejercicio E.4 · NFS de punta a punta (opcional)**

Solución razonada

Servidor: `nfs-kernel-server`, línea en `/etc/exports`, `exportfs -ra`. Cliente: `nfs-common`, `sudo mount -t nfs debian-lab:/home/marcos/ proyecto /mnt/lab`. Los ficheros son de `marcos` a ambos lados porque los dos usuarios tienen `uid 1000`. Línea de `fstab` con `_netdev` y `nofail`; `sudo umount /mnt/lab && sudo mount -a` la prueba.
**Por qué así** — Has montado la infraestructura de un laboratorio en miniatura: una casa compartida por red con permisos reales. El día que en el clúster te digan «tu `$HOME` está en NFS», sabrás exactamente qué significa — y por qué tu `uid` importa.